Malware Analysis - Notes

IMPORTANT WARNING

- These notes have been made starting from my notes taken during lessons and the power points published by the professor, then rearranged through the help of Claude to produce a narrative flow.
- All the content belongs to the Professor of the course.
- While I try to ensure everything is inside these notes, please don't use them as a "source of truth", and study from official sources as well.

MALWARE ANALYSIS part

CHAPTER 1: Introduction

Malware Analysis and Incident Forensics

The Anatomy of an Intrusion: The Cyber Kill Chain

Every cyberattack, however sophisticated or crude, can be decomposed into a sequence of discrete stages. This decomposition is not merely academic: it gives defenders a shared vocabulary for describing where in an attack's lifecycle a particular piece of evidence belongs, and it gives malware analysts a checklist of behaviors to expect and hunt for when dissecting a sample. The most widely adopted model for this decomposition is the **intrusion kill chain**, originally formulated by Lockheed Martin. It describes, stage by stage, the process by which perpetrators carry out a cyberattack, from the earliest scouting activities through to the final exploitation of the compromised environment.

The chain begins with **Reconnaissance**, the stage in which the attacker studies the target in search of vulnerabilities. This search is not limited to software flaws: it extends to the identification of weak points in an organization's technical stack (outdated software, misconfigured services) as well as its human element (employees who might be susceptible to social engineering). Reconnaissance is the intelligence-gathering phase that determines what avenues of attack will even be feasible.

Once a vulnerability or weakness has been identified, the attacker moves to **Weaponization**. This is the stage in which the attacker crafts an artifact capable of exploiting the discovered vulnerability — for instance, engineering a payload that triggers a buffer overflow in a specific piece of software. Weaponization transforms abstract knowledge of a flaw into a concrete, usable exploit.

The third stage is **Delivery**, the mechanism by which the weaponized artifact is transported to the victim. Delivery can take many forms: a phishing email carrying a malicious attachment, a booby-trapped USB drive left in a parking lot, a drive-by download from a compromised website, and so forth. The delivery mechanism is often what a defender first observes, since it is the point at which the malicious content crosses into the victim's environment.

Exploitation follows delivery: this is the moment at which the vulnerability identified during reconnaissance and targeted during weaponization is actually triggered, granting the attacker some initial foothold or code execution capability on the target system.

After exploitation succeeds, the attacker typically seeks to secure and extend that foothold. This is where the behavioral properties that malware analysts are trained to recognize come into play.

Persistence

Persistence is the property by which malware achieves and guarantees its own survivability within the compromised system over time. Consider a straightforward but illustrative scenario: the infected machine is rebooted, meaning the computer is fully shut down and then powered back on. If persistence has been correctly achieved, the malware will resume execution automatically the moment the system boots again, without any further action required from the attacker. In other words, persistence is about ensuring the malicious code survives beyond the singular moment of infection.

To achieve this survivability, malware typically must tamper with the operating system's configuration — modifying registry keys, scheduled tasks, service definitions, startup folders, library search paths, or similar mechanisms that the OS consults when deciding what code to execute automatically. This tampering process can vary enormously in character: it might be conducted in a very noisy, conspicuous fashion, or it might be executed with extreme care to avoid detection. Regardless of the specific technique, the changes made to achieve persistence are frequently designed to be hidden from casual inspection, since an administrator who stumbles upon an unfamiliar autorun entry might well investigate and remove it.

Uniqueness ("Single-Instance" Behavior)

A second behavioral property commonly exhibited by malware is **uniqueness**, sometimes described as "single-instance" execution. Malware authors generally want to guarantee that only one copy of their malicious code is running on a given machine at any time. This might seem counterintuitive at first glance — surely more running instances of malicious code would only benefit the attacker? — but in practice, multiple concurrent executions of the same malware are frequently meaningless or even actively harmful to the malware's own operation. Two instances might compete for the same system resources, interfere with each other's tasks, or produce undesired side effects. A vivid illustration of this danger is ransomware: if two instances of the same ransomware strain began encrypting the filesystem simultaneously, the result could be file corruption or an encryption process that breaks its own functionality, rendering the ransom note's promise of a decryption key useless even to the attacker. To prevent this outcome, malware commonly performs start-up checks — for instance, creating a named mutex or checking for a marker file — to verify whether the machine has already been infected. If an existing infection is detected, the new instance will typically decline to create a second infection, though it might instead choose to update the existing one.

Targeting

Not all malware is indiscriminate. Some threats are explicitly designed to infect only specific, pre-selected victims, and such malware needs a way to recognize whether it has landed on the intended target or on some other, irrelevant machine. This behavioral property is called **Targeting**. When a sample lands on a machine that is not the intended target, it might choose to terminate its own execution quietly, or it might instead disguise itself by behaving like an entirely different, more innocuous piece of malware or software, thereby throwing off analysts who happen to be observing it.

The mechanisms available to malware for recognizing the "right" victim machine are numerous. A classic technique, especially prevalent among banking trojans, is checking regional and language settings — a piece of malware targeting Eastern European banks, for example, might terminate immediately if it detects that the system locale is set to English (United States). Another approach is checking hardware and software characteristics of the machine, such as the presence of a specific CPU model, particular installed software, or particular configuration settings. Malware can also check for the presence of specific external devices, or search for known documents that would only be present on a machine belonging to

the intended target organization. In essence, the malware fingerprints the machine it has landed on in order to determine whether it matches the profile of the intended victim.

Obfuscation and Evasion

Eventually, defenders will notice something is wrong, and analysts will begin dissecting the malware to understand how it operates. Malware authors know this is inevitable and therefore invest considerable effort into making their creations difficult to analyze. This concern gives rise to two closely related properties: **Obfuscation** and **Evasion**.

Obfuscation refers to techniques that make data and code structures genuinely difficult to interpret, even when an analyst has full access to the binary. This can involve scrambling the malware's own code and data structures using complex, deliberately convoluted schemes, so that any easily discoverable hints of the sample's malicious nature are hidden from casual static inspection. Malware may also inject its code into the address space of other legitimate processes in order to conceal its real identity from process-monitoring tools, or it may employ a broader arsenal of techniques specifically engineered to hamper the analyst's ability to correctly understand its behavior. Malicious code might also be spread across multiple components or delivery stages, making it harder for an analyst to assemble the complete picture of how the attack functions, since no single artifact contains the whole story.

Evasion, meanwhile, refers to the malware's ability to actively hide its own presence and effects while operating, and to present a different, innocuous appearance if it detects that it is being analyzed — for instance, by a sandbox, a debugger, or antivirus software. The overarching goal of both obfuscation and evasion is to allow the malware to remain undercover on the infected machine for as long as possible, maximizing the window during which it can operate undetected. There are, however, notable exceptions to this "stay hidden" philosophy: ransomware, for example, has no interest whatsoever in remaining covert once it has finished encrypting the victim's files, since its entire business model depends on making its presence loudly and unmistakably known in order to extract a ransom payment.

Fingerprinting and Beaconing

Once the attack has succeeded in compromising the machine, the malicious software typically needs to inform the attacker that the operation was successful. This is accomplished by reusing many of the same techniques employed for targeting: the malware inspects the infected system and collects descriptive information about it, then transmits this description back to the attacker, commonly via a remote server under the attacker's control. This notification stage is referred to as **Beaconing** — the malware "beacons" its presence and characteristics back to its controller, confirming that the infection has taken hold and providing the attacker with the details needed to manage the newly compromised asset.

Communication

Beyond the initial beacon, malware frequently maintains ongoing **Communication** with external Command and Control (C2) infrastructure. This communication channel serves multiple purposes: it allows the malware to receive further instructions and commands from the attacker, to update its own internal components (effectively self-updating in the field), and to exfiltrate data harvested from the victim machine back to the attacker.

The methodologies used to implement this communication are impressively heterogeneous, reflecting the attacker's need to blend malicious traffic in with the ordinary noise of network activity. Communication may take place through direct links to attacker-controlled servers, but it can equally be routed through online storage services, from which the malware simply reads commands as though it were retrieving an

ordinary file — from a defender's vantage point, all that is visible is a request to a legitimate cloud storage provider, with nothing to obviously flag it as malicious. Social media platforms present another channel: fake accounts are sometimes used to host commands or facilitate data exfiltration, again blending in with ordinary internet traffic. Malware may also communicate through network services, most notably DNS queries, using the domain name resolution system as a covert channel for encoding and transmitting information. Whatever channel is chosen, the data exchanged over it is frequently obfuscated, ensuring that even if the traffic itself is intercepted, its contents remain opaque to an observer.

CHAPTER 2: x86 Assembly

The Nature of Binary Analysis

The vast majority of the malware samples an analyst will encounter arrive in the form of executables — binary content that a processor can directly execute. Fortunately, an analyst does not need to painstakingly examine every single byte of a malware sample; only specific portions of a binary typically warrant the deep, instruction-by-instruction scrutiny that reverse engineering demands, and there exists a range of tools designed to help focus that effort where it matters most.

At the most fundamental level, a computer executes a sequence of instructions. Malware, however, often "writes" these instructions in ways that diverge meaningfully from how a standard, benign program would be structured. A malicious sample might employ specific instructions that accomplish a given task in a highly inefficient or unusual way, purely because that unusual approach helps it evade signature-based detection or confuses automated analysis tools.

Disassemblers allow an analyst to examine code starting from the executable's declared entry point — the address, specified within the executable file's header, that dictates which instruction must be executed first when the program is loaded and run. A critical subtlety of the x86 architecture is that instructions are encoded as sequences of bytes, and the interpretation of any given byte depends entirely on the context established by what has already been read and decoded up to that point. Ordinarily, when reading a book, one begins at the first sentence and proceeds linearly to the end. Compiler-generated code behaves in a broadly analogous fashion: control flow, while it does branch and jump, tends to respect clean boundaries between instructions. Malware, by contrast, might begin reading its own instruction stream in an unusual order, jump abruptly to a location that is not the "obvious" next instruction, and then continue reading from what amounts to the middle of what a naive disassembler would have interpreted as a completely different instruction. The effect is somewhat like a reader whose eyes jump mid-sentence to a different point on the page and continue reading from there, deriving a sequence of words that bears no resemblance to what the original, linear reading would have produced. This deliberately anomalous structuring does not arise by accident — it exists specifically because the malware's author does not want the code to be easily analyzed. Malware may go even further and rewrite its own code at runtime, meaning that the bytes present in the file on disk do not even fully represent what will actually execute in memory. Because of these evasive properties, a careful analyst must, at least in principle, be prepared to analyze code one instruction at a time in order to achieve precision, rather than relying on the assumption that instructions will be laid out consecutively and unambiguously.

A useful class of tools that alleviates some of this burden is the family of **decompilers**, such as Ghidra and Hex-Rays, which attempt to reconstruct a high-level, C-like representation of a fragment of binary code. There are certain instruction sequences in assembly that are extremely verbose when read in their raw form but that correspond to easily recognizable, common patterns; decompilers are designed to recognize these patterns and collapse them into more legible high-level constructs. For an analyst working

through their first malware sample, however, even with these tools, the process can consume a great deal of time, and it is worth being candid about their limitations: decompilation is a process far from perfect. Information is irretrievably lost during the original compilation step — variable names, precise data types, and other identifiers that a human programmer relied upon simply do not survive translation into machine code, so a decompiler can only ever offer its best guess at reconstructing plausible substitutes. Furthermore, if obfuscation has been deliberately applied to the binary, decompiling the resulting obfuscated code will simply produce an obfuscated, and correspondingly unhelpful, reconstructed source — the decompiler cannot see through obfuscation any more effectively than a human reading raw assembly can. Malware authors are also known to specifically employ anti-analysis techniques engineered to confuse or break decompilers outright. Despite all these caveats, decompilation remains, on balance, very helpful for reverse-engineering professionals, since even an imperfect, obfuscated-looking C-like reconstruction is often faster to reason about than raw assembly.

It is worth stating plainly, as a foundational fact underpinning this entire subject: malicious code is not equivalent to compiler-generated code. Where a compiler produces code following predictable, well-understood idioms optimized for correctness and performance, a human or semi-automated malware-crafting process introduces obfuscation of control and data flows, subtle semantics such as an instruction that deliberately jumps into the middle of another instruction's byte sequence (exploiting exactly the byte-stream ambiguity described above), hand-written instruction sequences such as custom API-hashing algorithms used to resolve system calls without leaving human-readable strings in the binary, self-modifying code, and a range of other decoys designed specifically to mislead automated and manual analysis alike.

CPU and Memory Basics

To reverse engineer code effectively, an analyst must understand the substrate on which that code executes: the central processing unit and its relationship to memory.

Programmers, when writing high-level source code, think in terms of an essentially unlimited number of variables, each of whatever type is convenient for the problem at hand. The central processing unit, however, offers no such luxury. From the CPU's perspective, it is simply a collection of memory cells of specific, fixed sizes; there exist some cells of larger sizes as well, primarily for optimization purposes in domains like multimedia processing. Within the CPU itself, these cells are known as **registers**, and here the gap between the programmer's abstraction and the hardware's reality becomes especially stark: rather than an unlimited supply of named variables, the programmer working at this level has access to only a small, fixed number of general-purpose registers. On a modern (64-bit) CPU, there are additional special registers that are not directly controlled by the programmer — some exist purely to support debugging, and others are reserved for use by the operating system.

The registers directly usable by ordinary programs are known as the **general-purpose registers** (commonly labeled EAX, EBX, ECX, EDX, and so on in the 32-bit x86 world). The key architectural fact to internalize here is the tension between abstraction and hardware reality: high-level programming languages give the illusion of unlimited storage locations, while the CPU physically provides only a handful of extremely fast storage cells. Because registers offer dramatically faster access than main memory, any piece of data that will be accessed frequently benefits enormously from being kept in a register rather than being repeatedly fetched from and written back to memory.

This tension between what a programmer or compiler wants to express and what the hardware can actually provide is formalized through a contractual abstraction layer called the **Instruction Set Architecture (ISA)**. The ISA is an abstract model that describes precisely what a programmer needs

to know in order to program the machine: for x86, this includes the fact that there are eight general-purpose registers, the formats in which arithmetic instructions are expressed, and so on. The ISA functions as an interface, or contract, between software and hardware: humans (or, far more commonly, compilers acting on their behalf) produce ISA-conformant assembly code, and this assembly code is then translated by an assembler into a binary format that the physical hardware can directly consume and execute. Intel originally defined the x86 ISA for its 8086 processor, developed between 1976 and 1978. Over the following decades, later extensions to the architecture have consistently preserved backward compatibility with this original design — a design decision with real consequences for malware analysis, since even today, most malware samples encountered in the wild are still compiled as 32-bit binaries rather than 64-bit. The x86 ISA is also known by the alternate designation IA-32, and its 64-bit successor is referred to as x86-64, or x64 for short.

The prevalence of 32-bit malware, despite essentially all modern machines running 64-bit CPUs, deserves a moment's reflection. All contemporary machines are built around 64-bit CPUs, but these CPUs remain capable of executing 32-bit software, a capability that exists specifically to preserve backward compatibility with older programs. Windows, in particular, goes to considerable lengths to support the execution of older 32-bit software. This continued support for an older execution mode is convenient for a malware author for a very specific reason: it creates additional gaps and inconsistencies between how 32-bit and 64-bit code is monitored, sandboxed, and defended, gaps that a piece of malware can exploit to evade modern defenses. In addition, there remains a nontrivial fraction of machines in the world — particularly older or specialized systems — that are only capable of running 32-bit software at all, so 32-bit malware retains a broader potential victim pool. For both reasons, it remains convenient, from an attacker's perspective, to write malware that targets the 32-bit execution environment.

Word Size and Endianness

A **word** is understood to be the natural data unit for a given CPU design — the size of data that the processor's arithmetic and logic circuitry is fundamentally built to handle in a single operation. For x86 processors specifically, the native word size is 32 bits. This was not always the case: the original word size for the x86 lineage was 16 bits, and it is precisely for this historical reason, rooted in backward compatibility, that x86 instructions still assume, by default, that a "word" operand is 16 bits long, while a 32-bit operand is instead referred to using the distinct term "dword" (double word). The common IA-32 data type nomenclature, then, comprises byte (8 bits), word (16 bits), and dword (32 bits).

Memory itself, regardless of how large a logical data unit a CPU register might hold, always operates at the byte level — every individually addressable location in memory holds exactly one byte. This raises an immediate practical question: if a register can hold a 32-bit (4-byte) quantity with no difficulty, how does one write that same 32-bit quantity out to memory, which can only address and store data one byte at a time? The answer requires a rule that determines, precisely, in what order the individual bytes composing a multi-byte value should be written to (and subsequently read from) consecutive memory addresses. This rule is known as **Endianness**.

Consider a concrete example: suppose the goal is to store the dword value 0xABADCAFE at memory address 0x1000. This 32-bit value decomposes into four constituent bytes: AB (the most significant byte, MSB) and FE (the least significant byte, LSB), with AD and CA in between. There are exactly two conventions for how to lay these four bytes out across the four consecutive memory addresses 0x1000 through 0x1003. Under **Big Endian** encoding, the most significant byte is written first, so address 0x1000 would hold AB, followed by AD, CA, and finally FE at address 0x1003. Under **Little Endian** encoding, the convention is reversed: the least significant byte is written first, so address 0x1000 would hold FE, followed by CA, AD, and finally AB at address 0x1003. The x86 architecture uses Little Endian

encoding throughout. This detail is far from a mere curiosity for the malware analyst: any time raw memory or a hex dump is being read and interpreted as a multi-byte numeric value or pointer, the bytes must be mentally (or programmatically) reversed to reconstruct the intended value.

Registers in Detail

As already noted, there exists a fundamental mismatch between the programmer's mental model, in which arbitrarily many named variables can be created, and the CPU's physical reality, in which computation is confined to a small number of registers. Registers hold data with extremely fast access time compared to main memory, which is precisely why they are used as the CPU's primary working storage during computation.

Among the registers available on an x86 CPU, several categories are worth distinguishing. The **general-purpose registers** (GPRs) can store either ordinary data values or memory addresses (pointers) — the CPU imposes no rigid distinction between the two; it is purely a matter of how the surrounding code chooses to interpret the bits held within a given register. The **status register** holds a collection of truth values (individual bits, each with a specific meaning) reflecting the state of the processor immediately after the execution of the most recent instruction — this is heavily used, for instance, in evaluating conditional branches. The **program counter** holds the memory address of the instruction currently being executed, effectively tracking the CPU's position within the running program. Beyond these, there exist many other, more specialized registers serving various architectural purposes. On the x86 architecture specifically, the general-purpose registers are 32 bits wide.

Accessing Registers

The x86 architecture provides eight general-purpose registers: commonly referred to by the letters A, B, C, D, along with SI and DI (source index and destination index), plus two registers with special conventional roles. The **ESP** register conventionally points to the top of the stack, and the **EBP** register is often set equal to ESP at the beginning of a function's execution, establishing what is called a base pointer for that function's local data — though, when a function chooses not to use this convention, EBP remains available for general-purpose use like any other register.

The prefix **E** on a register name (as in EAX, EBX, and so on) indicates that the full 32-bit size of the register is being referenced; this E prefix stands for "Extended." When the E prefix is absent, only the least significant 16 bits (that is, the lowest 2 bytes) of the register are manipulated — so, for instance, AX refers to the lower 16 bits of EAX. Furthermore, for the four registers A, B, C, and D specifically, the two least significant bytes of that 16-bit portion can themselves be addressed individually: the high byte of that 16-bit segment is accessed using a name like AH ("A high"), and the low byte is accessed using a name like AL ("A low"). This layered addressing scheme — a full 32-bit register containing a 16-bit sub-register, which in turn contains two individually addressable 8-bit sub-registers — reflects the architecture's historical evolution from 8-bit, to 16-bit, to 32-bit generations, with each new generation extending rather than replacing what came before.

Two particularly important registers cannot be accessed directly by ordinary instructions: the program counter, referred to on x86 as **EIP**, and the status register, referred to as **EFLAGS**. Both are updated implicitly as a side effect of instruction execution rather than being directly writable the way a general-purpose register is.

The Instruction Cycle

The CPU operates according to the classic **fetch-decode-execute cycle**. During the **fetch** stage, the CPU reads the instruction located at the memory address currently held in EIP. During the **decode** stage, the control unit interprets the raw bytes just fetched to determine precisely what operation and operands the instruction specifies. During the **execute** stage, the specified computation is actually carried out, whether that involves the arithmetic logic unit (ALU) performing some calculation or simply moving data between locations. After execution completes, EIP ordinarily advances to point at the next instruction immediately adjacent in memory — unless the instruction that was just executed explicitly altered the control flow, for instance via a jump, in which case EIP is instead set to whatever destination address the jump specified.

Instructions on x86 vary considerably in their encoded size, ranging anywhere from 1 to 15 bytes, and are stored consecutively in memory along with any immediate operands — meaning literal data values or addresses — that they require as part of their operation. Register operands, by contrast, are not stored as separate bytes following the instruction; instead, they are embedded directly within the binary representation of the instruction's opcode itself. This is illustrated by the following pair of encodings:

```
b8 01 00 00 00     mov eax, 0x1
bf 01 00 00 00     mov edi, 0x1
```

Here, both instructions move the same immediate value (0x1, encoded as the four bytes 01 00 00 00 in little-endian order) into a register, but the specific destination register — EAX in the first case, EDI in the second — is encoded not as a separate operand byte sequence but as part of the opcode byte itself (b8 versus bf). This is a direct illustration of the earlier point that register operands live inside the opcode's own binary representation.

Memory Map of a Process

To fully appreciate how a running program is laid out, it is useful to consider a simplified view of the virtual address space of a 32-bit Windows program. In this simplified model, the lower 2 gigabytes of the address space are reserved for user-space code and data, while the upper 2 gigabytes are reserved for kernel-mode code, inaccessible to ordinary user-mode processes.

Within the user-space portion of the address space, several distinct regions can be identified. At the very base of the address range sits the **stack for the main thread**, a region of memory that, notably, grows toward lower addresses as it is used — meaning that as more data is pushed onto the stack, the boundary of the stack region extends downward, toward address 0x00000000, rather than upward. Adjacent to the stack lies the **heap**, which by contrast grows in the opposite direction, toward higher addresses, as dynamic memory allocations are made. This opposite-direction growth is a deliberate design choice: it allows the two regions to expand toward one another from a shared pool of available address space without immediately colliding, maximizing the flexibility available to a program that might need substantially more stack space, more heap space, or some mixture of both, depending on its runtime behavior.

Further along in the address space sits the **program image** itself, which comprises the PE (Portable Executable) header along with the code section, imports, and resources bundled into the executable file. Beyond this lies a region that is allocatable as stack space for additional threads or as additional heap, followed by dedicated areas for **DLLs** (Dynamic Link Libraries) that the process has loaded, each of which occupies its own region of the address space. Finally, near the very top of the user-accessible range, fixed addresses are reserved for structures such as the first Thread Entry Block, the Process Entry Block,

and a special page that the kernel shares directly with userland code for certain low-overhead communication purposes.

The critical, deliberately emphasized fact from this memory layout is that the **stack grows toward low addresses**. This single detail has significant practical consequences for reading stack-related assembly code and diagrams: when an analyst observes stack addresses in a debugger or in a diagram, addresses that are numerically lower correspond to more recently pushed, "deeper" positions on the stack, while addresses that are numerically higher correspond to older, earlier stack frames closer to the point where the thread began execution.

Memory Protection

The virtual address space described previously is not an undifferentiated expanse in which any code may read, write, or execute at will. The processor and operating system cooperate to enforce rules governing what may be done with each region of memory, and the mechanism that makes this enforcement possible is **paging**.

Under a paging scheme, the operating system divides the entire address space into fixed-size chunks called **pages**, conventionally 4 KB each. Each page carries metadata that answers two distinct questions. First, is this address valid at all — that is, does it correspond to a page that has actually been mapped into the process's address space, or is it unmapped territory that an access would be forbidden from touching? Second, for a page that is valid, what operations are permitted upon it: may it be read, may it be written, may its contents be executed as instructions? These read, write, and execute permissions are enforced at the granularity of regions rather than individual bytes, which is a natural consequence of the page-based organization. A further, security-critical rule layered on top of this is that pages belonging to kernel memory are accessible only from kernel code; any attempt by ordinary user-mode code to touch kernel pages is rejected outright.

This permission machinery is the foundation upon which a significant class of defensive mechanisms is built. Historically, a program's stack was treated as plain memory on which instructions could, in principle, be placed and executed. This permissiveness became a serious liability, because numerous exploitation techniques depend precisely on the attacker's ability to inject instructions into a writable region such as the stack or the heap and then redirect execution into that injected code. In modern operating systems, therefore, the stack is never executable: the pages comprising it are marked writable but not executable, so that even if an attacker succeeds in writing bytes there, the processor will refuse to interpret those bytes as instructions.

The security measure responsible for this behavior is **DEP (Data Execution Prevention)**. DEP enforces a strict separation between regions intended to hold data and regions intended to hold code: it prevents execution not only on the stack but also on the heap, ensuring that memory meant to store data cannot be hijacked as a staging ground for injected instructions. Operating systems may enforce a suite of such high-level security mechanisms specifically designed to hinder memory-vulnerability exploitation attacks, and DEP is one pillar of that suite.

A second major pillar is **ASLR (Address Space Layout Randomization)**. Rather than loading the various regions of a process — the program image, the DLLs, the stack, the heap — at fixed, predictable addresses on every execution, ASLR deliberately randomizes their placement each time a process is spawned. The practical consequence is directly observable: if the same program is executed twice, the addresses assigned to its regions will differ between the two runs. This randomization extends to dynamically loaded libraries as well — on Windows, where libraries take the form of DLLs, their load addresses are subject to the same per-launch randomization. The heap is likewise affected, so that

running a program twice and inspecting the addresses of heap-allocated variables will reveal specific variations between the two executions. The defensive value of this unpredictability is substantial: many exploitation techniques require the attacker to know in advance the precise address of some target (a function to hijack, a buffer to jump into), and if those addresses shift unpredictably on every run, the attacker's hard-coded assumptions break. Beyond DEP and ASLR, more recent Windows versions (Windows 10 and later) introduce additional protections such as heap allocation randomization and other hardening measures that further complicate the exploitation of memory vulnerabilities.

Memory Addressing

Having established how memory is protected, the next question is how instructions actually specify the memory locations they wish to operate upon. **Addressing modes** provide the vocabulary for expressing the addresses of data to be read from or written to main memory. The expressions used to form these addresses can incorporate immediate operands (literal constant values), registers, or a combination of both, and the real power of addressing modes lies in exactly this combinatorial flexibility: they allow operands to be assembled into complex expressions whose computed value depends on the current program state. Because a register's contents change as the program runs, an address expressed in terms of registers is effectively a dynamic address, resolved freshly each time the instruction executes. One important architectural constraint tempers this flexibility: an instruction can usually reference at most one memory operand, so one cannot, for instance, directly copy from one memory location to another in a single ordinary instruction without routing the value through a register.

A crucial point of interpretation underlies all of this. When an address expression is enclosed in square brackets, it denotes a **dereference** — that is, the instruction operates on the *contents* found at the computed memory location, not on the address value itself. So the instruction

```
mov eax, [0x1000]
```

does not place the number 0x1000 into EAX; rather, it reads the value stored in memory at address 0x1000 and places that value into EAX. Similarly,

```
mov eax, [esi]
```

treats the current contents of the ESI register as a memory address, reads the value stored at that address, and places it into EAX.

Common Addressing Modes

The square-bracket expression can be considerably richer than a bare constant or a single register. The architecture supports a family of addressing modes of increasing expressive power, several of which are encountered constantly in real code.

The simplest, an **immediate** address, uses a literal constant as the location, as in `mov eax, [0x1000]`. A **register** address uses the contents of a register as the location, as in `mov eax, [esi]`. Adding a constant yields the **register-plus-offset** mode, as in `mov eax, [esp-8]`, which reads from the location eight bytes below wherever ESP currently points. More elaborate still is the **register-scaled-plus-offset** mode, as in `mov eax, [ebx*4+0xff]`, in which the register's value is first multiplied by a scale factor and then offset by a constant. The most general form combines a base register, a scaled index register, and an offset: **base-plus-register-scaled-plus-offset**, as in `mov eax, [edx+ebx*4+8]`. In the last two forms, the offset field is optional.

This most general form is worth dwelling on, because it maps directly onto an extremely common high-level construct: iterating over an array. Imagine a loop walking through an array of 32-bit integers, each of which occupies 4 bytes. In the expression `[edx+ebx*4+8]`, the base register EDX would hold the base address of the array, the index register EBX would hold the current loop index `i`, the scale factor of 4 accounts for the fact that each integer element is 4 bytes wide, and the constant 8 serves as a fixed displacement into the structure. The processor computes the full effective address in a single step — base plus index-times-element-size plus displacement — which is precisely the arithmetic that accessing `array[i]` requires. Recognizing this idiom allows an analyst to reconstruct array accesses from otherwise cryptic bracket expressions.

Basic x86 Instructions

Every program, benign or malicious, is ultimately assembled from three fundamental kinds of instruction: those that **move data**, those that perform **arithmetic and logic**, and those that control **program flow**. Nearly any computation imaginable can be expressed as some arrangement of these three categories.

Data movement instructions are the most straightforward of the three. The workhorse of this category is **mov**, written in the form `mov dest, src`, which copies a value from a source to a destination. The source may be a register, the contents of a memory location, or an immediate constant; the destination may be a register or a memory location. The essential semantics are simple: whatever value the source denotes is copied into the place the destination denotes, leaving the source unchanged.

The other members of the data-movement family are **push** and **pop**, which exist specifically for manipulating the stack. These two deserve careful attention because they answer a recurring practical problem.

The Problem of Limited Registers and Local Storage

Recall the fundamental tension already established: programmers and compilers reason in terms of abundant named variables, but the CPU provides only a small handful of general-purpose registers. Two situations arise naturally from this scarcity. First, a function may need somewhere to keep its own local working data — storage pertaining to its own execution that must persist across the instructions making up that function. Second, and more acutely, a piece of code may simply run out of free registers: it needs to perform some computation, but every register it could use already holds a value that must not be destroyed.

The solution in both cases is to use the stack as overflow storage. When a register is needed but all of them are occupied, the code can save the current contents of some register onto the stack, freely reuse that register for the computation at hand, and then restore the saved contents afterward — effectively "borrowing" a register for a while. The two operations that make this saving and restoring clean are precisely push and pop.

Concretely, suppose that no free registers remain and the goal is to add to EDX the difference between EBX and ECX, a computation that naturally wants a scratch register to hold the intermediate difference. One approach is to carve out a dedicated temporary memory location, conceptually a slot called TMP, and write:

```
mov TMP, eax
mov eax, ebx
sub eax, ecx
```

```
add edx, eax
mov eax, TMP
```

Here EAX is first saved into TMP, then borrowed to compute `ebx - ecx`, the result of which is added into EDX, after which EAX's original value is restored from TMP. The stack offers a more idiomatic way to accomplish exactly the same borrowing, replacing the explicit temporary slot with push and pop:

```
push eax
mov eax, ebx
sub eax, ecx
add edx, eax
pop eax
```

The `push eax` saves EAX's current value onto the top of the stack; EAX is then freely used as scratch space for the subtraction and addition; and `pop eax` restores the original value from the stack back into EAX. The net effect is identical, but the stack-based version requires no pre-arranged temporary location, since the stack itself serves as an ever-available scratch area.

Arithmetic Instructions

Arithmetic instructions carry more analytical weight than their everyday simplicity might suggest, and they are relevant to code analysis for several interlocking reasons. They are used to compute offsets and function addresses, to modify the stack pointer during stack manipulation, and — significantly — they frequently do double duty as data-movement operations or as a means of updating the EFLAGS status register as a side effect of their computation. Because so many instructions update the flags implicitly, an arithmetic instruction is often placed not for the sake of its numeric result at all, but precisely to set the flags in preparation for a later conditional decision.

The core arithmetic instructions and their effects are as follows. The instruction `add dest, src` performs `dest += src`, adding the source into the destination. Its counterpart `sub dest, src` performs `dest -= src`, subtracting the source from the destination. The `inc dest` instruction increments its operand (`++dest`), while `dec dest` decrements it (`--dest`). Two unary instructions produce complements: `not dest` computes the one's complement of its operand (a bitwise inversion of every bit), and `neg dest` computes the two's complement (the arithmetic negation that turns a value into its signed negative).

Logic Instructions

The logic instructions perform bitwise Boolean operations. The instruction `and dest, src` computes a bitwise AND of its operands, `or dest, src` computes a bitwise OR, and `xor dest, src` computes a bitwise exclusive OR. Several common instances of these instructions recur throughout real-world code. A particularly famous idiom is zeroing a register with `xor eax, eax`: since any value XORed with itself yields zero, this instruction sets EAX to zero, and it is frequently preferred over the more obvious `mov eax, 0` because its encoding is shorter and faster. Logic instructions are also routinely used for extracting specific bits from a value and for checking conditions, and they feature prominently in obfuscation, where XOR in particular is a staple of simple encryption and data-scrambling schemes.

A critical caution must be sounded here, because it is a frequent source of confusion for anyone approaching assembly from a high-level language background. These operations are strictly **bit-wise**: they operate independently on each individual bit position of their operands. This is emphatically *not* the

same as the high-level logical operators `&&` and `||` found in languages such as C, which operate on whole truth values and, moreover, typically short-circuit their evaluation. The bitwise `and` and `or` of x86 have entirely different semantics from their high-level namesakes, and conflating the two will lead directly to misinterpretation of code.

Rotate and Shift Instructions

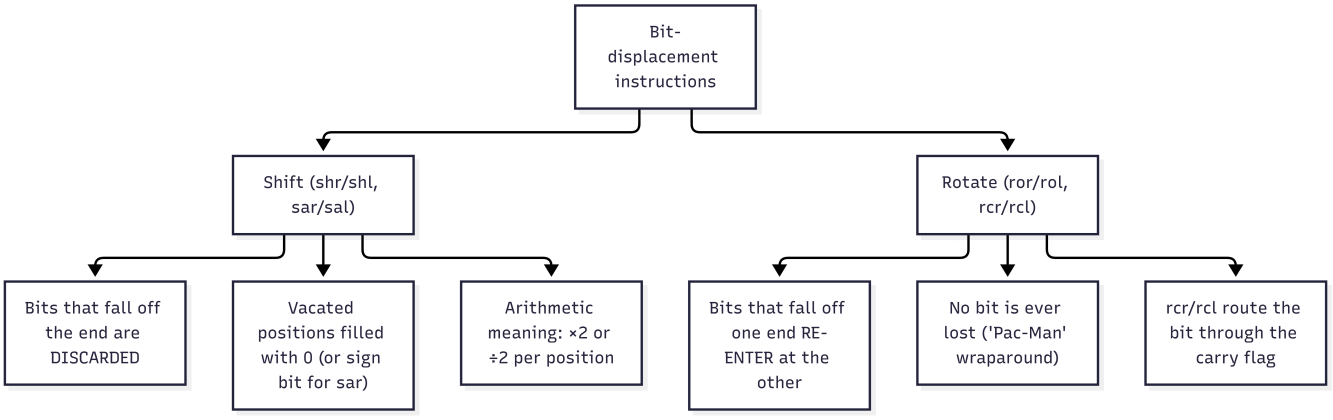
The final family in the arithmetic-and-logic category comprises the rotate and shift instructions, which are frequently encountered in bit-manipulation routines. Shift operations have an important arithmetic interpretation: shifting a binary value left by one position is equivalent to multiplying it by two, and shifting right by one position is equivalent to dividing by two, which makes shifts a fast substitute for multiplication and division by powers of two.

The architecture distinguishes carefully between several variants of these operations, and the distinctions matter.

The unsigned shift instructions are `shr dest, src` and `shl dest, src`, performing an unsigned shift to the right and to the left respectively. In an unsigned shift, bits that fall off one end are discarded, and vacated positions are filled with zeros. The signed shift instructions, `sar dest, src` and `sal dest, src`, perform signed shifts; the signed right shift in particular is designed to preserve the sign of a two's-complement number as it shifts, so that dividing a negative number by a power of two yields the correct negative result rather than a corrupted value.

Rotation differs from shifting in one essential respect: rather than discarding the bits that fall off the end, a rotation feeds them back in at the opposite end. A bit shifted out of the most significant position reappears at the least significant position, and vice versa — a kind of circular "Pac-Man" wraparound in which no bit is ever lost. The plain rotate instructions are `ror dest, src` and `rol dest, src`, rotating right and left. A further variant, `rcr dest, src` and `rcl dest, src`, performs a rotate *through the carry flag*: the carry bit is treated as an additional bit position participating in the circular rotation, so the bit shifted out goes into the carry flag while the previous carry value is shifted into the operand. For all of these shift and rotate instructions, when the source operand specifying the shift or rotation amount is absent, a default count of 1 is assumed.

The conceptual distinction between discarding bits and recirculating them can be summarized as follows:



Toward Program Flow Control

The third and final category of instruction governs **program flow control** — the means by which a program moves its execution from one point in the code to another rather than simply proceeding to the next adjacent instruction. A foundational fact must be kept firmly in mind when reasoning about any

control transfer: by the time an instruction's effect is being evaluated, the program counter has *already advanced* past that instruction to point at the next one. This "pre-advanced" state of the program counter is exactly what makes relative, offset-based jumps work correctly, since offsets are measured from the position the program counter has already reached — the detailed arithmetic of which forms the natural starting point for the study of branching.

Program Flow Control

Control flow is what elevates a program from a mere straight-line list of calculations into something capable of making decisions, repeating work, and organizing itself into reusable units. In x86, control flow can be implemented in three distinct ways, and understanding the differences among them is essential for following the logic of any binary.

The first is the **unconditional branch**: an instruction that replaces the current value of EIP with some desired destination address, causing execution to continue there regardless of any condition. The second is the **conditional branch**, which likewise replaces EIP with a desired address, but only when a condition holds — specifically, a condition evaluated over the value of one or more bits in the EFLAGS register. If the condition does not hold, the branch is not taken, and execution simply falls through to the next adjacent instruction. The third is **function calls and returns**, which are best understood as a special, structured type of unconditional branch, enriched with the extra machinery needed to remember where to come back to once the called routine finishes.

Implementing any of these mechanisms requires two fundamental primitives. The first is a way to specify the target of a branch — the place execution should jump to — which may be given either statically (a fixed address or offset baked into the instruction) or dynamically (an address computed at runtime and held in a register or memory location). The second is a way to evaluate a condition on the program state, so that conditional branches have something to test. As a convenience for human readers and assemblers, code locations in assembly can be annotated with **labels** — symbolic names standing in for addresses — so that a branch can refer to a destination by a meaningful name rather than by a raw numeric address.

Unconditional Branching

An unconditional branch, realized by the `jmp` instruction, can take its destination in one of two broad forms, and the distinction between them is a recurring source of confusion when reading disassembly.

The first form is an **offset**. Here the destination is specified either as a relative offset from the current EIP value, or as an absolute offset measured from the base of the current code segment. The second form is an **absolute address**, supplied directly as a register or memory operand, whose contents are taken to be the exact address to jump to.

The essential subtlety, and the reason offsets cause trouble, stems directly from the fact established earlier: by the time a jump is evaluated, EIP has already advanced past the jump instruction to point at whatever follows it. A relative offset is therefore added not to the address of the jump itself, but to the address of the instruction immediately after it. Consider the following assembly, shown alongside the machine code and the disassembler's interpretation that it produces:

```
                          0:  e9 00 00 00 00    jmp  5 <_main+0x5>
jmp 0x4                   5:  eb 05             jmp  c <L>
jmp L                     7:  83 e8 00          sub  eax,0x0
sub eax, 0                a:  ff e0             jmp  eax
jmp eax          ⟶       0000000c <L>:
```

```
L:                          c:  83 c0 00          add  eax,0x0
add eax, 0                  f:  ff 20             jmp  DWORD PTR [eax]
jmp [eax]
```

The instruction most worth tracing is the one at address 5: `eb 05`, written in source as `jmp L`. The opcode `eb` is a relative jump, and its operand `05` is the offset. Crucially, this offset is *not* added to 5 (the address of the jump) but to 7 — the address of the instruction immediately following the two-byte jump, which is where EIP already points once the jump is being carried out. The arithmetic is therefore 7 + 5 = 0xc, which is exactly the address of label L. This is the little piece of arithmetic that trips up newcomers: the result `c` seems not to match a naive "5 plus 5" expectation precisely because the base of the addition is the *already-advanced* program counter, not the jump instruction's own address.

By contrast, the jumps that take absolute addresses are entirely straightforward to read. The instruction at address a, `ff e0` or `jmp eax`, simply transfers control to whatever address is currently held in EAX — no offset arithmetic is involved. Likewise `jmp DWORD PTR [eax]` at address f jumps to the address *stored at* the memory location pointed to by EAX, dereferencing one level of indirection but again requiring no relative computation. The lesson for the analyst is clear: absolute-address jumps are read directly, while offset-based jumps demand the small but critical mental step of adding the offset to the address of the *following* instruction.

Conditional Branching

Unconditional jumps are only half of what control flow requires; the far more interesting half is the ability to jump *only when some condition is met.* Conditional branches accomplish this by evaluating conditions over selected bits of the EFLAGS register. Four flags in particular account for the overwhelming majority of conditional logic encountered in practice.

The **CF (carry flag)** is meant to signal errors — specifically overflows or borrows — arising in *unsigned* arithmetic. The **OF (overflow flag)** plays the analogous role for *signed* arithmetic, signaling when a signed operation has produced a result too large (or too small) to fit in the destination. The **SF (sign flag)** is set whenever the result of an operation is negative, mirroring the value of the result's most significant bit under two's-complement interpretation. The **ZF (zero flag)** is set whenever the result of an operation is exactly zero. These four flags, individually and in combination, encode essentially everything a program needs to know about how its most recent arithmetic turned out.

The flags are rarely useful in isolation; what makes them powerful is the ability to set them deliberately in order to compare two values. This is the purpose of the **cmp** instruction. The instruction `cmp dest, src` computes the difference `dest - src` — but, critically, it does *not* store that difference anywhere, leaving `dest` entirely unmodified. Its sole effect is to set the EFLAGS bits exactly as a real subtraction would have, so that the relationship between `dest` and `src` becomes legible in the flags. For instance, if `dest` and `src` are equal, the subtraction yields zero and ZF is set; if `dest` is less than `src`, the subtraction produces a negative or borrowing result that the sign, carry, and overflow flags collectively capture. Because `cmp` performs this comparison without the side effect of altering its operand, it can be freely placed before a conditional jump to set up a decision without disturbing the values being decided upon. The flags it leaves behind enable a whole family of useful comparisons.

The jcc Family of Conditional Jumps

Reading the flags directly would be tedious, so the architecture provides a large family of **jcc** instructions — "jump on condition code." A jcc instruction inspects the flags for the condition named by its **cc** suffix and jumps to a given offset accordingly. Two properties are worth stressing immediately. First, like the

relative jumps discussed above, conditional jumps take an *offset* as their destination; they never take an absolute address. Second, each condition comes paired with a logically negated counterpart, so that the analyst can always express both "jump if the condition holds" and "jump if it does not," choosing whichever reads more naturally.

The meaning of each conditional jump is understood relative to a comparison that has just been performed. Supposing the preceding instruction was `cmp D, S` — comparing a destination value D against a source value S — the principal conditional jumps behave as follows.

The instructions `je L` and its synonym `jz L` jump to label L when `D == S` (equivalently, when the zero flag is set, since equality makes the difference zero). Their negation, `jne L` / `jnz L`, jumps when `D != S`. The pair `jg L` and `jnle L` jump when `D > S` treating the operands as *signed*, with the negated form `jng L` / `jle L` jumping when `D <= S`. The pair `jge L` and `jnl L` jump when `D >= S` (signed), negated by `jnge L` / `jl L`, which jump when `D < S`. For *unsigned* comparisons, a parallel set of mnemonics exists: `ja L` and `jnbe L` jump when `D > S` with the operands interpreted as unsigned ("a" for *above*), negated by `jna L` / `jbe L`; and `jae L` and `jnb L` jump when `D >= S` unsigned, negated by `jnae L` / `jb L` ("b" for *below*).

The signed-versus-unsigned distinction embedded in this mnemonic scheme is not a pedantic detail. The architecture uses an entirely separate vocabulary — *greater/less* for signed comparisons, *above/below* for unsigned ones — precisely because the same bit pattern can represent two different numeric values depending on interpretation, and a comparison that is correct under one interpretation is wrong under the other. The following table consolidates the family:

Jump (and synonym)	Jump to L if (after `cmp D, S`)	Negated version
`je` L ; `jz` L	D == S	[D != S] `jne` L ; `jnz` L
`jg` L ; `jnle` L	D > S (signed)	[D <= S] `jng` L ; `jle` L
`jge` L ; `jnl` L	D >= S (signed)	[D < S] `jnge` L ; `jl` L
`ja` L ; `jnbe` L	D > S (unsigned)	`jna` L ; `jbe` L
`jae` L ; `jnb` L	D >= S (unsigned)	`jnae` L ; `jb` L

Other Comparisons

While `cmp` is the most direct way to compare two values, it is not the only instruction that sets flags in a way useful for branching, and real-world malware is unlikely to confine itself to the obvious choices.

A second common comparison instruction is **test**. Rather than subtracting, `test` computes the *bitwise AND* of its two operands, and like `cmp` it discards the result, leaving both operands unaltered and affecting only the flags. The most frequent use of `test` is checking whether a register holds zero. Writing `test D, D` — ANDing a register with itself — produces a result that is zero if and only if D itself is zero, which means the zero flag is set exactly when `D == 0`. A `jz` placed immediately afterward then branches on precisely that condition. This idiom, `test D, D` followed by `jz`, is the canonical way to ask "is this register zero?" and appears constantly.

The analyst must nonetheless remain alert to the fact that malware will not restrict itself to `cmp` and `test` when it wants to make a decision. Because so many ordinary arithmetic instructions update the flags as a side effect, the flags needed for a branch can be produced by an instruction whose apparent purpose is something else entirely. For example, one can decrement a value using `dec` or `sub`, and these instructions will set the zero flag when the result reaches zero; a subsequent `jz` can then branch on that

outcome without any explicit comparison instruction ever appearing. This is not merely an obfuscation trick — optimizing compilers do exactly the same thing, folding a comparison into a decrement that was going to happen anyway — so the analyst must read the flag-setting behavior of *every* instruction, not just the ones nominally designated as comparisons.

Intel vs. AT&T Syntax

Before proceeding to the structure of functions, it is important to address a purely notational matter that nonetheless causes real confusion: x86 assembly is written in two major syntax dialects, **Intel** and **AT&T**. The two describe the identical underlying machine instructions but render them very differently on the page. Intel syntax is dominant in the Windows world — the primary environment for the malware under study here — but AT&T syntax is encountered frequently enough (particularly in Unix-derived tooling) that an analyst must be able to read both. The relevant differences fall into four areas.

The first is **parameter order**. Intel syntax writes the destination first and the source second (`mov dest, src`), whereas AT&T reverses this, writing the source first and the destination second. This single difference is responsible for a great deal of misreading, since the very same `mov` line means opposite things depending on which convention is in force.

The second is how **parameter size** is conveyed. In Intel syntax, the size of an operation is derived from the register being used — referencing EAX implies a 32-bit operation, AX a 16-bit one, AL an 8-bit one. AT&T syntax instead attaches an explicit suffix to the instruction mnemonic: `b` for byte, `w` for word, and `l` for long (dword).

The third concerns **sigils** — the decorations placed on operands. Intel syntax autodetects the nature of each operand from context and uses no special prefixes. AT&T syntax marks operands explicitly: a `$` prefixes immediate constants, and a `%` prefixes register names.

The fourth is the notation for **addresses**. Intel syntax uses the square-bracket expression `[expression]` already discussed, adding a size specifier such as `byte`, `word`, or `dword` when the intended operand size would otherwise be ambiguous. AT&T syntax instead writes memory operands in the form `offset(base, index, width)`, packing the same base/index/scale/displacement components into a parenthesized form.

	Intel	**AT&T**
Parameter order	Destination, then source	Source, then destination
Parameter size	Derived from the register used	Suffix (b, w, l)
Sigils	Autodetect	`$` for immediates, `%` for register names
Addresses	`[expression]`; add byte/word/dword when ambiguous	`offset(base, idx, width)`

Functions and Stack Frames

With the full repertoire of data movement, arithmetic, and control flow in hand, it becomes possible to understand how code is organized into functions — and, just as importantly, how precariously that organization rests on convention rather than hardware enforcement.

A **function**, approximately defined, is a unit of code that manages register values and its own portion of the stack independently of other units. Two practical pressures give rise to this notion of independence. First, when one code unit calls another, the called unit should not carelessly clobber registers that the calling unit was still relying upon; without some discipline here, every function call would risk silently corrupting the caller's working data. Second, since CPUs come equipped with only a few general-purpose registers, a function frequently needs more working storage than registers can supply, and the standard remedy is to **spill** values — that is, to save them onto the stack and fetch them back later in the execution as the need arises. Both pressures point toward giving each active function a private, well-delimited region of the stack to work in.

That region is the **stack frame**. Conceptually, the stack is divided into frames, one per currently active function, so that at any moment the chain of frames mirrors the chain of calls that led to the present point of execution. A typical frame bundles together three categories of information. First, it holds the **arguments** that were passed to the function for this particular invocation. Second, it holds the function's **local variables** and any other temporary storage the function requires during its execution. Third, and indispensably, it holds the **return address** for the call — the record of where, in the calling function, execution must resume once this function finishes and exits. It is this stored return address that allows the structured "call and return" discipline to work at all, and, as will become evident, it is also precisely the piece of saved state that memory-corruption attacks most often seek to overwrite.

Local Storage

The notion of a function spilling register values onto the stack, introduced in the abstract, can now be made concrete. Recall the fundamental problem: suppose no free registers remain, yet a computation must be performed that requires a scratch register. The specific example already encountered — adding to EDX the difference between EBX and ECX, which needs an intermediate register to hold the difference — serves equally well here, and it illuminates exactly how the stack substitutes for a scarce register.

The naive approach dedicates a named temporary memory location, call it TMP, which serves as a holding area for whatever value currently occupies EAX, so that EAX may be "borrowed" for a while and then returned to its prior state:

```
mov TMP, eax
mov eax, ebx
sub eax, ecx
add edx, eax
mov eax, TMP
```

The idiomatic stack-based form replaces the hand-managed TMP slot with a push at the start and a pop at the end:

```
push eax
mov eax, ebx
sub eax, ecx
add edx, eax
pop eax
```

Here TMP is, in effect, a temporary location for saving and restoring the data currently in EAX, so that the code can borrow EAX for the span of the three middle instructions and then hand it back untouched. What makes the stack version superior is that it demands no pre-arranged, named storage: the stack is always present and always ready to accept another saved value.

It is worth watching the stack pointer behave as this executes, since the mechanics reveal exactly what push and pop do to ESP. Before the `push eax`, ESP points at the current top of the stack — in the illustrative layout, at the slot holding the value 0x1032, with the three slots above it empty. Executing `push eax` causes ESP to move upward in the diagram (toward lower addresses, as the stack always grows) and the saved EAX value to be written into the newly occupied slot directly above 0x1032. The middle instructions then run, freely trashing and reusing EAX. Finally `pop eax` reads that saved value back out of the slot ESP points to, restoring EAX, and moves ESP back down to where it began, leaving the stack exactly as it was found. The symmetry is total: whatever push disturbs, the matching pop restores.

Push and Pop Instructions

Having seen push and pop in service of register borrowing, it is worth stating their precise semantics, because the exact sequence of micro-steps each performs — and the fact that each can be decomposed into simpler instructions — is something an analyst must be able to recognize on sight.

The instruction **`push src`** performs two actions in order. First, it moves the top of the stack *up* by 4 bytes — meaning it subtracts 4 from ESP, since the stack grows toward lower addresses. Second, it copies the contents of its operand, which may be an immediate or a register, into the memory location now pointed to by the freshly updated ESP. The operand is thereby deposited at the new top of the stack.

The instruction **`pop dest`** performs the mirror image. First, it copies the 4 bytes currently pointed to by ESP into the destination register. Second, it moves the top of the stack *down* by 4 bytes — adding 4 to ESP — thereby discarding the slot that was just read. The order matters: the value is read out *before* ESP is adjusted, so that pop retrieves the value that push most recently deposited.

A revealing fact about these two instructions is that neither is truly primitive. Each can be realized by a short sequence of simpler instructions combining explicit stack-pointer arithmetic with an ordinary memory move. A `push ebp` is equivalent to subtracting 4 from ESP and then writing EBP to the top of the stack; a `pop edx` is equivalent to reading the top of the stack into EDX and then adding 4 to ESP. The following equivalence makes this explicit:

```
push ebp              sub esp, 4
xor ecx, ecx    ⟶    mov [esp], ebp
pop edx               xor ecx, ecx
                      mov edx, [esp]
                      add esp, 4
```

The `xor ecx, ecx` in the middle is incidental — it merely demonstrates that unrelated instructions may sit between the push and the pop — but the surrounding translation is the point. This decomposition matters enormously to the analyst for one reason above all: malware authors frequently use exactly this substitution to *hide* stack manipulation. A `push` or `pop` is immediately recognizable and draws attention; an equivalent `sub esp, 4` followed by a `mov [esp], ...` accomplishes the same thing while looking like ordinary, unremarkable arithmetic and data movement. An analyst who does not recognize the expanded form as a disguised stack operation will misread the code's intent.

Using the Base Pointer

The stack pointer ESP is a restless register: it moves every time anything is pushed or popped, which happens constantly throughout a function's execution. This restlessness creates a practical difficulty. When a function makes use of local variables stored in its stack frame, those variables need to be addressed somehow — and addressing them relative to ESP means chasing a moving target, since the very same local variable sits at a different ESP-relative offset at different points in the function, depending on how much has been pushed since the frame was established.

The remedy is to dedicate a second, stable register to the job: the base pointer, **EBP**. Upon entering a function that uses local variables, EBP is set to a fixed anchor point within the frame, after which every local variable can be referenced via a fixed offset from EBP that never changes for the duration of the function, no matter how ESP wanders. Consider the following prologue and subsequent access:

```
mov ebp, esp
sub esp, 8
mov [ebp-8], 0x1111
mov [ebp-4], 0xabadcafe
...
mov eax, [ebp-8]
```

The sequence begins by capturing the current stack top into EBP with `mov ebp, esp`, fixing the anchor. The `sub esp, 8` then lowers ESP by 8 bytes, carving out 8 bytes of room on the stack for two 4-byte local variables. Those two locals are now addressable at the fixed offsets `[ebp-8]` and `[ebp-4]`: the first is initialized to 0x1111 and the second to 0xabadcafe. Crucially, much later in the function — after arbitrary further stack activity has moved ESP around — the first local can still be retrieved with the identical expression `mov eax, [ebp-8]`, because EBP has not budged. Tracing the stack state confirms the arrangement: before the prologue ESP points near 0x3424; after `mov ebp, esp` and the `sub esp, 8`, EBP anchors at the original top (now holding, in the lower slots, 0xabadcafe and above it 0x1111) while ESP has descended two slots below it.

This illustrates precisely why EBP exists: accessing local variables through ESP is like chasing a moving target, whereas EBP provides a fixed frame of reference. There is, however, an important caveat for the analyst. EBP is an ordinary general-purpose register, and nothing compels a function to reserve it for this base-pointer role. If a function's code logic chooses to use EBP for some other purpose — as malware and aggressively optimized code both often do — then the convenient fixed-offset addressing is unavailable, and dereferencing local variables via `ESP+offset`, moving target and all, becomes the only option left. Recognizing whether EBP is or is not being used as a frame pointer is thus one of the first orienting judgments an analyst makes when reading a function.

Call and Ret Instructions

Function calls were earlier described as a special kind of unconditional branch, and the two instructions that implement them — **call** and **ret** — are where that "special" machinery lives. Their entire reason for existing is to solve the problem a plain jump cannot: a plain jump transfers control but leaves no record of where control came from, so there is no way to return.

A piece of code invokes a function using the **`call`** instruction. Unlike a jump, `call` does something extra *before* the control transfer takes place: it stores on the stack the address of the instruction that immediately follows the `call` itself. This stored address is the **return address** for the call — the place

where execution should resume once the called routine is done. With the return address safely on the stack, `call` then transfers control to the target function.

The called function (the callee) resumes the caller by executing **`ret`**. This instruction reads the address currently sitting at the top of the stack and populates EIP with it, causing execution to continue at exactly the instruction after the original `call`. In essence, `ret` is a jump whose destination is whatever address `call` left on the stack. The two instructions are thus a matched pair: `call` pushes the return address and jumps in; `ret` pops the return address and jumps back.

A conceptual warning accompanies all of this, and it is one the analyst must internalize deeply: **functions exist only at a logical level.** The processor has no innate concept of a "function" as a bounded entity. Code is simply laid out in memory as a continuous stream of bytes with no separators, no markers, and no inherent boundaries delimiting where one function ends and the next begins. The division of code into functions is a human and compiler convention imposed *on top of* an undifferentiated byte stream. The consequence for malware analysis is severe: adversaries can deliberately scramble the layout of their code — interleaving pieces of different logical functions, placing a function's body far from its entry, jumping in and out of regions in unexpected ways — making it genuinely difficult for an analyst (and for automated tools) to identify where function boundaries actually lie.

A Worked Function Call

Following the stack through a concrete call makes the matched behavior of `call` and `ret` unmistakable. Consider a caller that executes some instructions and then calls a function named `myfun`:

```
push eax            myfun:
add eax, 1          mov ebp, esp
call myfun          add esp, 8
sub eax, 1          ...
                    sub esp, 8
                    ret
```

Before the `call myfun` executes, ESP points at some stack top (in the illustration, the slot holding 0x2300). When `call myfun` runs, it pushes the return address — the address of the `sub eax, 1` instruction that follows the call — onto the stack, so ESP now points one slot higher (toward lower addresses) at a slot holding `<return address>`, and control transfers into `myfun`. The callee runs its body, and when it reaches `ret`, the instruction reads that `<return address>` from the top of the stack, loads it into EIP, and execution jumps back to `sub eax, 1` in the caller. The stack is left as it was before the call, ESP once again pointing at the 0x2300 slot. The arrows of the original diagram capture this round trip precisely: control dives from the `call` into `myfun`, and `ret` sends it back up to the instruction immediately after the call, with the stored return address serving as the thread connecting the two.

Calling Conventions

The `call` / `ret` pair explains *how* control gets into and out of a function, but it leaves two equally important questions unanswered. First, how are register values preserved across a function invocation, so that the caller's data survives the call? Second, how are parameters actually passed from caller to callee? Neither question has a single hardware-mandated answer; instead, the answers are fixed by agreement, and those agreements are called **calling conventions**.

A calling convention regulates several distinct matters at once. It specifies how parameters are passed — whether via registers, via the stack, or some combination — and in which order they are placed. It specifies which registers the callee is obligated to preserve on behalf of the caller, so that both sides know whose responsibility it is to protect any given register's value. And it specifies who performs the realignment of ESP upon return — that is, who is responsible for cleaning up the stack space that the arguments occupied once the call completes.

For the malware analyst, a nuanced point follows. Malware is free to use entirely custom, idiosyncratic calling conventions for its own internal functions, since those functions only ever call one another and can agree among themselves on any scheme they like. However, the moment malware needs to interact with the Windows API — which it almost always does, for anything from file access to network communication — it is forced to follow the *standard* conventions that those system functions expect. This creates a useful analytical foothold: calls into Windows APIs will obey recognizable conventions even when the surrounding custom code does not.

Windows Conventions

Within a single Windows program, different calling conventions can coexist, with different functions obeying different rules. Two conventions dominate.

The first is **cdecl**, the convention used by C programs. Under cdecl, arguments are pushed onto the stack in **right-to-left order**, meaning the last argument in the source-level call is pushed first and the first argument ends up on top, nearest the callee. Register preservation is split as follows: the caller is responsible for saving EAX, ECX, and EDX if it is using them (these three are therefore free for the callee to clobber), while the callee must preserve EBP, EBX, EDI, and ESI whenever it needs to use them. Under cdecl, the **caller adjusts ESP after the return**, cleaning up the pushed arguments itself. A 32-bit return value is delivered back to the caller in the EAX register.

The second is **stdcall**, the convention used by the Win32 API. In stdcall, the treatment of arguments, the division of caller-save and callee-save registers, and the return-value location are all identical to cdecl. The one decisive difference concerns stack cleanup: under stdcall, the **callee** adjusts the stack pointer upon return, rather than the caller. This is accomplished with the `ret N` instruction, where N indicates that N+4 bytes should be added to ESP *after* EIP has been updated with the return address — the extra 4 accounting for the return address slot itself, and the N accounting for the argument bytes the callee is cleaning up on the caller's behalf.

Example: cdecl

A concrete cdecl call makes the division of labor visible. Consider calling a function `subtract` corresponding to the C source `int __cdecl subtract(int arg1, int arg2) { return arg1 - arg2; }`:

```
mov ecx, 20             subtract:
...                     push ebp
push ecx                mov ebp, esp
push 10                 mov eax, [ebp+8]
push 30                 mov edx, [ebp+12]
call subtract           sub eax, edx
add esp, 8              pop ebp
pop ecx                 ret
```

```
cmp eax, ecx
jnz ErrorLabel
```

Reading the caller: ECX (holding 20) is first saved with `push ecx`, since ECX is a caller-save register the caller wants to preserve across the call. Then the two arguments are pushed right-to-left — `push 10` (arg2) followed by `push 30` (arg1) — so that arg1=30 ends up nearer the top. After `call subtract` pushes the return address and transfers control, the callee establishes its frame (`push ebp`, `mov ebp, esp`), reads its two arguments from fixed EBP offsets — arg1 at `[ebp+8]` and arg2 at `[ebp+12]`, the offsets reflecting that `[ebp]` holds the saved EBP and `[ebp+4]` holds the return address — computes `eax = 30 - 10`, tears down its frame with `pop ebp`, and returns the result in EAX via a plain `ret`. Back in the caller, the defining cdecl behavior appears: `add esp, 8` is the *caller* cleaning up the 8 bytes of arguments it had pushed. The subsequent `pop ecx` restores the saved ECX, and `cmp eax, ecx` / `jnz ErrorLabel` checks the returned result.

Example: stdcall

The stdcall version of the identical function, `int __stdcall subtract(int arg1, int arg2)`, differs in exactly one place:

```
mov ecx, 20              subtract:
...                      push ebp
push ecx                 mov ebp, esp
push 10                  mov eax, [ebp+8]
push 30                  mov edx, [ebp+12]
call subtract            sub eax, edx
add esp, 8   ← removed   pop ebp
pop ecx                  ret 8
cmp eax, ecx
jnz ErrorLabel
```

Everything on the caller side up to and including the call is identical. The difference is twofold and complementary. In the callee, the plain `ret` has become **`ret 8`**: after loading EIP with the return address, the callee adds 8+4 bytes to ESP, itself disposing of both the return address slot and the 8 bytes of arguments. Correspondingly, in the caller, the `add esp, 8` that cdecl required is now *gone* (struck through) — it would be redundant and indeed harmful, since the callee has already performed that cleanup. The stack diagrams for the two versions are identical right up until the return; only at the `ret` does stdcall's `ret 8` perform the extra ESP adjustment that cdecl deferred to the caller. Spotting whether cleanup happens at the call site (`add esp, N` after the call) or inside the callee (`ret N`) is therefore a reliable way for an analyst to tell which of the two conventions a given function follows.

Caller-Save vs. Callee-Save

Underlying both conventions is a general principle for dividing the responsibility of preserving registers, and it is worth stating in its own right because it recurs throughout all of reverse engineering.

Caller-save registers are those that the *caller* must push to the stack *before* making a call, if and only if the caller intends to use their values again after the call returns. The logic is that these registers are considered fair game for the callee to overwrite, so any caller who cares about their contents must protect

them itself beforehand. Under both cdecl and stdcall, the caller-save registers are typically EAX, ECX, and EDX.

Callee-save registers are the complement: these are pushed to the stack in the *prologue* of the *callee*, but only when the callee actually needs to use them. The callee, having borrowed them, is obligated to restore them before returning, so that from the caller's perspective these registers emerge from the call unchanged. Under both cdecl and stdcall, the callee-save registers are typically EDI, ESI, and EBX.

A performance observation explains why code tends to prefer one over the other. Compilers often favor callee-save registers for holding values that must survive across nested calls, because a value parked in a callee-save register is automatically protected by whatever functions get called in the meantime. Only when register pressure grows high — when there simply are not enough registers to go around — do compilers additionally press caller-save registers into service. In custom, hand-written malware code, however, no such tidy preference can be assumed: one may find any sort of assortment, since the author is bound by no compiler's habits.

EBP — Enter and Leave

The prologue and epilogue sequences that set up and tear down a stack frame are so stereotyped that the architecture provides single instructions to encapsulate each.

When EBP is used as a base pointer, a function prologue commonly performs:

```
push ebp
mov ebp, esp
```

This pair — saving the caller's EBP, then anchoring EBP to the current stack top — can be accomplished with the single instruction **`enter`**. The `enter` instruction additionally accepts an optional immediate operand specifying an amount to subtract from ESP, carving out room for local variables in the same stroke (equivalent to a following `sub esp, N`).

The symmetric teardown is governed by a strict requirement: to leave a function cleanly, **ESP must point to the caller's return address** at the moment `ret` executes, since `ret` takes whatever ESP points at as the address to return to. If ESP has been left pointing elsewhere (for instance, still down among local variables), EBP can come to the rescue in the function epilogue:

```
mov esp, ebp
pop ebp
```

The `mov esp, ebp` snaps ESP back up to the frame's anchor (discarding all locals at once), and `pop ebp` then restores the caller's saved EBP while advancing ESP to point exactly at the return address, ready for `ret`. This two-instruction epilogue can likewise be collapsed into the single instruction **`leave`**. Recognizing `enter` and `leave` — and their multi-instruction equivalents — lets an analyst immediately identify frame setup and teardown even when a compiler or author has chosen one encoding over the other.

LEA Instruction

The **LEA (Load Effective Address)** instruction occupies a peculiar and frequently misread niche. LEA evaluates an expression that defines a memory address and writes *that address* into a destination register. The crucial contrast is with ordinary data-movement and ALU instructions: those dereference a

given address and operate on the *contents* found at the pointed-to location, whereas LEA does not dereference at all — it simply *computes an address* and hands back the number. Despite borrowing the square-bracket syntax associated with memory access, LEA never actually touches memory; it performs only the address arithmetic.

This gives LEA two distinct uses. The first is genuine **address calculation**, the pointer arithmetic needed to figure out where some data lives without yet reading it. The second, and often surprising, use is as a compact way to evaluate **arithmetic expressions** that have nothing to do with memory at all, exploiting the fact that the address-arithmetic engine can compute `base + index*scale + offset` in a single instruction.

LEA: An Example

The distinction between LEA and MOV is sharpest in a side-by-side comparison. Suppose EBX = 0x1000, and the value 0xABADCAFE happens to be stored at address 0x1004. Then `mov eax, [ebx+4]` *dereferences* the computed address 0x1004 and copies its contents — 0xABADCAFE — into EAX. By contrast, `lea eax, [ebx+4]` computes the same address 0x1004 but copies *the address itself*, 0x1004, into EAX, never reading what lies there. Same bracket expression, entirely different result: one yields the stored value, the other yields the location.

The arithmetic-expression use is illustrated by a computation with no pointers in sight. Suppose EAX = 10 and EBX = 4, and the goal is to compute $C = A + 2 \cdot B - 1$. The single instruction `lea ecx, [eax+2*ebx-1]` produces exactly this result — $10 + 2 \cdot 4 - 1 = 17$ — in one stroke, treating the registers as plain numbers and the "address" expression as ordinary algebra. One constraint governs this trick: the scale factor used for the multiplication must be 1, 2, or 4 (the hardware-supported scaling widths), so LEA can cheaply multiply by these particular small constants but not by arbitrary ones.

REP Instructions

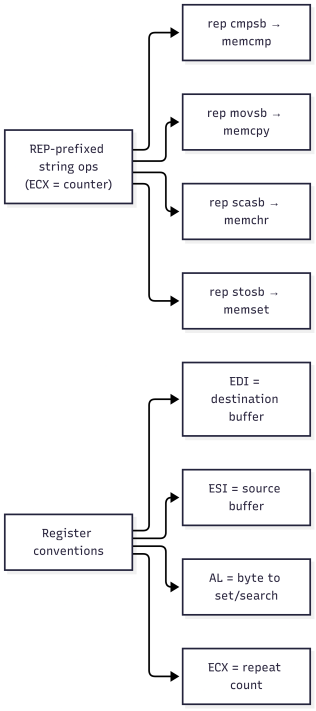
A final family of instructions provides compact, hardware-accelerated primitives for manipulating whole buffers of bytes, and recognizing them lets an analyst instantly identify common memory operations that would otherwise sprawl across an explicit loop.

The base instructions operate on individual bytes and come in four flavors. **`cmpsb`** compares bytes from two buffers. **`stosb`** initializes bytes in a buffer, filling them all with the same value. **`movsb`** copies one buffer into another. **`scasb`** searches a buffer for a particular byte. Each of these processes a single byte per execution; what turns them into bulk operations is the **`REP`** prefix placed in front of them. REP causes the instruction to repeat — advancing through the buffer — until either the ECX register, used as a counter, reaches zero, or (for the comparison and search variants) an additional condition on the zero flag is satisfied.

These string instructions draw their operands from a fixed set of registers by convention rather than from explicit operands. EDI holds the address of the first buffer, treated as the **destination**; ESI holds the address of the second buffer, treated as the **source**; AL holds the single byte to be set or searched for; and ECX serves as the repetition counter. Committing this register assignment to memory (destination in EDI, source in ESI, byte in AL, count in ECX) is essential, because the instructions mention none of it on the page.

The power of these primitives is that the entire family of core C standard-library memory functions can be implemented using nothing more than these four REP-prefixed instructions. Specifically, `memcmp` (compare two regions) corresponds to `rep cmpsb`; `memcpy` (copy a region) corresponds to `rep movsb`;

`memchr` (search a region for a byte) corresponds to `rep scasb`; and `memset` (fill a region with a byte) corresponds to `rep stosb`. When an analyst encounters one of these REP idioms, the correct mental translation is immediately to the corresponding named library function, collapsing what looks like obscure byte-shuffling into a single recognizable operation.



CHAPTER 1: Static Malware Analysis

Why Analyze Malware?

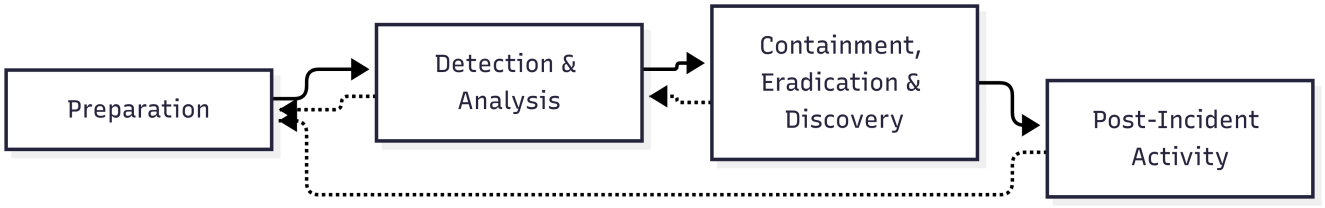
Malware analysis is the disciplined process of dissecting malicious software to understand its inner workings, its intentions, and its footprint on a compromised system. A piece of malware, by design, is a silent executable: once launched it produces no self-description of what it is doing, because it is engineered to leave as few traces as possible and to remain undetected for as long as it can. The analyst's job is to force that silence to give up its secrets. There are three principal motivations that justify the effort.

The first motivation is **monitoring and attack prevention**. By studying how malware operates, an organization can identify recurring attack patterns and translate them into detection rules that are then embedded into automated defensive devices such as intrusion detection systems. Understanding the mechanisms an attacker relies upon — for instance the exploitation of insufficient isolation between components — allows defenders to close exactly those gaps and to design IT systems that are structurally more resilient to forthcoming attacks. In short, comprehending an adversary's strategy is the precondition for building better defenses against it.

The second motivation is **incident response**. When an infection has already taken place, the defender needs to reconstruct precisely what happened. Because the malware advertises nothing about its activity, analysis is the only way to determine which assets — machines, files, accounts — have actually been compromised. This knowledge is essential to contain the further expansion of the attack, to eradicate it completely, and finally to recover efficiently. Without an accurate picture of the malware's behavior, containment and eradication become guesswork, and residual infected components may allow the attack to resurface.

The third motivation is **attribution**: understanding who the attacker is. The internal characteristics of a sample — coding style, reused components, referenced infrastructure — can help tie an incident to a known threat actor.

These three purposes are not isolated academic exercises; they map directly onto the recognized lifecycle of handling a security incident. The **NIST Incident Response Cycle** organizes this lifecycle into four connected phases that flow into one another and loop back for continuous improvement:



Preparation establishes the tools, procedures, and safe environments needed before an incident occurs. Detection and analysis is where malware analysis proper lives: the incident is recognized and the malicious artifact is understood. Containment, eradication, and discovery act on that understanding to stop the spread, remove the threat, and uncover any additional compromise. Post-incident activity closes the loop by feeding lessons learned back into preparation and detection, strengthening the organization against the next attack. Malware analysis is therefore a critical part of incident response, not a peripheral one.

What We Want to Extract: Signatures

The output of an analysis is not necessarily a sixty-page narrative report. Very often the practical deliverable is far more compact: a **signature**, understood as a set of information whose presence within a network or on a host reliably indicates the presence of the malware. Signatures — also called **Indicators of Compromise (IoCs)** — are what allow us to detect the same threat again elsewhere and to prevent further similar infections. Signatures come in three distinct shapes, each observing the malware from a different vantage point.

Host-based signatures describe artifacts that can be checked on a single machine. They identify files, processes, or registry keys on a victim computer whose presence betrays an infection. These are the traces the malware leaves behind on the endpoint itself.

Network signatures detect the malware by analyzing network traffic rather than local artifacts. They answer questions such as whether the malware inspects its surroundings and, crucially, whether it contacts an external **Command & Control** server — abbreviated **C&C** or **C2** — the remote infrastructure from which many malicious programs receive orders. Network signatures become considerably more effective when they are coupled with the insight gained from malware analysis, because analysis reveals what the traffic should look like and where it is destined.

Behavioral signatures shift the focus from the malware object to its effects: they describe what the malware does to the system rather than what the malware is. This is the domain of behavioral analysis, and it is conceptually different from the classic antivirus signature, which tries to match the static content of the file. A behavioral signature can catch a threat even when its binary form has changed, precisely because the behavior remains constant while the bytes may not.

A Necessary Warning: Handling Malware Safely

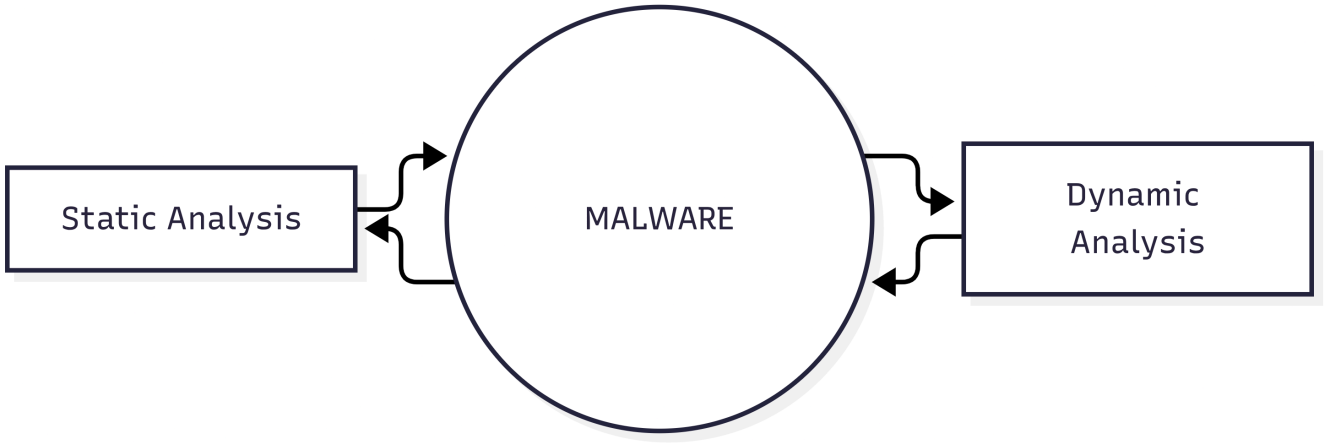
Before any technique is applied, the analyst must internalize that this work is inherently dangerous. The subject of study is software deliberately built to harm computers — possibly the very computer

performing the analysis. In many cases the sample must actually be executed to be understood, meaning the analyst voluntarily runs code written by criminals. Furthermore, some IP addresses referenced inside a sample may be live, and one must assume that behind such an address sits a real adversary.

From these premises follows a strict safety discipline. Analysis must always be performed inside a **virtual machine isolated from the main host**, so that any damage is confined to a disposable environment. That virtual machine should be **disconnected from the network** unless network communication is genuinely required by the specific experiment. If references to external IP addresses are found in the code, one must **never** attempt to contact them directly — doing so is comparable to a burglary victim traveling to the burglar's home address to ask how the break-in was done, an act that only exposes the investigator. And overriding everything, repeated deliberately because it is the single most important precaution: **keep a safe offline backup of your important data**, so that even a catastrophic escape of the malware cannot cause irrecoverable loss.

Static versus Dynamic Analysis

The techniques for extracting information from a malicious executable divide along a fundamental axis: whether or not the sample is run. A malware sample is an executable file containing binary code, written and compiled for a specific processor architecture, and it can be approached in two complementary ways that together form a cyclical, mutually reinforcing process.



Static analysis examines the malware *without running it*, inspecting the code and the structure of the file directly. Its instruments include the `strings` utility, PE inspection tools, and, at the more advanced level, a disassembler such as **IDA Pro** used to interpret assembly code.

Dynamic analysis does the opposite: it *runs* the malware and monitors the effects it produces on the running system. Because execution is involved, it relies on a virtual machine and the practice of taking snapshots so that a clean state can always be restored. Its instruments include monitoring tools such as **RegShot**, **Process Monitor**, **Process Hacker**, and **CaptureBAT**, as well as debuggers; for the examination of volatile memory it uses RAM-analysis frameworks such as **Mandiant Redline** and **Volatility**. Other forms of analysis exist beyond these two families, but they fall outside the scope of this treatment.

The two approaches feed each other: findings from static inspection guide what to watch for during execution, and observations from execution direct where to look in the code. Neither is complete on its own.

Basic versus Advanced Techniques

Cutting across the static/dynamic distinction is a second axis of difficulty, separating basic from advanced techniques.

Basic static analysis observes the malware without looking at its code in depth — relying instead on strings, PE inspection, and similar surface-level tools. It is quick and easy, but it fails against advanced malware and can miss important behaviors, since a determined author can hide precisely the details that matter. **Basic dynamic analysis** is likewise easy but demands a safe test environment, and it too is not effective on every malware, because some samples detect the observation and refuse to reveal themselves.

The advanced counterparts trade speed for depth. **Advanced static analysis** consists of reverse-engineering the malware code with a disassembler; it is complex and requires a solid understanding of assembly language and of how such code interacts with the operating system. **Advanced dynamic analysis** runs the code inside a debugger, allowing the analyst to examine the internal state of the running malicious executable and to watch how that state evolves instruction by instruction. This present material concentrates on the basic techniques, which are the natural entry point.

Expected Behaviors of Malware

Regardless of its specific purpose, malware tends to exhibit a recognizable repertoire of behaviors. Anticipating them lets the analyst know what to look for.

Persistence is the malware's drive to survive. After installing itself in the system, it performs actions that guarantee its survivability across events that would otherwise remove it — most notably reboots, but also inspections and other disruptions. To achieve this it often modifies the operating system configuration, and it takes care that these changes remain hidden from ordinary observation.

Uniqueness reflects that malware is often incompatible with itself. Multiple concurrent executions of the same sample are usually meaningless and can even be dangerous to the malware's own goals — imagine two ransomware instances competing to encrypt the same resources and corrupting each other's work. To avoid this, malware commonly performs checks at startup that guarantee only a single copy is running at any time. It is worth stressing that malware code is not written to be elegant; correctness of self-management, not beauty, is the concern.

Environment checks / Targeting capture the fact that modern malware is not meant to run on every machine it reaches. It is designed to operate only in an appropriate environment, and it verifies at startup that specific execution conditions are satisfied before acting. These conditions may include the current time or date, the system's localization, the availability of particular resources, or the presence of certain other processes. If the conditions are not met, the malware may stay dormant or exit.

Obfuscation and evasion describe the malware's effort to remain undetected for as long as possible by hiding both its presence and its effects. It may inject its code into other processes to conceal its true identity, it may mangle its own internal code to erase easily discoverable hints of its malicious nature, and it may deploy techniques specifically intended to hamper any attempt to analyze its behavior correctly.

Fingerprinting & beaconing refer to reconnaissance and announcement. The malware inspects the infected system and collects information about it — a fingerprint — and this information can then be used to "publicize" the malware's presence to a remote server, effectively announcing a successful infection.

Communication and control closes the loop with the attacker's infrastructure. Malware frequently interacts with external command-and-control services in order to receive instructions and commands, to

update its own internal components, and to exfiltrate stolen data. This communication may travel over heterogeneous types of links, and the data flowing on these channels is often obfuscated to resist inspection.

General Rules for Malware Analysis

Two pragmatic principles should govern the analyst's mindset. The first is **don't get caught in details**: it is neither necessary nor efficient to understand one hundred percent of the code, and the effort should concentrate on the key features that reveal what the sample does. The second is **try several tools**: if one tool fails to yield results, another should be tried, and one should not become stuck on a single hard issue but rather move along and return later if needed. Underlying both rules is the sober reality that malware authors are constantly raising the bar, so the analyst must remain flexible and outcome-oriented rather than perfectionist.

The Techniques of Basic Static Analysis

Basic static analysis, the focus of what follows, rests on a small set of concrete techniques: antivirus scanning, the computation of cryptographic hashes, and the examination of a file's strings, functions, and headers. Each of these is developed in turn.

Antivirus Scanning: An Obvious First Step

Submitting a suspicious file to antivirus engines is the natural opening move, and services such as **VirusTotal** make this convenient by accepting an uploaded file or URL, analyzing it against many engines, and returning a detection score together with community sharing of the sample. Yet this convenience carries two caveats. First, malware can easily change its signature and thereby fool signature-based antivirus detection, so a clean score is far from a guarantee of safety. Second, uploading a sample to a public service such as VirusTotal may itself alert the attackers that their malware has been caught, since they can monitor these platforms for the appearance of their own creations. The first step is obvious, but it is not free of consequences.

Hashing Samples

A **hash** condenses a file of any size down to a fixed-length fingerprint, computed with algorithms such as **MD5** or **SHA-1**. Because the fingerprint is derived from the file's contents, it uniquely identifies that file and serves as the de-facto standard way of naming and referring to samples. A **collision** — two different files producing the same hash — is theoretically possible, and MD5 collisions do exist, but they are uncommon enough that the hash remains a reliable identifier in practice. A terminological note deserves emphasis here: the analyst speaks of a **sample** rather than of *the* malware, because the hash identifies one specific file, and the same malicious program may exist in many slightly different files each with its own hash.

This last point exposes a limitation exploited by attackers. Because the hash changes completely when even a single byte of the file changes, an attacker can bypass hash-based detection simply by modifying a trivial detail of the code, producing a different hash that a hash-blacklist — and possibly an antivirus — will not recognize, even though the malware's actual behavior is unchanged.

Practically, most operating systems provide an `md5` utility as a standard command-line tool, so that hashing a file is a single invocation returning the fingerprint. On Windows, analysts who prefer a graphical interface can use a GUI-wrapped tool such as **HashCalc**, which computes MD5, SHA-1, and other digests for a chosen file at once.

The hash of a sample supports several uses. It can **label** a malware sample unambiguously; it can be **shared** with other analysts to identify the same malware across organizations; and it can be **searched online** to discover whether someone else has already identified and documented the file, sparing duplicated effort.

A more refined variant is the **import-hash** (often called *imphash*). Instead of hashing the whole file, the import-hash is calculated over the set of functions the sample imports from external libraries. Its value lies in clustering: similar samples belonging to the same malware family tend to share the same import-hash, because different versions of the same malware will most likely rely on the same set of APIs. Two files that differ byte-for-byte — and therefore have different MD5 hashes — can thus still be recognized as relatives through a shared import-hash.

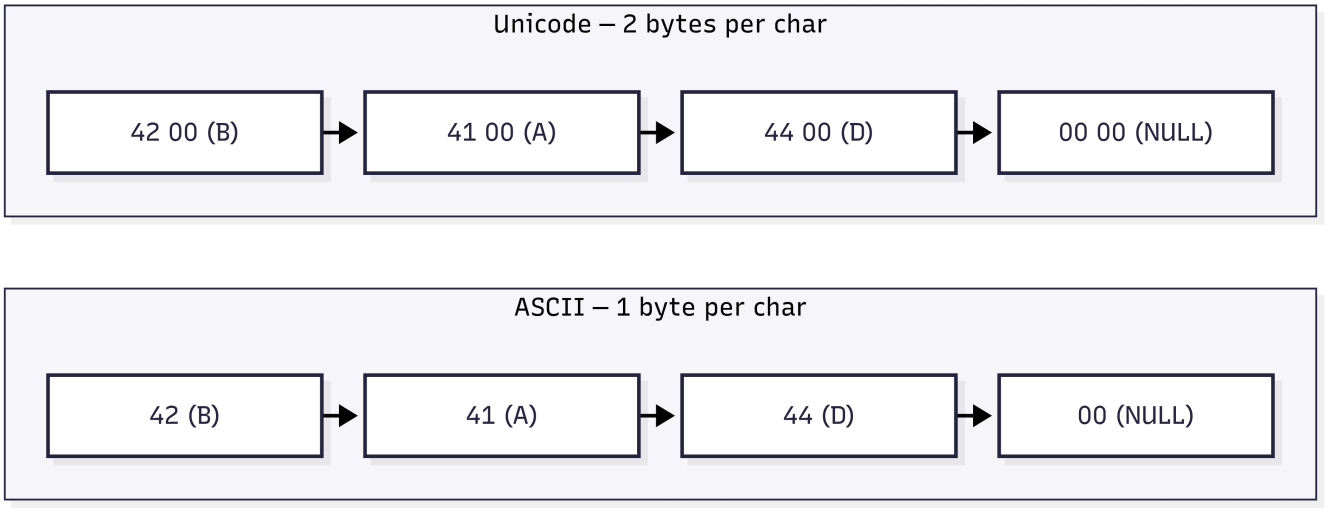
Strings

Among the most immediately rewarding artifacts to recover from an executable are its **strings**. A string is any sequence of printable characters embedded in the file, and such sequences frequently leak information the author never intended to expose to a casual observer: file paths, error messages, URLs, IP addresses, registry keys, function names, and fragments of user-interface text. Reading the strings of a sample is therefore a cheap and fast way to form a first hypothesis about what the program does.

To recover strings correctly one must understand how they are stored in memory, because a string is delimited rather than length-prefixed: every string is **terminated by a null value**, written `0x00`, which marks where the sequence of characters ends. The encoding of the individual characters, however, differs between the two conventions an analyst will encounter.

In **ASCII**, each character occupies eight bits — a single byte. The word *BAD*, for example, is stored as the three bytes `42 41 44` (the ASCII codes for B, A, and D) followed immediately by the null terminator `00`. The layout is compact: one byte per character, one terminating byte at the end.

In **Unicode**, each character occupies sixteen bits — two bytes. Microsoft refers to these as **wide characters**. The same word *BAD* now appears as `42 00 41 00 44 00`, each printable byte interleaved with a `00`, and the string closes with a double-byte null terminator `00 00`. Recognizing this interleaving pattern matters in practice, because a tool or an eye tuned only to ASCII may misread wide-character strings or miss them entirely.



The `strings` Command

The canonical tool for this task is the `strings` command. It is native to Linux and OSX and is also available for Windows, and its operation is simple: it scans a file and prints out every string it finds that

is three or more characters long. The three-character minimum is a heuristic that filters out much of the incidental noise while still capturing meaningful text.

The catch is that not every printed string is significant. The output mixes genuinely informative strings with **noise** — accidental byte sequences that happen to be printable but carry no meaning. Interpreting the output is therefore an act of judgment. Consider a representative run against a sample, `bp6.ex_`, which prints a list including short meaningless fragments such as `VP3`, `VW3`, `t$@`, `D$4`, `e-@`, and `M}C`, alongside several strings that genuinely tell a story:

- `GetLayout` and `SetLayout` are **Windows functions**, revealing that the program calls into the Windows API.
- `GDI32.DLL` is a **Dynamic Link Library**, the graphics-related library, indicating the program links against it.
- `99.124.22.1` looks like an **IP address**, a strong hint that the program has network functionality and a specific host it may contact.
- `Mail system DLL is invalid.!Send Mail failed to send message.` looks like an **error message**, suggesting the program has email-sending capability and reports failures.

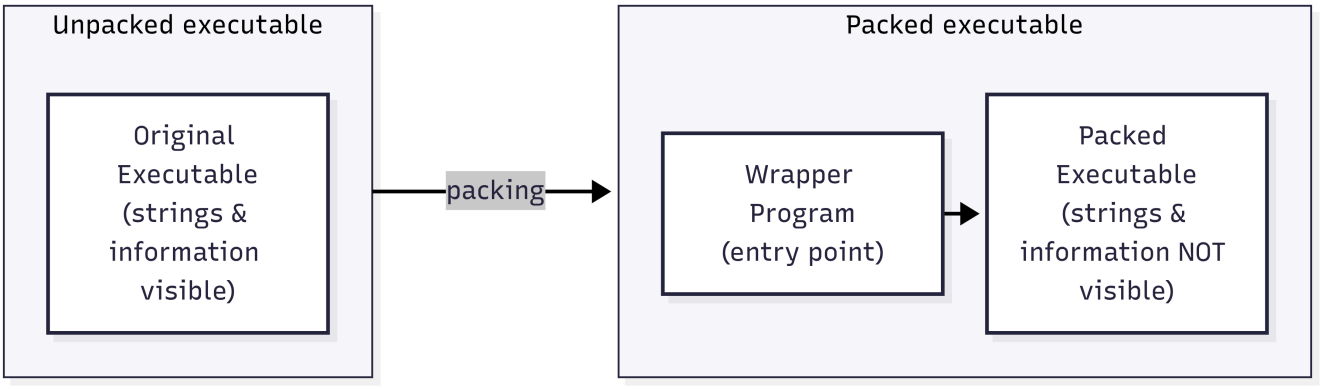
The short, cryptic fragments in **bold** — `VP3`, `VW3`, `t$@`, `D$4`, `e-@`, `M}C` — can safely be ignored as noise, while the recognizable API names, the library name, the IP address, and the error text together sketch a program that manipulates graphics, communicates over the network, and sends mail. From a handful of recovered strings the analyst has already inferred several concrete capabilities, and all of this without executing a single instruction.

Obfuscation Through Packing

The string-based approach is powerful, which is exactly why malware authors work to defeat it. The principal countermeasure is **packing**. Packing compresses the code of the executable much as a Zip archive compresses a file, and the immediate consequence for the analyst is that most strings and instructions become unreadable, because in their compressed form they appear as essentially random bytes rather than legible text or code.

An important historical nuance is that packing was not invented for malicious purposes. It was originally an obfuscation-and-compression mechanism designed to cope with limited memory and storage, shrinking a program's footprint. Attackers subsequently adopted it because the same compression that saves space also conveniently hides the original code of the malware, concealing the very strings and instructions that basic static analysis relies upon.

Mechanically, a packed executable is restructured into two parts. The original program — with all its strings and other information plainly visible — is compressed and stored as inert data, no longer directly executable in place. Wrapped around it is a small piece of code called the **wrapper** (or unpacker). When the packed file is run, execution begins in the wrapper, which decompresses the original program back into memory and then transfers control to it. The result is that static inspection of the file on disk shows the analyst only the wrapper; the meaningful body of the malware remains hidden inside the packed, unreadable region until runtime.



The practical lesson is that when `strings` returns almost nothing intelligible — only a short wrapper's worth of text and a sea of random-looking bytes — packing should be suspected. The absence of readable strings is itself an indicator.

Detecting Packers with PEiD

Because packing is so common, tools exist to detect it and, where possible, to name the specific packer used. A classic example is **PEiD**. Given a file, PEiD inspects characteristic markers and reports information such as the **entry point**, the **EP section** (the section containing the entry point), the **file offset**, the **first bytes** at the entry point, the **linker version**, and the **subsystem**. Its most useful output is a line identifying the packer signature — for instance recognizing that a sample was packed with a particular version range of **UPX**, a widely used packer. A telltale sign visible even without PEiD's signature database is an entry-point section bearing a packer's own name, such as a section labeled `UPX1`, since packers frequently rename the sections they create.

PEiD carries a serious warning of its own. Some of its plugins may **silently run the malware** in the course of trying to identify the packer. This means that merely attempting to detect the packer can inadvertently execute the sample, with all the danger that entails. The consequence is a strict operational rule: the testing virtual machine must be properly isolated before such tools are used, so that an accidental execution is contained.

File Execution in Windows

To interpret the headers that basic static analysis inspects, one must first understand what happens when an executable is launched. When a user double-clicks an executable file in Windows, the operating system — specifically the component known as the **loader** — examines the file in order to understand what to do with it. The loader must locate and correctly interpret everything the program needs to run.

An executable file is not a flat blob; it contains several distinct kinds of content. It holds **code**, the actual machine instructions to be executed; **data**, the values the program operates on; **other resources** such as icons, menus, and embedded media; and **metadata**, descriptive information about the file itself. For the loader to make sense of all this, the information must be organized according to a well-defined structure that the loader knows how to read. That structure, on Windows, is the Portable Executable format.

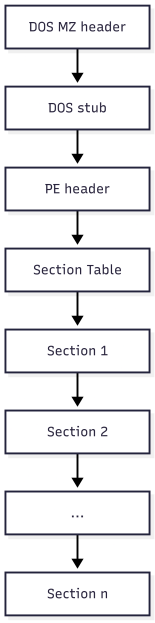
The Portable Executable (PE) File Format

The **Portable Executable (PE)** format is the file format used by Windows executable files, object code, and DLLs alike. Its single greatest convenience is precisely this universality: one file format serves several different usage scenarios, which means that executables (EXEs) and dynamic link libraries (DLLs) are treated in a very similar way despite their different roles. Structurally, the PE format is a data structure that carries all the information Windows needs in order to load the file into memory and

execute it. Almost every file executed on Windows is in PE format, which makes mastery of this format central to static analysis.

Overall Layout

A PE file begins with an **MS-DOS header and stub**, retained purely for backward compatibility. Following this legacy preamble come the **PE headers**, which contain the metadata used to load and execute the file correctly, and then a **section table** that describes the sections. Finally the file's actual content is organized into **sections**, each of which contains a specific kind of data — one section for executable code, another for global variables, another for resources such as multimedia, and so on. Each section is described by an entry in the section table, and usually at least two sections are present in any real executable.



Common Section Names

Certain section names recur so often that they function almost as a vocabulary, though their exact spelling depends on the toolchain. The most commonly present sections in an executable are the following. The **executable code section** is named `.text` under Microsoft conventions, or `CODE` under Borland. The **data sections** are named `.data`, `.rdata`, or `.bss` under Microsoft, or `DATA` under Borland; they hold initialized data, read-only data, and uninitialized data respectively. The **resources section** is named `.rsrc`. The **export data section** is named `.edata`, and the **import data section** is named `.idata`; these list, respectively, the functions the file makes available to others and the functions it needs from others. Finally the **debug information section** is named `.debug`.

A crucial caveat accompanies these names: they are actually **irrelevant to the operating system**, which ignores them and treats them as present only for the convenience of the programmer. The true nature of a section is determined by its characteristics flags, not by its name. The practical implication for the analyst is that section names cannot be trusted — one can encounter very different names in the wild, and packers in particular routinely assign their own idiosyncratic names to the sections they generate.

DOS Header and Stub

The **DOS header** occupies the first 64 bytes of the file. It exists solely for the eventuality that the program is launched from within DOS: in that case DOS can recognize the file as a valid executable and

run the accompanying **DOS stub**. The stub itself is trivial — it usually does nothing more than print a message such as "This program must be run under Microsoft Windows" and exit, a graceful refusal on legacy systems.

Two fields in this header are essential to the analyst. The first is the **magic signature** at the very beginning of the file, whose value is `5A4D` in hexadecimal, corresponding to the ASCII characters **"MZ"** (the initials of Mark Zbikowski, a Microsoft architect). The presence of `MZ` at offset zero is the first confirmation that a file is a DOS/Windows executable. The second essential field is **`e_lfanew`**, located at offset `0x3C`, which is a pointer holding the offset to the PE header. In other words, `e_lfanew` tells the loader where, further inside the file, the real PE header begins, allowing it to skip over the DOS compatibility material.

A subtle but important detail emerges when inspecting these values in a raw hexadecimal dump: multi-byte numeric values are stored in **little-endian** order, meaning the bytes appear reversed relative to how the number is written. The `e_lfanew` field pointing to a PE header at offset `0x000000E0`, for instance, appears in the file as the byte sequence `00 00 01 00` — the bytes laid out in reverse order. At the location it points to, one finds the **PE signature** `50 45 00 00` — the ASCII characters "PE" followed by two null bytes — marking the true start of the PE header. Reading such a dump, the analyst can visually confirm the chain: `4D 5A` ("MZ") at the top, the DOS stub text "This program must be run under Windows" in the middle, the `e_lfanew` pointer near offset `0x3C`, and the `PE` signature at the offset that pointer designates.

The Section Table

Once the three parts of the PE header have been parsed, the loader arrives at the **section table**, the directory that describes each section defined in the file. The table consists of a series of entries, one per section, and each entry is exactly **40 bytes** long. The number of entries is precisely the **NumberOfSections** value read earlier from the COFF File Header, so the two structures are read in tandem: the header states the count, and the table provides that many descriptors. Each 40-byte entry is an **IMAGE_SECTION_HEADER**, and its fields together tell the loader everything it needs to place the section correctly both on disk and in memory.

The fields of a section-table entry are the following.

The **Name** field holds the section name without its terminating character — the familiar `.text`, `.data`, `.rsrc`, and so on. As stressed earlier, this name is a convenience only and is not trusted by the loader.

The **VirtualSize** field gives the size of the section once it is loaded in memory: that is, how much space the loader will allocate for it in RAM.

The **VirtualAddress** field gives the **RVA** of the section — the address, relative to the image base, at which the section will reside in memory.

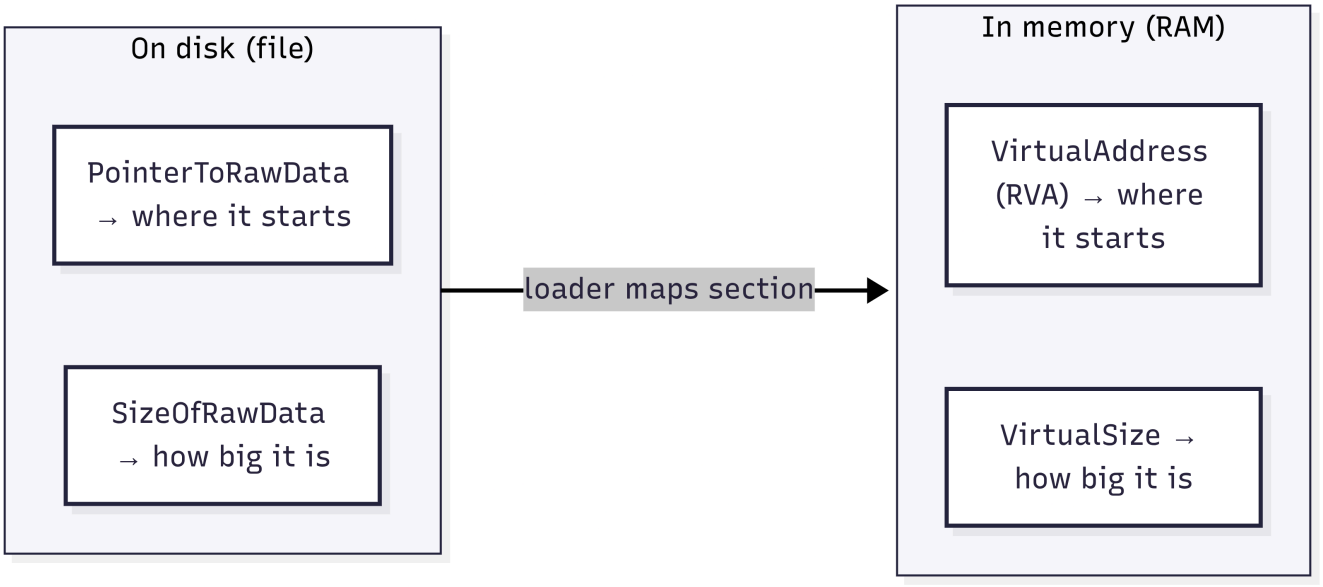
The **SizeOfRawData** field gives the size of the section's data as it exists in the file on disk.

The **PointerToRawData** field gives the offset from the file's beginning to the section's data on disk — where, physically in the file, the section's bytes start.

The fields **PointerToRelocations**, **PointerToLinenumbers**, **NumberOfRelocations**, and **NumberOfLinenumbers** relate to relocation and line-number information used chiefly in object files, and are of secondary importance for typical executable analysis.

The **Characteristics** field is, as in the file header, the most analytically significant. It contains flags describing the fundamental nature of the section: whether the section contains executable code, whether it contains initialized data, whether it contains uninitialized data, and whether it can be written to or read from. These flags, not the name, are what the loader and the analyst rely upon to know what a section truly is. A section flagged as both writable and executable, for instance, is a classic red flag, since legitimate code sections are normally executable but not writable.

The pairing of the two size fields and the two location fields is the conceptual key to the whole section table. **VirtualSize** with **VirtualAddress** describes the section *in memory*; **SizeOfRawData** with **PointerToRawData** describes the same section *on disk*. Because memory and disk use different alignment rules, these paired values usually differ, and the discrepancy between them is not an error but a designed feature of the format.



PE Sections

After the section headers come the sections themselves — the actual code, data, and resources the descriptors point to. The way these sections are laid out differs between the file on disk and the image in memory, and understanding both layouts is essential.

In the file on disk, each section begins at an offset that is some multiple of the **FileAlignment** value found in the Optional Header. Because a section's real data rarely fills an exact multiple of that alignment unit, the space between one section's data and the next is filled with `00` **byte padding**. This padding is not meaningful content; it exists only to satisfy the alignment requirement.

When loaded into RAM, the sections are aligned differently: they always start on a **page boundary**, so that the first byte of each section corresponds to the beginning of a memory page. The size of a page depends on the processor architecture — on x86 CPUs pages are **4 kB** aligned, whereas on IA-64 they are **8 kB** aligned. This in-memory alignment value is the **SectionAlignment** field, also found in the Optional Header. The two alignments — FileAlignment on disk and SectionAlignment in memory — are what make the on-disk and in-memory layouts of the same file diverge.

IMAGE_SECTION_HEADER: Virtual Size versus Size of Raw Data

The distinction between the memory description and the disk description of a section is not merely structural bookkeeping; it is a diagnostic tool. Restating the two fields plainly: **VirtualSize** measures the

section as it will exist in **RAM**, while **SizeOfRawData** measures the section as it exists on **DISK**.

For a well-behaved executable, these two values are normally equal, or very nearly equal, for the `.text` (code) section — the amount of code on disk matches the amount of code in memory, give or take alignment rounding. A striking departure from this equality is a hallmark of packing. In a **packed executable**, the **VirtualSize** of the `.text` section is much larger than its **SizeOfRawData**. The reason is intuitive once the packing mechanism is recalled: on disk the code sits compressed and therefore small, but at runtime the wrapper decompresses it, expanding it into a much larger region of memory that the section header must reserve in advance. A large virtual size backed by a tiny raw size means the section will "grow" dramatically when executed — exactly what an unpacker does.

A concrete section table for a hypothetical `PackedProgram.exe` makes the pattern vivid. Reading each section's virtual size against its raw size exposes several anomalies:

Name	Virtual Size	Size of Raw Data
`.text`	A000	0000
`.data`	3000	0000
`.rdata`	4000	0000
`.rsrc`	19000	3400
`Dijfpds`	20000	0000
`.sdfuok`	34000	3313F
`Kijijl`	1000	0200

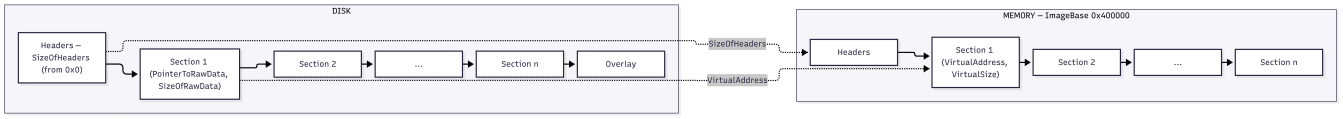
Two categories of suspicion appear at once. First, several standard sections — `.text`, `.data`, `.rdata` — report a nonzero virtual size but a **raw size of zero**, meaning they occupy space in memory yet contribute no data in the file; their contents must be produced at runtime by unpacking. Second, the section names themselves betray the packer: alongside the conventional names sit meaningless, machine-generated names such as `Dijfpds`, `.sdfuok`, and `Kijijl`, which no ordinary compiler would emit. Both signals — the virtual/raw size mismatch and the nonsensical section names — independently point to a packed sample.

Loading in Memory

Bringing the pieces together, one can now trace the complete transformation a PE file undergoes as it moves from disk to memory. On disk, the file begins at offset `0x0` with the **headers**, whose combined length is exactly the **SizeOfHeaders** value. The sections follow the headers on disk, each located by its **PointerToRawData** and extending for its **SizeOfRawData**. At the very end of the file, after the last section, there may be an **overlay** — appended data that lies beyond the region described by the section table and is not mapped into memory by the normal loading process (malware sometimes hides payloads here).

When the loader maps the file into memory, it places the image at the **ImageBase**, the preferred load address — commonly `0x400000`. The headers are copied first, spanning **SizeOfHeaders**. Then each section is placed at its **VirtualAddress** (interpreted relative to the image base) and given room equal to its **VirtualSize**. Because in-memory alignment (SectionAlignment, page-aligned) differs from on-disk alignment (FileAlignment), the sections end up spaced differently in memory than they were in the file: gaps open up between them to satisfy page alignment, and a section whose VirtualSize exceeds its SizeOfRawData is allocated more room than its disk bytes filled, the remainder being zero-initialized.

The relationships among the header fields become concrete in this mapping. **SizeOfHeaders** measures the header block that sits at the top of both layouts. **PointerToRawData** and **SizeOfRawData** locate and size each section *on disk*, while **VirtualAddress** and **VirtualSize** locate and size the same section *in memory*. The overlay at the bottom of the disk file has no counterpart in the mapped image.



The reason the padding and alignment exist is efficiency of memory protection and access: by making each section begin on a page boundary, the operating system can apply page-level permissions — read, write, execute — to each section independently, matching the permission flags stored in the section's **Characteristics** field.

Time Date Stamp

Returning to a field encountered in the COFF File Header, the **TimeDateStamp** deserves a dedicated treatment because of its forensic role. It shows when the executable was **compiled**, and this single fact carries investigative weight: older programs are more likely to be already known to antivirus software, since they have had time to be collected, analyzed, and added to detection databases. A recent compilation date, conversely, may indicate a freshly built sample not yet catalogued.

The stamp must nevertheless be treated with caution, because it is sometimes simply **wrong**. A well-known quirk is that all programs compiled with **Delphi** report the date **June 19, 1992**, regardless of when they were actually built, because of how the Delphi toolchain fills the field. Beyond such artifacts, the date **can also be deliberately faked** by an attacker who edits the field to mislead investigators. The TimeDateStamp is therefore a useful hint but never a proof.

Resource Hacker

The resources bundled into a PE file live in the `.rsrc` section, and a dedicated tool called **Resource Hacker** lets the analyst browse this section directly. Through it one can inspect the **strings, icons, and menus** embedded in the executable. These resources can be revealing: an icon impersonating a legitimate application, menu text in an unexpected language, or embedded strings can all contribute to understanding the sample's disguise and intent.

Clues in Libraries

Some of the most telling evidence about a program's purpose comes not from its own code but from the external functionality it borrows. The PE header **lists the libraries and functions that the program will load**, and the names of those imported functions can reveal a great deal about what the program does. Function names are often descriptive, chosen by the API designers to communicate their effect, and this descriptiveness works to the analyst's advantage. A single imported name can betray a whole capability: the presence of `URLDownloadToFile`, for example, indicates that the program **downloads something** from a URL, immediately suggesting network-retrieval behavior even before any code is examined. Scanning the import list is thus a fast route to a hypothesis about functionality.

Imports

An **import** is a function that a program uses but does not itself contain — a function that is stored in a different program, typically a library. Rather than reimplementing common functionality, a program borrows it from shared libraries, and the mechanism that connects the borrowing executable to the

borrowed code is called **linking**. Linking is what binds the external functions to the main EXE so that, at the appropriate moment, a call in the executable actually reaches the code residing in the library.

Linking can be performed in **three ways**: statically, dynamically, or at runtime. Each has different implications for how the sample behaves and for what the analyst will observe in the PE header, and each is examined in turn.

Static Linking

In **static linking**, all the code from the library is **copied directly into the executable** at build time. The library's functions become an integral part of the resulting file, indistinguishable from the program's own code. This approach is **rarely used for Windows executables** but is **more common in Unix and Linux** environments. Its principal consequence is that it makes the executable **large in size**, since it must carry a full copy of every library routine it uses. Its principal advantage is self-containment: static linking is the main way to go if one wants **self-contained code** that depends on no external files to run. For the analyst, static linking is inconvenient, because the library code is fused into the sample and there is no import entry announcing which library functions are present.

Dynamic Linking

Dynamic linking is the **most common method** on Windows. Here the library code is not copied into the executable; instead, the host operating system **searches for the necessary libraries when the program is loaded** and connects them at that point. The executable carries only references to the functions it needs, and the loader resolves those references against the actual libraries present on the system at load time.

A visible symptom of dynamic linking appears when a required library is absent. If a needed DLL cannot be found, the program fails to start and the system reports an error such as "The program can't start because `MSVCR100D.dll` is missing from your computer. Try reinstalling the program to fix this problem." This message is a direct consequence of the dynamic-linking model: because the library was expected to be supplied by the system rather than embedded, its absence is fatal at load time. For the analyst, dynamic linking is helpful, because the import table plainly lists the libraries and functions the sample depends on.

Runtime Linking

Runtime linking connects to libraries **only when they are actually needed**, rather than when the program starts. It is **unpopular in friendly, legitimate programs**, but it is **common in malware**, especially in packed or obfuscated malware, because it lets the sample avoid declaring its true dependencies up front in the import table — a form of concealment. It is not exclusively malicious, however; it is also used legitimately by software that supports **dynamic loading of plugins**, where the set of libraries to load is not known until runtime. Technically, runtime linking is **most commonly done with the `LoadLibrary` and `GetProcAddress` functions**: `LoadLibrary` loads a named library into the process at runtime, and `GetProcAddress` then retrieves the address of a specific function within that library so it can be called. The analytical significance is that a sample importing little more than these two functions may be hiding its real behavior behind runtime resolution, and the import table alone will not reveal what it ultimately calls.

Dependency Walker

To inspect a sample's dynamically linked functions, analysts use a tool called **Dependency Walker**, which displays the libraries a program depends on and the functions it draws from them. The tool

exposes a simple but powerful heuristic: **normal programs tend to have a lot of DLLs**, reflecting their reliance on the rich ecosystem of Windows libraries, whereas **malware often has very few DLLs**, because it deliberately minimizes its declared dependencies — often precisely because it resolves its real functionality at runtime or is packed.

The contrast is illustrated by two versions of the same-named file. The legitimate `SERVICES.EXE` expands in Dependency Walker into a long tree of dependencies — `MSVCRT.DLL`, `RPCRT4.DLL`, `SSPICLI.DLL`, `NTDLL.DLL`, `PROFAPI.DLL`, `CRYPTBASE.DLL`, and a large number of the modular `API-MS-WIN-CORE-*` libraries covering error handling, file access, handles, heap, I/O, the library loader, the local registry, process environment and threads, string handling, synchronization, and more. This dense forest of imports is typical of a genuine system binary. By contrast, a malicious file masquerading as `SERVICES.EX_` shows almost nothing: it imports only `KERNEL32.DLL` and `WS2_32.DLL`. The sparseness is itself the clue — a program presenting itself as an important system service yet declaring only two libraries is behaving unlike any legitimate counterpart, and the presence of `WS2_32.DLL` (a networking library) hints at network activity.

Dependency Walker also distinguishes, for each module, its **imports** and its **exports**. In its interface the imported functions (the parent's imports, marked **PI**) and the exported functions (marked **E**) of a selected library such as `MSVCRT.DLL` are shown separately, each with its ordinal, hint, and function name. Being able to read both lists lets the analyst see not only what a module takes from others but also what it makes available to them.

Common DLLs

Because so much of a program's intent can be read from which libraries it imports, it is worth knowing what the most frequently encountered DLLs provide. A handful of them account for the bulk of ordinary Windows functionality, and recognizing them lets the analyst infer capabilities at a glance.

`Kernel32.dll` is a very common DLL that contains core functionality, including the access and manipulation of memory, files, and hardware. Almost every program uses it.

`Advapi32.dll` provides access to advanced core Windows components such as the Service Manager and the Registry. Its presence suggests the program interacts with services or registry keys.

`User32.dll` contains all the user-interface components — buttons, scroll bars, and the components responsible for controlling and responding to user actions. A program importing it has, or interacts with, a graphical interface.

`Gdi32.dll` contains functions for displaying and manipulating graphics.

`Ntdll.dll` is the interface to the Windows kernel. Executables generally do not import this file directly; it is almost always imported indirectly by way of `Kernel32.dll`. Consequently, if an executable imports `Ntdll.dll` *directly*, it signals that the author intended to use functionality not normally available to ordinary Windows programs — capabilities such as hiding functionality or manipulating processes frequently reach for this low-level interface. A direct import of `Ntdll.dll` is therefore a suspicious sign.

`WSock32.dll` and **`Ws2_32.dll`** are networking DLLs. A program that accesses either of them most likely connects to a network or performs network-related tasks.

`Wininet.dll` contains higher-level networking functions that implement application protocols such as FTP, HTTP, and NTP. Its presence indicates the program speaks these protocols rather than merely opening raw sockets.

Example: Keylogger

The power of import and export analysis is best seen in a worked example. Consider a sample that turns out to be a **keylogger** — a program that secretly records keystrokes. Its behavior can be reconstructed almost entirely from the functions it imports and exports.

The sample **imports `User32.dll` and uses the function `SetWindowsHookEx`**, which is a popular way for keyloggers to receive keyboard inputs; installing a Windows hook lets the program intercept input events system-wide. It then **exports `LowLevelKeyboardProc` and `LowLevelMouseProc`**, the callback functions through which the hooked keyboard and mouse events are delivered, which the malware uses to capture the data and send it elsewhere. Finally, it **uses `RegisterHotKey` to define a special keystroke** — such as Ctrl+Shift+P — that acts as a trigger to **harvest the collected data** on demand. From these three imported and exported names alone, the complete operating logic of the keylogger emerges: hook the input, receive it through the callbacks, and retrieve the loot with a secret hotkey. No execution and no disassembly were required to reach this understanding.

Example: A Packed Program

The same import-based reasoning also detects packing. A **packed program** presents **very few functions** in its import table, because all the analyst can see is the **unpacker** — the wrapper — while the real program and its true imports remain compressed and hidden until runtime. A representative import listing for a `PackedProgram.exe` bears this out: from `Kernel32.dll` it imports only `GetModuleHandleA`, `LoadLibraryA`, `GetProcAddress`, `ExitProcess`, `VirtualAlloc`, and `VirtualFree`, and from `User32.dll` only `MessageBoxA`. This particular combination is a fingerprint of unpacking behavior. The pairing of **`LoadLibraryA` and `GetProcAddress`** is the runtime-linking mechanism the unpacked code will use to resolve its real dependencies once it is decompressed, while **`VirtualAlloc` and `VirtualFree`** are the memory-management functions used to allocate the region into which the original program will be unpacked and later to release it. A sample whose entire visible import set consists of these load-and-allocate primitives is almost certainly a packed one, showing the analyst nothing but the machinery of its own unpacking.

Exports

Imports and exports are two sides of the linking relationship, and the distinction between them aligns with the distinction between the two kinds of PE module. **DLLs export functions** — they make functions available for other modules to use — while **EXEs import functions**, consuming functionality provided by libraries. **Both exports and imports are listed in the PE header**, so the analyst can read either list from the same structure.

A useful expectation follows from this division: **exports should be rare in EXEs**, since an executable is normally a consumer of functionality rather than a provider of it. When an EXE does export functions, it is anomalous and worth investigating — as the keylogger example showed, an executable exporting callback functions such as `LowLevelKeyboardProc` is doing something unusual. The rule is a guide rather than a law, however, because **reality is sometimes different**: legitimate executables occasionally export functions, so an export in an EXE raises a flag without by itself proving malice.

CHAPTER 4: Basic Dynamic Analysis

From Static Dead-Ends to Running the Sample

Static analysis gives an examiner a disciplined checklist: extract strings, enumerate the imports, parse the PE header, inspect the resources, and reason about what a binary *might* do without ever executing it. Sooner or later, however, this approach reaches a wall. Three situations in particular force the analyst to change strategy. The first is **obfuscation**, where the meaningful logic of the program has been deliberately scrambled so that disassembly and string extraction reveal little. The second is **packing**, where the real code is compressed or encrypted on disk and only unpacked into memory at runtime, so that the static view shows a thin loader rather than the payload. The third is simpler and more human: the examiner has **exhausted the available static techniques**—every tool has been run, every obvious artifact has been catalogued—and the picture is still incomplete.

A particularly instructive case is a binary that **resolves its imports at runtime** rather than declaring them in the import table. Statically, such a sample appears to call almost no API functions at all, which makes it nearly impossible to guess how it interacts with its environment. The natural response is to stop reasoning about the program and instead *run it* and watch what it actually does.

This is **dynamic analysis**: deliberately executing a suspicious sample while monitoring the results. It is tempting to describe it as the technique that "shows you exactly what the malware does," but that claim deserves immediate qualification—**not really**. Dynamic analysis shows you exactly what the malware does *along the specific execution path it happens to take, in the specific environment you provide.* The central, honest goal is therefore more modest and more useful: to **understand the malware's behavior**. The behavioral approaches used to reach that goal fall into four families—**diffing, monitoring, tracing, and debugging**—which the following sections develop in turn.

The Controlled, Safe Environment

Running malware on purpose is only responsible inside a **controlled, safe environment**. The defining property of such an environment is **containment**: whatever the sample does must be prevented from escaping to production machines or to the wider world. The most important containment measure is to **disconnect the analysis machine from the Internet**. In the strongest form this means an **air-gapped** machine with no network connection of any kind—neither to the Internet nor to other machines. A slightly more practical arrangement places the analysis machine inside a dedicated subnet of machines reserved for malware analysis, which itself has no route to the Internet. On real hardware, disconnection can be as crude as unplugging the Ethernet cable or switching off Wi-Fi, which works but is cumbersome; for this reason virtual environments are more common, though the principle is identical—the machine must be cut off from the Internet.

There are three distinct reasons the Internet connection must be severed, and each matters independently.

The first reason is **containment of replication**. If the sample is the kind of malware that tries to propagate automatically—replicating onto other local machines or attacking other hosts on the network—then letting it reach the network is both unethical and operationally damaging. You would be participating in spreading the infection.

The second reason is **operational security against the attacker**. The moment a live sample is allowed to communicate with the outside world, it may signal back to its operator that it has executed and that a particular machine has been reached. This hands the adversary a valuable piece of intelligence. For sophisticated campaigns, or when the defender works for a government, a large corporation, or any organization holding sensitive data, tipping off the attacker that they have been detected—or that a specific host has been compromised—is a serious mistake. Even when the sample has

already run and infected some machines, so that the warning is partly moot, a controlled environment is still required for the third reason.

The third reason is **fidelity of observation**. The entire purpose of the exercise is to let the environment yield clues about what the sample is doing. If the sample is let loose, those clues are lost: the analyst does not get all the information they need, because the behavior leaks out of view instead of being captured.

Severing the network does not mean the sample must be starved of all communication. A productive compromise is to build a **dummy network**: a simulated environment that answers the sample's network requests convincingly enough that the malware believes it has connectivity, while nothing actually reaches the Internet. Later sections develop exactly how such fake services are provided; for now the key idea is that disconnection and interaction are not mutually exclusive—the analyst can work around the air gap.

Why Behavior Can Still Hide From You

Dynamic analysis lets the examiner sidestep the showstoppers of static analysis—obfuscation, packing, and the exhaustion of static tooling—but it introduces a new adversarial problem. Malware may actively **resist dynamic monitoring**. A sample may detect that it is running in an environment that is "not the right one," or, worse, in an environment conspicuously full of the tools an analyst uses. A common way to express the intuition is this: an ordinary victim—think of a relative's home computer—does not have professional reverse-engineering tools installed, and malware will happily infect such a machine; but a machine loaded with analysis tooling looks nothing like a victim, and a well-written sample will detect those tools and **refuse to run**. The consequence is that dynamic analysis is powerful but must be conducted with full awareness of what the environment reveals about itself. The remainder of this chapter builds up the techniques, the tooling, and the environmental precautions needed to cover these bases one step at a time.

The Four Behavioral Analysis Techniques

All dynamic analysis seeks the same thing—to understand what a sample will do to a machine—but there are several ways to observe it, and they differ sharply in how much they reveal and how much effort they demand. Four techniques form a ladder: **diffing**, **system monitoring**, **API tracing**, and **debugging**. Moving down the ladder, each technique is a more powerful and more precise form of analysis; moving up it, each is easier to use and operates at a higher level of abstraction.

Diffing

Diffing is the coarsest and simplest technique. The method is to capture a **snapshot of a clean system state**, execute the malware, capture a second snapshot of the now-**compromised system state**, and then compare the two—quite literally a "spot the difference" exercise between before and after. Any large, persistent change stands out plainly, and that is the technique's chief advantage: **artifacts can be observed easily**, with only a few minutes of work needed to produce an initial characterization.

The weakness of diffing follows directly from its before/after nature. It can **miss any evidence that is created during the malware's activity and then erased**, whether the deletion is a deliberate anti-forensic act by the malware or simply the normal cleanup of a short-lived operation. Some effects are **permanent**, and those survive into the final snapshot; others are **transient**, and those vanish before the second picture is taken. A network connection that is opened and then closed is the classic example—it leaves no trace in the after-snapshot even though it was a meaningful behavior. Diffing therefore captures the most visible, lasting effects but little else.

Two tools exemplify the approach. **Regshot** captures two instances of the Windows registry and reports which keys were created and which were modified between them. **Autoruns** reports which software has been registered for automatic startup; if a new automatic-start entry appears, the malware has achieved **persistence**—a foothold that causes a new program to launch every time the machine reboots. The pros, cons, and tools of diffing can be summarized as follows: it observes artifacts easily, it can miss created-then-erased evidence, and its representative tools are **regshot** and **autoruns**.

System Monitoring

Monitoring is the first genuinely live technique. Starting again from a clean system state, the analyst **records every individual change to the system and to network traffic that appears after the suspicious file is executed**. Rather than comparing two static pictures, monitoring continuously captures the changes as they happen—changes to files, changes to the registry, network communication—so that the most visible effects are logged in real time, including the transient ones that diffing would lose.

Monitoring of this kind requires **cooperation from the operating system**: the OS must expose facilities for collecting the relevant events, and all the major operating systems—Windows among them, but also Linux—provide such facilities to ease the work. The advantage is completeness at the event level: monitoring **can collect all manifested changes**. The corresponding drawback is volume. Monitoring is a "shotgun" approach that gathers **far too much information**, so the analyst must **weed out the irrelevant data** and decide what, in a large stream of operations, actually matters.

The need to filter is not a minor inconvenience; it is where inexperience produces wrong conclusions. Two recurring misreadings illustrate the point. First, a sample's startup frequently causes Windows to spawn supporting processes such as `conhost.exe` and `cmd.exe` (the Windows command-line interpreter). A novice reports these as malicious activity, when in reality they are the ordinary scaffolding Windows needs to support a command-line application—noise that should simply be discarded. Second, the `CreateFile` operation is routinely misread. In Windows terminology, **`CreateFile` does not necessarily mean "create"**; it means "create *or* open." Opening a file that already exists still appears in the log as `CreateFile`. The analyst must therefore filter by the operation's flags—distinguishing, for example, the case "if the file does not exist, create it" (worth recording) from the routine loading of a system library such as `kernel32`, which in turn loads further standard libraries (not worth recording). Here the examiner is overwhelmed by information and must discard aggressively; success depends on skill, on the quality of one's filtering scripts, on experience, and on how event-intensive the particular sample is—some samples are quiet, others will flood the log.

Monitoring also invites a specific evasion called **API hammering**. The malware deliberately issues a very high-frequency burst of API calls—"hammering" the event stream—choosing operations that execute almost instantly on a normal machine but introduce large latencies when monitored. A routine that would add perhaps ten seconds to normal execution can stretch to ten minutes under monitoring, because the monitoring overhead is multiplied across the hammered operations. The technique both slows the analyst and buries genuine behavior in artificial events, so it must be anticipated.

The principal tools for system-wide monitoring are **Process Monitor (procmon)** for registry, file, process, and thread activity, and **Wireshark** for sniffing and tracing network communication. Monitoring's profile is thus: it collects all manifested changes, it often produces too much information requiring the analyst to weed out irrelevant data, and its tools are **procmon** and **Wireshark**.

API Tracing

API tracing is more fine-grained than monitoring. Instead of recording only the *changes* a sample makes, it **hooks and records every important API call the suspicious process makes**—every interaction with the system, not merely the ones that leave a visible mark. At first this sounds like even more data, and it can be, but tracing comes with powerful ways to reduce the stream to signal.

The first reduction is to **record only the APIs the malware calls directly**. When a program calls a function in a Windows library, that function typically calls many further APIs internally, producing a large cascade of calls. By restricting attention to the calls made directly by the sample, the analyst collapses this cascade and keeps only the decisions the malware itself made. The second reduction is **categorization**: focus on specific, security-relevant classes of API—**file, process, registry, services, network**. The third reduction is **curation**: concentrate on APIs notoriously associated with malicious activity. An analyst begins with a published list of perhaps three or four hundred such APIs and, over time, refines it into a personal curated set. Tracing also offers a clean defense against API hammering: when a sample hammers two or three specific APIs, the analyst simply disables logging for exactly those APIs and steps around the technique.

The result is **rich visibility into activity beyond the typical file/process/registry/network picture** that monitoring provides—getting the examiner closer to the kind of interpretation normally required by static analysis. The cost is that **there is still often too much information to weed through**, and **API-specific interpretation can take a great deal of time**—though still less than full static analysis. Representative tools include **Rohitab API Monitor** and **WinApiOverride** (note that more than one tool shares the "API Monitor" name), alongside academic prototypes such as a tracer developed at this institution and the open tool **TinyTracer**.

Debugging

Debugging is the most powerful and most demanding technique, operating one level finer than individual API calls. A debugger lets the analyst **control execution at the granularity of individual instructions**. The central mechanism is the **breakpoint**: the examiner directs the program to run normally but to halt when it reaches a chosen instruction, at which point the program's state—its memory and registers—can be inspected. This capability makes debugging a **superset of the functionality of an API monitor**: anything a tracer can observe, a debugger can observe, plus the full internal state at any chosen point.

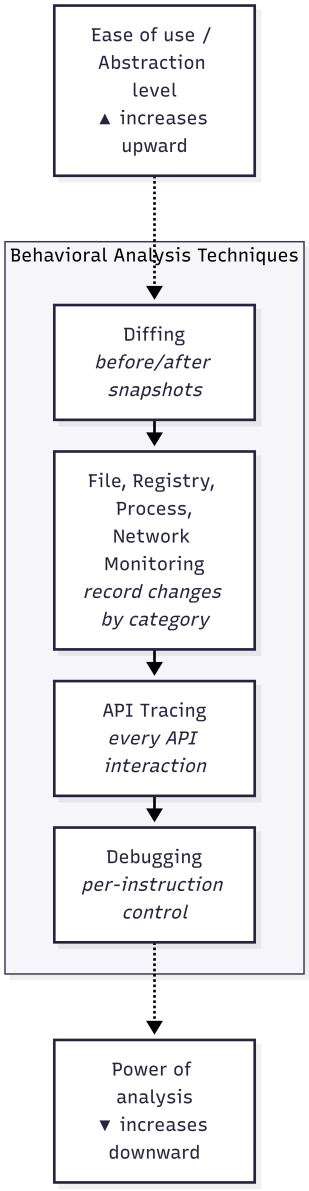
The price of that power is effort and fragility. Choosing useful breakpoints presupposes some **advanced static analysis**—the examiner must read the code well enough to know where to stop—and debugging is typically **done in conjunction with basic static analysis and assembly reading**. Debugging also provokes the strongest evasion: **malware will frequently change its behavior or refuse to run when it detects that it is being debugged**, which forces the analyst to devise workarounds, such as patching the sample to neutralize anti-debugging checks.

The debuggers used in practice each have a niche. **IDA** is the natural default for a course of this kind because it is both a disassembler and a debugger in one; its debugger once required a paid license, but after the NSA released **Ghidra** for free, IDA made its debugger and other features freely available. **OllyDbg**, historically significant and now continued as **x64dbg**, excels whenever a sample does unusual things with its own code—self-modifying code or complex packing schemes. **Immunity Debugger** is optimized for memory-dump and forensic work and carries useful plugins. **WinDbg** is the hardcore Windows reverser's tool: it is primarily command-line rather than graphical, but it is uniquely powerful because it can inspect what happens inside Windows drivers—that is, it can debug the **Windows kernel**. Windows normally protects the kernel from such inspection, but certain operations are

permitted, and WinDbg can even attach from the outside to an entire virtual machine, which is exactly what is needed when analyzing **rootkits**—malware that compromises the kernel rather than user space. WinDbg is the ultimate debugging tool, but it is reserved for when it is genuinely needed; for malware of ordinary complexity, IDA is sufficient and excellent. The debugging profile is therefore: it provides a superset of an API monitor's functionality, it typically must accompany static analysis and assembly reading while malware may alter its behavior under a debugger, and its tools are **IDA, OllyDbg, Immunity Debugger, and WinDbg**.

Comparing the Techniques

The four techniques form a single spectrum. Diffing is a no-brainer—run the sample and compare before with after—but it yields little beyond the most visible effects. File/registry/process/network monitoring focuses on event categories and gives good results but already carries noise. API tracing shows almost everything—nearly every interaction with the operating system, unless the tool can be evaded—at the cost of an even larger data volume. Debugging shows truly everything but overwhelms the analyst with data. The two axes along which they trade off are the **power of the analysis** and its **ease of use / level of abstraction**, which move in opposite directions.



In practice an examiner needs **almost all of these methods**, chosen according to the situation. Diffing and simple event-detection tools give a fast first assessment of what a normal execution looks like. When a sample then refuses to execute on the analysis machine, API tracing can reveal that it was tipped off

by something it found on the host—a detail exposed through a particular API call. The analyst then descends to the debugger to work around the obstacle: patch the offending sequence, force the sample down a different path, and finally observe the full range of behaviors. The guiding principle is to match the technique to the question, moving down the ladder only as far as the evidence requires.

The Fundamental Limits of Dynamic Analysis

Dynamic analysis is powerful, but it is bounded by limits that are not peculiar to malware—they are the same limits that govern program analysis and software testing in general. In software testing the two broad families of technique are **static testing**, where code is inspected without running it, and **dynamic testing**, where the program is executed. Static testing is attractive because it can reason about all of the code, but it finds only specific classes of bug and suffers from both **false positives** and **false negatives**, because it works on an approximation of the real execution rather than the real thing. Dynamic testing is attractive because a real defect, when hit, actually manifests, but it suffers from the **coverage problem**: you only exercise the parts of the program for which you have supplied an input. Any path you never reach is never tested, and you remain ignorant of what lives there.

Both limits reappear exactly in malware analysis. Static malware analysis gives only an **approximation**, one that a malware author can deliberately make coarser by hiding things, and that is further constrained by the limitations of the tools. Dynamic malware analysis, meanwhile, inherits the coverage problem in a particularly sharp form: **in general, only a single execution path—one execution trace—is examined**. There are paths in the sample's potential behavior that a normal run simply never activates.

Some of those unreached paths are harmless—dead code, logic that exits when some component is absent, or fail-safe branches—and ignoring them costs nothing. But many are reached only under conditions the analysis environment fails to meet, and these fall into two opposite categories.

The first category is the **conspicuous environment**, which drives the sample toward an **evasive outcome**. The analysis machine is too obviously an analysis machine: it is littered with tools, it has only one or two CPUs when a real modern machine has at least eight cores, its disk is suspiciously small, it has only four gigabytes of RAM, or it bears other hallmarks of virtualization. Detecting any of these, the malware chooses the evasive branch and withholds its real behavior. This is the problem of the environment containing **too much** that betrays it. Beyond coarse fingerprinting, the limits worth remembering are that a single path is examined, that the **analysis environment is possibly not invisible**, that it is **possibly not comprehensive**, and that there are **scalability issues** when many samples must be processed.

The second category is the **incomplete environment**, where the problem is what is **missing** rather than what is present. Consider a hypothetical sample that activates only by exploiting a vulnerability in a specific version of Java—say Java 16. If that exact version is not installed, the malicious path never runs, and no amount of monitoring will reveal it. The remedy here is not to hide artifacts but to **arrange the environment to meet the sample's expectations**: recover the required dependency and install the vulnerable software. Discovering the requirement is itself part of the work—an API call that checks the program-files directory for, say, an Oracle Java 16 folder is often the clue that tells the analyst what to provision. Resolving these cases is a matter of modifying the environment, patching the sample, or a mix of the two.

How Observation Is Made Possible: Interposition

The practical question beneath every dynamic technique is: *how* do you actually see the events? The answer begins with help from the operating system, but it can also come from **hardware**—and, better, from **virtualized hardware**, where a hypervisor confers monitoring capabilities—and from lightly **patching the program** itself. The general name for the act of observing an event, and of making that observation possible, is **interposition**.

Interposition is the ability to **intercept an event, optionally inspect or check it, and then let the event proceed**. It is a broad and old idea in systems and software security. Intrusion-detection schemes, for instance, interpose on network requests of a given type and, if a request matches a blacklisted pattern, change the outcome so the forbidden action never happens. In other words, by interposing one can not merely observe but also *alter* the operation of an event. For the purposes of dynamic malware analysis the goal, for now, is just to **log** events—but the mechanism is the same, and to perform interposition one needs **instrumentation**.

Instrumentation: Modifying Something in Order to Observe

To interpose you must modify something. Instrumentation is that modification, and it can be applied at three levels: the **program**, the **operating system**, or the **hardware**.

Program Instrumentation

Program instrumentation modifies the execution of the application's own instructions, so that the analysis operates in the **same address space as the sample**. The canonical example comes from debuggers and the **software breakpoint**. To stop execution at a specific instruction, the debugger replaces the bytes of that instruction with one particular opcode that the CPU recognizes as a halt-and-trap instruction. When the CPU reaches it, an exception is raised, and the debugger takes over control of execution. A software breakpoint is thus literally a one-byte patch to the code: the original instruction's first byte is overwritten with the breakpoint opcode, interrupting execution exactly where desired. This is instrumentation at the level of individual instructions, and it underlies manual analysis with a debugger.

When per-instruction control is more than needed—when seeing the API calls would suffice—there are two or three ways to interpose on API calls. Windows provides a native mechanism called **Detours**. Detours installs a **trampoline**: the prologue of the target API is replaced with a jump to the monitoring tool, which records the call and then jumps back into the original API code. Because it rewrites the function entry itself, this form of instrumentation **reliably captures all API calls**. A looser, lighter family of methods works instead at the level of the import/address information: when Windows reports that, for example, `CreateFile` is loaded at some address, the instrumentation substitutes a different address pointing at a small stub that registers the call and then invokes the real API. This address-rewriting approach is easier but comparatively easy for a malware author to evade, whereas the trampoline approach is far harder to evade. More aggressive program instrumentation is possible too—beyond breakpoints, one can make larger software modifications or even **emulate the instructions** outright.

The crucial caveat is that **most of these modifications are not invisible**. A sample can compute a **checksum of its own code** and, through such an integrity check, detect that its bytes have been altered—because it knows exactly what opcodes it was compiled with, it can validate the expected sequence, notice that one byte has changed, see the checksum differ, conclude that it has been tampered with or patched, and refuse to run. Program instrumentation therefore grants the finest and most precise control, but it can require significant effort and must be used carefully to avoid detection. In summary, program instrumentation operates in the sample's own address space, is the basis of manual debugging and of Detours-style API hooking, modifies the binary under analysis—inserting breakpoints, rewriting

functions, using debug registers—and is **not invisible**, so malware can detect the analysis artifacts, often demanding substantial manual effort.

OS Instrumentation

Instrumentation can instead live inside the operating system where the sample runs. On Windows this takes the form of **system-call hooks**: every Native API invoked—whether by the malware directly or by the libraries it uses—can be guaranteed to be logged. Because the instrumentation sits below user space, it is **somewhat invisible to user-mode malware**, which is an advantage. It has corresponding weaknesses. It **can cause problems when the malware runs in the OS kernel**, and it has **limited visibility of the activity inside the program**—for example, it cannot set function breakpoints, because it does not see the program's internal control flow, only its crossings into the OS. The signal it provides, while guaranteed, is therefore **coarse-grained**: real but not always detailed enough.

Windows also offers a complementary, more modern facility: **ETW (Event Tracing for Windows)**. ETW is a parallel stream of events that naturally accompanies process execution; the analyst can tap this list as another form of OS-provided logging. It is **hard for malware to evade** and is useful as a complement, though it is not very precise on its own. OS instrumentation, then, operates in the OS where the sample runs, uses Windows system-call hooks, is somewhat invisible to user-mode malware, can misbehave when malware runs in the kernel, and offers only limited visibility into the program's internals.

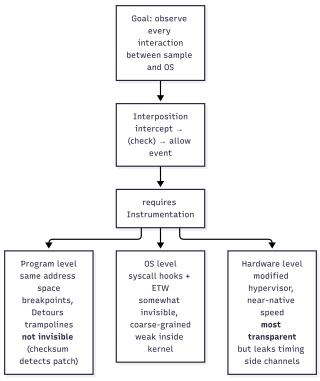
Hardware Instrumentation

The strongest level is instrumentation of the hardware. One does not modify a real CPU; instead a **hypervisor provides a virtual machine (a virtual processor, and sometimes a whole OS environment) that is modified for monitoring**. Historically this was done by **software emulation of the executed instructions**—emulating an entire virtual machine instruction by instruction. The modern approach, the subject of a dedicated later lecture, is different: security architects adapt advanced CPU features—the **virtualization extensions** and the **page tables**—to introduce observation capabilities while the code runs at **near-native speed**, with no emulation. For a normal application you cannot tell the difference in performance.

The great virtue of hardware instrumentation is **transparency**. The analysis **observes activity "from the outside,"** so it is **transparent to the sample (and to the guest OS)**; of the three levels it is the most transparent. The sample runs inside a virtual machine whose entire state can be observed externally. The trade-offs are that an **OS environment must still be provided**, that a **limited (minimal) environment could be detected but is faster**, and that a **complete environment is more comprehensive but slower**. This line of development leads directly to **sandboxes**, discussed shortly.

Transparency is powerful but not perfect. Vendors have for years modified hypervisors to make them as realistic as possible—patching BIOS data, presenting a slightly more realistic graphics device, and generally narrowing the gap with a real environment—yet the result is still imperfect, because this kind of monitoring introduces **side channels**. A side channel is an **unintended source of information leaking from the execution**. Hardware-assisted monitoring subtly changes the memory behavior of the application—polluting certain caches in characteristic ways—so that a careful adversary can infer the presence of the analysis not from files, artifacts, or devices, but from **timing**: some instructions execute slightly slower under observation.

The classic illustration of a timing side channel comes from cryptography. In the late 1990s it was shown that many cryptographic implementations were broken not because the underlying algorithms were flawed, but because the **code executed at different speeds depending on the secret key**. If the time taken by a cryptographic primitive depended on whether, say, a particular key bit was 0 or 1, then across a large number of iterations—hundreds of thousands—an attacker could amplify a timing difference of about a microsecond, imperceptible to a human, into full recovery of every bit of the key. The same style of inference applies in reverse to malware analysis: a sample can use timing side channels to detect that it is being observed. This is an active research area—work on exactly these detection-and-evasion dynamics has been presented at venues such as Black Hat—and it describes a world in which adversaries continually find new ways to detect the analysis system. The defender's response is, first, to build the **most transparent environment possible**, and second, to remain aware of these evasive sequences so that in the best case they can be seen, patched, and worked around. When the analyst succeeds, they can observe essentially every interaction the process has with the operating system.



What to Observe: System Calls and the Windows API

Once the analyst has succeeded in building an observation capability—by hardware instrumentation, a modified operating system, tooling, or any combination—attention turns to *what* to watch. The answer follows from a simple architectural fact: a **process interacts with the operating system through system calls**, and it needs the OS for **every interaction with its environment**—the file system, the network, the registry, and so on. Anything malicious a sample wishes to do, including stealing or exfiltrating data, ultimately requires OS support. Therefore the productive focus is the sample's interactions with the operating system.

On Windows these interactions can be intercepted at two layers, and the distinction matters. At the lowest user-accessible layer live the **Native APIs**, exposed by `ntdll.dll`, where "NT" stands for *native*. Many of these system calls are officially **undocumented and can change without notice**; Microsoft's own documentation declares them private and not for public use. In practice they are only "somewhat" undocumented, because reverse-engineering communities have reconstructed the precise semantics and arguments of perhaps ninety percent of them—but officially they are off-limits. Above this layer sit the **Windows APIs**: a collection of **stable user-mode shared libraries**—every library except `ntdll`—which developers are expected to use. These are the documented, stable surface. Of course, a determined program **can bypass the Windows APIs** and call the Native layer directly, which is itself a signal worth noticing. Monitoring at the Windows API layer already yields good signal; from that signal the analyst can then sort activity into the relevant categories.

The Report: Categories of Activity to Collect

The output of behavioral analysis is naturally organized into categories, each with its own dedicated tools. **File activity** covers reads, writes, creations, openings, and the like. **Registry activity** covers

manipulation of configuration keys and values. **Service activity** covers the starting and stopping of Windows services through the Service Manager. **Process activity** covers process start and termination and inter-process communication. **Network activity** covers API calls and packet logs. Much of the analysis amounts to performing these categories of tracking by hand—track file activity, track registry activity, track process activity, and so on—which is precisely the set of manual tasks that a sandbox tries to automate.

Sandboxes

A **sandbox** is an all-in-one, off-the-shelf solution for basic dynamic analysis: the analyst feeds a sample to it and receives a report describing the sample's behaviors across all of the categories above. Sandboxes can also perform static analysis—a desirable feature for users—but that is a bonus rather than a requirement, since static analysis does not need a sandbox. Internally a sandbox is usually a **virtualized environment**, which handles the no-Internet problem elegantly: the machine remains cut off from the real Internet, but **simulators pretend to provide network services**. If the malware performs a DNS lookup or issues an HTTP request, the sandbox replies with a plausible response—imperfect, but enough to convince the sample that connectivity exists.

Sandboxes represent years of work by specialized companies. Among the historically notable examples are **Norman Sandbox, GFI Sandbox, Anubis, Joe Sandbox, ThreatExpert, BitBlaze, and Comodo Instant Malware Analysis**; in current practice, **Joe Sandbox** is regarded as one of the best tools available, and **VMRay** is likewise excellent. They are **expensive but easy to use**, with different subscription tiers; government-grade licenses can run into the hundreds of thousands per year. Some offer a **free tier**. In exchange, they **automate dynamic analysis** and **produce a clean report**—for instance, Joe Sandbox Cloud publishes example reports—so that running a sample yields a very detailed account with little effort.

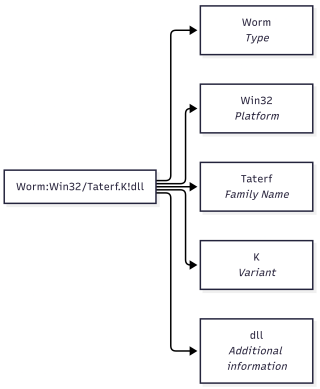
A better sandbox also mounts better defenses against classic fingerprinting. The standard artifacts and tactics malware uses to detect analysis are countered, and good sandboxes offer **cookbooks**: the user can instruct the sandbox to build a virtual machine with specific software installed—a particular Java version, a specific old Office release—so that samples with environmental prerequisites will activate. This is what justifies the price: sandboxes automate a great deal of dynamic analysis and generate good reports, **but they still require a capable operator**. The purpose of learning the underlying techniques by hand is precisely so the analyst is not helpless when a sandbox falls short.

The limits of blind sandbox trust are concrete. A sandbox may be unaware of specific logic embedded in a sample—for example, a requirement that a particular file with particular contents be present on the desktop, without which the behavior changes. A skilled analyst knows to tweak the sandbox to satisfy such conditions; an analyst who simply trusts the tool can be misled, and very high-quality malware will deliberately resist sandboxes. The sound workflow is therefore to **make a first run in a sandbox**, and if little malicious behavior is observed, treat that as a clear indicator that the full story has *not* been obtained, and move to in-depth manual analysis. Sandboxes are among the best first tools available and combine well with the manual techniques described throughout this chapter.

Malware Classification

A useful by-product of sandboxes—and of antivirus engines—is a **tentative family classification**: many sandboxes attempt to give a name to the sample under analysis. The naming conventions derive from the **Computer Antivirus Research Organization (CARO) Malware Naming Scheme**. A canonical CARO-style name such as `Worm:Win32/Taterf.K!dll` decomposes into five parts: the **Type**

(`Worm`), the **Platform** (`Win32`), the **Family Name** (`Taterf`), the **Variant** (`K`), and **Additional information** (`dll`).



In practice, vendors do not agree on a single convention. A few representative conventions illustrate the divergence: Symantec uses the form `Prefix.Name.Suffix`, as in `Infostealer.Banker.C`; Avira uses `Prefix:Name [Type]`, as in `Win32:Zbot-BS [Trj]`; and Kaspersky uses `[Prefix:]Behaviour.Platform.Name[.Variant]`, as in `Trojan.Win32.Genome.taql`.

The deeper problem is that **classification schemes are hardly coherent**. The same specimen receives many different names across engines—twenty engines can yield twenty different labels, thirty sandboxes can yield twenty-five different names. A single real sample scanned across many antivirus products demonstrates this plainly: one engine calls it `Win32/Kido.worm`, another `Worm/Conficker.Z`, another `Worm/Win32.Kido.gen`, another `Win32:Rootkit-gen [Rtk]`, another `Worm/Downadup`, another `Worm.Generic`, another `Win32.Worm.Conficker.B`, another `Trojan.Dropper`, another `NetWorm.Win32.Kido.A`—all describing the same piece of malware, updated around the same dates. Better dedicated tools exist for grouping malware into families, but it is worth knowing that many sandboxes will also assign some label, and that a given sandbox may at least recognize, in its own naming scheme, that two related samples belong to the same family. Names from any single engine should therefore be treated as tentative hints, not ground truth.

Analysis Environments

The choice of where to detonate a sample is a trade-off among realism, safety, speed, and convenience. There are three options: a dedicated **real machine**, a **virtual machine**, and—covered above—a sandbox built on virtualization. Each has a distinct profile.

Real Machines

A real, physical machine dedicated to malware analysis has one overriding strength: it is the **most realistic object available, the best approximation of an actual victim**. Because it is genuine hardware, it does not suffer the **virtualization artifacts** that sophisticated malware looks for, and some malware **detects virtual machines and refuses to run properly in one**—so a real machine sidesteps that evasion entirely. It also avoids analysis-induced slowdowns. In an incident-response context this realism is especially valuable: an examiner can take a machine from the victim's own perimeter—one of the hosts of a breached organization—and run the sample on it, obtaining the most realistic possible result, provided snapshots and backups have been taken first.

The disadvantages are practical and significant. A real analysis machine normally has **no Internet connection**, so parts of the malware that depend on connectivity may not work; providing a convincing simulator is harder on bare metal, often requiring a second machine cabled directly to the first to act as the fake network—on the assumption that this second machine does not itself become infected. The

deeper problem is cleanup: after a single run the disk should be **wiped and the machine re-imaged**, which is time-consuming. Industry tools exist to optimize re-imaging, but it remains demanding enough that a real machine cannot be used casually for everyday analysis.

Virtual Machines

Virtual machines are the **most common method**, for two reasons. The first and most important is **time**: with a VM the analyst acquires a snapshot, performs the analysis, then discards the dirtied machine and restores the clean state with the press of a single button. The second is **isolation**: a VM protects the host from the malware. Of the two, time is the greater everyday advantage—even more than security.

A strong practical recommendation follows from the time advantage: always take a snapshot of the machine while it is **clean, pristine, and working**, before any analysis begins. Many people install VirtualBox or VMware, create a VM, and then never create a snapshot—which squanders the main benefit. Every time a sample is dissected, the environment becomes polluted: the wallpaper may change, new entries appear in autorun, and so on. Work is always best conducted from a clean state, and with a hypervisor restoring that state is a single click. The same ease is now available on KVM, which has graphical front-ends as well.

The isolation claim deserves a careful qualification, because it is somewhat divisive. A VM is isolated from the host **unless the hypervisor has a bug**, in which case a **VM-escape attack (also "VM-escape" / "VMScape")** becomes possible. If an attacker breaches the isolation layer the hypervisor imposes on the guest, two things happen in sequence. First, the attacker **escalates privilege**, because the hypervisor runs at a CPU ring higher than the guest, so breaking out is already an elevation. Second, from that vantage the attacker can **compromise the host machine**.

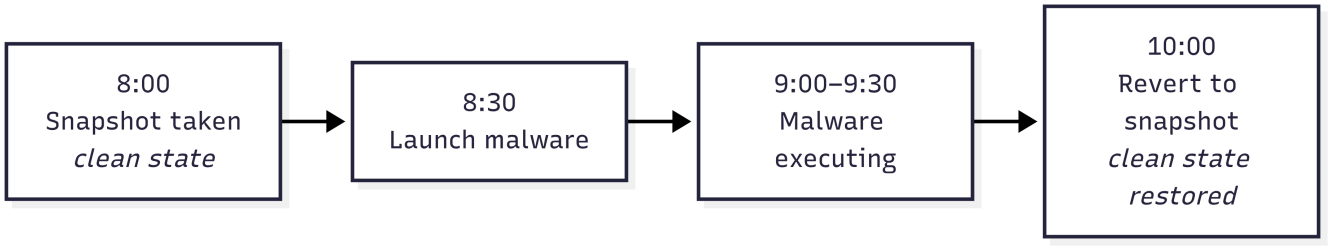
Whether malware does this in the wild is a matter of calibrated judgment. For a course environment, students are more than safe: everyone is running a generic VirtualBox, and across a class of fifty people there will be several slightly different minor releases, so a single bug cannot be exploited uniformly on every machine. The risk changes entirely when the target is an industry. A valuable target runs an **enterprise-class hypervisor**—VMware ESX, Hyper-V, or similar—and finding a bug there is far more profitable, because it can compromise valuable assets. If an attacker knows which hypervisor version a specific victim uses, a sample can be **weaponized against that precise version**. This matters because virtualization is no longer merely an analyst's tool: real workloads increasingly run in the cloud, whether rented from providers like Amazon or Azure or hosted in an organization's own facility, where virtualization is used for efficiency—four physical servers running twenty server applications instead of twenty physical machines. Virtual machines were in fact designed for performance in the first place. The honest assessment is that VM-escape attacks are possible: not for consumer-grade malware, but credible for APTs or highly complex attacks against very valuable financial targets. A statement that would have been dismissed as a joke five years ago—"VM escape is realistic"—can no longer be dismissed; it is unlikely but genuinely possible.

Far more dangerous in everyday practice than VM escape is **naive VM usage**—using a VM incorrectly. Two concrete failure modes recur. In the first, a researcher sets up a Windows host and a Windows guest with a **shared folder**, and places a malware dataset in that shared folder. Initially the host's Windows Defender wipes the shared folder; the researcher then disables Defender on the host to stop this. Now, when a sample is run in the guest and tries to replicate across the network, it treats the **shared folder as a network volume** and plants a file there; later the host user opens that folder, and if an exploitable vulnerability is triggered, the host is compromised—because the malicious file was planted on something

shared with the host. In the second failure mode, the VM's network adapter is placed in **bridge mode**, which connects the guest directly to the real network card, so that other people on the same network—family members running Windows, for instance—are attacked by the sample communicating directly through the host's network interface. The lesson is that one should worry more about *how* a VM is used than about VM escape.

Snapshots

If the VM is trusted, snapshots are the feature that makes it worth using. The analyst can **take a snapshot, run the sample, and revert**, and can maintain **multiple layers of snapshots**. Snapshots serve three purposes: to **protect a clean installation** of the analysis environment, to **keep track of ongoing progress during analysis**, and to **hop instantly from job to job** when several samples must be analyzed at once. During an investigation one can snapshot after reaching a finding, continue, snapshot again after the next finding, and so branch into parallel lines of work. The intended rhythm of a snapshot-based session is: take the snapshot, launch the malware, let it execute, and revert to the snapshot when done.



Snapshots come in two forms. **Offline snapshots** capture only the disk and are cheap enough to keep by the hundreds—VirtualBox can keep up with that many. **Live snapshots** are taken while the machine is running and capture the raw RAM as well, so execution can resume from exactly where it left off—invaluable when in the middle of a debugging session. Live snapshots are costly, however, because each requires a dump of RAM and further burdens the disk, so they should not be overused.

Oracle VirtualBox

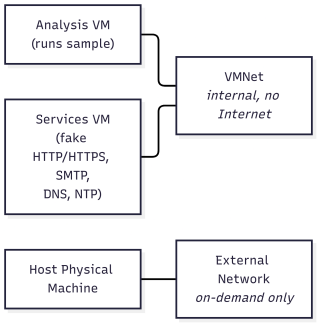
For analysis on a native Intel or AMD machine, **Oracle VirtualBox** is the typical choice. It is **free**, it can **read and write VMware disks**, it offers **reasonable performance** for the goals of this kind of work, and it supports taking **several snapshots and reverting to them** as needed. (On Apple Silicon or other ARM CPUs, VirtualBox is not available and an emulator is required instead.) A representative VirtualBox Manager view shows a library of guests—an XP Professional machine branching into SP2 and SP3 subtrees, each with different Internet Explorer versions saved as separate snapshots, down to a "Current State (changed)"—which is, in effect, a cheap do-it-yourself sandbox-with-cookbook: the analyst has created several snapshots and, in each, varied the installed software (different browser or Office versions) so that samples with different prerequisites can be exercised.

Configuring the Network in VirtualBox

Network isolation can be achieved at several granularities. The crudest is to **disconnect the virtual network adapter while the VM is running**—literally the small blinking network icon in the corner of the window, which toggles the interface and produces an air-gapped machine on demand. More robust is to configure the machine's networking mode itself. **Host-only networking** allows traffic to reach the host but **not the Internet**. Conceptually, the virtual machine's NIC connects to the host physical machine's NIC, while the host's second NIC—the one facing the external network—is left out of the guest's reach.



More complex setups use **ad-hoc networking to fake external services**. A typical arrangement has an **Analysis Virtual Machine**, where the sample runs, and a separate **Services Virtual Machine**, which provides fake network services, both connected to an internal virtual network (a **VMNet**) that never touches the Internet; the host physical machine keeps its own NIC facing the real external network, used only on demand. The services VM can be quite capable—offering **HTTP and even HTTPS** (with fake but realistic-enough SSL certificates to satisfy basic malware), **SMTP**, and **network time** servers—enough to make the malware believe it has Internet access, which in turn unlocks behaviors that would otherwise stay dormant.



Risks of Using a VM, and Preparing the Windows Machine

Three residual risks must be kept in mind when using a VM for malware analysis. **Malware may detect that it is in a VM and run differently. Virtualization environments may have bugs**, which in some cases malware may exploit to spread to or affect the host. And—reassuringly for a teaching setting—the samples analyzed in this course are **harmless**.

Because most malware encountered in practice is **Windows-based**, analysis is typically performed in a **Windows VM**—a Windows 10 VM in this course. Two preparation points matter. Some old malware **will not run without administrator privileges**, and the machine's **internal safety protections, such as Windows Defender, must be disabled**, otherwise samples are automatically quarantined or deleted; these protections are managed through the Group Policy editor, `gpedit.msc`. If using the course-provided VM, these services and group policies are already disabled for exactly this reason—but that same configuration makes the machine **very vulnerable and several years out of date**, so it must be used only for the course or for experimentation, never for real work or for entering sensitive information such as login credentials, which on such a machine would effectively be given away. The disabling of security patches is deliberate: active protections would interfere with analysis, and the old configuration also preserves backward compatibility with textbook exercises.

Launching the Sample

Dynamic analysis means running the thing—but what "running" entails depends on the form the malware takes. An executable is straightforward: you click it and it starts. In reality, however, **much malware is not an executable but a DLL—a library**. DLL malware is usually not **first-stage** malware (the component that initially infects the machine) but **second-stage**: something dropped onto the machine by the operation of the first stage. A typical chain begins with an executable or an exploit for a document reader—a PDF viewer or Microsoft Word—which then drops a DLL onto the disk; some other

process is later forced, through techniques known as **injection**, to load that DLL, at which point the payload activates.

To analyze such a DLL in isolation, rather than re-infecting the machine and catching it in context, the examiner needs a support environment that loads and runs the library. **EXE files can be run directly, but DLLs cannot.** The standard mechanism is the Windows tool **`rundll32.exe`**, invoked as:

```
rundll32.exe DLLname, Export arguments
```

The **Export** value is one of the functions the DLL exports—exactly the kind of name found earlier with tools such as Dependency Walker, PEview, or PE Explorer. The complication is that **many malicious DLLs do not have an export**, so `rundll32` has nothing to call. There is a workaround: one can supply a **non-existent export name**, and Windows will still load the DLL—but the analyst must be careful **not to click OK on the resulting warning dialog**, because doing so terminates the program before the malicious code can be observed. By tweaking the use of `rundll32` in this way, the DLL can be loaded and its malicious behavior observed even without a usable export; a dedicated later lecture covers this in full.

Process Monitor

Process Monitor (procmon) monitors registry, file system, network, process, and thread activity. It is extremely detailed: it **captures all processes, not just one**, which means it **keeps every recorded event**. Two operational cautions follow. Because it retains everything, it **consumes a great deal of memory**—running it for ten or fifteen minutes can exhaust all RAM and crash or badly lag the virtual machine—so it must be run only as long as needed. And because it records everything, the display must be filtered to find items of interest.

A simple experiment—launching `calc.exe` while procmon captures—makes the noise concrete. The log immediately fills with events that have nothing to do with any malicious intent: the calculator process issues `RegOpenKey`, `RegQueryValue`, and `RegCloseKey` operations against Windows error-reporting keys, `ReadFile` against `C:\Windows\System32\calc.exe`, and `CreateFile` / `QueryBasicInformation` / `Load Image` against `WindowsCodecs.dll`. This is exactly the `CreateFile`-means-open phenomenon in action: the calculator is not "creating" `WindowsCodecs.dll`, it is loading it. Some keys, such as those under `Software\Classes`, are routine and can be ignored; others genuinely matter. The analyst's task is to filter by event type.

The Process Monitor Toolbar

The procmon toolbar exposes the controls that make filtering workable. The key buttons are **Start/Stop Capture**, **Erase** (clear the current events), **Filter**, and a set of **default filter toggles** for the main categories—**Registry, File system, Network, Processes**—which can be switched on and off to show only the categories of interest.

[Photo: Process Monitor toolbar annotations] The toolbar, from left, exposes open/save, then the Start/Stop Capture control, the Erase (clear) control, and the Filter control, followed by the four category toggles (Registry, File system, Network, Processes); knowing these four controls is enough to drive the tool for basic analysis.

Filtering With Exclude

A powerful technique is to **hide normal activity before launching the malware**. Before detonating a sample, the analyst clears the log and right-clicks each benign **Process Name**, choosing **Exclude**, so that everything already running in the system is suppressed and only the sample's activity remains. The most useful filter dimensions are **Process Name, Operation, and Detail**. The Process Monitor Filter dialog lets the analyst specify conditions of the form "Process Name is `calc.exe` → Include," and comes pre-populated with sensible exclusions— `Procmon.exe` itself and the `System` process are excluded, operations beginning with `IRP_MJ_` and `FASTIO_` are excluded, results beginning with `FAST IO` are excluded, and paths ending with `pagefile.sys` are excluded. Filtering can be further refined to a single process, or a process and its children, to make the output genuinely manageable, and background activity that predates the sample can be excluded wholesale as another way of minimizing noise.

Process Explorer

Where procmon logs historical events, **Process Explorer** performs **real-time inspection of running processes**, showing what is running and the state of each. It uses a **color code** that conveys a process's nature and lifecycle at a glance: **services are pink** (long-running programs), **processes are blue** (with the currently selected process highlighted), **new processes flash green briefly** when created, and **terminating processes turn red** for a few seconds before disappearing.

Process Explorer also exposes per-process detail. It can show the **loaded DLLs** for a process—enabling this view (View → Lower Pane View → DLLs) lists every library a process has loaded, with full paths—and it can reveal whether an executable is legitimate, including whether it carries a valid **digital signature**.

Properties: DEP, ASLR, and the Limits of Verification

The **Properties** sheet for a process shows its **DEP and ASLR status** and offers a **Verify** button that checks the Windows signature of the file **on disk**. The crucial limitation is that Verify checks only the disk file, **not the memory image**, so it **will not detect process replacement**. Process replacement is a technique in which something is loaded in memory that looks like a legitimate Windows component: an attacker hollows out a process's context and either removes the original code and loads its own, or keeps the original code and loads additional code alongside it, so the process appears to be a legitimate system process while in fact being abused. Because Verify never inspects what is actually resident in memory, it is blind to this—so Process Explorer alone cannot catch it, and other means are required.

Strings: Image vs. Memory

One of those other means is the **Strings** tab of the Properties sheet, which can display the printable strings either from the on-disk **Image** or from the live **Memory** of the process. **Comparing Image strings to Memory strings** is diagnostic: if the two sets are very different, that discrepancy can **indicate process replacement**. The appearance of entirely new strings in memory that do not exist in the image is a sign that something has been tampered with—though not a certain one, since legitimate unpacking and runtime behavior also introduce new strings. It is an indicator to be weighed, not a proof.

Detecting Malicious Documents

The same tooling detects malicious documents. The procedure is to **open the document—for example a PDF—on a system running a vulnerable application**, such as an old version of Adobe Reader. The analyst then **watches Process Explorer to see whether the document launches a process**: a malicious document exploits a vulnerability in the viewing application to start a **second-stage payload**, which may appear as another executable or as a library being loaded into the victim

process. When such a process appears, the **Image tab of its Properties sheet shows where the malware is** on disk. Beyond Process Explorer, a comparable tool called **Process Hacker** offers the same kind of visibility, and these tools come into their own when applied to real cases—such as detecting DLL-injection scenarios, which later hands-on sessions cover in depth.

The Windows Registry

The registry is best understood as a **huge configuration file**—a database in which Windows stores everything about the configuration of the system and of applications. It is the **repository for configuration and control of Windows systems**, and it holds two broad classes of setting. **System-wide** settings govern the whole machine and all its users—which device drivers to load, how to configure the memory manager and process manager, and so on—and applications read these system-wide settings. **Per-user settings** hold per-user preferences and records such as the most-recently accessed documents.

Malware cares intensely about the registry, for two main reasons. The first is **persistence**: through the registry a sample can arrange to be restarted at every boot, and some malware goes further, saving its own data—or even its **entire payload**—inside the registry. The second is **environmental reconnaissance**: the sample reads the registry to decide whether the machine it has landed on is the "right" one, checking installed applications, specific values such as the keyboard layout, how long the machine has been in use, the installation date, or the history of installed applications. The registry is one convenient place to inspect all of this.

Structure: Keys and Values

A **registry key is a container consisting of other keys (subkeys) or values**—structurally much like a file-system path, with further keys and name/value pairs nested inside it. A **registry value stores data whose type can be `REG_SZ` (string), `REG_DWORD`, `REG_BINARY`, and others** such as `REG_QWORD`. Thus a key is analogous to a directory path, and inside it live pairs of names and typed values (strings, numbers, boolean 0/1 values, DWORDs, QWORDs).

The Root Keys

The registry is organized under a handful of **root keys**, each storing a particular class of information, and some of which are **links** to locations under other root keys.

Root Key	Stored Information	Link
`HKEY_CLASSES_ROOT` (HKCR)	File association and Component Object Model (COM) object registration (e.g. ProgID and CLSID)	Merged
`HKEY_CURRENT_USER` (HKCU)	Data associated with the currently logged-on user	Yes
`HKEY_LOCAL_MACHINE` (HKLM)	Global settings for the machine	No
`HKEY_USERS` (HKU)	All the accounts on the machine	No
`HKEY_CURRENT_CONFIG` (HKCC)	Current hardware profile	Yes

The two roots encountered most often in malware analysis are **`HKEY_CURRENT_USER` and `HKEY_LOCAL_MACHINE`**. `HKCU` is the one malware touches most, because **modifying it requires no privileges**: operating on the current user's part of the registry needs no administrator rights, and there

the malware is free to open, modify, create, and delete. `HKLM`, by contrast, is generally readable but requires **administrator rights** to modify. For malware to write to `HKLM`, either it is already running as administrator, or it must escalate privilege to become administrator—which is exactly the kind of sensitive operation that triggers Windows' **User Account Control (UAC)** prompt asking whether to authorize the action.

Links and Hives

Several root keys are implemented as **`REG_LINK` links** to canonical locations. `HKEY_CURRENT_USER` is a link to `HKEY_USERS\<SID>`, the Security ID of the current user. `HKEY_CURRENT_CONFIG` is a link to `HKLM\SYSTEM\CurrentControlSet\Hardware Profiles\Current`. And `HKLM\SYSTEM\CurrentControlSet` is itself a link to `HKLM\SYSTEM\ControlSet00X`, where `X` is a number.

A **registry hive** is a "logical group of keys, subkeys and values in the registry that has a set of supporting files containing backups of its data"—for example, `HKLM\SAM` is backed by the file `C:\windows\system32\config\SAM`. Alternatively, a hive may be **constructed dynamically in memory** rather than persisted to a supporting file.

Regshot

Regshot is the diffing tool for the registry. It takes a snapshot of the registry, allows a second snapshot to be taken later, and reports what changed—keys added, values added, and values modified—so the analyst can snapshot before running a sample, snapshot again afterward, and read off the differences. Its typical output lists, for instance, "Values added: `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\ckr: C:\WINDOWS\system32\ckr.exe`" alongside "Values modified" and a total change count. It is an open-source tool with a wealth of plugins.

Persistence

Persistence is the set of techniques a malware uses to **survive after a reboot**. The main avenues are the **registry key**, the **file system** (startup locations, DLL search-order hijacking, and trojanizing system files), the **Master Boot Record (MBR)**, and the **Basic Input/Output System (BIOS)**. In practice the first two—registry-based and file-system-based persistence—are the ones most commonly studied, and they are developed below; MBR and BIOS persistence operate at lower levels.

Persistence Through Registry Keys

A table of **frequently used registry keys** separates those requiring administrator privileges from those that do not. (Neither list is comprehensive, and the keys listed are not necessarily more important than others omitted.)

Administrator privilege is required to update `HKLM`:

```
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\"Shell" and "Userinit"
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows\"Appinit_Dlls"
HKLM\System\CurrentControlSet\Control\Session Manager\KnownDlls
HKLM\System\CurrentControlSet\Services
HKLM\Software\Microsoft\Windows NT\CurrentVersion\Image File Execution Options
HKLM\Software\Microsoft\Windows\CurrentVersion\Explorer\Browser Helper Objects
```

Without administrator privileges, malware can persist with these keys:

```
HKCU\Software\Microsoft\Windows\CurrentVersion\Run
HKCU\Software\Policies\Microsoft\Windows\System\Scripts\Logon
HKCU\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\Shell
```

The `...\CurrentVersion\Run` and `Run Once` keys are **auto-start paths**: anything placed there runs at every startup of the machine (`Run` on every boot, `Run Once` a single time). `Appinit_Dlls` and related entries force applications to load malicious DLLs; the `Services` key registers services; the `Browser Helper Objects` key is a target for malicious browser extensions exploiting browser vulnerabilities. The `HKCU` variants—`Run`, `Run Once`, and the logon/Winlogon paths—are the ones usable **without administrator rights** and are the relevant ones for low-privilege persistence. Many other keys are just as relevant for configuration, locale, hardware, and the list of devices that have been plugged into the machine, giving access to a great deal of system information.

Persistence Through the File System

The simplest file-system technique uses **startup locations**—folders whose contents Windows launches automatically at logon. There are two, corresponding to the per-user and all-users scopes:

```
For the logged-in user:  %USERPROFILE%\Start Menu\Programs\Startup
For all users:           %ALLUSERSPROFILE%\Start Menu\Programs\Startup
```

Placing an item in the per-user folder causes it to start automatically when that user logs in; the all-users folder applies to every user of the machine. These paths are expressed through shell environment variables, and both resolve to concrete directories on the Windows VM under the user's and the all-users' profile.

Windows Services

The other major persistence avenue is the **Windows service**. A service is a **long-running executable that runs without user interaction**, much like a Unix daemon: it runs for the entire power-cycle of the machine—from boot until shutdown or explicit termination—and typically provides auxiliary features that applications need or performs periodic tasks (such as Windows Update checking for updates). Services **can be automatically started when the computer boots**. The Windows API `CreateService()` registers a service, with companion APIs to query, stop, start, and restart it—a self-contained Windows feature, where Linux achieves the same end with different software. Registered services are found under the registry key `HKLM\System\CurrentControlSet\Services`.

Services are abusable because they **can run DLLs**: an attacker can create a malicious DLL and have a service run it, achieving yet another form of persistence either by tampering with the registry to register the DLL or by having malware take advantage of the service infrastructure.

svchost

The generic host for DLL-based services is `svchost.exe`. `C:\Windows\System32\svchost.exe` is a **generic host process for services that run from DLLs**, and **multiple instances are often running**, with **one instance containing a group of services**. The service groups are listed in the registry key `HKLM\Software\Microsoft\Windows NT\CurrentVersion\Svchost`.

`svchost` is a classic cover for malware, in several ways. It is **common for malware to name itself `svchost.exe` but run from somewhere other than `C:\Windows\System32`**—for example from `C:\Windows`. Because a healthy system already shows many legitimate `svchost` instances, spotting one more by name alone is difficult; the giveaway is usually a wrong path or a different file size. Alternatively—and more stealthily—malware **simply registers a new DLL for the real `svchost` to run as a service**, or creates a normal `svchost` with no DLL and then manipulates that process to plant its code inside it. In short, `svchost` can be made to load a malicious DLL on demand—spawning a process that runs `svchost` with the malicious DLL as an argument—giving the attacker another persistence mechanism. For all these reasons, `svchost` is a recurring name to scrutinize carefully in malware analysis.

Network Analysis

The final category of behavior is network activity, captured with dedicated packet-capture tools. There is little new theory here—the value is in the tooling and in the fake-network techniques that make a disconnected sample communicate.

Packet Sniffing With Wireshark

Wireshark captures and inspects the packets a sample sends and receives on the network interface. A live capture shows each packet's time, source, destination, protocol, and summary information, with a filter bar (for example, filtering on `http`) to narrow the view, a packet-detail pane breaking each frame down by layer (Frame, Ethernet, IP, and up), and a raw hex/ASCII pane at the bottom. Because every interaction a sample has with the network passes through such a capture, Wireshark is the workhorse for the network-activity category. One particularly useful application is inspecting **DNS queries**, which are worth watching closely because—as later material shows—even a DNS query can be abused to **exfiltrate information**.

Follow TCP Stream

Wireshark's **Follow TCP Stream** feature reassembles a full conversation from its individual packets, presenting the complete request/response exchange as readable text—for example an HTTP `GET` with its headers followed by the server's `HTTP/1.1 200 OK` response and body. Beyond reading the conversation, the analyst **can save files directly from streams here too** (via Save As), extracting, say, an image or executable that was transferred over the connection.

Talking to the Malware Directly: ncat and FakeNet

Network tools do more than observe—they let the analyst **play the role of the remote server**. Sometimes a sample opens a socket and waits for an inbound connection; the analyst can then connect to it and send bytes to see how it responds. Other times the analyst wants to intercept the sample's outbound communication and supply crafted responses. The tool for this is **ncat** (included with **nmap**), which can be combined with **FakeNet**. A representative session runs `ncat -l 80` to listen on port 80; when a browser or the malware issues an HTTP request to the local address, ncat receives and displays the raw request (`GET / HTTP/1.1`, `Host:`, the `User-Agent`, and so on), letting the analyst see exactly what the sample sends and reply as a fake server would. This simulates being the malware's command-and-control endpoint closely enough to elicit and inspect its behavior.

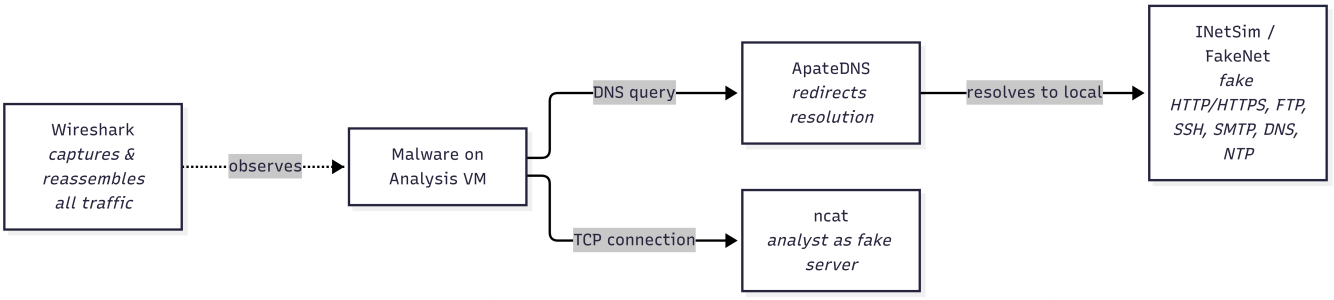
Redirecting DNS With ApateDNS

To make a sample reach a fake server in the first place, its name resolution must be redirected. **ApateDNS** intercepts DNS requests and returns answers the analyst controls, so that whatever domain the malware looks up resolves to a chosen address—typically the local analysis machine. Its capture window lists each domain requested and whether a response was returned ("FOUND"), while a status pane shows the server setting DNS to a loopback or local address and "Sending valid DNS response of first request." The analyst sets the **DNS Reply IP** (defaulting to the current gateway/DNS) and a count of **NXDOMAIN** responses, selects the interface, and starts the server. With DNS pointed at the analysis environment, the malware's requests land on the local fake services instead of the real Internet.

INetSim

The most capable simulator is **INetSim**, typically run on a small Linux distribution (Kali is common, but minimal distributions of only two to three hundred megabytes suffice). INetSim stands up a convincing set of fake network services, so the analysis VM is configured to use the simulator's machine as its DNS server and network provider; done properly, the simulator machine can also host a DHCP server so that DNS and everything else are configured automatically on the analysis VM. A representative setup has a Kali machine running INetSim as the services provider and a Windows attacker/analysis machine whose TCP/IPv4 settings point its **Preferred DNS server** at the simulator's address.

INetSim is convincing enough to deceive real client software. Pointed at it, a browser requesting an arbitrary domain such as `fred.com` receives INetSim's default page—"This is the default HTML page for INetSim HTTP server fake mode"—so the browser is **fooled** into believing it reached a live site. It even deceives a port scanner: running an intense `nmap` scan against `fred.com` reports a long list of open ports (21, 53, 113, 80, 443, 25, 110, 995, 990, and many more), because INetSim answers on all of them, **fooling nmap** into seeing a fully populated host. The services it can present are broad—**HTTP and HTTPS** (with fake but realistic-enough SSL certificates), **FTP, SSH, SMTP**, network time, and more—which is enough to make malware believe it has genuine Internet access and to deceive even classic port scanners.



Using the Tools Together

The tools of basic dynamic analysis form a coherent kit, each covering part of the behavioral picture. **Procmon** logs system activity—with the discipline of filtering on the malware's executable name and clearing all events immediately before running it. **Process Explorer** gives real-time process inspection. **Regshot** captures registry changes by diffing. A **virtual network with ApateDNS and INetSim** supplies fake connectivity so network behavior manifests. And **Wireshark** records and reconstructs the traffic. Used in combination—a sandbox or Regshot for a fast first characterization, procmon and Process Explorer for live system behavior, and the ApateDNS/INetSim/Wireshark stack for network behavior—these tools let an analyst observe essentially the full range of a sample's interactions with the system and the network, while the environmental precautions of earlier sections keep that observation both safe and faithful.

CHAPTER 5: Malware Evasion

1. The Evasion Problem

For malicious software, remaining undetected for as long as possible is not a luxury but a survival requirement. A sample that is recognized quickly is a sample that fails: antivirus engines carry signatures that match known malicious code, and network-level detection mechanisms can flag and block a suspicious executable in transit or at runtime. Every hour that a sample stays unrecognized is an hour in which it can achieve its objective, whether that is exfiltrating data, encrypting files for ransom, or establishing persistence. From the author's perspective, therefore, the entire lifecycle of the malware is designed around delaying recognition, and a central part of that effort is directed at the very systems built to study it.

To appreciate why evasion exists, one must first understand how malware is analyzed, because evasion is defined precisely in opposition to those analysis techniques. Analysis proceeds along two complementary avenues — static analysis and dynamic analysis — which are not mutually exclusive but reinforce each other.

1.1 Static Analysis and Its Limits

Static analysis examines a sample without executing it. It is extremely attractive from the standpoint of computational efficiency: the executable is passed through a pipeline that inspects its strings, its imported functions, and other structural features that can hint at malicious intent, often without any need to reason about the code in fine detail. A revealing import table or a suspicious set of embedded strings can already raise a strong alarm.

The fundamental limitation is that **not everything about a program is visible statically**, and what is visible is often expensive to extract without yielding a precise verdict. Static techniques can suggest that a sample is *likely* suspicious, but they rarely establish malice with the certainty required to block execution outright. This uncertainty has a serious practical cost: blocking on weak static evidence risks denying legitimate software. Many benign applications deliberately employ obfuscation to protect intellectual property, and these legitimate programs can look, under a static lens, much like malware trying to hide. A vendor whose signed, legitimate product is blocked will — with justification — complain that the antivirus is broken. Static analysis thus takes the analyst only so far; beyond that point, dynamic analysis becomes necessary.

1.2 Dynamic Analysis and Externally Observable Behavior

Dynamic analysis requires actually running the sample and observing what it does. The object of study here is the sample's **externally observable behavior** — an expression that must be understood precisely. It does *not* mean inspecting, in detail or at all, how the internal functions of the sample call one another. Instead, it means watching how the sample interacts with the surrounding system: the API calls it issues, the files it creates, the registry keys it reads and writes, the network connections it opens. From these interactions an analyst builds a behavioral profile constructed entirely from what is visible from the outside.

Only after a sample has been profiled at this behavioral level does the question arise of whether it warrants deeper, manual analysis — the painstaking, instruction-level dissection performed by a human expert. Very few samples actually reach this stage, reserved for cases that genuinely merit it, such as a newly emerged malware family or a strain that looks meaningfully novel.

The reason automated dynamic analysis must carry the bulk of the load is sheer volume. An enormous number of new samples surface every single day, because malware authors can generate as much malware as they wish. Even before the current era of AI-assisted tooling, this was possible through techniques that **diversify the implementation** of a sample: one can produce many distinct instances of what is functionally the same code, such that it is not easy to tell they are the same malware at all. Faced with this flood, defenders need to analyze quickly — not as quickly as static analysis, but fast enough to keep pace. Dynamic analysis is the compromise, and it is exactly this dynamic, running-the-sample step that evasion attacks.

1.3 Evasion and the Notion of an Artifact

When a sample is executed for analysis, that sample may actively attempt to impede the analysis. The classic term for this behavior is **evasion**: the sample tries to fool whatever analysis system is attempting to observe it. To evade, however, the sample must first *detect* that it is being analyzed — it must distinguish an analysis environment from an ordinary laptop or workstation.

The possibility of such detection rests on a principle that is best captured by analogy with the **Heisenberg effect** in physics: the act of observing a system inevitably changes that system. When a sample runs inside a sandbox or any other analysis environment, something about that environment differs, however subtly, from execution on an ordinary machine. Any such difference is called an **artifact**. An artifact is like a faint blur in a photograph — a small, telltale sign that betrays the presence of an analysis environment. Malware can react to artifacts, and much of it does: not all malware checks whether it is being observed, but most of it makes some attempt to do so, and sophisticated samples make many such attempts.

This sets up the defining dynamic of the field, a genuine **cat-and-mouse game**. Defenders harden their analysis systems against the artifacts they know about; attackers probe for the ones that remain. The decisive asymmetry is that a defender must cover every possible discrepancy, closing all of them, whereas an attacker needs to find only a single unprotected feature — one weak link — to detect the analysis environment and escape. Understanding evasion therefore requires knowing, in the attacker's own terms, exactly which artifacts exist and how each one is probed. As with good defense in general, one must know the enemy.

2. What Distinguishes an Analysis System

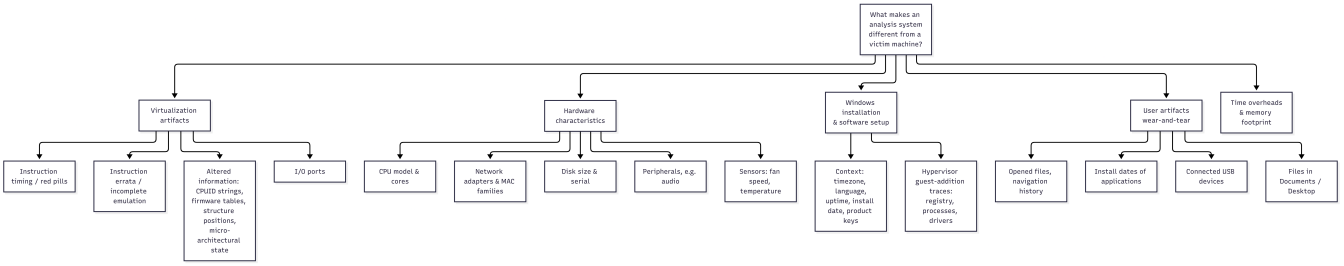
Before cataloguing specific evasion techniques, it is worth asking what, fundamentally, makes an analysis system — a sandbox, an antivirus dynamic engine, or similar — different from an ordinary victim machine. The answer begins with a structural requirement that every such system shares.

An analysis system must start each sample from a **clean state**, **restore** that state once execution finishes, and **contain** the effects of execution so that a running malicious sample cannot cause lasting damage or escape. The most effective and practical way to satisfy all three requirements at once is to use a **virtual machine**. Virtualization provides exactly the isolation, snapshotting, and reset capabilities that clean-state analysis demands.

Herein lies the first and most heavily studied source of artifacts. Although virtualization is remarkably performance-efficient — it was engineered at the hardware level specifically to minimize the performance penalty, which is why cloud infrastructure relies on it so pervasively — it was **not designed for security**. Its purpose was efficient resource sharing, not perfect indistinguishability from bare metal. As a consequence, a virtualized environment inevitably leaves artifacts that betray the presence of

virtualization. A malware author's natural first question is therefore whether the code is running inside a virtual machine at all.

Detecting virtualization, however, is only the beginning. Beyond it lie several further dimensions along which an analysis system can be distinguished from a genuine target, and the sophisticated sample will exploit whichever proves weakest.



The reason these hardware, installation, and usage features matter so broadly is that they are directly relevant to **evasive malware** — malware whose only concern is to escape analysis and whose detection of a virtualized or atypical environment is enough to make it withhold its payload. There is, however, an important subtlety that modern conditions impose. Virtualization is now ubiquitous in legitimate settings; even recent versions of Windows employ virtualization by default to protect the machine. Consequently, merely detecting virtualization no longer, by itself, proves that a sample is inside a sandbox — it may simply be running in the cloud or on a personal machine that happens to be virtualized. A careful modern author treats the detection of virtualization as grounds for *suspicion* that prompts further checks, rather than as a conclusive signal, and adjusts the detection strategy accordingly.

A second category of adversary must also be distinguished: **targeted malware**. Where evasive malware wants only to avoid analysis systems, targeted malware additionally wants to avoid the *wrong victims*. The same features that reveal a hypervisor can reveal whether the current physical machine is the intended target at all — whether, for example, the CPU model, installed software, or organizational fingerprint matches the specific victim the campaign is after. Targeted malware has every incentive to stay dormant on anything other than its intended target, because activating on a non-victim achieves nothing and only risks exposure: a wrong victim may be the one running a detector, and a security provider will then obtain signatures to detect the campaign. Targeted malware thus hides even more aggressively than purely evasive malware, remaining under the radar for as long as possible.

With this framework in place — artifacts as the fundamental enabler, virtualization as the richest source of them, and the evasive-versus-targeted distinction shaping how aggressively a sample reacts — the remainder of this chapter examines each dimension in turn, beginning with the virtualization artifacts that the literature has studied since the late 2000s.

3. Virtualization Artifacts: Red Pills

The most thoroughly studied dimension of evasion is the detection of virtualization itself, a line of research that reaches back to the late 2000s, with foundational papers appearing around 2008 and even earlier. The goal of such detection is narrow and precise: to determine whether code is executing inside a virtual machine or natively on physical hardware. Detecting a virtual machine does not, on its own, prove that the environment is a sandbox — but if the environment *is* a sandbox, it is almost certainly virtualized, which makes virtualization detection the natural first probe.

3.1 The Red Pill Concept

The terminology comes from the *Matrix* films, in which the red pill reveals reality while the blue pill sustains a comforting illusion. Researchers adopted this naming with deliberate cleverness. A **red pill** is

a sequence of instructions capable of detecting whether the executing code is running inside a virtual machine — a short routine that pierces the illusion of bare-metal execution. (By symmetry, as will be seen later, a *blue pill* is the defender's counterpart: a mechanism that sustains the illusion of a genuine machine so that the sample never realizes it is being watched.)

A red pill can take one or more of several distinct forms, and it is worth enumerating them before examining the most important in detail. First, it can exploit **instruction timing**: certain instructions take measurably longer to execute inside a VM. Second, it can exploit **instruction errata** — incomplete or incorrect emulation of the instruction set. Third, it can read **information** that is altered under virtualization, such as CPU identification data, firmware tables, the contents and memory positions of specific structures, or the micro-architectural state of the machine. Fourth, it can probe **I/O ports** whose behavior differs under a hypervisor.

3.2 Why Some Instructions Take Longer Inside a VM

The timing red pill rests on a structural fact about how virtualization works. A virtual machine is designed so that the overwhelming majority of instructions run directly on the physical CPU without intervention — an addition or a subtraction cannot corrupt the global system state, so there is no reason to intercept it, and letting it run natively is what keeps virtualization fast. The situation changes when the guest executes a **privileged** operation or queries information about the system. In those cases a component of the hypervisor, the **Virtual Machine Monitor (VMM)**, must step in to supervise and, where necessary, rewrite the interaction. This transfer of control from guest to VMM is a **VM exit**: the guest is paused, the VMM performs work, and control is eventually returned.

The canonical example is a request for CPU information. When the guest asks for processor identification, the hypervisor intervenes to adjust the returned data — for instance, to signal to the operating system that it is in fact running under a hypervisor. Performing this interception costs the VMM real work, and that work manifests as **extra execution time** for the triggering instruction. A sample can measure this overhead and detect the hypervisor very reliably.

3.3 Instruction Errata and Incomplete Emulation

A second avenue arises when virtualization does not rely on hardware-assisted extensions. Modern CPUs provide such extensions — Intel's are called **VT-x** — but in the past, and still today in cross-architecture settings, virtualization was achieved through **emulation**. Running a course virtual machine on an ARM-based MacBook, for example, entails emulating an x86 CPU, and emulation is where correctness gaps appear. Certain features may simply be unavailable or incorrectly reproduced: hardware breakpoints that trigger on memory accesses, for instance, have been found unemulated by some emulators, and a sample that depends on such a feature will immediately notice its absence.

This is the principle behind the WOOT 2009 work *A Fistful of Red-Pills*, which automatically generated procedures to detect CPU emulators by finding instructions that were not emulated with full fidelity. Such discrepancies work fine for ordinary programs — which is why they go unnoticed in general use — but malware deliberately seeks out exactly these corner cases. The mindset is identical to vulnerability discovery: one does not probe the ordinary paths that everyone exercises, but the overlooked edges that implementers never thought about.

3.4 Altered Information and Micro-Architectural State

The hypervisor's interception of CPU queries not only costs time; it also changes the *content* of what is returned. Instead of reporting the true CPU model or the genuine motherboard information, a virtualized

query yields hypervisor-specific strings. Beyond CPU identification, other informational sources are perturbed under virtualization: firmware tables such as SMBIOS strings and ACPI tables carry telltale values, and certain data structures occupy unusual positions in memory, with the Interrupt Descriptor Table being a classic example whose location differs under virtualization.

The subtlest source of all is the **micro-architectural state** of the machine, an area in which very few have demonstrated practical attacks against malware analysis, though its existence is well established. The distinction at work is that between the instruction set architecture — the abstract contract telling software what operations it may perform — and the concrete *implementation* of that architecture by a given CPU. Assembly language has no notion of caches, TLBs, or load ports; yet these structures are real components of the processor's implementation and carry observable state. The insight, demonstrated among others by research groups working in this space, is that observing malware through virtualization tricks necessarily disturbs this hidden-but-present state. Because analysis code runs in the VMM alongside the sample, it alters, for example, cache utilization in ways that a genuine system would not. A sophisticated attacker can mount a **side-channel attack** that reconstructs cache utilization and, from the anomaly, infers the presence of the monitoring VMM — a difficult but demonstrated technique. This line of work is explored in the *My Ticks Don't Lie* research on timing-based hypervisor detection.

3.5 Example: CPUID

The `cpuid` instruction is the workhorse of value-based virtualization detection, and it is simple to use. It retrieves processor identification and feature information. The caller first loads a value into register `EAX` to select which record is requested, then executes `cpuid`; the instruction writes its results, all at once, into the four registers `EAX`, `EBX`, `ECX`, and `EDX`.

Two request types matter for evasion. Setting `EAX = 0x1` asks for feature information. Among the results written to `ECX`, the **31st bit** is set to 1 when the code is running under a hypervisor — this is the *hypervisor-present bit*. Setting `EAX = 0x40000000` instead asks for hypervisor brand information, which is written into `EBX`, `ECX`, and `EDX` as ASCII bytes forming 12-byte strings. The characteristic brand strings include:

```
"KVMKVMKVM\0\0\0"   (KVM)
"Microsoft Hv"       (Hyper-V)
"VMwareVMware"       (VMware)
"XenVMMXenVMM"       (Xen)
"prl hyperv  "       (Parallels)
"VBoxVBoxVBox"       (VirtualBox)
```

The hypervisor bit

To test for virtualization, one sets `EAX = 1`, executes `cpuid`, and inspects bit 31 of `ECX`. In GCC/Clang inline assembly (AT&T syntax):

```
size_t eax, ebx, ecx, edx;
__asm__ volatile (
    "cpuid"
    : "=a"(eax), "=b"(ebx), "=c"(ecx), "=d"(edx)    /* outputs */
    : "a"(1)                                         /* input: EAX = 1 */
```

```
    : "memory" );                                   /* clobber */
printf("%d\n", (int)(ecx >> 31));
```

Reading this inline-assembly block requires understanding its four parts, in order: the instruction (`cpuid`); the **outputs**, which bind the contents of `EAX` , `EBX` , `ECX` , `EDX` to the C variables `eax` , `ebx` , `ecx` , `edx` ; the **input**, `"a"(1)` , which places the value 1 into `EAX` before execution; and the **clobber** list `"memory"` , which merely informs the compiler of side effects and can otherwise be ignored. The result of interest is the 31st bit of `ECX` . It is extracted by shifting `ECX` right by 31 positions, which moves that single bit into the lowest position while prepending zeros; the `printf` therefore prints `0` or `1` . A value of `1` means a hypervisor is present. Compiled and run inside a virtual machine, this program prints `1` .

This same code reveals the defender's obligation. A sandbox designer who virtualizes an analysis environment must modify the hypervisor so that, when the VMM services the `cpuid` instruction, it **patches the output** — writing `0` into the hypervisor bit — or the sandbox will be trivially detected. Value-based red pills of this kind are, in that sense, fixable: the VMM controls the answer and can lie convincingly.

The hypervisor brand

Detecting the brand string proceeds analogously but reads the `EAX = 0x40000000` leaf. The following MSVC example uses Intel-syntax inline assembly:

```
#include "stdafx.h"
#include <iostream>
#include <string>
using namespace std;

int main()
{
    bool IsUnderVM = false;
    __asm {
        xor   eax, eax
        mov   eax, 0x40000000
        cpuid
        cmp   ecx, 0x4D566572
        jne   NopInstr
        cmp   edx, 0x65726177
        jne   NopInstr
        mov   IsUnderVM, 0x1
    NopInstr:
        nop
    }
    cout << IsUnderVM;   // prints 1 if "VMwareVMware" was detected
    return 0;
}
```

The logic loads `0x40000000` into `EAX` and executes `cpuid` , after which the 12-byte brand string is spread across `EBX` , `ECX` , and `EDX` . The code then compares only **two** registers, `ECX` and `EDX` , rather

than all three. This shortcut is sound because brand strings such as `"VMwareVMware"` are built from a repeating token — the six characters `VMware` appear twice — so examining eight of the twelve bytes already identifies the hypervisor uniquely. The hex constants are the little-endian ASCII encodings of the string's middle and tail: `0x4D566572` encodes the bytes that read as `reVM`, and `0x65726177` encodes `ware`, which together with the `VMwa` held in `EBX` reconstruct `VMwareVMware`. If both comparisons match, `IsUnderVM` is set to 1; if either fails, control jumps to `NopInstr`, `IsUnderVM` remains 0, and the program prints `0`. (The leading `xor eax, eax` is, strictly speaking, unnecessary here, since the following `mov` overwrites `EAX` immediately — a small sloppiness that does not affect correctness.)

3.6 Timing CPUID: Execution Latency as a Red Pill

The informational red pills above read *what* `cpuid` returns; the timing red pill measures *how long* it takes. As established, `cpuid` causes a VM exit under a hypervisor, handing control to the VMM, and that detour costs cycles. The challenge is that this overhead is far too small to measure with ordinary clocks. The CPU operates at gigahertz frequencies and completes an enormous number of operations per unit of time; conventional timers lack the resolution to see a single instruction's cost.

The solution is the `rdtsc` instruction, which reads the **Time Stamp Counter (TSC)** — a hardware-provided counter that measures CPU cycles rather than wall-clock time. Despite its name, the TSC is not a time value in seconds; it is a CPU-dependent cycle count. Crucially, on processors of roughly the last decade the TSC is **invariant**: it advances at a constant rate independent of the core's current frequency, so that dynamic frequency scaling — for power saving or turbo boosting — does not change the pace at which the counter ticks. It is not a clock of time, but it is a reliable clock of cycles.

The baseline here is empirical. A native `cpuid` costs on the order of 80–100 cycles, and because the CPU executes other instructions in parallel and out of order, the measured difference between two TSC readings around it typically lands somewhere between roughly 60 and 130 cycles. Under a hypervisor, the same measurement rises to around 500, 1000, or more — **an order of magnitude greater**. That gap is easily measured.

The following GCC example (compiled with Intel syntax, `-masm=intel`) implements the measurement:

```
int elapsed; // gcc -masm=intel
__asm__ volatile (
    "rdtsc ;"
    "mov %0, %%eax ;"
    "mov %%eax, 1;"
    "cpuid ;"
    "rdtsc ; "
    "sub %0, %%eax ;"
    "neg %0 ;"
    : "=r"(elapsed) :
    : "rax", "rbx", "rcx", "rdx");
```

Here `%0` is not a value but a placeholder instructing the assembler to allocate a temporary location (a register or stack slot, at the compiler's discretion) bound to the C variable `elapsed`. The difficulty the code navigates is that **both** `rdtsc` and `cpuid` overwrite register `EAX`, so the first timestamp must be saved before `cpuid` destroys it. The sequence therefore proceeds: execute `rdtsc` to read the current counter (`rdtsc` writes the TSC into `EDX:EAX`); save that starting value out of `EAX` into `%0`; load `1`

into `EAX` to request feature information; execute `cpuid`, which is the instruction being timed; execute `rdtsc` again to read the ending counter into `EAX`.

At this point `%0` holds the start and `EAX` holds the end. Rather than computing `end - start` directly, the code computes `start - end` with `sub %0, %%eax` and then negates the result with `neg %0`, which yields `end - start` more straightforwardly. Note that although the TSC is 64 bits wide, a short interval's difference fits comfortably in 32 bits, so inspecting only the low half in `EAX` suffices. If `elapsed` exceeds a threshold of around 500, the code concludes it is running inside a virtual machine. Because individual measurements fluctuate, such checks are normally repeated several times and averaged.

3.7 Al-Khaser: CPUID Latency in Practice

The averaging approach is exactly what the open-source **Al-Khaser** project implements. Al-Khaser is a collection of a great many evasion tricks, written in portable C using intrinsics, and it is an excellent resource for studying how evasions are built in practice — though the quality is uneven, some techniques being cleaner than others. Its `cpuid`-latency check reads as follows:

```
/*
CPUID is an instruction which cauz a VM Exit to the VMM,
this little overhead can show the presence of a hypervisor
*/
BOOL rdtsc_diff_vmexit()
{
    ULONGLONG tsc1 = 0;
    ULONGLONG tsc2 = 0;
    ULONGLONG avg  = 0;
    INT cpuInfo[4] = {};

    // Try this 10 times in case of small fluctuations
    for (INT i = 0; i < 10; i++)
    {
        tsc1 = __rdtsc();
        __cpuid(cpuInfo, 0);
        tsc2 = __rdtsc();

        // Get the delta of the two RDTSC
        avg += (tsc2 - tsc1);
    }

    // We repeated the process 10 times so we make sure our check is as much
reliable as we can
    avg = avg / 10;
    return (avg < 1000 && avg > 0) ? FALSE : TRUE;
}
```

This version uses the `__rdtsc()` and `__cpuid()` intrinsics — convenience functions exposed through the Windows C runtime rather than raw inline assembly — but the logic mirrors the assembly above. It brackets a `cpuid` call between two TSC readings, accumulates the delta over ten iterations to smooth

out short-term fluctuations, and divides by ten to obtain an average. The verdict follows from the threshold: if the average lies strictly between 0 and 1000 cycles, the routine returns `FALSE`, meaning no hypervisor is detected; otherwise it returns `TRUE`, flagging virtualization. The lower bound `avg > 0` guards against a degenerate or negative result, which would itself indicate that something abnormal — such as time manipulation — is occurring.

Taken together, the `cpuid` family of red pills illustrates the two complementary strategies of virtualization detection: reading values the VMM must forge (the hypervisor bit and brand strings) and timing operations the VMM cannot service for free (the VM-exit latency). Both are reliable, both are well documented in tools like Al-Khaser, and both are, for a sandbox designer, fixable only by careful and complete intervention in the hypervisor — patching every forged value and masking every measurable overhead.

4. Hardware Characteristics

Beyond the question of whether a machine is virtualized at all, a sample can inspect the machine's **hardware characteristics**, because certain hardware details betray a VM — or, for targeted malware, betray that the machine is not the intended victim. These checks are valuable precisely because sandboxes are often resource-constrained or generically configured, and because a given campaign may expect a very specific hardware profile at its target organization.

Several hardware features are commonly inspected. The **CPU model and core count** form a classic consistency check. When provisioning an analysis VM, operators tend to be greedy with resources: to keep the host responsive while running many VMs, each guest is often given only one or two cores. Yet the reported CPU model may be a high-end Intel part that genuinely ships with twelve or sixteen cores. A sample that reads both facts sees an inconsistency — a flagship processor exposing only two cores — that a real machine would not present. The **number of network adapters** and the **MAC address family** are similarly revealing. A single network adapter can be unusual on hardware that would normally carry both a wired port and a Wi-Fi card, and, more decisively, hypervisors emulate network devices that are assigned MAC addresses drawn from vendor-specific ranges typical of virtualization products. **Disk size and serial number** offer another signal: a drive reported as, say, 50 GB makes little sense on a modern machine, where such a capacity is implausibly small. More sophisticated checks examine **peripherals** such as the audio card — whether it is a simple device of the kind a hypervisor exposes or a richer modern one — and even **sensor measurements** like CPU temperature and fan speed. In a hypervisor these sensors are frequently either unavailable or emulated with fixed values, so that whether the CPU is idle or under heavy load, the sample reads the same unchanging fan speed and temperature.

All of these features are clearly relevant to **evasive** malware, since they build confidence that the environment is virtualized. As noted earlier, however, the mere detection of virtualization is no longer conclusive on its own, given how widely legitimate systems now use it. Their greater discriminating power often lies in service of **targeted** malware: these same properties can reveal not just a hypervisor but whether the current physical machine matches what is expected at the organization under attack. A campaign against a specific laboratory, for instance, might decline to activate on a machine whose CPU model is a year or two older than the equipment that organization is known to run — because those machines are simply not its intended victims, and firing on the wrong host would risk exposure for no gain.

4.1 Example: MAC Addresses

Because emulated network adapters receive identifiers from characteristic vendor ranges, the leading bytes of a MAC address can name the hypervisor directly. The families typically checked include:

```
"\x08\x00\x27"   (VirtualBox)
"\x00\x05\x69"   (VMware)
"\x00\x0C\x29"   (VMware)
"\x00\x1C\x14"   (VMware)
"\x00\x50\x56"   (VMware)
"\x00\x1C\x42"   (Parallels)
"\x00\x16\x3E"   (Xen)
```

A sample enumerates the machine's adapters, reads each MAC address, and compares its prefix against this list; a match identifies the virtualization platform. Al-Khaser contains ready-made code for exactly this check.

4.2 The Defender's Counter: Blue Pills

A natural question follows from all of this: if a sample can read these properties to understand its environment, can the defender not *alter* them so that everything appears normal? The answer is yes, and this is the essence of the blue pill. For years the defensive strategy has been to catalogue the typical queries that malware performs and to build countermeasures for each one. There are two ways to do so. The first is to change the property at its source — reconfiguring the virtual network card, for example, so that its MAC no longer begins with VirtualBox's `08:00:27`. The second applies when the hypervisor does not permit such a change: one instead modifies the **execution environment** so that whenever the sample calls an API to query the property, the returned result is rewritten on the fly. Intercepting the answer to the malware's query, rather than changing the underlying hardware, is the canonical blue-pill approach.

This strategy was realized in a research system named **BluePill**, presented at Black Hat Europe in 2019, which demonstrated that defeating the classic, well-known evasion sequences can be carried out to a considerable extent and with good effect. Its limits, however, define the deeper problem of the field. Masking every *known* artifact is achievable, but it does nothing for the artifacts that are not yet known. The defender who merely patches known checks is always one novel sequence behind; escaping that trap requires a fundamentally different design, a theme that recurs at the close of this chapter.

4.3 Example: The Speaker Check and the Limits of Mere Presence

The hardest lesson of hardware fingerprinting is that **exposing a device is not the same as emulating it convincingly**, and a clever attacker can probe the difference. The **TeslaCrypt** ransomware illustrates this with its audio-device check. The reasoning behind it is sound: a server has no need for an audio card, and sandboxes typically run on servers — often analyzing hundreds of samples at once across many virtual machines rather than one at a time. The absence of an audio card is therefore itself suspicious. A defender can respond by emulating the *presence* of an audio card, but TeslaCrypt's authors anticipated this. Reporting that a device exists is one thing; actually exercising it is another. TeslaCrypt uses **DirectShow**, part of Windows' DirectX multimedia framework, to invoke a slightly more complex feature of the audio subsystem than mere playback — it builds a filter graph and calls `AddFilter`. If the sandbox has emulated the device's existence but not this richer behavior, the operation fails in a way that exposes the fake, and the virtual machine is detected.

```
{
    wchar_t* filterName = L"random_name";

    IGraphBuilder *pGraph;
    CoCreateInstance(CLSID_FilterGraph, NULL, CLSCTX_INPROC_SERVER,
                     IID_IGraphBuilder, (void**)&pGraph);
    if (E_POINTER != pGraph->AddFilter(NULL, filterName))
        ExitProcess(-1);

    IBaseFilter* pBaseFilter;
    CoCreateInstance(CLSID_AudioRender, NULL, CLSCTX_INPROC_SERVER,
                     IID_IBaseFilter, (void**)&pBaseFilter);
    // in case of no audio device, CoCreateInstance returns
    // VFW_E_NO_AUDIO_HARDWARE 0x80040256
    // but the result is not checked and it fails later

    pGraph->AddFilter(pBaseFilter, filterName);

    IBaseFilter* pBaseFilter2;
    pGraph->FindFilterByName(filterName, &pBaseFilter2);
    if (NULL == pBaseFilter2)
        ExitProcess(1);

    FILTER_INFO info = { 0 };
    pBaseFilter2->QueryFilterInfo(&info);
    if (0 != wcscmp(info.achName, filterName))
        return;

    IReferenceClock* pClock;
    if (0 != pBaseFilter2->GetSyncSource(&pClock))
        return;
    if (0 != pClock)
        return;

    CLSID clsID;
    pBaseFilter2->GetClassID(&clsID);
    if (clsID.Data1 == 0)
        exit(1);

    if (NULL == pBaseFilter2)
        exit(-1);

    IEnumPins *pEnum = NULL;
    if (0 != pBaseFilter2->EnumPins(&pEnum))
        exit(-1);
    if (0 == pBaseFilter2->AddRef())
```

```
        exit(-1);
}
```

The structure of this COM-based routine is a chain of audio-subsystem operations, each of which must succeed with the exact result a genuine device would produce. A filter graph is created through `CoCreateInstance`; an audio-render filter is instantiated — and tellingly, the code ignores the `VFW_E_NO_AUDIO_HARDWARE` error that `CoCreateInstance` returns when no audio hardware exists, allowing the routine to fail later in a more informative way. The filter is added to the graph and then located again by name; its filter information, synchronization clock, class identifier, and pins are each queried and validated. At every step a result that deviates from what a real, fully functional audio device would return causes the routine to exit or return, flagging the environment as an inadequately emulated sandbox. The broader principle, and the reason this was such effective engineering on the attacker's part, is that emulating a real machine faithfully is extraordinarily difficult: a defender who exposes a device must also make it behave correctly under probing, and any gap between appearance and behavior is a detectable artifact.

5. Windows Installation and Software-Setup Artifacts

Even a sample that cannot conclusively fingerprint the virtualization layer can turn to the **software setup** of the machine, and in particular to the rich set of information that a Windows installation carries. Historically, when an analyst or a sandbox operator needed to examine a sample — perhaps on a freshly provisioned machine running a newly released Windows build — the resulting system came laden with contextual data, much of which is anomalous in an analysis environment.

5.1 Context Information

The **install date** is among the most telling. If a machine was installed today and a sample reaches it tomorrow, the odds that a genuine user is running software on a one-day-old system are low. It is not impossible — a victim's computer might have been replaced after a theft or failure — but to a sample it is grounds for suspicion that the host is not a genuine, long-lived machine.

The **uptime** is equally revealing. Consider a machine infected within the first minute since boot. For that to reflect reality, a user would have to log in, open their mail client, read a phishing message, click its payload, and infect the machine, all within sixty seconds — implausibly fast. A sample observing such a short uptime will reasonably conclude that a sandbox run has just been started on it. The healthy range is bounded on both sides: an uptime of a few minutes is suspicious, but a continuously running uptime of ten days is also abnormal unless the target is a server.

Other context fields extend the picture. The **time zone** is checked by malware with geographic targeting; European banking trojans, for example, historically verified that they were in the expected European time zone and stayed dormant in US or Asian zones. The system **language** serves a similar purpose. Some samples go so far as to inspect the **product key**, flagging non-genuine copies of Windows — which is doubly ironic, since many analysts themselves run non-genuine copies on their analysis machines.

5.2 Guest-Addition Traces

A further, very practical source of artifacts is the **guest additions** package that hypervisors install inside the guest to improve usability — shared folders, drag-and-drop, a shared clipboard, and smooth graphics. These conveniences are universally welcomed by analysts (older hypervisors offered no such thing, forcing users to set up network file systems merely to move a file in or out), but they leave conspicuous traces. A guest running these additions exhibits characteristic **processes**, installed **drivers**,

telltale **files**, and **registry entries** — all innocuous to the analyst who relies on them, but all suspicious to a sample looking for signs of a hypervisor.

5.3 Example: VirtualBox and the Registry

The Windows registry is where many of these traces concentrate, and VirtualBox provides a clear example. Several **key values** betray it directly:

```
HARDWARE\DEVICEMAP\Scsi\Scsi Port 0\Scsi Bus 0\Target Id 0\Logical Unit Id 0
    => Identifier     = "VBOX"
HARDWARE\Description\System => SystemBiosVersion = "VBOX"
HARDWARE\Description\System => VideoBiosVersion  = "VIRTUALBOX"
HARDWARE\Description\System => SystemBiosDate     = "06/23/99"
```

The disk drive, rather than appearing as an ordinary serial-attached or NVMe device, is exposed as a SCSI disk whose identifier is literally `VBOX`. The BIOS and video-BIOS version strings carry `VBOX` and `VIRTUALBOX`, and the system BIOS date is the famously anachronistic `06/23/99` — a value no modern physical machine would report. Any of these is a clear teller of VirtualBox, and tools exist that attempt to rewrite these strings to grant a cloak of invisibility; such tools work for simple malware.

A set of **registry keys** is likewise characteristic, most of them tied to guest additions:

```
HARDWARE\ACPI\DSDT\VBOX__
HARDWARE\ACPI\FADT\VBOX__
HARDWARE\ACPI\RSDT\VBOX__
SOFTWARE\Oracle\VirtualBox Guest Additions
SYSTEM\ControlSet001\Services\VBoxGuest
SYSTEM\ControlSet001\Services\VBoxMouse
SYSTEM\ControlSet001\Services\VBoxService
SYSTEM\ControlSet001\Services\VBoxSF
SYSTEM\ControlSet001\Services\VBoxVideo
```

Detecting these is mechanically simple, as Al-Khaser's implementation shows. The routine holds a blacklist of registry key paths and checks each one for existence:

```
VOID vbox_reg_keys()
{
    /* Array of strings of blacklisted registry keys */
    const TCHAR* szKeys[] = {
        _T("HARDWARE\\ACPI\\DSDT\\VBOX__"),
        _T("HARDWARE\\ACPI\\FADT\\VBOX__"),
        _T("HARDWARE\\ACPI\\RSDT\\VBOX__"),
        _T("SOFTWARE\\Oracle\\VirtualBox Guest Additions"),
        _T("SYSTEM\\ControlSet001\\Services\\VBoxGuest"),
        _T("SYSTEM\\ControlSet001\\Services\\VBoxMouse"),
        _T("SYSTEM\\ControlSet001\\Services\\VBoxService"),
        _T("SYSTEM\\ControlSet001\\Services\\VBoxSF"),
        _T("SYSTEM\\ControlSet001\\Services\\VBoxVideo")
    };
```

```
    WORD dwlength = sizeof(szKeys) / sizeof(szKeys[0]);

    /* Check one by one */
    for (int i = 0; i < dwlength; i++)
    {
        TCHAR msg[256] = _T("");
        _stprintf_s(msg, sizeof(msg) / sizeof(TCHAR),
                    _T("Checking reg key %s "), szKeys[i]);
        if (Is_RegKeyExists(HKEY_LOCAL_MACHINE, szKeys[i]))
            print_results(TRUE, msg);
        else
            print_results(FALSE, msg);
    }
}
```

The only element that may be unfamiliar is the `_T(...)` macro and its `TCHAR` type, which simply select the appropriate wide-character or narrow representation for Windows strings; the substance is a plain existence query. The loop computes the array length by dividing the total size by the size of one element, then iterates, calling `Is_RegKeyExists` against `HKEY_LOCAL_MACHINE` for each blacklisted path and reporting a positive result when the key is present. Such routines reliably strip away the most conspicuous guest-addition traces, which is exactly why cleanup tools target them.

6. Applications

A sample can also reason about the **software installed** on the machine, and here the presence of a given program may play one of three distinct roles.

First, a program may be a **necessary condition to trigger a payload**. Targeted malware seeking to infect, say, the machines of a specific municipal authority knows that those machines run a particular software product; if that product is absent, the malware withholds its malicious behavior because it is not on the right victim. This is not exclusive to targeted campaigns: even general-purpose malware may refuse to execute unless a specifically vulnerable component is present — a particular JVM, a vulnerable browser, a given Office version — because that component is the vehicle for its exploit. Lacking it, the sample simply stops before reaching, and failing at, the exploitation stage.

Second, a program may be an **adversary to disarm**. The clearest case is an antivirus product. If the sample recognizes the specific defensive software and knows how to neutralize it, it activates dedicated code to impede that product; if it cannot, it may give up and decline to infect the machine.

Third — and most relevant to purely evasive malware — the presence of a program may be a **sufficient condition for evasion**. Finding an analysis tool such as IDA, or a debugger, installed on a machine is, to a sample, strong evidence that the host belongs to an analyst rather than an ordinary victim. Few ordinary users have IDA installed; its presence marks the machine as an analysis environment that the sample wishes to avoid. This observation inspired a defensive idea sometimes described as **vaccination**: deliberately filling genuine machines with these artifacts — installing IDA, hypervisor traces, and the like — so that malware, perceiving an analysis host, avoids the machine, effectively yielding a free antivirus. The idea works against some malware, but it is not robust: sufficiently capable malware probes to confirm that the tools are genuinely functional, detects that the artifacts are mere decoys (just as TeslaCrypt detected the fake audio card), and may then wipe the machine in retaliation.

At a professional level, analysis rarely relies on a single VM. Analysts dissect malware using **multiple VM images**, each configured with different Windows versions and different applications and versions — some deliberately vulnerable, others current. Some vendors have optimized this provisioning to the point of maintaining base models that are customized on the fly, instantiating a VM with precisely the desired software at the desired version. In all cases these images must be sound with respect to the context information discussed above, such as install dates and product keys.

6.1 Al-Khaser and the Case of Furtim

Al-Khaser checks for a long list of analysis tools and debuggers by process name, a list that has changed little over the years because the approach is not especially clever:

```
const TCHAR *szProcesses[] = {
    _T("ollydbg.exe"),          // OllyDebug debugger
    _T("ProcessHacker.exe"),    // Process Hacker
    _T("tcpview.exe"),          // Part of Sysinternals Suite
    _T("autoruns.exe"),         // Part of Sysinternals Suite
    _T("autorunsc.exe"),        // Part of Sysinternals Suite
    _T("filemon.exe"),          // Part of Sysinternals Suite
    _T("procmon.exe"),          // Part of Sysinternals Suite
    _T("regmon.exe"),           // Part of Sysinternals Suite
    _T("procexp.exe"),          // Part of Sysinternals Suite
    _T("idaq.exe"),             // IDA Pro Interactive Disassembler
    _T("idaq64.exe"),           // IDA Pro Interactive Disassembler
    _T("ImmunityDebugger.exe"), // ImmunityDebugger
    _T("Wireshark.exe"),        // Wireshark packet sniffer
    _T("dumpcap.exe"),          // Network traffic dump tool
    _T("HookExplorer.exe"),     // Find various types of runtime hooks
    _T("ImportREC.exe"),        // Import Reconstructor
    _T("PETools.exe"),          // PE Tool
    _T("LordPE.exe"),           // LordPE
    _T("SysInspector.exe"),     // ESET SysInspector
    _T("proc_analyzer.exe"),    // Part of SysAnalyzer iDefense
    _T("sysAnalyzer.exe"),      // Part of SysAnalyzer iDefense
    _T("sniff_hit.exe"),        // Part of SysAnalyzer iDefense
    _T("windbg.exe"),           // Microsoft WinDbg
    _T("joeboxcontrol.exe"),    // Part of Joe Sandbox
    _T("joeboxserver.exe"),     // Part of Joe Sandbox
    _T("ResourceHacker.exe"),   // Resource Hacker
    _T("x32dbg.exe"),           // x32dbg
    _T("x64dbg.exe"),           // x64dbg
    _T("Fiddler.exe"),          // Fiddler
    _T("httpdebugger.exe"),     // Http Debugger
};
```

This enumeration captures the asymmetry at the heart of the field. A defender must cover as many of these tools as possible, since it is unknown which the malware will check; an attacker needs to find only a single gap — one analysis feature that is not hidden — to detect the environment. Defenders must cover everything; offenders need only one weak link.

The **Furtim** malware is the exemplary demonstration of an attacker exploiting this asymmetry to the fullest. Its very name derives from the Latin for *furtively*, reflecting its extreme caution. Furtim first detected and disabled an unusually wide range of antivirus products — including some so obscure that even seasoned practitioners were unaware they existed — and then searched for a large set of tools that analysts launch manually, reportedly amounting to several hundred checks in total. At the time of its release no publicly known sandbox on earth could run it without being detected and evaded. Even the best sandbox of the day, **Joe Sandbox**, is said to have taken on the order of two weeks to adapt its platform to run the sample, and in less sophisticated sandboxes Furtim still evades analysis today.

Most distinctive was Furtim's treatment of the human analyst. Rather than exiting immediately upon detecting a tool such as IDA — which would at least signal plainly that evasion had occurred — it deliberately continued executing as though the tools were no insult to it, only to crash, seemingly without reason, several subroutines later. The intent was to **frustrate** analysts: to let them reach a point of apparent progress and confidence before revealing that the malware had been in control the whole time. This design reflects authorship by people with long experience in the security industry, who understood that the analyst's scarcest resource is time, and that wasting it is itself an effective defense.

7. User Artifacts and Wear-and-Tear

The final dimension of setup-based detection concerns how *used* a machine looks. A **fresh Windows installation is suspicious**, because a genuine victim's machine accumulates a history that a just-provisioned sandbox lacks. Sophisticated malware inspects the **recently opened files**, the **navigation history**, the installed **applications and their install dates** — including whether everything was installed at once or anything was installed recently — the **recently connected USB devices** (Windows records a list of these in the registry), and the **number and type of files** in the Documents and Desktop folders.

The governing concept is **wear-and-tear**, by direct analogy with clothing: garments worn over time acquire stains, wear, and tears, and a Windows installation accumulates a comparable history of use. A sandbox that wishes to pass as genuine must therefore exhibit a realistic wear-and-tear state, and manufacturing one convincingly is extraordinarily hard. The **Spotless Sandboxes** work (IEEE S&P 2017) quantified the stakes: a properly written sample could detect in excess of 95% of analysis systems using as few as **three** wear-and-tear checks. This result is not an indictment of the industry's competence. It reflects, rather, the genuine difficulty of emulating what a real, lived-in machine looks like — a difficulty so great that one can train models (in that work, classical machine-learning models) to distinguish real from synthetic usage and thereby evade analysis.

8. Timing Attacks

Of all the players in the evasion game, one has been present from the very beginning: **time**. Playing with time is the oldest technique for detecting malware analysis systems, and it remains among the most effective. Three broad options are available.

8.1 Timing an Instruction Sequence

The first option generalizes the `cpuid`-latency red pill already seen. Wherever a sample knows roughly how long an instruction sequence should take — or holds some other baseline or reference — it can time that sequence's execution and compare. The `cpuid` case, which takes measurably longer under a hypervisor, is one instance of a general method.

This same method detects **binary instrumentation**, a monitoring technique that may be either dynamic or static. The idea of instrumentation is to modify a program's instructions so as to add extra instructions into the execution flow. In **dynamic binary instrumentation**, a component injects additional instructions — meant to perform monitoring — into the stream of instructions as the program runs. Each injected instruction costs time, so a sample that times a sequence against a known baseline can notice the inflation and infer that it is being instrumented.

8.2 Exhausting the Analysis Budget with Sleeps

The second option exploits the fact that a sandbox will not run a sample forever. Because so many samples must be processed, each is granted only a limited window — typically five to ten minutes, sometimes as little as three or four, at least on a first pass. There is also little to gain from running longer: studies show that for a sample that is not actively evading, the first few minutes are enough to observe its behavior.

Malware turns this budget into a weapon by planting **sleep** operations. The naive version simply sleeps for ten minutes and waits out the sandbox — and it works, unless the sandbox is clever enough to identify a large sleep, intercept the operation, and shorten it to a millisecond. Malware authors respond by replacing one big sleep with **many tiny sleeps** that sum to the same large delay, making the sequence far harder to detect and patch away. For the malware this costs little: activating after twenty minutes is no hardship if the sample is running undisturbed on a real machine, but it reliably defeats a time-bounded sandbox.

Should the sandbox grow clever enough to skip or shorten all sleeps, the malware has a further move: it can **check whether time has been manipulated**. Knowing that a normal execution should have slept, say, fifteen minutes, the sample performs its sleeps — which may be skipped — and then reads the current time. If the elapsed difference is not at least the expected fifteen minutes, time has been accelerated, and the environment is exposed. The defender can counter by accumulating all the sleep time it skipped and subtracting it whenever the sample asks for the time, so that the clock appears consistent. This is a genuine cat-and-mouse exchange, and it is tricky for the defender, because time must be emulated correctly and consistently across **all** the time sources that Windows exposes — and there are several, not just one. All of them must report mutually consistent values, or the inconsistency itself becomes the artifact. For some malware this time emulation succeeds; for other cases it does not, and defeating the strategy may require manual intervention.

8.3 Long Computations

The third option is the hardest to patch. Instead of sleeping, the sample simply performs a **very long computation** in the initial stages of its execution — computing an enormous number of digits of π, or any other arbitrarily expensive mathematical problem. The work is genuine and takes as long as the author wishes. Running this in a sandbox, the sample does nothing suspicious while the analysis budget simply runs out, and the sandbox concludes there is nothing to see — the malware has evaded it without a single overtly malicious action.

This points toward an interesting line of defensive research: writing analysis that recognizes a sample doing nothing useful and **fast-forwards** past it, skipping the stalling computation to reach whatever the sample intends to do next. There is considerable room for such techniques in this domain.

8.4 The Impossibility of a Transparent Sandbox

The overarching lesson of the timing attacks — and of evasion as a whole — is that **no analysis system is perfectly transparent**. Whatever system one designs, some imperfection will remain, and the attacker will target precisely that imperfection, because this is the nature of cybersecurity as a cat-and-mouse game. Harden a system along one dimension and attackers shift to another: years of focus on buffer-overflow defenses merely pushed attackers toward use-after-free bugs, leaving defenders nearly back at square one against the new threat.

This leaves two strategic paths. The first is to keep **fixing every issue** as it is discovered — patching each artifact until the next evasive sample reveals a new one, then patching again, indefinitely. The second, more ambitious, is to devise a **design that is virtually unaffected** by these issues, or that requires only minimal changes because most evasions simply do not apply to it. Pursuing either path demands intimate knowledge of how evasions are implemented, which is why the manual analyst must know exactly where to look.

9. Spotting an Evasive Sequence

When automated analysis is defeated and a sample must be dissected by hand, the analyst needs to know where evasive behavior hides. Several surfaces deserve close attention.

Special instructions are the first sign: sequences involving `cpuid`, `rdtsc`, or `int` warrant careful study of the surrounding subroutines, since they may form an evasion that must be disarmed. (A further family of special instructions used specifically against debuggers is deferred to the dedicated treatment of debuggers.)

Library calls are a broad and important surface. Evasive logic frequently surfaces as calls that touch files, the registry, events in the graphical user interface (such as which windows are visible or in the background), hardware features, drivers, processes, pipes, DLL handling, the network, IPC objects, and time sources.

System calls are harder to handle. More sophisticated samples bypass the standard libraries and issue system calls directly, talking to the kernel without the intermediating API, which makes their behavior more difficult to observe and interpret.

WMI — Windows Management Instrumentation — is a querying facility, examined in detail below, through which a sample can interrogate the operating system about hardware and software.

The **process environment** is the final, often overlooked surface. The **Process Environment Block (PEB)**, a specific structure in the process's memory layout, exposes information such as the number of CPU cores available to the process and whether the process is being run under a debugger. Reading these values requires no API at all — they are simply fields in the PEB — so a sample can consult them silently. For this reason, an analyst debugging a sample will often go straight to the PEB to alter the reported core count or to hide the debugger's presence, a task for which debugger plugins exist.

10. WMI Queries

The **Windows Management Instrumentation** subsystem supports SQL-style queries to the operating system about both hardware and software. A sample can ask the OS for information in a familiar `SELECT ... FROM ...` form, drawing on a wide range of classes:

```
SELECT * FROM   (examples of what can be checked)
    Win32_Bios                      (SerialNumber)
    Win32_PnPEntity                 (DeviceId)
```

```
    Win32_NetworkAdapterConfiguration (MACAddress)
    Win32_Processor                (NumberOfCores, ProcessorId)
    Win32_LogicalDisk              (Size)
    Win32_ComputerSystem           (Model, Manufacturer)
    Win32_NTEventLogFile           (FileName – related to VBOX)
    MSACPI_ThermalZoneTemperature  (CurrentTemperature)
```

The significance of WMI for evasion is that it lets a sample obtain much of the fingerprinting information discussed throughout this chapter **without calling the specific APIs** one would normally associate with each property. Rather than invoking dedicated functions to read the core count or disk size, the sample issues a single generic WMI query. The same facility can enumerate running processes, list installed antivirus products, and much more. An analyst must therefore be aware that whenever a sample uses WMI, these queries deserve the same scrutiny as direct API calls. It is through WMI that a sample can, for example, read the CPU core count, the disk size, or the BIOS serial number while sidestepping the obvious interfaces. (Thermal data such as `MSACPI_ThermalZoneTemperature` is a partial exception, being inherently sensor-oriented, but the structural hardware facts are readily obtained this way.)

10.1 Al-Khaser: Checking Win32_Bios

A representative WMI-based check inspects the BIOS serial number for strings characteristic of hypervisors and emulators:

```
// Get the value of the Name property
hRes = pclsObj->Get(_T("SerialNumber"), 0, &vtProp, 0, 0);
if (SUCCEEDED(hRes)) {
    if (vtProp.vt == VT_BSTR) {

        // Do our comparison
        if (
            (StrStrI(vtProp.bstrVal, _T("VMWare"))  != 0) ||
            (wcscmp(vtProp.bstrVal, _T("0"))        == 0) || // VBox (serial is just
"0")
            (StrStrI(vtProp.bstrVal, _T("Xen"))     != 0) ||
            (StrStrI(vtProp.bstrVal, _T("Virtual")) != 0) ||
            (StrStrI(vtProp.bstrVal, _T("A M I"))   != 0)
            )
        {
            VariantClear(&vtProp);
            pclsObj->Release();
            bFound = TRUE;
            break;
        }
    }
    VariantClear(&vtProp);
}
```

The routine retrieves the `SerialNumber` property of the BIOS object returned by the WMI query, confirms that the result is a string (`VT_BSTR`), and then performs a series of case-insensitive substring

comparisons. A serial containing `VMWare`, `Xen`, or `Virtual` points to the corresponding platform — the last of these catching **Virtual PC**, a historical emulator, rather than VirtualBox. A serial that is exactly the string `"0"` identifies VirtualBox, whose BIOS serial is simply zero. The string `A M I` matches the American Megatrends BIOS vendor, common in emulated firmware. If any comparison succeeds, the code clears the variant, releases the WMI object, sets `bFound` to `TRUE`, and breaks out of the enumeration, having concluded that the BIOS serial reveals a hypervisor or emulator. These are the common BIOS serial numbers found across virtualization and emulation products, and matching against them is a compact, reliable fingerprint.

11. In-Guest Agents and Their Inherent Weakness

The discussion so far has concerned two main attack surfaces: the artifacts of the **execution technology** (the imperfections and overheads of virtualization) and the artifacts of the **software setup** (the installation, applications, and usage of the guest). There is, however, a more fundamental weakness that afflicts sandboxes and dynamic-analysis systems generally — one that precedes any specific artifact. For such systems, operating **inside** the virtualized guest is already a position of weakness.

To understand why, consider how monitoring has historically been performed. A sandbox must acquire execution information about the sample — which processes it starts, which files it creates, and so on. Network activity can sometimes be observed from outside the VM, at the firewall level, but for everything else the classical approach places a piece of software **inside** the virtual machine. These in-guest monitors are called **agents**, and for years they operated in **user space**, most often as an **injected DLL**: the agent forces the program under analysis to load a library that intercepts and records what the program does. Many antivirus products have historically worked the same way.

Kernel-space agents once offered a stronger vantage. Because any DLL present in a process's memory can be discovered by querying that memory and enumerating loaded modules, a user-space agent must work to hide itself; a kernel-resident monitor avoided much of this visibility. In the past, this was comparatively easy. Today it is far harder. Windows has substantially **hardened the kernel**: drivers must be signed with a certificate, and the system no longer permits arbitrary modification of internal structures of the kind that monitoring once relied upon — for instance, to intercept which files a process accesses. Two mechanisms enforce this. **PatchGuard** (Kernel Patch Protection) detects, at runtime, any attempt to modify the Windows kernel; a rootkit-style patch of a system call's implementation to intercept events triggers a blue screen of death. PatchGuard embodies on the order of two hundred and growing distinct checks, and defeating it is so difficult that some practitioners have built entire careers as PatchGuard-bypass experts. In parallel, Windows now offers sanctioned facilities through which security products can observe kernel events via standardized mechanisms, such as **Event Tracing for Windows (ETW)**, with some practitioners even beginning to experiment with **eBPF for Windows**.

The bottom line is structural and does not depend on any of these details. An in-guest agent — whether it runs, with difficulty, in the kernel or, more visibly, in user space — **shares the execution context with the sample under analysis**. In security, any scheme in which the attacker coexists with the defender in the same space is, in a deep sense, destined to struggle: the attacker can detect that the defender is operating in the same arena. One may always study how malware fingerprints systems and then fix or randomize the agent's implementation to be less detectable, but in the end the agent still shares the room with the sample, and there is a real chance that a future sample — which knows how the current defenses are built, while the defender cannot know how the next sample will be built — will defeat it.

12. Virtual Machine Introspection

The response to this structural weakness was to move the analysis **outside** the virtual machine, so that the monitor is no longer visible to the sample. This idea was initially developed for intrusion-detection systems, which sought greater resistance to attack while retaining the ability to observe events, and it was introduced in the NDSS 2003 work on a Virtual Machine Introspection based architecture for intrusion detection. Performing analysis from outside the guest requires a special virtual machine and the cooperation of the hypervisor's **VM monitor**.

For this design to work and to resist evasion and subversion, three capabilities are crucial.

Isolation requires that the program under analysis cannot access the code or state of the monitoring system. Achieving this would, in effect, require a vulnerability in the hypervisor to breach — and hypervisors are, by construction, well suited to providing exactly this isolation.

Inspection requires that the monitoring system has full access to the state of the analyzed machine. In principle one can always read the memory and registers of the virtual machine, so everything is inspectable; doing so in practice is not simple, but the tooling has improved markedly.

Interposition is the most important and most subtle capability: the ability to step in at specific operations, shepherding or altering an interaction as it happens. One needs to interpose at particular events — for example, the kernel transitions caused when a system call is issued, or attempts by the sample to modify sensitive properties or access regions that could subvert the monitoring mechanism. Not every operation requires interposition, but the capability to intervene at chosen moments is essential.

13. Ether (CCS 2008)

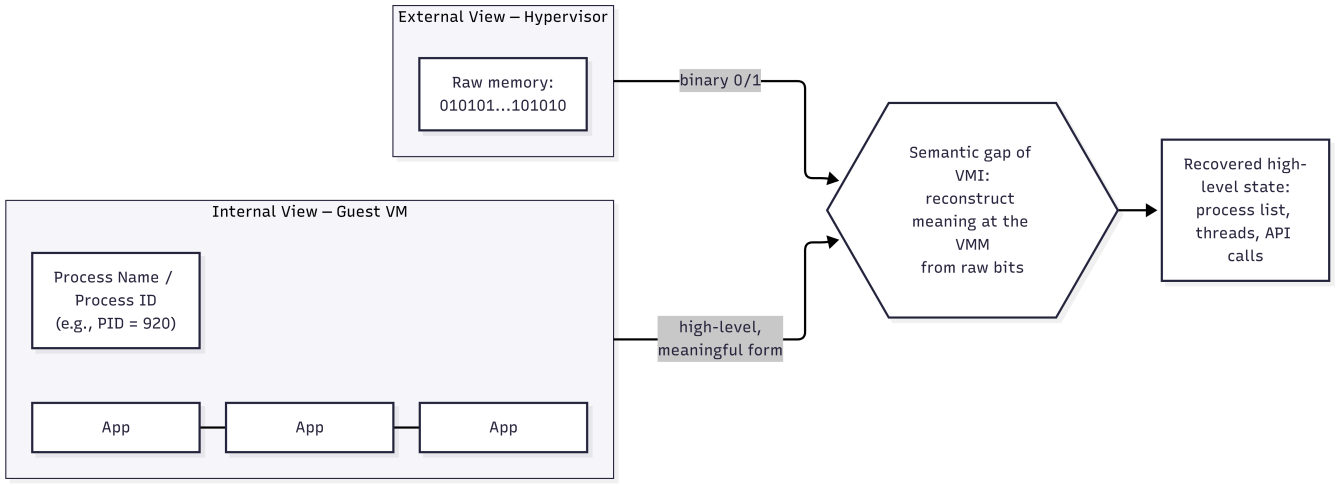
The first solid realization of this design for malware was the **Ether** malware analyzer, which pioneered the use of Intel's **VT-x** virtualization extensions to build a transparent analysis system. Ether's analyzer resides in the **VM monitor**, and therefore runs at a privilege level that the analyzed code cannot reach. Its authors identified **five requirements** for a system that seeks to hide the memory and CPU changes its very presence inevitably causes.

The first is **higher privilege**: the analyzer runs at a privilege level the analyzed code cannot attain. The second is **privileged access to side effects**: if the analysis introduces side effects, they should be visible only at a privilege level the analyzed code cannot reach. The third is **same basic instruction semantics**: instructions must behave exactly as they would natively, with side effects permitted only during exceptions. The fourth is **transparent exception handling**: when an exception occurs, the analyzer can reconstruct the expected context where needed, so the analyzed code never sees an anomalous state. The fifth is **identical timing information**: access to time sources is shepherded so that values can be forged, defeating the timing red pills discussed earlier.

Ether's design was genuinely promising, yet it was undermined by **practical limitations** — above all, the imperfections of VT-x. The extensions were engineered for near-native speed and efficient cloud workloads, not for security, so corner cases inevitably remain. The requirement of privileged access to side effects, in particular, did not anticipate that someone would later demonstrate a **side channel**: the design alters cache utilization, and a user-space program can reconstruct that cache utilization without violating any of the five properties, thereby inferring the monitor's presence. Likewise, although time sources can be shepherded, doing so in a way indistinguishable from legitimate use proved so difficult that the timing guarantee is, in practice, broken. Even so, a system of this kind remains in a far better place than the standard virtual machine with a DLL injected into the sample. Ether advanced the state of the art substantially, and the good part of the industry — the better sandboxes — has adopted the Virtual Machine Introspection idea it helped establish.

14. The Semantic Gap Problem

The principal cost of Virtual Machine Introspection is the **semantic gap**. An in-guest DLL is comfortable to work with: it can call the Windows APIs and obtain high-level information about processes directly. With VMI, by contrast, the monitor sees only raw memory — a sea of zeros and ones — and must **reconstruct** the high-level meaning itself. To list the running processes, it must know where the kernel resides, what the kernel's structures look like, how to walk from those structures to the list of processes, and how to move from a given process to that process's memory. VMI methods therefore incur a semantic gap whenever they try to inspect high-level concepts of the guest — API calls, threads, process identities — from the low-level external view.



Bridging this gap requires detailed knowledge of how the operating system is implemented. On Linux the source is available, which eases the task; on Windows it demanded an enormous amount of reverse engineering, which dedicated tools nevertheless accomplished. Over the years the tooling has become very capable — to the point that from the VMI paradigm one can not only read guest state but even **inject a thread** into a guest application to make it run code on the analyst's behalf. On a Linux host, for example, one can open a terminal and issue a command that lists the processes running inside a guest VM, complete with their PIDs and names, because the VMI infrastructure looks up the guest's memory and recovers that information. The implementation complexity of a VMI-based system is nonetheless extreme: not long ago, merely obtaining the list of processes was a hard problem, whereas today far more is possible. The reward for that complexity is substantially **better transparency**, which greatly aids automatic analysis.

15. Automatic and Manual Analysis

VMI-based systems thus face higher implementation complexity than in-guest agents but offer better transparency, and sandbox vendors have begun adopting VMI-based or hybrid approaches. In academic research, the reference system is **DRAKVUF** (introduced at ACSAC 2014), developed in Poland alongside a security operation center that continues to advance VMI-based systems.

Automatic analysis, however, is only one part of the story. There will always be malware that escapes even a good automatic system, and every new family eventually demands **manual analysis**; one cannot rely on the sandbox alone to produce a report. When dissection is required, VMI is **no lifeboat** for the analyst's conspicuous manual work. Analysts still, as a matter of course, work on their own laptops with classic virtualization products, and so they must **dismantle the very same evasions again — this time by hand**, confronting the identical tricks that the sandboxes worked so hard to bypass. The sandbox may offer indications — noting, for instance, that a sample was clearing a particular check — but much of the work remains, and it is daunting precisely because analysts are a scarce resource.

Software can be run as much as the infrastructure allows; human experts are few, which is why any research that economizes their time is highly valued.

The latest trend addresses exactly this duplication: **out-of-VM debugging of a process running in a VM** — applying Virtual Machine Introspection not only to automatic analysis but to the manual dissection itself, through a debugger. The **BluePill** project was an earlier solution aimed at sparing analysts the repeated dismantling of evasions, and in recent years the technology has advanced to where dissection through a hypervisor is becoming feasible. It is ongoing work, not yet as plug-and-play as one would wish, but it is maturing, and it promises to spare analysts the toil of defeating the same evasions over and over.

CHAPTER 7: Advanced Static Analysis

What Advanced Static Analysis Is, and Why It Matters

Basic analysis — whether static or dynamic — is deliberately shallow. It is the triage step: run a few automated tools, pull the obvious strings, check the imports, drop the sample into a sandbox, and read the report. For many samples this is enough. But every now and then a sample resists this treatment: the basic results come back inconclusive, the threat is new, or something about the file simply warrants a closer look. When that happens, the analyst escalates to **advanced static analysis** — a careful, controlled reading of the program's actual code, performed *before* the sample is ever executed in a live environment.

The defining idea of advanced static analysis is that you examine the sample's logic without having to dissect every subroutine in full detail. You open the executable in a disassembler such as IDA, and you begin by skimming. The goal of this first pass is to build a mental map: which subroutines are *bulky*, full of basic blocks and branching logic, and which are *thin*, containing perhaps a single API call or two. This distinction already tells you where the interesting behavior is likely to live.

The reason API calls are the anchor of this process is that **API calls characterize how a sample interacts with its environment**. A program leaves its mark on a system through the operating-system services it invokes — files it touches, registry keys it writes, network connections it opens, processes it spawns. A sample that makes no API calls is, from an analyst's standpoint, almost inert: it cannot affect the system in any observable way, so there is comparatively little to learn from it. Conversely, the set of APIs a program calls is effectively a description of what that program *does*.

The workflow therefore proceeds roughly as follows. You open the code and skim it. You identify the API calls that are visible. For each relevant call, you try to recover its arguments — and with practice, you begin to recognize recurring values and constants: a sensitive filesystem path, a flag that requests executable memory (a strong hint that the sample intends to place and run code somewhere), a well-known registry location. These are the *tells*. Connecting these dots gives you a first, genuine glimpse of the malware's behavior — far deeper than anything basic static tools could reveal. It is not the whole picture; some behavior only becomes visible when the program actually runs. But a great deal can be recovered statically. As a bonus, this reading lets you extract **Indicators of Compromise (IoCs)** — distinctive paths, names of inter-process communication objects, and other artifacts peculiar to particular malware families, which let you recognize related samples in the future.

This is one disciplined approach, reflecting common practice, rather than *the* single correct method. The governing principle is: **start with a high-level view, and add detail only where it matters.** Experience is what teaches you where that is.

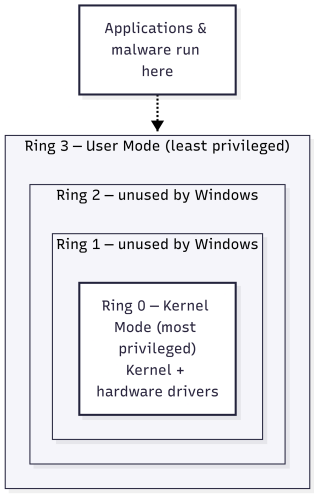
Throughout this chapter we adopt one simplifying assumption: the malware's activity is **not concealed or obfuscated**. API-call obfuscation — the deliberate hiding of which functions a sample invokes — is a real and important topic, but it is set aside here. We assume that whatever a sample does with the operating system is plainly visible in the disassembly.

The Privilege Model: Rings, User Mode, and Kernel Mode

To understand how malware interacts with the system, we first need to understand the privilege structure the CPU and operating system enforce. Modern processors define a hierarchy of **protection rings**, and these rings are genuine *privilege boundaries* — enforced by hardware, not merely by software convention. Code confined to a given ring is physically prevented by the processor from reaching resources that belong to a more privileged ring.

There is a deliberate inversion in the numbering that trips people up: the **lower** the ring number, the **higher** the privilege. Ring 0 is the most privileged, innermost layer, and it is where the **kernel** runs. Rings 1 and 2 exist in the hardware but are not used in practice by Windows. Ring 3 is the least privileged, outermost layer, and it is where ordinary user-space code runs — all normal applications, and virtually all malware along with them.

Conceptually, the model is a set of nested shells:



Almost all code — including the bulk of the operating system's own user-facing components — runs in user mode. Only the core of the operating system and its hardware drivers run in kernel mode. Not every part of the OS needs ring 0; plenty of it runs in user space. But the critical core does, because it must be protected not only against *security* problems but also against ordinary *compatibility* and *stability* problems — ensuring, for example, that a misbehaving application cannot overwrite kernel memory.

Two restrictions define what it means to run in ring 3. First, **user-mode code cannot access hardware directly.** There do exist indirect ways to issue commands to hardware, and in the earliest CPUs one could write directly to I/O ports — but over time this has become steadily more restricted, and today the ability to interact with hardware is confined to the kernel. Second, **certain CPU instructions are privileged** and may only be executed in kernel mode. Instructions that manipulate control registers (the `CR` registers) and other special registers used by the OS for tasks such as process scheduling simply cannot be issued from ring 3.

User-Mode Processes

In user mode, **each process is an island.** It has its own memory — an isolated address space — along with its own security permissions and its own resources. One process cannot reach into another's memory

unless that other process explicitly chooses to share it. This isolation is what makes the system robust: when a process does something invalid — corrupts its own memory, executes an illegal instruction, or otherwise crashes — the operating system can simply terminate it and reclaim its resources, with no harm to the rest of the system.

Crossing the Boundary: Calling the Kernel

A user-mode program cannot jump directly to an arbitrary point in the kernel. Allowing that would be catastrophic for both security and system safety — there are obvious ways to abuse such freedom. Instead, the processor architects and OS designers defined a small number of controlled **gate mechanisms**. Whenever user code needs a kernel service, it must pass through one of these well-defined interfaces, which deliver control to exactly the right kernel code.

The mechanism works through **system calls**, each identified by a number. On Linux these numbers are fixed, publicly documented, and stable across the years: a given feature — read, write, send on a socket — has one well-known value. To invoke a system call, you place its number in a specific register and then execute a special instruction. The exact instruction depends on the architecture and era: historically Windows used `int 0x2e` (the software interrupt instruction `int` with operand `0x2e`); with the move to faster hardware gates, `sysenter` (Intel) and `syscall` (AMD, and later the common standard) replaced it. At the beginning these differed between vendors; today they are effectively equivalent.

Whichever instruction is used, the program sets up the required register state and the **hardware performs the transition on its behalf.** The processor consults predefined structures that only the kernel is permitted to write, and through them the kernel tells the hardware precisely what to do. The program thus enters ring 0 — but it does so only at a controlled entry point, and it will leave only at a controlled exit point. Entry and exit are never arbitrary.

For the malware analyst, the appearance of one of these instructions — `sysenter`, `syscall`, or `int 0x2e` — is significant: **it indicates that the code is invoking kernel-level functionality directly.** This is a hallmark of highly sophisticated malware, and it is rare, for a very practical reason. Windows changes these system-call numbers from version to version. Some values are stable; many are not, differing across Windows releases. None of this is officially documented — but excellent guides exist, produced by people who reverse-engineer the Windows kernel and catalog the numbers for each edition. A sophisticated malware author willing to track this can use direct syscalls, partly because such calls are also harder for security solutions to monitor. In most cases, though, malware does not bother: it reaches the kernel indirectly, through the higher-level APIs described below.

Kernel-Mode Execution

Once execution enters the kernel — whether through one of these gates or as part of the normal running of kernel tasks — the rules change. **Everything running in the kernel shares resources and memory.** There is no per-process isolation at this level, so kernel code must be written with extreme care. When kernel code fails — executes an invalid instruction, or corrupts memory — the result is the **Blue Screen of Death (BSoD)**: the system crashes and the machine must be rebooted. In the era of Windows 95 and 98 this was a daily occurrence; a single buggy driver could produce a crash every few minutes. It is far rarer today, but the fundamental fragility remains, because the kernel is so easily contaminated by errors.

Because kernel code is presumed to be well-tested and trusted, it runs with **fewer security checks than user mode — or, more precisely, security mechanisms of a different kind.** Chief among these is **PatchGuard** (Kernel Patch Protection), an *integrity* mechanism that attempts to detect

unauthorized modification of kernel code. If some memory is overwritten by mistake, or if an attacker tries to alter the implementation of a system call inside the kernel, PatchGuard detects the change and deliberately raises a Blue Screen of Death.

In the past, many security products — antivirus engines among them — shipped kernel drivers to perform their work. This is less common now, precisely because PatchGuard constrains what such drivers may do. In its place, Windows offers sanctioned components that security solutions can use. One such component is **AMSI** (the Antimalware Scan Interface), which functions as a kind of built-in malware-inspection subsystem: a security product can register for events, and Windows will inform it when relevant activity occurs. A consequence for the analyst is that malware authors increasingly try to **disable or circumvent these systems** rather than evade a kernel driver.

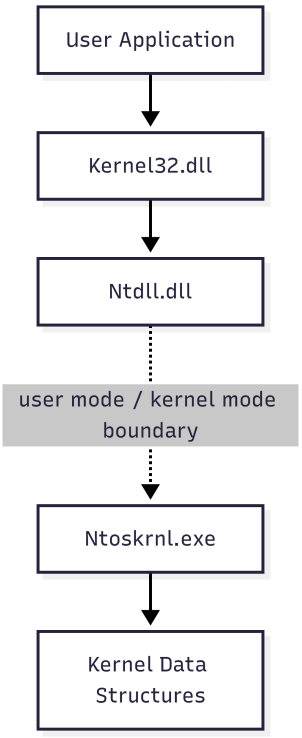
Finally, there is malware that runs *entirely* in kernel mode. The most common example is the **rootkit**. Such malware is extremely sophisticated and operates in a different league from user-mode malware; it is a specialized discipline unto itself, with relatively little publicly documented and only partial reference implementations available. It is not the focus of this course, but it is a rich area for independent study.

The Two Interaction Layers: The Native API

Setting aside the obfuscated direct-syscall approach already described, user-mode code interacts with the operating system through **two levels of API**: the low-level **Native API** and the higher-level **Windows (Win32) API**.

The Native API is the lower of the two. It is, in effect, the library-level embodiment of the gate mechanism described above — a set of low-level system calls implemented in a single library, `ntdll.dll`. This library is the component that actually talks to the kernel. It manages the interactions between user space and the kernel, and the functions it exports *are* the Native API.

The relationship between the layers is shown in the standard diagram of the user-mode/kernel-mode divide:



The kernel itself is represented here by the executable `Ntoskrnl.exe`. That exact name is a convention of the diagram rather than what you will necessarily find on a given machine — several variants exist,

some of which exploit features available on more recent hardware — but on any system *some* `ntos`-based process is running, and that is the kernel. Its user-mode counterpart is `ntdll.dll`, which knows exactly which kernel version is running and how to talk to it. Concretely, this means `ntdll.dll` contains the correct system-call numbers for all the low-level calls that need to be made.

The functions exposed here form the **NT API**, and they are recognizable by their `Nt` prefix — for example `NtWriteProcessMemory` or `NtCreateThreadEx`. These functions are not intended for average developers. Normally they are meant to be used by *other* libraries. Even a library that sounds very low-level from its name — `kernel32.dll`, which essentially every Windows application uses — is itself a Win32 library that relies on `ntdll.dll` underneath. Whenever `kernel32.dll` needs to talk to the kernel, or needs to allocate very large chunks of memory, it does so through these low-level native calls.

You rarely see the Native API used by legitimate software — at most a handful of functions. In malware, by contrast, you can encounter samples written almost entirely against these APIs. Official documentation is limited, but there is a great deal of high-quality documentation that is the product of reverse engineering, accumulated over many years — in some cases understanding the behavior better than Microsoft's own engineers, having identified gaps between a format's specification and its actual parser implementation. Dedicated websites document many of the native functions well. There is also an open-source reimplementation of the Windows kernel, **ReactOS**, whose codebase is often the quickest way to see how a given NT function is actually called and what it expects.

Popular Native API Calls in Malware

Why would a malware author accept the extra difficulty of using the Native API? The payoff is **information**: these calls can return far more than their Win32 equivalents. They give access to low-level properties of running processes and of the hardware itself — which is exactly what a sample wants when, for instance, it is trying to fingerprint a sandbox. A representative family of such calls is the set of query functions used to obtain information about the system, processes, threads, handles, and other items:

- `NtQuerySystemInformation`
- `NtQueryInformationProcess`
- `NtQueryInformationThread`
- `NtQueryInformationFile`
- `NtQueryInformationKey`

The power of these calls is considerable. `NtQuerySystemInformation` alone accepts so many information-class flags that one can derive well over fifty distinct configuration details from it — where the corresponding Win32 APIs would surface only a fraction. This ability to dig deeper and lower is precisely why they are handy, both to malware authors and to Windows power users. A common refrain in online discussions — on Stack Overflow and similar sites — about checking, say, whether a process is actually executing something, is that it cannot be done properly with the standard APIs; you must drop down to these native calls. For the analyst, a core function such as `NtQuerySystemInformation` should therefore be read as a signal that the sample is collecting detailed, low-level reconnaissance of its host.

The Windows (Win32) API

Where the Native API is low-level and sparingly documented, the **Windows API** — commonly called the **Win32 API** — is the standard, broad interface through which normal programs interact with Windows. It provides applications with functions to access operating-system services and resources, and in practice it exposes essentially everything Windows can do. Programs normally reach OS resources through API functions **exported by system DLLs**.

The scale of this interface is enormous. An analysis of the system DLLs across Windows versions found on the order of **100,000 distinct APIs**, with roughly **60,000** widely available across the common releases. The purpose of offering so much is **code sharing**: a developer need not reinvent a piece of functionality every time, because a standard implementation is already provided by the operating system. This was a major reason for Windows' success in the 1990s — developers could ship small executables that nonetheless carried rich functionality, which mattered greatly in the era of floppy disks and slow internet connections. It also sidestepped a persistent pain point on other systems, where an application might depend on one specific library version being installed, or else resort to bundling static libraries. Windows made this simple and, above all, **stable**: code written in 2005 will generally still run in 2026.

What follows is not a programming course; the aim is only to cover the parts of the Win32 API that are relevant to malware analysis — namely the data types, the concept of a handle, and a set of low-level functions for files, the registry, networking, and execution.

Data Types and Hungarian Notation

The Windows API defines **its own names for standard C data types**. You will rarely see a plain `int`; instead you encounter names like `WORD` and `DWORD`. This renaming arose from a desire to provide a compatible, uniform abstraction across architectures and across the system's long evolution — Windows descends from earlier systems and has always aimed for broad compatibility (there was even a Windows edition for Intel's Itanium, a 64-bit CPU incompatible with 32-bit code). Whatever the internal reasons, Windows gave every machine-level type a consistent Windows name: for example, every 32-bit unsigned integer is a `DWORD`.

Layered on top of this is **Hungarian notation**: variable and parameter names are given a prefix that indicates their type or intended use. This is genuinely helpful during analysis, because the name alone carries a hint about the type — you often do not need to read the full prototype to understand what a parameter is. The common prefixes are:

Type (prefix)	Meaning
`WORD` (`w`)	16-bit unsigned value
`DWORD` (`dw`)	32-bit unsigned value
Handle (`H`)	A reference to an OS object
Long Pointer (`LP`)	Points to another type

Thus a parameter named `dwSomething` is a 32-bit unsigned integer; a parameter named `hObject` is a handle to the object the API will act on; and a parameter named `lpBuffer` is a pointer to a mutable buffer. When you pass something mutable — an array, a buffer — you will see the `LP` prefix in the prototype.

Handles

A **handle** is a reference to an item that has been opened or created in the operating system — a process, a menu, a file, a window, and so on. Handles behave like **immutable pointers to those OS objects**, but with important restrictions. You **cannot perform pointer arithmetic** on a handle; it is not the base of an array of objects, but a single, one-time value. Conceptually it is similar to a Unix file descriptor, except that it applies to a great many kinds of objects rather than just files. The rationale for immutability is simple: if you ask the operating system for information about, say, another process, you

are *querying* it — there is no meaningful sense in which you would modify the handle itself, so by definition you receive a read-only reference.

Handles are useful precisely because you can **store one and reuse it later** to refer to the same object, and the handle remains valid for the duration of the execution. You might enumerate the processes running on a system and hold a handle to each, then query each handle's properties; likewise you can hold a handle to a file, or to a window. As a concrete example, the `CreateWindowEx` function returns an `HWND`, a handle to the newly created window; to do anything to that window afterwards — such as destroying it with `DestroyWindow` — you must pass that handle.

A handle can be **duplicated**, which is the closest thing to sharing one, but otherwise it is the only means of accessing its object. It is also good practice to **check whether a handle is valid**, and you will frequently see malware authors open a resource and then compare the result against `INVALID_HANDLE_VALUE` — a comparison worth noting during analysis.

File-System Functions

The most fundamental Win32 file operations are `CreateFile`, `ReadFile`, and `WriteFile`, which provide ordinary file input and output. These need little explanation.

More interesting — and more frequently abused by malware — is the **memory-mapping** pair, `CreateFileMapping` and `MapViewOfFile`. Together these load a file's contents into RAM so that it can be accessed as memory rather than read sequentially. `CreateFileMapping` creates a *file-mapping object*, and `MapViewOfFile` maps a view of that object into the process's address space — making the contents visible — and returns a **pointer to the base address of the file in memory**. Once mapped, the file can be read, and in some cases modified with the changes flowing back to disk, by indexing into it as though it were one enormous array. This is exactly how one efficiently accesses a very large configuration file when only specific parts are needed.

The reason this matters for malware is that **Windows itself uses memory mapping to load code.** When a library is loaded, Windows is in effect performing a memory mapping of the library's code into memory. Malware authors have combined `CreateFileMapping` and `MapViewOfFile` — and their Native API counterparts — to implement **custom loading of code that bypasses the Windows loader entirely**. The recipe is not especially difficult: map memory, grant it write and execute permissions, and you have executable code sitting in memory that a security solution may well miss, because it never saw the code being loaded through normal channels. This technique underlies many code-injection variants. The practical takeaway is that `CreateFileMapping` / `MapViewOfFile` can be read as a possible signal of manual code loading, not merely of file access.

Special Files: UNC Paths, Extended Paths, and Namespaces

Windows was designed with interoperability in mind — among workstations and servers, in enterprise settings with many shared files and services — and this shows up in the variety of **special paths** it supports.

The first category is paths that point to resources on **another machine**. You have likely written these in the file manager without thinking about it: two backslashes, the server name, and then the path on that server, as in `\\server\share\file.txt`. These are **UNC paths** (Universal Naming Convention). For malware, encountering a path of this form is meaningful: it may indicate an attempt to **spread across the network** — writing or dumping something onto shared resources on other computers.

The UNC convention also permits a trick for encoding very long paths. Normally a path is limited by `MAX_PATH`, which on Windows is **260 characters**. Prefixing a path with `\\?\` **bypasses normal Win32 path processing**, allowing extended-length paths beyond that 260-character limit, for both local and UNC paths.

Related to these are **namespaces** — special folders within the Windows file system. The root namespace, written `\`, is the lowest-level namespace and contains everything. More dangerous from a security standpoint is the **device namespace**, written `\\.\`, which is used for **direct disk I/O**. Through it, one can obtain direct access to the raw disk — even to its boot sector. In older Windows versions this required only administrator rights, with no special authorization: a program of just three API calls — open the device at a path such as `\\.\PhysicalDisk1`, then write a handful of bytes to the boot sector — could render a machine unbootable, requiring an expert to recover it. This is exactly what the **Witty worm (2004)** did: for sheer destruction, it wrote to `\\.\PhysicalDisk1` to corrupt the disk. Any such path is highly relevant as an indicator of compromise, and during analysis it is a clear sign that something destructive is underway.

Special Files: Alternate Data Streams

A final special-file feature is the **Alternate Data Stream (ADS)**: a second stream of data attached to a filename, kept *in addition to* the file's primary, visible content. You address it by appending a suffix to the filename, using a colon — `File.txt:otherfile.txt`. When you open the file by its ordinary name you see the original contents; the additional stream is reachable only through the suffixed name.

The mechanism can be demonstrated at the command line. Writing `echo 1 > foo` creates a normal file `foo`; a directory listing shows it with its small size. Writing `echo 2222... > foo:bar.txt` adds a *hidden* second stream named `bar.txt` to `foo`. Crucially, a subsequent directory listing of `foo` is **unchanged** — the extra stream does not appear in its size or in the listing — yet the stream is fully retrievable, for example by opening `foo:bar.txt` in Notepad.

This is a feature of the **NTFS file system** specifically. For most practical purposes NTFS is the only Windows file system in use today, but for many years Windows ran on FAT and FAT32; NTFS only became standard with Windows 2000, and installation on FAT32 remained possible until relatively recently. On any NTFS volume, malware can use Alternate Data Streams to conceal additional content from casual inspection.

The Windows Registry

The **Windows Registry** is one of the most important components of the operating system for a malware analyst — and one with no real equivalent on Linux or macOS. You can picture it as a gigantic, hierarchical collection of configuration data in which Windows stores all manner of information: desktop preferences, mouse settings, the list of USB devices ever attached, the list of installed updates, and dozens of other categories. It stores both OS and program configuration settings.

The registry's prominence has historical texture. In years past there was a cottage industry of "registry cleaner" tools built on the belief that a smaller registry made Windows run faster; half the time these tools destroyed the machine instead, trimming keys the system needed. The risk was serious enough that Windows eventually introduced the **System Restore** feature, which could roll back to a previous registry state.

Structurally, the registry is arranged as a hierarchy that maps neatly onto a file-system metaphor. The **root keys** are like top-level drives; **keys** within them are like folders (and are indeed shown as folders in

the Registry Editor); and inside keys are **values**, which are like files whose contents may be numbers, Boolean values, or strings. A value is a name/data pair with an associated type.

The Five Root Keys

The registry is split into exactly **five root keys**. Note the Hungarian notation: `HK` stands for "handle to a key."

- **`HKEY_LOCAL_MACHINE` (HKLM)** — settings that are global to the local machine (system-wide settings for the entire computer).
- **`HKEY_CURRENT_USER` (HKCU)** — settings specific to the current user: their preferences, their applications, their startup programs.
- **`HKEY_CLASSES_ROOT`** — information defining types, such as details of installed components.
- **`HKEY_CURRENT_CONFIG`** — settings about the current hardware configuration, specifically the differences between the current and the standard configuration.
- **`HKEY_USERS`** — settings for the default user, new users, and current users.

A normal user can modify only `HKCU`; touching the machine-wide or OS-configuration keys requires administrator rights, and doing so carelessly can break the system. For malware dissection, the first two keys are by far the most important. `HKEY_CLASSES_ROOT` is worth knowing because malware can do advanced things through it — the COM-object example later in this chapter is one such case.

Example: The Run Key and Persistence

The single registry location you will encounter most often as a malware analyst is:

```
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
```

This key lists executables that start when a user logs on. Opening it reveals a set of value pairs. The `(Default)` value has no name, a type of `REG_SZ` (a string), and typically no data set. Below it is a list of auto-start programs, each a name mapped to an executable path.

A closely related key is **`RunOnce`**, whose entries execute at the next reboot and are then **deleted**. For a program to run again it must either live in `Run` or re-create its `RunOnce` entry each time. The historical rationale for `RunOnce` was the once-common need to finish an installation after a mandatory restart — far less necessary today, as modern software rarely demands a reboot.

Malware uses these keys to achieve **persistence**. The scenario is typical: a malware runs on a machine — perhaps because an inattentive user clicked something — but the user may shut the machine down, so the malware needs a mechanism to run again automatically. The simplest such mechanism is to place itself in a `Run` key: create a value whose name and data point to the malware's path. The entry can be **disguised** to look legitimate. Looking at a real `Run` key — with entries like `Adobe ARM`, `BoxSyncHelper`, `itype` (a Microsoft keyboard/mouse utility), `SoundMan`, and `SunJavaUpdateSched` (the Java auto-updater) — most appear benign, and indeed usually are. But a capable malware author can craft a name that mimics an installed application, changing only a letter or two, such that neither the user nor a cursory review notices. Because defenders also inspect these keys, attackers often go further — abusing a *legitimate* program to load malicious code rather than registering an obviously new executable. (The browser is a classic vehicle for loading an infected library; such techniques are covered later.)

Common Registry Functions

Three Win32 functions dominate registry code. Before querying or setting a value, you must first **open** the key:

- **`RegOpenKeyEx`** — opens a registry key for editing and querying.
- **`RegSetValueEx`** — adds a new value to the registry and sets its data.
- **`RegGetValue`** — returns the data for a value entry in the registry.

The documentation often omits the trailing `W` (wide-char) or `A` (ASCII) character, writing `RegOpenKeyEx` for what is really `RegOpenKeyExW`. This points to two pervasive Windows naming conventions, discussed next.

The `Ex`, `A`, and `W` Suffixes

Windows was not built all at once; it is the product of continuous releases and improvements. Yet the platform prizes **backward compatibility**: if an API changed and software had been built around it, that software would break. Where other systems sometimes forced users to install an old library version or lose an application, Windows took a different approach — when it updates a function in a way incompatible with the old one, it keeps the old function and gives the new one the **same name with an `Ex` suffix** ("extended"). The registry APIs were among the first to need this. Functions updated significantly more than once can even carry two `Ex` suffixes.

The second convention concerns strings. **Every Win32 API that takes or returns a string exists in two versions:** one for **ASCII** strings (suffix `A`) and one for **wide-char** strings (suffix `W`). A wide-char string uses two bytes per character rather than one, allowing non-Latin alphabets. (This predates true Unicode, which arrived much later, but it is Windows' long-standing wide-char mechanism.) The documentation frequently omits the `A` / `W`; functionally the two are the same call, differing only in how you must encode the string you pass.

Concretely, an ASCII string can be turned into a wide-char string by inserting a zero byte after every character. So in memory a wide-char string appears as recognizable ASCII letter values **interleaved with zeros**, terminated by a double-zero. For example, the letter `A` (ASCII `0x41`) becomes the two bytes `41 00`. When you see this interleaved pattern in a memory dump, you are looking at a wide-char string. In disassembly, a wide-char string literal is sometimes marked in C with an `L` before the quotes (to signal "long"), but a disassembler typically just renders it as a plain string.

Reading Registry Code in Disassembly

Consider a sequence that opens the `Run` key and writes a value. The calling convention pushes arguments in reverse order, so the parameters appear *above* the call, pushed from last to first:

```
0040286F  push  2                   ; samDesired
00402871  push  eax                 ; ulOptions
00402872  push  offset SubKey       ;
"Software\Microsoft\Windows\CurrentVersion\Run"
00402877  push  HKEY_LOCAL_MACHINE  ; hKey
0040287C  call  esi                 ; RegOpenKeyExW
0040287E  test  eax, eax
00402880  jnz   short loc_4028C5
```

A crucial subtlety: `RegOpenKeyEx` does **not** return the opened handle as its return value. Its return value is a **status code**, and the handle is written through a pointer passed as the last parameter

(`phkResult`). That is exactly why the code does `test eax, eax` after the call — it is checking whether the *status* is zero (success), not inspecting a handle. Annotating the five parameters of `RegOpenKeyExW` makes the structure clear:

```
lea   ecx, [esp+7E8h+phkResult]
push  ecx                           ; phkResult  -> location for the output handle
push  20006h                        ; samDesired  -> KEY_WRITE (the access level)
push  0                             ; ulOptions
push  offset aSoftwareMicros        ; "Software\Microsoft\Windows\CurrentVersion\Run"
push  HKEY_CURRENT_USER             ; hKey
call  ds:RegOpenKeyExW
```

whose C prototype is:

```
LSTATUS RegOpenKeyExA(
    HKEY   hKey,
    LPCSTR lpSubKey,
    DWORD  ulOptions,
    REGSAM samDesired,
    PHKEY  phkResult
);
```

Reading this efficiently is a matter of looking only at the familiar places. You inspect the **first argument** — a standard, recognizable constant such as `HKEY_CURRENT_USER` or `HKEY_LOCAL_MACHINE` , which the disassembler labels for you. You then look at the **subkey string** pushed as the second argument, since that tells you *which* registry path is targeted. The third (`ulOptions`) and fourth (`samDesired` , the access level — here `KEY_WRITE`) arguments can usually be skipped, though `samDesired` is a low-level detail of what access is requested. The fifth argument (`phkResult`) is the pointer to a buffer where the API writes the handle — the handle you will need for the next call.

Once the key is open, the value is written with `RegSetValueEx` , and the parameters again decode cleanly:

```
mov   ecx, [esp+7E8h+phkResult]
sub   eax, edx
sar   eax, 1
lea   edx, ds:4[eax*4]
push  edx                           ; cbData    -> size of data to be stored
lea   eax, [esp+7ECh+pszPath]
push  eax                           ; lpData    -> pointer to data to be stored
push  REG_SZ                        ; dwType    -> type of data (a string)
push  0                             ; Reserved
push  offset ValueName              ; "System"  -> name of the value
push  ecx                           ; hKey      -> handle to the key
call  ds:RegSetValueExW
...
mov   edx, [esp+7E8h+phkResult]
```

```
push  edx                       ; hKey
call  ds:RegCloseKey
```

The important elements here are the **`hKey`** handle obtained from the earlier open, and the **data being written** (`lpData`) together with its name. For a persistence entry, `lpData` will be a substring or full path pointing to where the malware is installed and must run, and the value name is whatever label the malware chooses. After writing, the handle is passed to **`RegCloseKey`** to release it. The practical reading strategy is the same throughout: identify the key, the targeted path, and the data being written, and skip the low-level parameters that do not change the behavioral story.

The Networking APIs

Windows exposes several networking layers; two are relevant here, at two different levels of abstraction.

Winsock: The Berkeley-Compatible Socket API

The lower-level option is **Winsock**, Windows' implementation of **Berkeley-compatible sockets**. It lives primarily in the library **`ws2_32.dll`** , and it is almost identical to Unix sockets — if you know those, you know this. Being low-level, it gives you direct control over every byte you read and write. The core functions are:

Function	Description
`socket`	Creates a socket
`bind`	Attaches a socket to a particular port, prior to the `accept` call
`listen`	Indicates that a socket will be listening for incoming connections
`accept`	Opens a connection to a remote socket and accepts the connection
`connect`	Opens a connection to a remote socket; the remote socket must be waiting for the connection
`recv`	Receives data from the remote socket
`send`	Sends data to the remote socket

One important initialization detail distinguishes Windows from Unix: the **`WSAStartup`** function must be called before any other networking function, to allocate resources for the networking libraries. When debugging, setting a breakpoint on `WSAStartup` is useful, because the start of networking activity should follow shortly after.

Server and Client Sides

The distinction between the function sequences comes down to a single question: **do you initiate the connection, or wait for one?**

A program on the **server side** maintains an open socket waiting for connections. It calls, in order, `socket` , `bind` , `listen` , `accept` — the `listen` / `accept` pair meaning "I am waiting, and someone will contact me" — and then uses `send` and `recv` as necessary.

A program on the **client side** connects to a waiting socket. It calls, in order, `socket` , `connect` — `connect` meaning "I am attempting to reach someone" — and then likewise uses `send` and `recv` .

For malware, the presence of `accept`, `connect`, `recv`, and `send` suggests the sample communicates with a **command-and-control server**. It might be a **remote access trojan** that receives commands a few bytes at a time, parses each command, and performs the corresponding functionality.

The WinINet API

The higher-level option is the **WinINet API**, whose functions live in `Wininet.dll`. It sits above Winsock and implements application-layer protocols such as **HTTP and FTP** directly, so you need not deal with raw bytes. Its characteristic functions are:

- `InternetOpen` — initializes use of the API.
- `InternetOpenUrl` — connects to a URL.
- `InternetReadFile` — reads data from a URL.

This is ideal for "quick and dirty" tasks: a sample can open and download a file in a single high-level call. Many simple malware families use exactly this — connecting to a website, downloading a second-stage component, and running it — without the complexity of raw sockets. A dedicated helper such as `URLDownloadToFile` does the whole job in one call. Finding these functions is a strong sign of interesting network activity in a sample. As always, you do not need to be able to write the code — only to grasp what the sample is doing.

Execution Mechanisms

We usually imagine a program as a single entity that does all its work by itself. In reality, while `jmp` and `call` transfer control *within* the current thread, Windows provides a range of mechanisms to execute code in **other threads or processes** and to coordinate that execution. Just as legitimate software uses these mechanisms, malware uses them to orchestrate its own execution — spreading work across multiple threads and processes, loading libraries, and coordinating through inter-process communication objects. The mechanisms covered here are **DLLs, processes, threads, mutexes, services, the Component Object Model (COM), and exceptions.**

Dynamic-Link Libraries (DLLs)

DLLs enable **code to be shared** and reused by other applications. Using the DLLs that ship with Windows keeps a program's own code smaller, and software companies can likewise ship their own custom DLLs to share components across a product suite. Malware authors use DLLs in several ways — three in particular:

Using Windows DLLs to interact with the OS. This is the obvious, unavoidable use: calling Windows functionality through its libraries.

Storing malicious code in a DLL. Why split the malicious logic out of the main executable? At least two reasons. First, a security solution may focus only on the delivered executable; if that executable looks unremarkable and merely drops a couple of DLLs on disk, a weaker solution may not scrutinize those DLLs — where the real malicious code waits as "step two." (A good solution inspects them.) Second, Windows malware rarely does everything in one shot; splitting functionality into a DLL that is then loaded by another application enables multi-stage execution. There are powerful tricks here around **load paths**: when a program requests a DLL without specifying a full path, Windows runs a **path-resolution (search-order) mechanism**, and an attacker who places a malicious DLL with the same name as a legitimate one, earlier in the search order, gets the malicious version loaded instead — **DLL search-order hijacking**. Even more directly, a process can be made to load a DLL with a **single API call**: one

process can ask another to load a given DLL. This capability exists because it was introduced to make software flexible and interoperable — the machinery behind, for example, embedding an Excel table in a PowerPoint slide and editing it in place relies on many layers of shared code. This is the recurring tension between the system architect, who devises ten convenient, easy-to-use features, and the security researcher, who turns one of them into a weapon by exploiting an unanticipated corner case. Forcing another process to load a DLL — in any of its many covert forms — is one of the major vectors for infecting machines.

Using third-party DLLs. Rather than using Windows' own communication APIs, a sample can borrow the DLLs of a browser or other trusted application and use *that* program's implementation to send data over the network (for example, using a Firefox DLL to connect to a server instead of the Windows API). A security solution tends to be less suspicious of a digitally signed, long-trusted application, lowering its guard and failing to track closely what that borrowed code is doing.

Basic DLL Structure

A DLL is **very similar to an EXE** and shares the same **PE (Portable Executable) file format** — the same sections, exports, and imports. In fact, **a single flag (one bit) in the PE header** discriminates between a DLL and an EXE; with a hex editor you can flip it and even force a DLL to run standalone.

Two characteristics distinguish DLLs from EXEs in practice. First, the **exports/imports ratio is reversed**: a legitimate DLL has **many exports** (the functions it shares — a normal system DLL may export three or four hundred) and **few imports**, whereas an executable imports heavily (a large program may import hundreds of functions from many libraries) and typically exports nothing. Second, the **entry point** is not a standard `main`. A DLL's entry point is a function conventionally called **`DllMain`** — `DllMain` is not exported, but it is specified as the entry point in the PE header.

`DllMain` is a short callback — perhaps twenty lines of assembly or fewer — that is invoked on **four events**: when the library is **loaded** by a program, when it is **unloaded** (often because the program is exiting), when a **thread is created** in the program, and when a **thread terminates**. It receives a reason parameter (`dwReason`) whose value — 0, 1, 2, or 3 — tells it which of these events occurred, so the DLL can, for instance, perform one-time initialization of its internal structures on first load.

Malware uses this differently: it can register behavior to fire on *every* thread creation, routing hijacking or tampering through `DllMain`. The analytic consequence is that **when you analyze a DLL, you examine `DllMain` first**, to see which code runs there. That tells you whether the malware has automated features triggered by these load/thread events, or whether it is instead a library meant to be invoked through specific exports.

Processes, Threads, and Exceptions

Processes

A **process** is the box or container that lets a program run — it holds the program's resources, such as handles and memory. But a process does not itself execute code; the actual code is run by the CPU on execution units called **threads**. A program being executed by Windows is a process, and each process has its own isolated resources.

The threads of a process **coexist in the application's address space**, sharing its memory and resources. This sharing is the source of concurrency hazards: one thread must not ruin the work of another, so the usual synchronization discipline applies. The security angle is that a process can run

entirely on its own, or it can have **some of its code execute inside another process**. Malware — and legitimate software too — can ask another process to start a thread, supplying that process with the code to run. This is a technique to be abused, so it is important to watch which threads a sample creates: `CreateThread` for threads created locally within the application, and `CreateRemoteThread` for threads created in *other* applications.

The first thread of a process is created automatically when the process starts: the loader loads the executable, finds the start function (the entry point), and the initial thread begins execution there.

Creating a New Process

A new process is created with **`CreateProcess`**. In its ASCII form the prototype takes many parameters, most of which can be passed as `0`:

```
BOOL CreateProcessA(
    LPCSTR                lpApplicationName,
    LPSTR                 lpCommandLine,
    LPSECURITY_ATTRIBUTES lpProcessAttributes,
    LPSECURITY_ATTRIBUTES lpThreadAttributes,
    BOOL                  bInheritHandles,
    DWORD                 dwCreationFlags,
    LPVOID                lpEnvironment,
    LPCSTR                lpCurrentDirectory,
    LPSTARTUPINFOA        lpStartupInfo,
    LPPROCESS_INFORMATION lpProcessInformation
);
```

The parameter that matters most is **`lpCommandLine`**, which contains the command line to execute — *what* program to run, *where* the file is, and any arguments. The `dwCreationFlags` parameter can be important in other contexts but can be treated as `0` for now.

The more dangerous detail is the **`STARTUPINFO`** parameter. It contains handles for the standard input, standard output, and standard error streams — and these handles **could be set to a socket**. This is the classic construction of a **remote shell**: a process is created whose input and output are wired directly to a socket, exchanging bytes with a remote attacker. Most of the time `STARTUPINFO` is used benignly, simply directing where a child process's input, output, and error streams go — but the ability to redirect them to a socket is what makes `CreateProcess` plus a crafted `STARTUPINFO` a tell for a remote-shell capability.

Threads

Returning to threads: multiple threads live in the same process box and share almost everything. **What they do not share is the context, and (normally) the stack.**

Each thread normally has its **own stack**, because otherwise calling a function in one thread would overwrite the stack memory of a function in another thread — the threads would trample each other. With separate stacks they stay out of one another's way.

The subtler point concerns the **registers**. Registers are thread-specific in effect, but the CPU does not have separate physical register banks per thread — there is only one set of registers. So every time the operating system schedules a thread to run and later stops it, it must **save that thread's registers**

somewhere and restore another thread's. That saved register state is the thread's **context**: the register state, including the instruction pointer and the stack pointer. The OS saves and restores each thread's CPU context in a dedicated structure when switching between threads on a CPU core. A thread is created with `CreateThread` to run from a designated start function, and the OS schedules it independently; with multiple CPUs, multiple threads may execute in parallel.

Because context is just data that can be read and written, it is a powerful abuse surface. From software you can **query a thread's current state, suspend the thread, obtain its context, and change its registers** — this is one way to **hijack a thread**: the thread is running some code; you suspend it, change its instruction pointer, and resume it, so it continues executing code of your choosing. Consequently, you should look carefully at where a sample uses threads, watching for the family of thread-specific APIs — `CreateThread`, `ResumeThread`, `GetThreadContext`, `SetThreadContext`, and the like. In analysis these are consistently informative.

Exceptions

Software is designed to run until it fails. It may fail for the classic reasons — a null-pointer dereference, access to invalid memory, a division by zero — or a program may detect a broken condition in its own code and **explicitly raise an exception** (much as one might call `abort` or throw from a constructor).

Windows provides mechanisms to handle exceptions; the common one is **Structured Exception Handling (SEH)**. In code you recognize SEH by instructions that access the special segment register `FS`. `FS` is a register like `EAX` or `EBX`, but in user mode it can only be read, and it is used to access **thread-specific data**. You will see sequences such as:

```
01006170  push   offset loc_10061C0     ; (1) the handler to install
01006175  mov    eax, large fs:0        ; read current SEH chain head
0100617B  push   eax                    ; (2) link to previous record
0100617C  mov    large fs:0, esp        ; make this record the new head
```

Here `fs:0` means reading at **offset zero** from the memory pointed to by `FS`. At that offset you find an address that leads to a **Structured Exception Handling record** on the stack. The mechanism works as follows: whenever a hardware trap occurs — a fault such as division by zero, or an explicit interrupt such as a breakpoint — Windows invokes the structured exception handler. Crucially, this is a **chain**: there may be multiple SEH records linked together, each offering a handler for a particular kind of error. When an exception occurs, Windows uses `FS:[0]` to find the first SEH registration record on the stack and then **traverses the chain** looking for a handler suitable for the error at hand. If a suitable handler is found, control transfers to it; if none is found, the program is terminated. It is directly analogous to `try` / `catch`: if there is a `catch` for the exception type, it handles it; otherwise the program dies.

A second, related mechanism exists — **Vectored Exception Handling (VEH)** — with the same goals and principles but a somewhat different implementation. The conceptual model is identical.

Exceptions are especially relevant in malware because they can be used as an **anti-debugging** technique. A sample can deliberately raise an exception. Under normal programming, a debugger is exactly where a developer would intercept an exception to investigate the error. But malware *expects* not to be running under a debugger: it raises an exception, and if that exception is caught (by the debugger), the malware infers that it is being debugged. A skilled analyst therefore recognizes that when the debugger reports such an exception, the right move is often to **ignore it and let it pass through** — the malware will have registered its own SEH handler to deal with that exception during normal execution. These lookup

sequences are thus worth recognizing during static analysis, even though the evasive behavior fully manifests only dynamically.

Mutexes

Mutexes are synchronization objects used to coordinate access to shared resources. Because they are **inter-process communication (IPC) objects**, they can be shared across threads *and* processes for coordination. The basic principle is familiar from any concurrency course: a mutex is a resource you can lock — you acquire it, hold it for a while, then release it, while any other entity that tries to acquire it must wait. This is the classic producer/consumer coordination, usable within one process or across processes sharing memory.

With malware, however, mutexes are often used for a different purpose: to ensure that **only one instance of the malware runs on a machine**. Imagine an impatient user who double-clicks the malicious executable twenty times waiting for something to appear; twenty concurrent instances would only interfere with one another and might prevent the malware from cleanly compromising the machine. The fix is for the malware to check whether a particular mutex already exists — if it does, another instance is already running, and this one can exit.

A key practical detail: **mutexes can be named**, and such names are **often hard-coded** — which makes them an excellent source of IoCs for identifying a malware family. (Mutexes can also be anonymous, but the interesting case for analysis is the named one.) The relevant functions are:

- `CreateMutex` — creates a mutex, or opens an existing named mutex.
- `OpenMutex` — opens an existing named mutex.
- `WaitForSingleObject` — waits to acquire ownership of a mutex (a blocking call); any subsequent threads attempting access must wait.
- `ReleaseMutex` — called when a thread is done using the mutex.

Note that the `WaitForSingleObject` / `ReleaseMutex` *locking* pattern is not typically what malware uses mutexes for — the single-instance check is.

The Malware Single-Instance Check

Consider a sample that checks for a mutex named `HGL345` — a peculiar, distinctive name. If the mutex already exists, another instance is running and the malware exits; if not, it creates the mutex and continues. In disassembly:

```
00401007  push 1F0001h             ; dwDesiredAccess
0040100C  call ds:__imp__OpenMutexW@12   ; OpenMutexW(x,x,x)
00401012  test eax, eax
00401014  jz   short loc_40101E
00401016  push 0                   ; int
00401018  call ds:__imp__exit      ; another instance exists -> exit
0040101E  push offset Name         ; "HGL345"
00401023  push 0                   ; bInitialOwner
00401025  push 0                   ; lpMutexAttributes
00401027  call ds:__imp__CreateMutexW@12 ; CreateMutexW(x,x,x)
```

The logic reads cleanly: `OpenMutexW` tries to open `HGL345`; `test eax, eax` sets the zero flag if `eax` is zero (meaning the open failed and the mutex does *not* exist); `jz` therefore branches forward to the creation path when there is no existing instance. If the open *succeeds* (a non-zero handle), the mutex already exists, so the fall-through path calls `exit`. Otherwise `CreateMutexW` creates the mutex and execution proceeds.

For the analyst, the recipe is: spot `CreateMutex` or `OpenMutex`, look nearby for an `ExitProcess`, `exit`, or an infinite sleep (`Sleep(INFINITE)`), and infer a likely single-instance behavior. You then note the mutex name in your report as an indicator of compromise for that family.

Services

Services are a Windows mechanism with no exact Linux equivalent (Linux has daemons managed by init/systemd and similar). Generally, services are **background programs that run without user interaction** — anything from an update checker to routine maintenance. They do not need a logged-in user.

Services are managed by the **Service Control Manager (SCM)**. A crucial property is that **services can run independently of user logon** — they can start automatically when Windows boots and operate even while the machine is sitting at the login prompt with nobody signed in. Scheduling is flexible, and services can run under different security policies.

This is where services become security-relevant. Each service runs **under a configured security account**, and some run as `SYSTEM` (LocalSystem) — a special account, comparable to `root`, with extensive local privileges and access to many protected resources, in some respects *more* powerful than an ordinary administrator. An administrator attempting to touch certain paths or resources may be prompted for confirmation or refused outright; a service registered as needing high privileges is simply entrusted to perform such actions. Other services run under restricted service accounts or specific user accounts, and creating or configuring a service requires appropriate privileges.

For malware, this yields a potent alternative to registry-based auto-run: **install a service to establish persistence**, potentially **with elevated privileges**. If done right — perhaps by abusing another application — the malware can register a service that runs effectively as `SYSTEM`. This is also an alternative to defeating **UAC** (User Account Control), the elevation prompt Windows shows to authorize privileged actions. UAC bypasses exist and work reasonably well on some versions, but they are detectable and version-dependent; abusing a service is often a cleaner route to privilege.

Service API Functions

Service manipulation shows up through a recognizable API sequence:

- **`OpenSCManager`** — returns a handle to the Service Control Manager.
- **`CreateService`** — adds a new service to the SCM, and can specify whether the service starts automatically at boot time.
- **`StartService`** — starts a stopped service (for example, one configured to start manually).

Seeing `OpenSCManager` in particular should immediately direct your attention to services. A second-stage payload running after an earlier component may call `StartService` to launch a service that other malware components previously created. Static analysis of these calls is well complemented by dynamic tools that show services being registered in real time.

svchost.exe

One service-hosting arrangement was especially beloved by malware authors. The parameter `dwServiceType` passed to `CreateService` determines the `SERVICE_` type:

- **`WIN32_OWN_PROCESS` (0x10)** — the service runs in its own dedicated process.
- **`WIN32_SHARE_PROCESS` (0x20)** — the service can share a process with other services.

Windows lets multiple services run together, rather like threads within a single process. Historically almost all services ran under one process, and that host executable is **`svchost.exe`** — a Windows service-host executable that loads and runs services implemented as **DLLs**. Today there are multiple `svchost.exe` instances, and different services may run within the same instance or in separate ones. The critical point is that **the actual service code resides in a DLL, not in `svchost.exe` itself.**

Two facts make this dangerous. First, `svchost.exe` was intended for Windows' own maintenance services, not for applications — but its implementation also allows loading arbitrary DLLs, and malware authors exploit this. Second, because `svchost.exe` is *always* running, nobody glancing at Task Manager finds its presence suspicious. So malware creates a **malicious DLL and loads it into `svchost.exe` as a shared service**, blending in. To detect this, an analyst must open the service configuration and inspect its **`ServiceDll`** entry to identify the DLL actually being executed, then open that DLL in a disassembler or security solution. While `svchost.exe` is intended for Windows services, that does not prevent malware from disguising malicious activity through it.

Service Information in the Registry

Every created service is also reflected in the registry, under:

```
HKLM\System\CurrentControlSet\Services\<ServiceName>
```

The relevant values decode as follows: a `Start` value of `0x03` means "Load on Demand"; a `Type` value of `0x20` means `WIN32_SHARE_PROCESS` (a shared service); and `ImagePath` gives the executable or driver path. In the example of the `Browser` service, `ImagePath` is `%SystemRoot%\System32\svchost.exe -k netsvcs` (confirming it runs under a shared `svchost.exe` group), `Start` is `0x00000003`, `Type` is `0x00000020`, and `ObjectName` is `LocalSystem`. Reading these registry values is therefore a static way to understand how a service is configured to run, who it runs as, and what code it ultimately executes.

The Component Object Model (COM)

The **Component Object Model (COM)** is the final execution mechanism, and it is a classic means of sharing functionality — but it is **more structured than plain DLLs**. Where DLLs share essentially *raw code*, COM lets software components expose functionality through **standardized interfaces**, including **across process boundaries**. You may have encountered COM under the older name **OLE**, typical of Office interoperability (the machinery behind embedding and editing one application's content inside another).

Any thread that intends to use COM must first **initialize the COM library**, generally by calling **`OleInitialize`** or **`CoInitializeEx`**. Seeing one of these is the signal that a sample is entering the COM world.

Identifiers: GUIDs, CLSIDs, and IIDs

From the operating system's perspective, every COM class and interface is identified and accessed via a **Globally Unique Identifier (GUID)** — a machine-wide unique identifier that can be enumerated to discover what capabilities exist. These identifiers divide into two main types:

- **Class Identifiers (CLSIDs)** identify classes of objects, and are registered in the registry under `HKEY_CLASSES_ROOT\CLSID`.
- **Interface Identifiers (IIDs)** identify the interfaces that objects expose, and are registered under `HKEY_CLASSES_ROOT\Interface`.

COM Object Activation

An application instantiates a COM object with `CoCreateInstance`, specifying a **CLSID** (which class) and a requested **IID** (which interface). Windows then uses the **CLSID registration** to locate the corresponding implementation. Two registration subkeys determine *how* the implementation is loaded:

- `InprocServer32` → a **DLL loaded into the calling process**.
- `LocalServer32` → an **EXE running in a separate process**.

Malware Relevance and COM Hijacking

Two abuses matter. First, malware can simply use COM objects to **access Windows functionality** — reaching capabilities through the COM framework rather than through direct API calls.

Second, and more significantly, there is **COM hijacking**: malicious modification of COM registrations that causes legitimate applications to load **attacker-controlled code**. The deeper opportunity lies in **privilege-boundary abuse**. A COM object may be entitled to perform elevated actions that the calling application is not — and there is often no clear mechanism prohibiting the caller from exercising those elevated privileges. The security checks are typically written only for the *expected* usage patterns: an object advertises that it offers certain features, but nothing prevents a caller from using *other* features present in the same implementation. By modifying how a COM object is registered, an attacker obtains a wealth of what are best described as **escalation primitives** — without having to defeat UAC or register a privileged service. All that is needed is a vulnerable COM object, and Windows contains a great many that have not all been discovered yet. This makes COM a rich, and still under-studied, domain for both offense and defense.

This marks the end of the material covered in the lecture. The master deck continues with a series of **assembly patterns** — global vs. local variables, `switch` constructs, arrays, and structs — but these were **explicitly deferred to the hands-on lab sessions** rather than treated in this lecture; the lecturer set them aside as better suited to lab work than to the theory lesson. They complete the picture of what Windows offers for static analysis and will be handled there.

INCIDENT FORENSICS part

CHAPTER IF_1: Attack Killchain

The individual cyber-attacks observed over the last decade differ from one another in countless technical details, yet they share a structural resemblance that makes them worth studying as a single family. No defender can realistically hope to design a bespoke countermeasure for every conceivable attack, because each attack is, in its particulars, unique. What is achievable, and far more valuable, is a *methodology*: a

disciplined way of decomposing attacks into their constituent stages so that the common points become visible and can be defended systematically. Such a methodology does not promise protection against every possible threat. It leaves open space for highly targeted or technically extraordinary attacks that will always exist at the margins. This residual risk should not be read as a failure of the approach, because the overwhelming majority of attacks encountered in daily practice rely on common, repeatable techniques. Being able to recognise and block the vast majority of common attacks is already an extremely important result, and it is the result this methodology is designed to deliver.

Before introducing the model itself, it is essential to characterise precisely the kind of attack it is meant to describe. Three traits define the attacks that matter most today.

The Profile of a Modern Dangerous Attack

Precise Targeting

The attacks of primary interest are those directed at a specific, deliberately chosen target rather than scattered indiscriminately across the Internet. Mass attacks, once the norm, have become comparatively rare. In the early ransomware era the dominant strategy was to spray malware across the network in the hope of infecting as many personal computers as possible, on the reasoning that if ten million machines were compromised and even one percent of their owners paid the ransom, the campaign would be profitable. This indiscriminate approach proved far less effective than expected, and attackers — ransomware operators included — shifted toward precise targeting. A widespread, uncontrolled attack with no clear victim still exists, but it has become unusual. Precise targeting is therefore the first defining characteristic of the attacks examined here.

Full Control of the Target

Most dangerous attacks today aim to grant the attacker *full control* of the compromised machine. This marks a departure from older attacks, in which all the logic needed to carry out a well-defined, pre-decided action was baked into the malware itself. Such self-contained attacks still exist — malware crafted for one specific machine or one narrow class of machines — but they are now relatively unusual and tend to be reserved for targets with specialised hardware, such as IoT devices or industrial control systems. For the vast majority of targets, which are ordinary personal computers and servers, obtaining full control is a central goal. The attacker wants the freedom to decide *later* what to do with the machine. This matters because, at the moment the attacker first breaches a system's defences, they do not yet know what lies beyond them. They compromise an initial machine and only afterward discover what is visible and what can be done from that new vantage point. The ability to load new modules, extend the capabilities of the infected machine, and freely execute commands from within is therefore fundamental to the modern attacker's way of operating. The technical embodiment of this trait is the Remote Access Trojan (RAT), which combines remote access with privilege escalation and supports the multi-step compromises that characterise serious intrusions.

Long-Term Persistence

The third common trait is long-term persistence. Historically this was associated with so-called Advanced Persistent Threats (APTs) — groups of attackers, frequently state-sponsored, acting from political motivations. Today persistence is sought not only by those groups but also by purely financially motivated criminals. Persistence is valuable because it lets the attacker fully exploit the compromised asset over time: having obtained full control, the attacker can steal information, use the machine as a staging point to infect other machines, and — once the original objectives are met — even sell the access to another party who may find further uses for the compromised asset. The longer the attacker can

remain inside, and the more complete their control, the more freedom they enjoy, and the easier it becomes to achieve a strong return on their initial investment.

Reading the Three Traits from the Defender's Side

Each of these characteristics shapes how the defender must study and respond to the attacker, and they do not all cut in the same direction.

Precise targeting works in the defender's favour. An attacker who selects targets at random is almost impossible to characterise or profile: there is no motive to understand and no way to anticipate the next move, because the victim was chosen by the roll of a die. When an attacker targets a specific organisation, by contrast, that choice implies a motivation bound to that organisation. Understanding the nature of that motivation reveals which of the defender's own assets hold value for the attacker — and therefore what must be defended most carefully. This is precisely the concern of *cyber threat intelligence*: to defend against an adversary, one must know who they are, which assets they can bring to bear, and how they wield them, so that a defensive strategy can be made as effective as possible.

Full control works against the defender. Knowing that the attacker will try to seize the maximum level of control over a compromised machine or network means that detecting their presence will be difficult. To take full control of, say, a personal computer, the attacker must overcome every defensive measure in place, and in particular the limitations imposed by access control. Even on a machine where ordinary employees operate with restricted administrative privileges, the attacker may succeed in *escalating* those privileges to obtain full control. Once operating as a full administrator, the attacker can hide their presence with relative ease. Full control of the target therefore implies that identifying a compromised machine becomes harder.

Long-term persistence also works against the defender. An attacker intent on remaining inside the organisation's assets for as long as possible will do their utmost to protect their foothold. This means concealing their presence, but it also means engineering a "plan B." If the defender detects the intrusion and expels the attacker — by disconnecting or resetting the compromised machine — the attacker's entire investment is lost. To guard against this, a persistence-seeking attacker will typically install backdoors or other mechanisms to regain control of a compromised asset *even after* their presence has been identified and seemingly removed. Such mechanisms are almost always present; sometimes they are more visible, sometimes less, but the attacker who values persistence almost always leaves a way back in.

The Rational, Cost-Constrained Attacker

Underlying this entire analysis is a single, powerful assumption: the attacker behaves *rationally*. The attacker is making an investment — spending money, time, and effort, and placing their reputation and freedom at risk — in order to obtain a return. For the venture to make sense, the expected return must exceed the cost of the initial investment. Crucially, the attacker is also *cost-constrained*: they do not command infinite resources and cannot spend without limit. There is a ceiling on the investment they are willing to commit against a given target. Faced with such constraints, a rational attacker acts to maximise return relative to the resources invested, exactly as any disciplined actor would: a plan is made, money is committed, and it is committed only because the return is expected to outweigh it.

This assumption is the lever on which the entire defensive methodology rests. The attacker's rationality originates in their motivation and propagates through every subsequent stage. A commercially motivated ransomware actor with modest capabilities will not attempt to breach a state-of-the-art, heavily defended target; they will prefer a softer target where the probability of success — and therefore of reward — is higher relative to cost. An attacker motivated by politics is a different matter entirely: a state-aligned

actor seeking, for instance, to undermine confidence in another country's ability to defend its IT assets may command an enormous budget and can afford to build the best possible attack. The scale of resources available thus changes, completely, the decisions the attacker makes at every stage of the model. For the defender, the practical consequence of the rationality assumption is simple and far-reaching: *anything that raises the attacker's cost or lowers their probability of success is a legitimate and effective defensive act*, because it degrades the very cost-effectiveness calculation on which the attacker depends.

The Cyber Kill Chain

To study how an attack unfolds, and to find opportunities to disrupt it, the activity of the attacker is described through a well-defined model. A shared model is valuable precisely because it exposes commonalities: when different attacks are described in the same terms, it becomes far easier to recognise what the corresponding phases of distinct attacks have in common. The most famous such model is the **Cyber Kill Chain**.

The Cyber Kill Chain was introduced more than a decade ago by Lockheed Martin, a US corporation active principally in the defence sector, which adapted the "kill chain" concept long used in military planning to the field of cyber security. Since then it has become the de facto standard for representing, at a basic level, the progression of an attacker. The model is deliberately straightforward: it divides the attacker's activity into **seven phases**, arranged on a logical timeline. There are no sharp time boundaries between the phases; the idea is simply that, over the course of an attack, the attacker passes through these phases in order to fulfil their original goals.



Like any model, the kill chain is not perfect. It is not formally specified, which leaves gaps and cases where its application is slightly ambiguous. This should not be regarded as an obstacle. The questions posed in cyber threat intelligence rarely have a single precise answer; the aim is to raise the right questions and obtain answers that are *credible to a certain degree of confidence*, while remaining ready to revise those answers if later evidence contradicts them. Judged against that standard, the kill chain is quite effective.

One further caveat is in order. The original paper states that during an attack the attacker goes through all seven phases. This is not strictly true: some attacks may skip phases, a point especially relevant to attacks built heavily on social engineering. Even so, with a sufficiently flexible reading, social-engineering attacks can be made to fit the model. Two examples make the point.

Consider the classic ransomware attack. An employee inside a company receives an email, opens the attachment, and the ransomware encrypts the machine and displays its ransom note. This maps cleanly onto the model: there is a **delivery** phase (the malware arrives as an attachment to a — probably phishing — email), an **exploitation** phase (the malware installs itself, possibly using a privilege-escalation technique to gain additional rights and encrypt the machine's contents), and an **action on objectives** (the contents are encrypted and the ransom note shown). This is exactly the kind of attack the model was born to represent.

Now consider an attack like the recent fraud against a bank, in which the chief executive was induced to execute bank transfers on the basis of a WhatsApp message and a falsified phone call from someone impersonating a trusted lawyer. At first glance this does not fit. Yet with flexibility it does: here the **delivery** occurs through instant messaging or an ordinary phone line; the **exploitation** is not technical but social, exploiting the executive's inability to distinguish his friend's real voice from a falsified version

of it. It is still an exploitation — not of a vulnerability in code, but of a vulnerability in a human being. Approached with an open mind, the model can represent even these peculiar attacks.

Phase 0: Motivation and Requirements

Before Phase 1 there is a hidden, non-explicit **Phase 0** in which the attacker defines the requirements of the process they are about to carry out. The analogy is to building software: constructing an application requires many steps, but before even the first requirements interview there must be an initial *willingness to answer a question*, where the answer takes the form of a working system. Here, the answer to the question is the construction and execution of an attack kill chain, and the original question is intimately bound to the attacker's **motivation**.

That motivation may be to encrypt the target and demand a ransom, to exfiltrate strategically important information, to steal secret industrial knowledge, or merely to seize control of a machine in order to use it as a stepping stone to attack someone else. These motivations are *not* part of the kill chain. The model assumes the attacker already has a motivation and describes only what happens after that motivation crystallises into the decision to attack a target. Phase 0 is nonetheless where the attacker's rationality takes root: the cost-effective behaviour that runs through every later phase ultimately stems from the original intention and the resources that intention justifies.

Phase 1: Reconnaissance

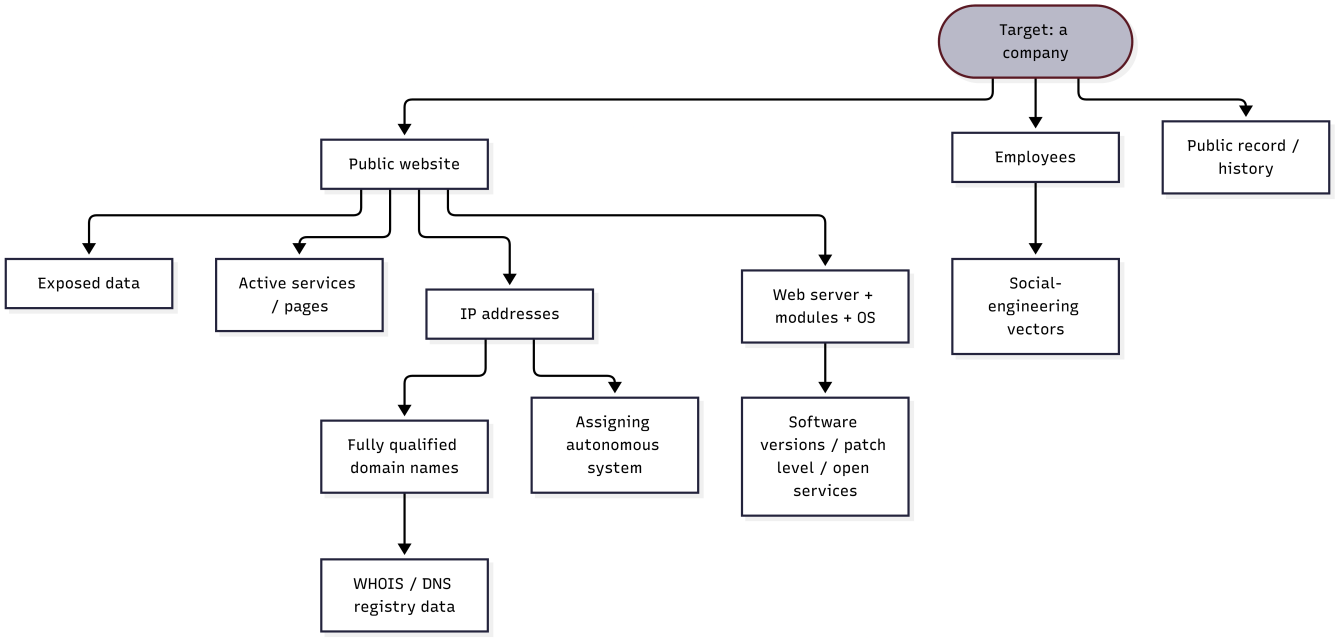
At the outset of the first phase, the attacker is assumed to possess nothing more than an Internet connection and a decision: they want to attack a specific target in order to achieve a well-defined goal. From this minimal starting position the attacker must now *plan* the attack — deciding what is needed to turn an intention into a real operation with a realistic chance of impact. Reconnaissance is the phase in which this planning is grounded in knowledge of the victim, and it is a widely underrated phase, even though the quality of everything that follows depends on it.

The Target Surface

The first thing the attacker does is study the target in order to define its **target surface**. The target surface is an abstract concept: it is the set of all assets belonging to the target that can be reached, directly or indirectly, from the attacker's standpoint through the kinds of connections available to interact with the target. It is best understood not as a single object but as an expanding web of interconnected assets, each of which may reveal further assets.

Consider a company as the target. The most obvious element on its target surface is its **website**, publicly available on the Internet and directly reachable from the attacker's position. But a website is not a single thing; it decomposes into many assets. There are the **data** exposed through its pages. If the site contains active pages offering services, those **services** are themselves part of the target surface. The website is implemented on IT assets that are connected to the Internet through **IP addresses**, which are in turn mapped to **fully qualified domain names (FQDNs)** — and both the IP addresses and the domain names are part of the surface. Each of these assets opens the door to further information. A domain name was registered by someone, and DNS registries have traditionally been open, so querying the WHOIS system can yield information including details of the individuals who registered the domains; this is precisely why much of this information has in recent years begun to be *redacted*, requiring appropriate authorisation to access. Likewise, an IP address was assigned by someone, and whether static or dynamic makes little practical difference: one can follow the trail — through the autonomous system that announces it, for instance — to harvest further information.

The website also runs on a **web server**, a piece of software executing on an operating system. Techniques exist to fingerprint which specific web server is in use, which modules are configured alongside it, the version of each software component, whether those components have been patched, which operating system the machine runs, and whether other services are exposed on it. The amount of information that can be collected in this way is genuinely impressive, and this only scratches the surface, because a company's total exposure is typically very large. Even the **employees** themselves are part of the target surface: although obtaining a list of a company's staff is not trivial, harvesting data from social networks, scraping professional networking sites, or reading open job postings can yield at least a partial roster. Employees belong to the target surface because they can be attacked through social-engineering approaches. Finally, there is the accumulated public record — newspaper articles about past events, and the publicly documented history of an established company — which frequently contains precious information.



Public Data and the Transparency–Security Tension

A particularly instructive class of reconnaissance sources is data that organisations are *legally obliged* to publish. In several jurisdictions, transparency regulations place enormous quantities of sensitive information in the public domain. A striking example is that of a national anti-corruption authority that collects daily records of every contract signed by any public or private entity, together with other parties. The volume of highly sensitive data in such a database is vast, and — most strikingly — a large part of it is publicly queryable precisely because of transparency rules. One can, for instance, retrieve all the contracts a given organisation signed with its counterparts over the last year. The only systematically redacted elements tend to be personal information about the individuals who executed the contracts, and material tied to ongoing judicial proceedings; the remainder is openly available. A rough analogue in the United States is the Securities and Exchange Commission, but with a crucial difference: the SEC does not place a comparable proportion of this information openly on the Internet. Proportionally, the data made public under some European transparency regimes can be substantially larger than what is made public elsewhere — so much so that the data has been repurposed for ends entirely unrelated to its original purpose, such as a doctoral study deriving the carbon footprint of companies from the kind and volume of contracts they signed.

This illustrates a genuine and difficult tension: **transparency and security generally act in opposite directions**, and striking the right balance between them is hard. For decades of relatively

peaceful globalisation it was reasonable to assume that most parties could be trusted so long as a contract linked them; as the geopolitical situation shifts, the threshold between openness and protection may deserve to be reconsidered.

A second example in the same vein, more directly technical, concerns the procurement process of public administrations. For transparency reasons, the technical documents produced when an administration issues a call to acquire a service or system are often published. These documents necessarily describe not only *what* is being bought but also how the acquired system must integrate with what already exists — and in doing so they frequently include sensitive details about internal IT assets. There are cases where such documents effectively contain full maps of an administration's internal network. This information is not secret in itself, but it is the kind of information that, once exposed, dramatically simplifies the attacker's work.

Security by Obscurity, Revisited

These examples underline the real take-home message of the reconnaissance phase. A well-known maxim of the field states that **security by obscurity does not work**: one cannot make an asset secure merely by hiding it, and this is the very foundation of cryptography, which is entirely convincing as the right way to protect information. However, this maxim does *not* imply that hiding things is useless. The correct reading is precise: you cannot *rely* on hiding an asset to protect it, but hiding still confers real advantages. Hiding an asset means, in effect, removing it from the target surface, or at least making it harder for the attacker to define that surface accurately.

One might object that hiding an asset does not prevent a determined attacker from eventually finding it; it only delays them. This is true — and it is exactly the point, given the rationality assumption. Because the attacker chooses their actions on a cost-effectiveness basis, anything that makes their work harder plays directly on that calculus: it increases the cost of building an effective plan and lowers the expected return. Obscurity, therefore, is not a substitute for security, but it is a legitimate instrument for shifting the attacker's cost curve unfavourably.

Defending the Reconnaissance Phase

The kill chain is most useful when read from the defender's side: for each phase, the question is what can be done to hamper the attacker's progress to the next phase, or at least to raise the cost of completing the current one. For reconnaissance, several measures follow.

The first is to **reduce the public footprint to its bare minimum** — exposing only what is genuinely required to fulfil business goals. If a publicly reachable website is needed, then it should exist; but it should be maintained with care. A well-defined process should periodically review what is exposed and remove information that was useful once but is no longer needed. It is often said that whatever is placed on the Internet is there forever; this is true only to a point. Not everything persists: although historical versions of websites are sometimes preserved by archival services such as the Wayback Machine, these archives do not capture everything. It therefore remains worthwhile to ensure that any information or service is exposed only for the minimum time strictly necessary. The common failure mode is the opposite — material is published and then forgotten, remaining online indefinitely. Several well-known attacks began exactly there: in a widely studied breach of a large US retail chain, the attacker gathered usable reconnaissance from an outdated document that had remained on the company's website far longer than it was ever needed.

The second measure is to **monitor for reconnaissance activity**, though here the limits are real. If an attacker consults information that a third party published about the organisation on an external social-

media channel, there is essentially nothing the organisation can do to observe that access. Within the organisation's own perimeter, however, monitoring is possible. The data accessed on one's own website can be tracked, and anomalous accesses can be flagged. Web traffic is commonly tracked already — for performance monitoring, error detection, and marketing analytics — and the same instrumentation can be turned to security. If a PDF document has sat published and unviewed for two years and then suddenly receives a connection from a previously unknown IP address, that access looks strange. There may be little to *do* about it directly, beyond removing the document if it is useless; but the mere knowledge that someone retrieved a document from which they may have derived information about a future target can prove valuable later.

That value is realised in the **incident-reconstruction phase** — a further implicit stage (conceptually a "Phase 8") in which the victim, after an attack, must rebuild everything that was damaged and, as part of cyber threat intelligence, reconstruct the entire kill chain to understand what happened and how the attacker reached their objective. During this reconstruction, the defender will also revisit the reconnaissance phase. Discovering, for instance, that six months earlier someone uploaded a document containing the email address of the very employee through whom the attack began is extremely informative: it reveals that the attacker is capable of using social-engineering techniques against targets identified by studying the organisation's personnel. Acting on this, the organisation can try to reduce the exposure of its employees — for example by carefully managing which email addresses are publicly available. In practice, most organisations do little about this, because it is difficult and expensive. Monitoring is itself hard: distinguishing a potentially malicious access from benign noise is far from simple, because every published resource is continuously accessed by a multitude of crawlers and robots that scan the web to map available information. One can configure `robots.txt` to discourage scanning, but that only restrains the well-behaved bots. The sheer volume of legitimate-looking automated access makes it genuinely difficult to isolate the accesses with potentially malicious intent.

The Output of Reconnaissance: Finding a Vulnerability

By the end of the first phase, the attacker has a **plan**. Having defined the target surface, what they are searching within it is a **vulnerability** — a defect that can be exploited to gain some privilege on the target infrastructure. The privilege in question varies with where the vulnerability lies on the surface: it might be access to the internals of the target infrastructure, access to the inbox of an employee, or something else entirely. The target surface of even a small company is typically huge and contains a large number of vulnerabilities of widely varying seriousness; not all vulnerabilities are grave.

Among all the vulnerabilities discovered, the attacker selects the one that, from their point of view, offers the greatest probability of successful exploitation at the lowest cost — because the attacker is cost-effective. If two distinct vulnerabilities offer the same level of privilege once exploited, but one resides in a system about which the attacker knows nothing while the other is a vulnerability for which the attacker already possesses working exploitation code, the attacker will choose the second without hesitation, because it is easier and more cost-effective to use. At the close of reconnaissance, then, the attacker holds a plan that reads, in essence: *in the next step I will exploit that vulnerability to gain this advantage.* With the plan fixed, the attacker proceeds to the second phase, where the plan must be turned into real, running tools.

Phase 2: Weaponization

With a plan in hand, the attacker must now **weaponize** it: identify and acquire all the technical assets needed to turn the plan into a real, executable attack. This is, by a wide margin, the phase that demands genuine technical skill. The precise nature of the assets required depends entirely on the plan written

during reconnaissance. The attacker might need a tool to launch and manage a phishing campaign; the exploitation code for a known vulnerability, or for a zero-day; or the means to place a convincing voice call and persuade a victim to surrender a secret code. All of these are technical assets, and — importantly — all of them can be acquired from outside.

The Shifting Economics of Weaponization

Over roughly the last ten to twenty years, the technical skills required for weaponization became available on a **black market**. On the dark web, criminals could hire people with specific competencies to supply the exact pieces of code needed to implement parts of their plan, and the criminal's own task reduced to gluing those pieces together. This arrangement widened the pool of attackers dramatically: whereas in the 1990s and early 2000s only skilled individuals could mount a cyber-attack, by the 2010s a non-technical person could become an effective cyber-criminal simply by hiring skilled actors on black markets. In that world, all one needed was *money* to invest.

A further shift is now under way, and it is substantial. Much of the weaponization work is beginning to be performed by **artificial intelligence**. It is now possible to download an open-weight model free of charge and run it locally — not yet on a laptop, since a large machine is still required, though that constraint may ease in the near future — and ask it to identify vulnerabilities in a specific application and to build the exploitation code for them. Commercial services guard against this: asking a hosted assistant for such material typically runs into guardrails that block the request. Open-weight models impose no such restriction, or impose guardrails that are easily circumvented, and versions with the guardrails stripped out circulate freely. The practical consequence is that the need for the black market is rapidly decreasing, and the barrier to becoming an effective cyber-criminal is collapsing. A documented case illustrates the point: a university student assembled a set of open models, directed them to scan the website of a bank, let the models discover the vulnerabilities and generate the exploitation code, and used the result to compromise the bank and deploy ransomware. The student had no difficulty identifying the vulnerability or obtaining the exploit at near-zero cost — the models supplied the "brains." (The attacker was arrested within days, having left an open, poorly protected infrastructure that let law-enforcement agencies identify him; but the ease of the weaponization itself is the lesson.)

The trajectory is therefore clear and, viewed soberly, alarming. The field has moved from a reality in which only highly skilled people could attack, through a reality in which the unskilled could *buy* skill on the black market and needed only money, to an emerging reality in which an attacker needs neither deep skill nor much money — only enough computational resources to run a state-of-the-art model.

Zero-Days and the Value of Secrecy

Weaponization is often "outsourced" precisely because unknown vulnerabilities are extraordinarily valuable. A **zero-day** — a vulnerability unknown to the software's producer and to the defensive community — can be sold for very large sums, and its value holds *only as long as it remains secret*, because disclosure triggers patching that destroys its utility. The scale of this market is considerable. Public brokers openly advertise payouts for top-tier mobile exploits, for instance offering substantial sums for full chains against iOS or Android (remote code execution combined with local privilege escalation and sandbox escape), with the highest tiers reaching into the millions, and with the understanding that the end user is a government. That such capabilities command these prices reflects the simple fact that a working, secret exploit against a widely deployed platform confers enormous power for as long as nobody else knows of it.

What Is a Vulnerability?

To reason about weaponization it is necessary to define a **vulnerability** precisely. Restricting attention to software vulnerabilities (setting aside human-factor vulnerabilities for the moment), the concept is best understood through the gap between what software was *meant* to do and what it *actually* does.

Building software begins with collecting **requirements**, which define exactly the functionalities the software should possess at deployment. These requirements are analysed, a plan is made, the software is engineered and tested, and a product is delivered to the client. In theory, if the set of initial requirements is drawn as one region, the delivered software should fall exactly within that region. In reality, what is delivered is a different set of features that only *partially* intersects the original requirements. Three regions result, and the picture — though exaggerated, since in practice the intersection is usually large — makes the point sharply:

- The **intersection** contains the features that correctly implement the original requirements. This is what the client wanted and received.
- The **missing region** consists of requirements that were not implemented at all, or were implemented differently from what was intended. These are what we ordinarily call **bugs**.
- The **extra region** consists of features that are undesired, were never designed, and are typically undocumented, yet remain present in the delivered software.

[Photo: Vulnerability Venn diagram — two overlapping sets, "requisites/requirements" and "implemented functionalities." The left-only crescent is *bugs*, the overlap is correctly implemented *features*, and the right-only crescent is *undesired / non-designed / undocumented functionalities*; the union of the two crescents constitutes the *potential vulnerabilities*.]

It may seem strange that undesired, unrequested features should exist at all, but it happens constantly. Requirements rarely specify **test code**, yet test code is written to verify that the implemented features match the requirements; at delivery this test code should be removed, but in practice much of it remains inside the shipped software. Similarly, a developer may implement a function, then the next day devise a cleverer implementation that saves memory or runs faster, write the new version, and keep *both* behind an `if` clause intended to be resolved later — only to forget about it, leaving the superseded code in place forever. Such undesired code usually does not disturb the client, because if the application is used correctly there is no legitimate execution path that reaches it.

The code is nonetheless present, and herein lies the danger. Both the region of **buggy** functionality and the region of **undesired** functionality may contain **potential vulnerabilities**. The attacker may exploit these defects to trick the application into doing something it was never designed to do — for example, by diverting the program's flow of execution toward undesired code that happens to implement the ability to write data to disk, a capability otherwise absent from the application. Once execution is diverted there, the attacker gains the power to write to the file system.

Tracking Vulnerabilities: CVE and the NVD

When a vulnerability is discovered, it is usually recorded in a public database. This is not true of the very oldest vulnerabilities, but most vulnerabilities in widely used software are tracked. The principal such database is the **National Vulnerability Database (NVD)** maintained by NIST, alongside a few similar databases. For each vulnerability, the record contains several important items of information.

First and foremost is a **unique identifier** — the CVE identifier — which makes it possible to track exactly which versions of the software are affected and which attacks exploited the vulnerability to carry out malicious actions. The record provides a textual **overview** describing the vulnerability, and then a set of **metrics**, among which the most important is the **severity**. Severity is a theoretical evaluation of

how serious the impact of an attack exploiting the vulnerability could be. Different vulnerabilities carry different impacts according to what they permit: a vulnerability that merely allows the attacker to read the contents of a small region of memory, without any ability to write, has limited impact, whereas a vulnerability that permits remote execution of arbitrary commands has extreme impact because it lets the attacker do essentially anything. These judgements are expressed through a standardised scoring system (CVSS), which decomposes the assessment into factors such as the access vector (for example, whether the vulnerability is network-exploitable), the access complexity, whether authentication is required, and the resulting impact on confidentiality, integrity, and availability. A record also indicates whether a patch is available and, when it is not, what measures can be taken either to reduce the chance of exploitation or to reduce the impact if exploitation occurs.

Patches

A **patch** is a piece of code released by a software producer in response to the identification of a new vulnerability. Concretely, suppose a new vulnerability is discovered in the Linux kernel. At time zero the information is made available to the Linux development community, which works for some interval and produces code that fixes the vulnerability. Once applied, the patch alters the running code of the vulnerable software so that the vulnerability is no longer present — and this matters greatly, because the moment a patch is applied, the attacker's ability to impact that system drops to zero.

Producing a patch is not always easy, because not all defects are alike. A defect may be a single wrong instruction or condition in the code, which is cheap and quick to fix. Or the defect may be far more complex, lying in the very **architecture** of part of the software, in which case restructuring the software to remove the vulnerability can be expensive. When a fix is costly, it may delay the release of the patch, or push the developer toward a different approach entirely. The developer might decide that removing the vulnerability is too expensive to do immediately, defer the true fix to the next major version eighteen months away, and in the meantime release a fix that makes exploitation impossible **not by removing the vulnerability but by removing the parts of the code that allow the attacker to reach the vulnerable code**. This strategy is effective, but it does not offer the same guarantees as removing the defective code outright: there may still be other, unconsidered execution paths that allow the attacker to reach and exploit the vulnerability. Moreover, when the patch is released the attacker can tell which strategy was chosen, and therefore knows whether the vulnerability is still present and merely harder to reach.

Vulnerability Disclosure Strategies

A central question is: **at what point should knowledge of a vulnerability be made public** — that is, when is the right moment to publish it on a database such as the NVD? There is no single answer; the alternatives span a spectrum, with two extremes and a pragmatic middle.

At one extreme is **full disclosure (full transparency)**: as soon as a vulnerability is discovered, it is made public, so that everyone using the software knows it is defective, understands they are at risk, and can take countermeasures. The decisive disadvantage is that criminals learn of the vulnerability at the same moment as everyone else, and a race begins to build exploitation code against it. At this point there is no patch available, because the producer did not know the software was vulnerable; criminals and producer learn simultaneously, and it becomes a race — exploit on one side, patch on the other — in which whoever arrives first wins. Those in the middle, running the vulnerable software, must decide what to do in the interim; the right choice depends on the nature of the vulnerability, its prospective impact, and whether effective mitigations exist, and may even involve shutting down a service until a patch becomes available.

At the opposite extreme is **full secrecy**: the finder does not make the vulnerability public, but instead provides the producer with all the information and keeps it secret until a patch is available. Assuming no criminal independently discovers the same vulnerability, this prevents attacks. The problem lies in what the producer will actually do with the information. Fixing a problem that only one person in the world is aware of is not necessarily a high priority: the company's resources are typically committed to building the next version, selling the product, and implementing new features, so the fix becomes a background task handled in spare time, and the patch may not appear for months. This is a poor outcome, because in the meantime some criminal may independently reach the same conclusion, identify the vulnerability, and begin exploiting it.

The approach actually used is therefore **in the middle (coordinated disclosure)**. The information is kept secret and shared only with the producer, but the producer is granted a limited **grace period**, on the understanding that the information will be kept secret until that period expires, after which the vulnerability will be loaded into the public databases. A common arrangement is a maximum of **90 days**: "I know this problem exists in your software; you have at most 90 days to produce the patch; after 90 days I will publish the information on the NVD." This strikes a good balance and pushes the producer toward a time-bound yet sustainable patch process. It deliberately does not force the producer to *rush*, which is itself desirable, because rushing a patch introduces its own problems: every change to software may create new issues, so solving one problem can open others, including regression errors. (One often observes a point release followed a day later by a further point release — a "patch of the patch" — precisely because the first patch contained an error.) A relaxed but bounded deadline lets the producer plan the effort required to deliver the patch safely while ensuring it is delivered within a reasonable time.

How the Attacker Treats a Vulnerability: Exploitation Over Time

From the attacker's side, the existence of a vulnerability is treated according to two main situations, and the dynamics of exploitation over time reveal several instructive phenomena.

Consider first the common case: a previously **unknown** vulnerability is published today in the NVD, and a patch is made immediately available. Is the vulnerability still exploitable? Absolutely — because the availability of a patch does not mean everyone applies it promptly. Numerous studies examine the **distribution of time-to-patch**, which depends on many factors: whether the software runs on personal computers or servers, whether it is managed or unmanaged, whether it is consumer or professional software, and more. What all these distributions share is a **long tail**. The patch is applied more or less quickly by the vast majority, but there is always a long tail of installations that remain unpatched for an impressively long time, and the attacker knows this. Consequently, when the vulnerability becomes known at time $\mathbf{T_0}$, the number of attempted attacks attributable to it rises immediately to a large value; because the patch is available at the same moment, people begin applying it, and the attack curves rise quickly to a certain level and then converge to a **plateau**. When these lines become horizontal, the vulnerability is no longer practically exploitable, because essentially everyone has applied the patch. The lines are never exactly flat, owing to the long tail — there are always some systems that remain vulnerable and on which the exploit still works.

[Photo: Attacks-over-time graph (attacks on a log scale versus time in weeks, centred on the disclosure instant t_0). Most CVE curves start near t_0, rise steeply to a high plateau, and flatten as patching completes; a few curves (e.g. a zero-day) begin rising *before* t_0, and at least one post-disclosure curve with no available patch keeps climbing for weeks after t_0.]

A subtlety concerns *which* systems populate the long tail: they are typically unmaintained, unpatched systems, which seldom hold information valuable to the attacker. If the system is not valuable enough for

its owners to maintain, it is unlikely to be valuable to the attacker. Some residual value remains, but it is rarely worth spending resources on.

Some curves depart from this standard shape. In certain cases the number of attacks rises above zero *weeks before* the time at which the vulnerability becomes public. What is happening is that some criminal knows of the vulnerability before the general public — the signature of a **zero-day**. Zero-days are vulnerabilities discovered by criminals, who obviously do not inform the vendor and instead begin exploiting them, producing attack curves that climb before T_0. As soon as a previously unknown vulnerability is exploited, after only a few attacks alarms begin to ring and awareness spreads, so the zero-day quickly transitions into the public-disclosure situation. In the case of **public disclosure with no patch yet available**, the number of attacks continues to grow, because there is no patch to apply to block them, and this growth persists for weeks after public release.

How Many Vulnerabilities, and How Old?

A natural question is how many vulnerabilities are exploited in a single attack. The answer is rarely just one. Often more than one vulnerability is exploited in a specific **chain**, each granting an increment of access, until the attacker reaches the level of control needed to take over the target machine or software. And these vulnerabilities are by no means all new. Statistics make this plain: old vulnerabilities are widely exploited. Data from 2015 — old, but qualitatively still valid — showing the distribution, by year of disclosure, of the vulnerabilities used in attacks launched that year, reveals that attacks in 2015 still relied on vulnerabilities disclosed as far back as 1999, more than fifteen years earlier. How can systems remain vulnerable to a fifteen-year-old defect? Several realities sustain this. Some systems **cannot** be patched. Others remain unpatched out of plain **laziness**: an administrator who runs a server behind multiple layers of security, is confident that no one can reach it, and knows it hosts no critical service, may simply not bother to patch — or may be willing to patch but unwilling to spend a week verifying that everything still works, and so leaves the machine unpatched because the probability of anyone reaching it is close to zero. Close to zero, however, is not zero. A vulnerability is therefore not necessarily present only in readily reachable machines; sometimes it is buried behind layers of defence, but it is still there, and an attacker who reaches that point through earlier steps can exploit it.

Time to Weaponization

Two further empirical observations frame this phase. First, **a small number of vulnerabilities account for a large fraction of all exploits**: plotting the cumulative share of exploits against the top exploited CVEs produces a steeply rising curve that quickly approaches one hundred percent, meaning a handful of well-known issues — classic privilege-escalation bugs, memory-disclosure flaws in legacy server products, well-known protocol weaknesses in older cryptographic implementations — dominate real-world exploitation. Second, **time to weaponization is critical**: plotting the percentage of CVEs exploited against the number of weeks elapsed since publication shows a curve that climbs very steeply in the first weeks and then rises more gently, meaning that a large share of eventually-exploited vulnerabilities are weaponized and used very soon after disclosure. The defensive implication is direct: the window between disclosure and patching is exactly the window in which the attacker is most likely to strike, so the speed of patch deployment is itself a security-critical quantity.

A Weaponization Case Study

A concrete illustration ties these threads together. In a highly targeted campaign attributed to a state-aligned group, the attackers chained two zero-day exploits — one in a widely deployed browser plug-in and one local privilege-escalation flaw in the operating system. The high-level flow was: the victim clicks a link to an attacker-controlled website; an HTML/JavaScript launcher page serves a Flash exploit; the

exploit triggers the browser-plug-in vulnerability and executes shellcode; the shellcode downloads and runs an executable payload; and the payload exploits the local privilege-escalation vulnerability to steal a System-level token. This is weaponization in full: a plan realised through acquired exploit code, assembled into a working chain, with secret zero-days providing the critical advantage. With the weapon built, the attacker must next get it to the target — the delivery phase.

Phase 3: Delivery

In the delivery phase the attacker takes the assets prepared during weaponization and moves them onto the machines where the attack will be executed, there to exploit the vulnerabilities identified earlier. Delivery can occur through several means, but today the distribution is highly skewed.

Delivery Vectors

By a wide margin, the most common delivery approach is **email**. The malicious payload is carried either directly, as an attachment, or indirectly, as a link that must be clicked. The dominance of this channel is not a matter of impression but of measurement: among detected malware, the overwhelming majority — on the order of nine in ten cases — is delivered by email, with the web accounting for a small fraction and all other vectors for a sliver. Within email delivery, the techniques are refined: generic **phishing** casts a wide net, **spear phishing** is tailored to a specific individual or organisation, and the **watering hole** compromises a legitimate site that the intended victims are known to visit, so that the malware is delivered when they browse to it. The files used as carriers follow their own distribution, dominated by lightweight script formats and office documents: scripting files such as JavaScript account for the largest share, followed by other scripting formats, Windows executables, and Microsoft Office documents, with archives and PDFs making up much of the remainder. The practical lesson is that the attacker prefers the carriers most likely to slip past filters and most likely to be opened by an ordinary user — scripts and documents rather than bare executables.

The second most common approach is the use of **compromised identities**. Here the attacker purchases a compromised identity from a black-market vendor and uses it to log in to a system with whatever limited privileges that identity confers. Operating from inside with legitimate credentials, the attacker then executes the exploitation code locally, escalates privileges to a higher level, and from that pivot point moves freely within the compromised target.

Crucially, both principal delivery methods — phishing and compromised identities — are tightly bound to **social engineering**. This is precisely why one so often hears that *the human is the weak link*. Far from being a platitude, this is a statement supported by the statistics, and it carries direct, practical consequences for how organisations must defend themselves.

The Evolution of Anti-Phishing Training

If people tend to open attachments in phishing emails, the obvious response is to train them not to, and organisations have invested ten to fifteen years in exactly this effort. The classical approach relied on **education**: lectures and best practices teaching employees to be suspicious of messages containing grammatical or syntactic errors, to scrutinise the sender's email address, to look for tell-tale phrases in the body, and to examine the company logo for misalignment or subtle differences from the original. These are all *technical signs* of a falsified, poorly crafted phishing email.

The difficulty is that these defects are disappearing. Artificial intelligence now allows attackers to craft messages that are perfectly aligned with the visual standards of virtually any company in the world, so the grammatical errors and misplaced logos that once betrayed a phishing email are vanishing. The

message simply looks perfectly legitimate. Compounding this, the availability of compromised identities means the message may genuinely originate from a **legitimate source**, removing even the possibility of catching it by inspecting the sender. Training employees to spot signs of falsification in the artefact itself is thus becoming a losing strategy.

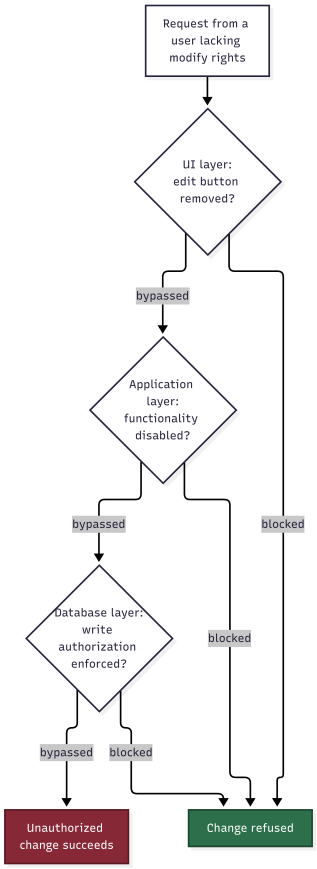
For this reason, training is shifting away from using the employee as a **monitoring sensor** aimed at the email, and toward monitoring the **process**. Rather than looking for cosmetic defects, employees are now taught to recognise the characteristic **psychological techniques** of phishing, which persist even when the email is visually flawless, because they are intrinsic to the manipulation rather than to its presentation. A standard phishing message tends to place the reader under **pressure**. It frequently appears to come from someone higher in the organisational hierarchy, so that the reader feels they are receiving an order from a superior. It often pretends to involve a procedure that departs from the standard — framed as a humanitarian reason, or as a favour: *"I am bypassing the usual checks, just for you, to speed things up."* And it commonly invokes **urgency**: *"reset your account password within the next two hours, or your account will be blocked."* These psychological levers remain present regardless of how legitimate the email looks, and they exist for a single purpose — to strip away the defensive caution that people naturally exercise when they are not under pressure.

The corresponding countermeasure is to train people to **avoid, at any cost, diverting from standard procedures**. Standard procedures exist precisely because they embed well-defined safeguards against this class of incident, and a phishing message always works by inducing the victim to step outside those procedures in order to bypass the checks they contain.

The fraud against the bank's chief executive discussed earlier is the exemplary case. A clear countermeasure procedure was in place: any request to execute a money transfer had to be double-checked on a **second channel initiated by the bank itself**. The attackers used psychological pressure to make the executive diverge from this procedure. Rather than verifying the request on the out-of-band channel *he* was supposed to initiate, he relied on an out-of-band channel initiated by **someone else** — in fact the attacker — and that single substitution defeated the one check that would have prevented the incident.

Defense in Depth

This episode illustrates a broader methodology known as **defense in depth**. The principle instructs that when a security measure is placed — for instance while designing a system — it should not be made effective at a single level only, such as the user interface. If a user who lacks modification rights on a document should not be able to modify it, simply removing the edit button from the interface is not sufficient protection. The functionality must be removed, or at least made inaccessible, at the **application layer** beneath the interface, and the constraint must also be enforced at the **database layer**, so that a modification request arriving from a user without the appropriate right cannot change the record. Enforcing the same policy at multiple independent layers is what it means to implement defense in depth.



Although defense in depth originated with software systems, its logic applies equally to **human-based processes**. Placing a single check in a critical human-controlled process is generally not enough. A double check is a good first step, but a third and a fourth can be added — as many as remain sustainable from a cost standpoint. Every additional check slightly delays the final effect of the process and therefore slows it down, but each one improves the process's resilience, making it harder for an attacker to overcome *all* the checks and bring the attack to success. The fraud succeeded because a single verification was circumvented; a process built in depth, with independent and redundant verifications, would have forced the attacker to defeat several of them at once.

Once the weapon has been delivered to the target, it must actually be triggered against the chosen vulnerability — the task of the next phase.

Phase 4: Exploitation

By this point the attacker has made a plan during reconnaissance, acquired the technical assets during weaponization, and delivered those assets to the intended target. In the **exploitation** phase the attacker runs the delivered assets to gain the privileges required to act on the target. This step is often more or less automatic and frequently reduces to the execution of a piece of software — but not always. Sometimes exploitation proceeds not by defeating code, but by **asking the victim to do something**, and understanding this broadens the notion of exploitation considerably.

Attack Methods

Exploitation can be achieved through four broad kinds of attack method. The first is a **zero-day exploit**, which leverages a vulnerability unknown to the producer and the defensive community and therefore has no patch. The second is the exploitation of a **known software vulnerability**, relying on the reality — established earlier — that a large population of systems remains unpatched long after a fix is available. The third is **misconfiguration**, in which the software itself is not defective but has been deployed or set up in an insecure way, leaving an opening the attacker can use. The fourth is **bad user**

behaviour, in which the decisive weakness is not in the software at all but in the choices of the human operating it. The last two methods make clear that exploitation is as much a human and organisational problem as a technical one.

Exploiting the Human: Permissions and Consent

The human dimension of exploitation is visible in the ordinary mechanism of **permission requests**. Most software installed on a personal computer or smartphone today asks for permissions the first time it runs. An application that genuinely needs the device's location will present a prompt — "this application is requesting access to the GPS system; do you allow it?" — and the user clicks yes. A music player, by contrast, has no need of the user's location, yet many applications and websites request it anyway. It is common to encounter a service — a flight-booking website, say — that prompts for location on every page even though the location is not required to deliver the service, and the task can be completed without ever granting it. The problem is the standard user: when someone installs a game or an application, they simply want to use it, so every request is answered with an automatic "yes," without any examination of what the software is actually asking for. Attackers know this, and in most applications the mechanism is used merely to collect information the user would otherwise never surrender. The impact of such over-collection is not direct, not easily measurable, and does not prompt complaints — which is precisely what makes it effective.

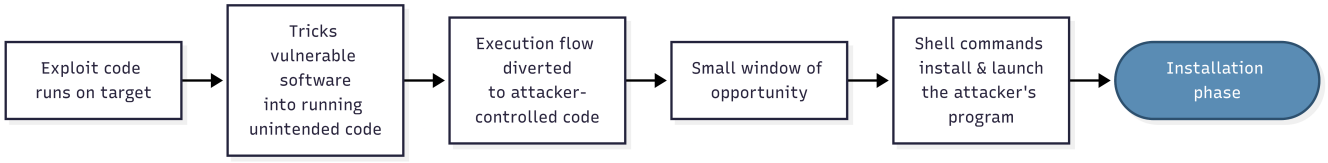
The more dangerous version of the same phenomenon concerns **administrator privileges**. Some software, when installed, requests administrator rights. There are legitimate reasons for this: a program may need administrative privileges to modify the configuration of the operating system, which is common for software that installs low-level drivers operating at the kernel level, or that installs services intended to run continuously in the background. These reasons are real, but they should not be the norm. The difficulty is that very few users are able to judge whether a given piece of software genuinely needs the level of access it is requesting. Unable to make an informed decision, most users simply enter their password and grant consent, because the alternative is not using the software they wanted. This mechanism — the operating system asking for the user's password before allowing software to adopt administrative privileges — is the solution operating-system producers devised to make consumer-grade systems more robust while still leaving the user enough flexibility to use them freely.

The contrast with the enterprise model is instructive. A laptop issued by a company is used with **user-level privileges**, and the user is given no administrator rights. To install software, the employee must raise a ticket with the IT service; the request is reviewed, the software is checked, and it is installed — typically remotely — by the IT team only if it is genuinely needed and deemed secure. This is done precisely to prevent the user from granting administrator consent without understanding the consequences, applying the principle of least privilege. On a personal, consumer machine no such intermediary exists: the only thing the operating system can do when software requests elevated privileges is **ask the user**, and then live with the risk that the user is unable to make an informed decision. If the user says yes without understanding, they are exposed — and a great deal of malware exploits exactly this human "vulnerability" to infect standard personal computers. Here the exploited defect is not a flaw in code but a flaw in the decision process of the person at the keyboard.

The Mechanics of Technical Exploitation

When exploitation is technical, the sequence is as follows. A piece of code is run on the target machine that tricks the vulnerable software into executing lines of code that were never intended to implement any legitimate functionality. Typically this diverts the program's flow of execution to a location where the attacker's own code can run. When this diversion succeeds, the attacker is handed a **relatively small**

window of opportunity in which they can do essentially whatever they wish. What the attacker typically does in that brief window is run **shell commands** that, more or less visibly, install and launch the attacker's program on the target machine.



This act of installing and launching the attacker's program is exactly what carries the attack into its fifth phase. The window of opportunity opened by exploitation is short and leaves traces; the attacker's immediate priority, therefore, is to convert that fleeting access into a durable, concealed presence — the work of installation.

Phase 5: Installation

The official kill chain names the fifth phase **installation**, though it is equally well described as **persistence**, and persistence is exactly its purpose. Having gained a foothold through exploitation, the malware now does its utmost to establish itself durably inside the compromised machine.

Surviving the Machine

The first task is to ensure that the malware's own code **survives everything reasonable** that can happen to the machine, and the most elementary such event is a **reboot**. A machine sitting in an office will typically be shut down at the end of the business day and powered up again the next morning; when it comes back up, the malware must reload itself automatically, without requiring the human victim to do anything at all. Numerous techniques exist to achieve this — many of them specific to the Windows platform and studied in detail later. The malware may configure the operating system to run it at every reboot; it may install itself as a continuously running **service**; or it may place itself inside a **dynamically loaded library** that is automatically executed by some benign piece of software. The list of available techniques is long, but whichever is chosen, the attacker's overriding concern is to carry out the installation **transparently**, making the change to the operating system's configuration as invisible as possible.

Hiding the Traces

Having guaranteed its persistence, the malware next hides all the reasonable traces of its presence. This matters because the execution of exploitation code typically **leaves traces**: during exploitation the attacker diverts the flow of an existing program into a handful of lines of attacker-supplied code, and in so few lines there is little opportunity to conceal one's tracks. Trace removal is therefore a task performed during installation. If the exploitation step or the process of establishing persistence wrote temporary files or altered temporary configuration, those artefacts are removed; if anything was written to local logs, the relevant portions of those logs are redacted so as not to leave them for a monitoring agent to find.

The Payload

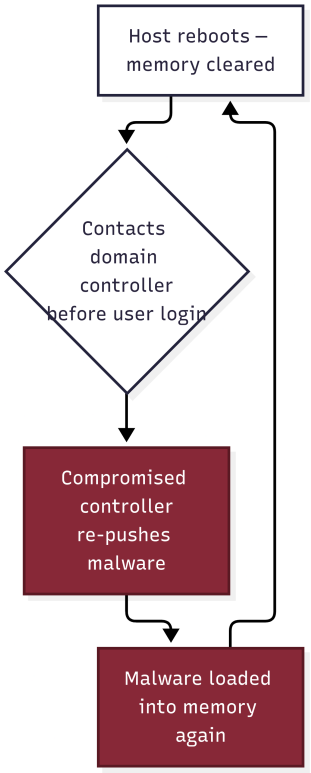
During installation the malware frequently **downloads the payload** from some location on the Internet. The payload is the part of the malware that contains the actual code implementing the actions on the objective — the work of the final phase. Sometimes the payload is delivered earlier, during the delivery phase, carried alongside the exploitation code; but in that case it is delivered in an **obfuscated or encrypted** form and is only decrypted and made executable at this stage. Quite often, however, the

payload is not delivered in the fourth phase at all, but is instead downloaded here from the Internet. When it is downloaded, the network connection used must be adequately **protected and hidden**, otherwise it could be captured by a network-monitoring agent or blocked by a firewall. For this reason the attacker tends to use channels that resemble ordinary, benign traffic inside the target: at a targeted company the attacker will do its best to conceal the communication and make it look like noise, so that it is neither blocked nor noticed. The specific concealment techniques belong properly to the next phase.

Fileless Malware and External Persistence

Some malware carries concealment to the extreme: it **writes nothing to disk** and operates only in memory. The motivation is that catching traces of malware in memory is technically more complex than detecting its presence on a hard drive or solid-state drive — feasible, but harder. This raises an obvious question: how does a purely in-memory malware achieve persistence across reboots, when memory is cleared on restart? The answer is that such malware must go one step beyond, and several mechanisms exist.

The most common approach is to have the malware **reinstalled at every reboot by an external entity**. Consider machines configured inside an **Active Directory forest**, a system frequently used by companies that adopt Microsoft software — the vast majority of which run Windows. In such an environment a central controller governs all the machines in the same security domain. The malware can be pushed from the compromised controller onto the individual hosts and merely placed in memory, because every time a machine reboots it will in any case contact the main controller — to check for updates or commands — before allowing the user to log in. At that moment the compromised controller re-pushes the same malware. The malware is thus persistent on the single host and user machine, but its persistence is performed **externally**, by the controller.



More exotic cases exist. Some malware installs its persistence mechanism in the **boot sectors** of the hard drive. In very limited cases, malware embeds itself in the **firmware** of the hard drives or the motherboard and loads itself into memory at every reboot. These are rare exceptions rather than the norm.

The Point of No Return for Detection

At the end of this phase the malware is solid, persistent, and well hidden inside the compromised asset — and this has a sobering consequence for the defender. If the incoming infection is **not** identified and blocked at delivery, during exploitation, or while installation is still ongoing, then once the installation phase completes it becomes **extremely difficult** to identify that the machine has been infected. The reasoning is direct: if the malware reached this stage and deleted the traces it left behind, it has already overcome every limitation imposed by the local antivirus or endpoint detection and response (EDR) software. All the local countermeasures have already been put out of the game; and if they were surpassed during the earlier phases, there is no reason to expect them to intercept the malware's presence now. Exceptions exist, but at this point identifying the machine as compromised is extremely difficult. From this well-established, concealed position, the malware proceeds to reach out to its operator — the command-and-control phase.

Phase 6: Command and Control (C2)

With the malware installed, persistent, and concealed, the attack enters the **command-and-control** phase, commonly abbreviated **C2** or written C&C. At the start of this phase the payload cleans up whatever remains of the infiltration procedure and may, if necessary, leverage a local exploit to escalate its local privileges further. From this moment onward, detecting the intrusion becomes extremely difficult. The defining event of the phase is that the installed malware, which on its own does not yet know what to do next, **opens a connection toward the attacker** — toward some IT asset controlled by the attacker — for two distinct purposes: to announce itself, and to await orders.

Beaconing

The first activity is **beaconing**. Having installed itself, the malware on the compromised machine simply announces its presence to the attacker — the equivalent of waving a hand and saying, "I did it; I successfully compromised the machine." The beaconing message may also convey basic information about the compromised target, such as the machine's internal IP address or the username under which the malware is currently executing; it carries a limited amount of information about the host. Beaconing is performed **periodically** toward a target on the web whose address is written inside the malware's code — but written in a well-concealed form, because the concealment of this destination is central to the malware's survival.

A great many techniques exist to implement beaconing in a non-obvious way. The obvious implementation would be to hard-code an IP address and have the malware periodically send a message to it, waiting for that address to reply "received — I know you are alive." This naive scheme is rarely used, for a simple reason: if the defender captures that IP address, the malware can be immediately neutralised by blocking all access to the address. To avoid this single point of failure, malware instead uses more elaborate channels. It may write messages on **social-media accounts**; it may use open online services such as paste sites to write on publicly readable documents; it may write on shared documents in cloud-storage services such as Dropbox or Google Drive. In each case the beacon hides inside traffic to a popular, legitimate destination that the defender cannot casually block.

Beaconing Through DNS

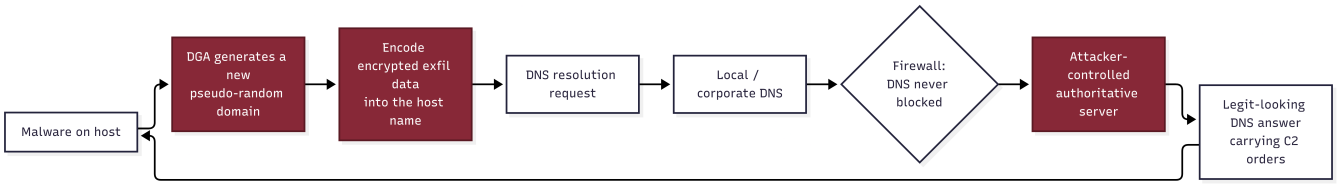
A particularly widespread technique is **beaconing through the Domain Name System (DNS)**. The underlying constraint is that every time the malware wishes to send or receive a message over the Internet, it must do so in a way that will not be blocked by the firewall of the compromised company; the network flow should resemble a perfectly legitimate flow as closely as possible. This is why standard

protocols such as HTTP and HTTPS are favoured — and why DNS is especially attractive, because DNS is a protocol that firewalls essentially never block: if DNS were blocked, nothing would work any more.

Attackers therefore found a way to use DNS to convey messages to a command-and-control server. The idea is relatively simple. The malware performs a **host-resolution request**, handing the operating system a fully qualified domain name for a domain controlled by the attacker. If the domain is uncommon, the request is handled by the local DNS server, which forwards the query toward the **authoritative server** for that domain — and that authoritative server is controlled by the attacker. The firewall sees only a perfectly legitimate DNS request travelling toward an external DNS server, and the server replies with a perfectly legitimate-looking answer. The subtlety is that **anything can be encoded** in both the request and the response: the malware can encode, within the host portion of the name for its attacker-controlled domain, the information it wishes to exfiltrate. To an observer it looks like a request for a strange, pseudo-random name under a domain never heard of before — not especially suspicious, unless that domain is already known to be associated with criminal activity. And moving from one domain name to the next is trivial.

Domain-Generation Algorithms and Sinkholing

Attackers push this idea to its logical extreme with **domain-generation algorithms (DGAs)**. The malware contains a predefined function that, each time it is executed, generates a new domain name to be used for a DNS query. The malware takes this seemingly randomly generated domain name, attaches as the host portion the information it wants to exfiltrate — encrypted so that its content is not obvious — and performs a resolution request on the resulting fully qualified domain name. Every time a new beaconing request is issued through the DGA, the domain name **changes**. The attacker has pre-registered all the domains the DGA is capable of generating, so the beacons always reach a server under their control.



This design defeats the obvious defensive countermeasure. Blocking requests to a known malicious domain is called **sinkholing** that domain: if one knows that a specific kind of malware sends its beacons to, say, a particular domain, one configures the DNS server so that any request for that domain is answered with the **loopback address**, which severs any connection from the malware to its C2 server. Sinkholing works beautifully against a fixed destination — but against a DGA it fails, because the defender does not know in advance which domain names will be contacted. To learn them, one must obtain a sample of the malware, **reverse-engineer its code**, and understand how the DGA works. This is precisely one of the reasons that malware analysis remains a compelling and necessary discipline.

Receiving Orders and Downloading Modules

Once the malware has beaconed its presence and the command-and-control server has acknowledged it ("I know you are alive"), the malware enters a **waiting state**, awaiting orders from the C2. These orders may instruct the malware to execute any of several functionalities, depending on what it is capable of doing against the objective. Among these functionalities is the ability to **download additional modules**, and this is what makes the malware able to **evolve over time**: it can change its own code, implement new features, or improve the execution of features it already possesses. A modular, updatable

implant is far more valuable to an attacker than a fixed one, because it can adapt to the environment it finds and to the defender's responses.

Concealed Channels: Steganographic C2

Two examples show how thoroughly the command channel can be concealed, in both cases using **steganography** — the hiding of information within ordinary-looking data.

The first is an old downloader from the early 2000s that used steganography to encode its commands inside an **image header**. The BMP format — an old, now outdated image format once common in the Windows world — stores a file as a block of metadata (the header) followed by the data (the pixels). The malware altered a carefully chosen subset of bits within the header so as not to change the visible content of the image, while encoding the beacon there at a low bit rate. The file was, to all appearances, an ordinary image.

The second is a more recent trojan that used a **social-media stream as its command channel**. The attacker published what looked like ordinary memes — standard pictures — on a public feed; within the pixels of those images, steganography encoded a set of commands, each carrying a distinct meaning for the malware that monitored the feed. Visually the posts looked perfectly legitimate, yet each one delivered an instruction to the compromised target. The commands supported by such an implant are exactly those one would expect of a flexible espionage tool: capturing a screenshot, retrieving the list of running processes, capturing the contents of the clipboard, retrieving the username of the infected machine, and listing the filenames present in predefined locations such as the desktop or the user's application-data folders.

With a reliable, concealed command channel in place and the ability to receive orders and new modules, the attacker is finally positioned to do whatever the operation was for — the actions on the objective.

Phase 7: Actions on Objectives

The seventh and final phase, **actions on objectives**, is where the attacker finally does what the entire operation was for. Its content depends entirely on the goal of the malware. The attacker may exfiltrate data of any kind present on the machine, install and use additional software, impersonate the user, encrypt the contents for ransom, or pursue whatever other end motivated the attack in the first place.

Data Exfiltration

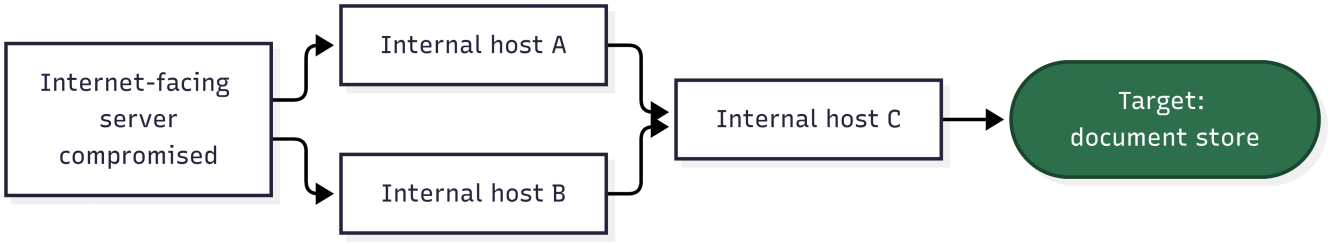
A central class of objective is **data exfiltration**, and the key insight is that virtually any information on a single personal computer is potentially valuable. Passwords, private encryption keys, documents, and information about the surrounding network are all prizes. Moreover, once the malware has obtained administrative privileges, it can do essentially anything one might imagine: install and run further software, impersonate the legitimate user, act as a **proxy** that relays the attacker's traffic through the victim's machine, and even monitor the victim's physical surroundings through the device's webcam. Full administrative control turns the compromised machine into a general-purpose instrument of the attacker's will.

Lateral Movement

A particularly important kind of action on objectives is **lateral movement** (also called lateral spread). In lateral movement the attacker uses the compromised machine as the **starting point for a new attack**: from this fresh vantage point, the attacker surveys the surroundings for other machines, treating them as a newly visible portion of the attack surface to be studied in order to plan the next step. The

compromised personal computer, in other words, may be merely an **access point to a larger network**, and the attacker will take their time exploring the surroundings to infiltrate further resources, remaining in place as long as possible.

A concrete scenario makes the pattern clear. Suppose the attacker's goal is to exfiltrate a specific document from a company. The attacker plans an initial attack, discovers a server the company exposes on the Internet, finds that server has a vulnerability, and proceeds through weaponization, delivery, and exploitation to gain access to it. This server is typically installed inside the **internal network** of the target company, and beyond its Internet-facing connection it also has access to other private hosts that are not directly visible from the Internet. The target document is probably **not** present on this first machine; it is buried somewhere deeper inside the company's internal IT assets. From this new vantage point, however, the attacker can move in a second step to another internal machine, compromise it, and continue through several lateral movements, progressively converging toward the server or client where the sought-after information is actually stored.



This reveals a fundamental point — and one of the main limitations of the kill chain itself. Modern attacks typically do **not** reach the attacker's final goal in a single pass through the chain. A real attack is carried out as a series of steps, each implemented through its own iteration of the cyber kill chain. The kill chain models one such step cleanly, from beginning to end, but a complete attack moves through several steps that are sometimes executed in **sequence** and sometimes in **parallel**. The kill chain should therefore not be read as a simple linear sequence, but as one building block within a **directed graph** — a graph, in general — that represents the whole attack.

The Timing of the Phases

The seven phases differ enormously in how long they take, and these characteristic durations have direct, practical consequences for defence.

Phase	Typical duration
1 · Reconnaissance	days / months
2 · Weaponization	days / months
3 · Delivery	minutes / hours
4 · Exploitation	milliseconds / seconds
5 · Installation	milliseconds
6 · Command and Control	years
7 · Actions on Objectives	years

The reasoning behind each is instructive. **Reconnaissance** credibly takes days or months, because studying what is visible, deciding which is the softest point to attack, and determining which target offers the best return all take time. **Weaponization** is similar: IT assets must be acquired — possibly bought on a black market — which requires human time and spans days or more. **Delivery** takes minutes or

hours when carried out by email, the lower bound depending on whether a human must read the message and click an attachment; when the email is interpreted automatically by a system, it is even faster. **Exploitation** happens through code executing inside the CPU, so it takes mainly seconds or less to finalise, and **installation** is likewise a matter of milliseconds. The last two phases are entirely different in character: **command and control** and **actions on objectives** can continue for **years**, precisely because the attacker intends to persist as long as possible, remaining until caught and expelled from the system.

Timing and the Choice of Countermeasures

These timings determine the *kind* of countermeasure that can realistically be deployed at each phase. One cannot expect to block an exploitation or installation technique by having a human expert watch what is happening on a machine, because **milliseconds are not in the human domain**. Humans can act effectively in the phases that unfold over days, months, and hours — reconnaissance, weaponization, and delivery — and in the long-running final phases that span years. Indeed, many successful compromises are actually caught at the command-and-control or actions-on-objectives level, over their long duration: one morning a technician on the network team notices a strange network flow in the previous night's logs, decides it is worth investigating, and discovers that it is in fact a communication from an infected machine to an external command-and-control server. How long it takes for this to happen — days, weeks, months, or years — depends on the case. This is exactly why the attacker takes such care to **conceal all external communication**: hiding it makes it hard for the monitoring team to identify and block the attacker in these two final phases.

At the millisecond-scale phases of exploitation and installation, by contrast, the only viable approach is to rely on **automatic software** that monitors what is happening inside the server, endpoint, or personal computer and blocks suspicious actions in real time. This is why there remains a thriving market for what were once called antivirus products and now carry more elaborate names such as EDR (endpoint detection and response) or XDR (extended detection and response). Such software monitors the environment and looks for **anomalous behaviours**. Whenever a program attempts an action that is strange in context — writing a file into a directory where files should never be written, overwriting the memory region belonging to another program, or calling a dynamic-link library that is not installed — these anomalies are caught at this level, because they are the potential signature of an exploitation technique that has just been activated.

The overarching discipline, therefore, is phase-by-phase: for each stage of the kill chain one must correctly identify the appropriate countermeasures, and then identify which **technologies** can implement those countermeasures, or which kind of **human beings** must oversee those tools for them to be effective. A machine is needed where the timescale is milliseconds; a human is needed where the timescale is days or years; and an effective defence weaves the two together across the whole length of the chain.

CHAPTER IF_2: Models for Cyber Threat Intelligence

Knowing the Adversary

A long-standing principle in the literature of defense is to *know your enemy.* The intuition is simple: if you understand who is acting against you and how they operate, you are better prepared, and you are ultimately able to respond more effectively. In the cyber domain this principle is operationalised into three questions that a defender must be able to answer about any adversary — **who they are**, **which goals they pursue**, and **how they act** — because the answers together determine the fourth and decisive question: **which preventive measures are most effective**.

Knowing *who* the adversary is does not mean knowing the personal identity of the individual at the keyboard. It means characterising the *nature* of the adversary well enough to narrow the options available for countering their activity. The posture a defender adopts against a competitor who has hired someone to damage them is very different from the posture adopted against an established cybercrime gang that has been attacking organisations across many sectors for a decade. The two adversaries differ in resources, persistence, and intent, and these differences justify different defensive decisions.

Knowing *which goals* the adversary pursues is even more important, because a rational adversary acts only to reach a clear objective. That objective might be to steal specific information, to block or delay the victim's operations, to extort money, or to damage the victim's public reputation. Motivations are tightly coupled to the nature of the adversary. A ransomware gang is driven by money and gains nothing from merely damaging a company's reputation; an adversary whose goal is reputational harm is therefore more plausibly a competitor in the same market segment, who would benefit directly from the victim losing clients and contracts, or an actor with a political motivation. Reading the motivation backwards in this way constrains the space of plausible adversaries.

Knowing *how* the adversary acts translates motivation into the concrete high-level activities the defender should expect. An attacker whose goal is to steal intellectual property will try to compromise the victim's assets in order to reach a protected file server and exfiltrate the protected data. An attacker whose goal is reputational damage will not necessarily compromise any system at all; they may instead run a fake-news campaign. Understanding how the attacker will act is the penultimate step toward the real objective of the whole exercise: identifying the **most effective preventive measures**. Only by knowing the tactics and techniques that will be used against you can you determine which countermeasures will actually work. Answering all of these questions has a single practical purpose — understanding, in advance, what you should do to be prepared when the attack arrives.

Defining Cyber Threat Intelligence

Cyber Threat Intelligence (CTI) is the discipline that collects and **analyses information about adversaries in order to answer these questions**. One precise definition captures its essential character:

> Cyber Threat Intelligence is **analysed information about adversaries that reduces the uncertainty of a specific decision made by a specific consumer**.

Every clause of this definition carries weight. The raw material is *information*, but information alone is not intelligence; it must be *analysed*. The subject of that analysis is *adversaries*. The purpose is to *reduce uncertainty* — not to eliminate it, which is impossible — and the uncertainty in question is not abstract but attached to a *specific decision* that a *specific consumer* has to make. CTI is therefore best understood as a **methodological tool for taking informed decisions**, not as an end in itself.

The decision-centred framing is what gives CTI its meaning. Consider the board of an organisation, which controls the budget. If the board decides to invest in new technologies to defend a particular class of application, but that application is not what the attacker is actually interested in, the decision has little impact on the organisation's real security. The decision failed because it was uninformed. In a well-run organisation every important decision is an informed decision: the decision-maker works to reduce the uncertainty surrounding the choice as much as possible. A residual uncertainty always remains — this is the nature of any enterprise — and accepting a decision under uncertainty means accepting a certain level of risk. You can only verify whether a decision was correct *a posteriori*, after events unfold. The best you can do at the moment of decision is to reduce the risk to a minimum, and reducing that risk is precisely the direction in which threat intelligence works.

Because of this, the value of a well-designed CTI process is felt at every level of an organisation. It produces practical, technical outputs for the operators, but it also produces information for the top of the organisation, and it is there — where the most consequential decisions are made — that the information matters most.

What Cyber Threat Intelligence Is Not

The definition above is still abstract, and in practice there is considerable confusion about what CTI is, even among experts. It is clarifying to state plainly what CTI is **not**.

CTI is **not a feed**. A subscription that delivers a continuous stream of indicators delivers *threat data*, not *threat intelligence*: nobody has analysed it for you, and nobody who produced it knew which decisions you need to make. Relying on an external feed is like trying to govern by only reading newspapers — you are merely buying newspapers, and there is a great deal of work between reading the articles and being able to take decisions based on them. Data from a feed is not tailored to your specific case. When you subscribe, the data may describe *an* adversary, but not necessarily *your* adversary; it may describe an actor targeting other organisations, and even when it concerns an actor targeting you, parts of it may relate to motivations that are not the ones relevant to you. Data must be **contextualised** before it is useful.

CTI is **not a tool**. Installing a platform — for example MISP, a well-known open-source CTI platform — does not create a CTI capability. A platform of this kind is an organised knowledge base: it needs data, and even with data it is not enough, because a *process* is made of tools **and of people** who look at the data and transform it into intelligence. The tool provides the technical substrate needed to manage the typically large volume of ingested data, but without the people who actually do the work of turning data into intelligence, the tool is only one ingredient of the game.

CTI is **not the news**. Knowing that a breach happened is *awareness*; knowing what it implies for your own exposure is *intelligence*. The fact that there are actors in the world willing to encrypt everyone's data for ransom is awareness of a risk, but it does not mean that risk is directed at you or can affect you negatively. Knowledge of an external actor that attacks other companies — even companies similar to yours — is not automatically interesting; what makes it intelligence is understanding how knowing that actor helps you set up the right countermeasures.

CTI is **not attribution theatre**. Publicly naming a state actor is the most *visible* output of CTI and almost never the most *useful* one. Most of the value a CTI process produces is created far below the level of attribution.

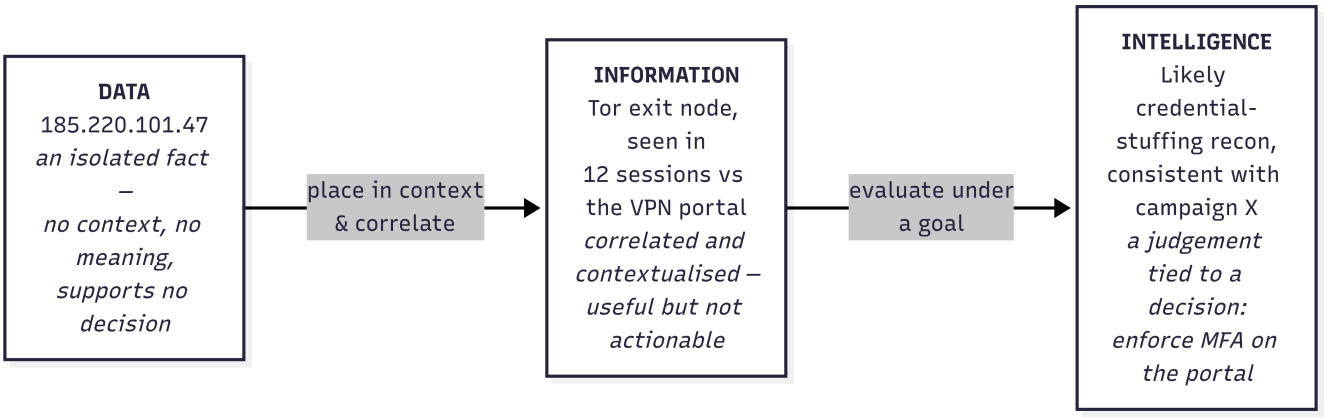
From Data to Intelligence

The same object can carry completely different meanings depending on how much it has been processed. A single IP address illustrates the progression from data, through information, to intelligence.

As **data**, an IP address such as `185.220.101.47` is an isolated fact. It is an IT asset that was involved in some cyber incident, somewhere in the world, affecting someone. By itself it has no context, and without context it has no meaning for you and supports no decision.

As **information**, the same address becomes "a Tor exit node, seen in 12 sessions against our VPN portal." The fact has now been placed in context and correlated with something we ourselves observed: twelve authentication requests toward our VPN gateway, arriving through an address linked to a well-known anonymity service. This is more useful — but it is still **not actionable**, because it is not yet tied to anything we want to achieve.

As **intelligence**, the address is interpreted in the light of a goal: "likely credential-stuffing reconnaissance, consistent with campaign X." We have connected a low-level observable to an objective — preventing our accounts from being compromised through credential-stuffing by a particular adversarial group — and this is what makes it intelligence, because it now supports an informed decision, such as enforcing multi-factor authentication on the portal.



The danger of stopping at the data stage is concrete. A common practice is to ingest a feed item such as "this IP address was used for command-and-control by an attacker," automatically install a firewall rule to alarm on it, and consider the job done. This can seem reasonable, but the rule rests on no analysis: you do not know whether the address is still connected to a live threat, nor whether that threat is acting against *you*. You are blindly trusting an external source, and such rules frequently land on firewalls, IDS and monitoring systems and stay there, unused, because the information had no context. The failure can be worse than wasted effort: there have been cases in which CTI feeds delivered the IP addresses of content delivery networks, and clients who blocked them disrupted legitimate services for hours or days. Blocking an address you never actually receive traffic from is not harmless — it is a decision taken purely on trust, outside any reasonable intelligence process.

The contrast between two organisations facing the *same* piece of data makes the point sharply. **Organisation A blocks the IP**: the ticket is closed in four minutes, the adversary rotates infrastructure the same day, nothing is learned, and the same work repeats the following week — the organisation remains vulnerable because, from data that was only ever data, it learned nothing new. **Organisation B asks who is behind it**: it pivots on the IP, finds nine sibling domains, recognises a campaign targeting its own sector, and changes an access-control decision — enforcing MFA — rather than a mere firewall rule. The adversary sees roughly the same immediate effect (the attack does not land), but to be effective again against Organisation B, merely changing the IP address is no longer enough; the attacker must overcome a genuinely harder obstacle. This is the whole point of CTI: taking decisions on the basis of **contextualised information**, so that the best possible decision shifts the cost of the next move onto the adversary's shoulders instead of leaving it on your own.

The Intelligence Funnel

Viewed from the standpoint of the data itself, running a CTI process means progressively refining a large, messy input into a small, decisive output. The stages form a funnel.

At the mouth of the funnel is **noise**: everything that is collected according to the Priority Intelligence Requirements (discussed later). By filtering out what is not understandable, the noise is reduced to **data** — artefacts grouped by their defining characteristics but still without context or meaning. Correlating data and assigning it a purpose turns it into **information**: data *with* a purpose. Checking which pieces of information actually help to take a specific decision raises information to the level of **intelligence**.

Finally, when the decision supported by that intelligence can be made effective against the adversary, you have **actionable intelligence** — intelligence-led, evidence-based assessments that can be initiated and acted upon.



Two properties of this funnel explain much about how CTI works in practice. First, the **upper stages are comparatively easy** because they are essentially technical: collecting data and removing noise can be done at scale with limited skill — you must understand which internal and external data you can collect and how to normalise sources, but these are technical problems that can be solved. Second, as you move toward the **lower stages the complexity rises and human intervention becomes indispensable**: distilling genuinely actionable intelligence requires knowledge that is difficult to acquire and comes mostly from experience. This is why a full CTI process is not for every organisation. It demands data, tools, and scarce human experts simultaneously, so only organisations with a strong motivation — those protecting extremely valuable assets — can realistically sustain one.

Who Can Build a CTI Capability

Because the lower end of the funnel is so demanding, maintaining a full in-house CTI process is realistic only for organisations that combine three things: a strong motivation (extremely valuable assets to protect), the economic power to acquire data and tools, and the ability to attract experts who are scarce on the job market. In the Italian landscape, the national cybersecurity agency runs a full-fledged CTI process because doing so is part of its mission. In the private sector, only the largest companies can sustain one — a defence-and-aerospace group such as Leonardo not only runs CTI but sells it as a service to others, and a major energy company such as ENI has both the economic power to build the capability and a strong motivation, since a global operating picture is fundamental to its business.

As organisations descend the scale of mission, size, and risk appetite, a point is reached where a company concludes it is not mature or large enough, and does not face a high enough level of risk, to justify so complex a process. Such organisations are not left defenceless; they typically **externalise** the function. The important caveat is the trade-off this introduces. When you buy CTI from a third party, the quality of the result depends on how much the provider can **customise** the intelligence to your case — and the real value of CTI lies precisely in being strictly tailored to you. A provider, however, must operate at scale to be cost-effective, so it is pushed toward products that are as general-purpose as possible. Bought-in CTI is therefore usually better than having none at all, but it will be less mature and less effective than intelligence tailored specifically to your organisation.

A middle path exists. Organisations lower on the maturity scale can pool their efforts by forming a sector association and funding a dedicated office — an **ISAC** (Information Sharing and Analysis Center). The logic is that organisations in the same market segment tend to face the same adversaries, so most of the intelligence they need is shared: rather than each paying for a perfectly tailored service, they are similar enough to share the peculiarities the service should have, and so they pay once, together.

Who Consumes CTI

Intelligence produced by a CTI process is used by many different players inside an organisation, and the **same output** can serve very different consumers taking very different decisions on very different timescales. Each consumer can be characterised by the decision they take, the product they need, and the time horizon over which that decision — and the value of the intelligence behind it — remains relevant.

Consumer	Decision they take	Product they need	Horizon
SOC L1 / L2	Escalate or close	Enriched, contextualised indicators	Minutes
Incident Response	Contain, scope the intrusion	Actor TTP profile, related indicators	Hours – days
Threat Hunting	Which hypothesis to test	Prioritised TTPs, hunt packages	Days
Detection Engineering	Which detection to write	ATT&CK coverage-gap analysis	Weeks
Vulnerability Mgmt	What to patch first	Evidence of active exploitation	Weeks
Red / Purple Team	Which adversary to emulate	Emulation plan	Months
Risk and CISO	Where to invest	Organisational threat profile	Quarters
Board / executives	Budget and priorities	Executive brief, BLUF format	Years
Legal / Compliance	Whether to notify	Dated, factual reconstruction	Legal deadlines

The **Security Operations Center (SOC)** is the most obvious consumer: it takes the output of the CTI process and acts on it directly. If the intelligence attached to an IP address is a technical decision such as "block this address," the SOC implements it; on the basis of the intelligence it may also escalate a case. The actions of a SOC play out on a human timescale of minutes, because the information it receives is technical and is used to gain an advantage in a short time frame. That advantage is also short-lived: the value of this kind of information **decays rapidly**, which is exactly why the decisions must come quickly.

At the level of **Incident Response**, CTI helps responders deal better with an ongoing incident — knowing how a specific ransomware gang acts against its victims helps them find the infection vector or block the spread across client machines. **Threat hunting** deserves special clarification, because it is the opposite of threat monitoring. Suppose intelligence tells you that a particular malware calls out to a specific command-and-control IP address and stores itself in a specific Windows directory. With this you can take two different kinds of action. The first is **monitoring/prevention**: set up a network rule so that as soon as a connection to that C2 address appears, you raise an alarm and block it — acting as an incident unfolds. The second is **threat hunting**: not assuming you are already infected, you proactively check whether you *are* a victim by launching a script across thousands of desktops looking for that specific file name in that directory. If it is found on one or more machines, you have discovered that you were the victim of a so-far-silent incident you had no prior clue about. Monitoring watches for a threat arriving; hunting actively looks for a threat already inside.

Other consumers use intelligence on progressively longer horizons. **Vulnerability management** receives intelligence that someone is actively exploiting a newly discovered vulnerability, and uses it to raise the priority of patching that vulnerable software relative to other work. The **red team** needs to mimic the real adversary during its exercises — not run random attack scripts, but reproduce the activities a real attacker targeting the organisation would plausibly perform — and it draws the necessary knowledge of

likely techniques from CTI. At the decision level, the **Chief Information Security Officer (CISO)** decides where it is best to invest the annual budget, and the **board** decides how large that budget should be. If intelligence indicates a strong campaign of attacks building against the organisation's market segment, the board may decide to give the security office more resources — "next year, their budget plus 20%." In short, CTI is a process that helps decisions be taken at **every level** of the organisation, each for its own purpose.

The Three Levels: Tactical, Operational, Strategic

Because different actors at different levels take different decisions from the same process, the **level of abstraction** of the intelligence and the way it is used vary enormously — even when it derives from a single underlying event. A phishing campaign against the finance department, told to three different consumers, produces three different products.

At the **tactical** level, to the SOC, the goal is to *block and detect*: the useful product is the set of sender domains and URL patterns, the attachment hashes and a YARA rule. It is machine-readable and valid for days. At the **operational** level, to incident response and hunting, the goal is to *understand the campaign*: the actor uses OAuth consent-phishing rather than attachments, and the post-access behaviour involves mailbox rules and forwarding. This is human-readable and valid for months. At the **strategic** level, to the CISO, the goal is to *decide where to invest*: the organisation's sector is being systematically targeted, and identity — not the endpoint — is the contested ground. This is decision-oriented and valid for years.

The goals of the three levels can be stated compactly. The **strategic** level (executives) provides an understanding of identified and credible threats, correlated to business impact. The **operational** level (managers and analysts) enables the formulation of approaches to dealing with threats and the prioritisation of team activity. The **tactical** level (security operators) provides an understanding of how to mitigate threats and enables the tools to do the heavy lifting.



This is not fundamentally different from how any business takes decisions. A winery, each year, decides which grapes to grow over the coming five years — a decision taken not on a morning's whim but on the basis of market intelligence collected beforehand, precisely to reduce the risk that a bad choice harms revenue. Cyber threat intelligence plays the same game: the board takes decisions, valid for years, on how to reduce the risk that an unknown actor on the other side of the world compromises the organisation's IT systems and damages the business.

The Intelligence Cycle

Intelligence is obtained from data through a process that is **not linear**, even though the funnel might suggest that it is. It is better understood as a cycle that iterates on data in order to improve the process itself. Practitioners across intelligence disciplines — market intelligence, cyber intelligence — refer to the same cycle of a few steps, and the reason the cycle exists is to serve a very concrete goal, which is worth making explicit before listing the steps.

Reconstructing the Attacker's Playbook

The whole process, however abstract it sounds, serves an extremely practical end. To see why, think from the attacker's side. When an adversary decides to target you, they begin with a reconnaissance phase in which they study the attack surface and decide a plan of action; then, in a weaponisation phase, they turn that abstract plan into something concrete. By the end of weaponisation — before the active part of

the attack begins — the attacker effectively possesses a *manual of instructions* that describes every step of the attack. This manual is an abstraction that does not literally exist in such a clean form, but the attacker nonetheless has a clear idea of the sequence of steps needed to reach the final goal.

If you could read that manual, you could build a perfect defence. Suppose the plan said: at a certain point the attacker will deliver a malicious email attachment with specific characteristics — then you improve your mail server configuration so that an attachment matching those characteristics is caught. If the attachment, once opened, would exploit a vulnerability in a particular process — then you configure the endpoint security system to detect malicious activity against that process. After installation, the malware would write a file in a particular directory — then you reconfigure your tooling to check for exactly that file. Finally the malware would beacon to a particular social-media feed — then you block all connections to that feed. With the attacker's full instruction manual in hand, you could in theory assemble a defence plan that is **100% effective**.

The problem is that you do not have the plan. What you *can* do is form **hypotheses** about its contents. You will never have exact access to every step the attacker will take, but information about the attacker can be inferred from data about their previous actions. If you know the attacker is known to deliver phishing emails carrying a password-protected ZIP attachment whose password is written in the email body, that is precious: you do not know for certain that this step appears in the manual written for *you*, but because the attacker used it before, there is a high probability they will reuse the same technique against you — because a rational attacker acts in a cost-effective manner and reuses what already works.

This is the practical reason for running a CTI process. Through CTI you are, in effect, asking a precise question: *which actions will the attacker take against us — what is written in that manual?* The more you can reconstruct the attacker's key tactics by profiling them — rebuilding the instruction manual, which is called the **playbook** — and the more you set up defences that replay that playbook locally, the better prepared you are. Being prepared against an adversary whose behaviour you *cannot* anticipate leaves open a mass of possibilities that can hardly be managed efficiently: if you do not know what the attacker will do, there is no principled way to decide whether to buy that antivirus or to upgrade the firewall. You run the intelligence cycle precisely to rebuild the hypothetical playbook, and rebuilding it lets you take informed decisions by prioritising defensive interventions. The better you are at rebuilding the playbook, the better the decisions you take will be.

The Six Steps

The cycle consists of six standard steps.

1 — Direction. The first stage states the goals of the CTI process. You define requirements — what you want to know — by posing the questions the process should answer, expressed as Priority Intelligence Requirements. Once you have the list of questions, you decide which data is needed to answer them and you produce a collection plan.

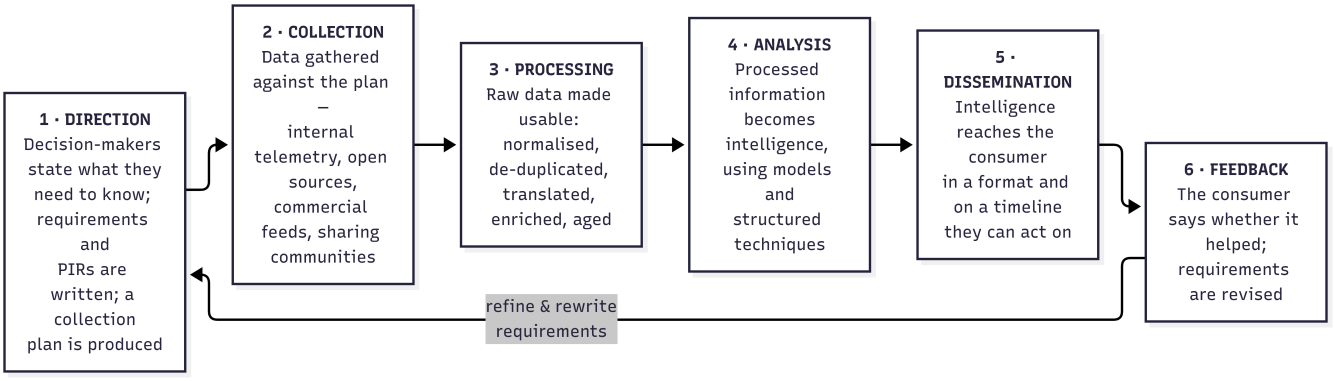
2 — Collection. You gather data *against the plan* — not at random, but exactly the data needed to produce the intelligence that answers those questions. Data may come from internal telemetry, external sources, commercial feeds, and so on, but what is collected was defined in the first step.

3 — Processing. This is the stage that transforms raw data into usable information. Practical steps include normalising the data, removing duplicates, and translating it where needed. A particularly important aspect is that data **ages**: a piece of data that is crucial now may be useless in a month, so data carries a notion of freshness. Some data lasts forever with no deadline; some is useful only for a very restricted time frame. Aging must therefore be managed.

4 — Analysis. Processed information becomes intelligence through correlation, using various models and structured techniques (several of which are studied in their own right). At the end of this fourth stage you hold the intelligence in hand — the information that answers the questions posed at the first stage.

5 — Dissemination. The intelligence reaches the consumers who originally expressed the requirements, in the format and on the timeline they can act on.

6 — Feedback. You ask the consumers whether the intelligence was actually actionable: were the decisions it drove effective, are there further questions to answer, and do the original questions need revising — is a question still valid now as it was a year ago? This feedback is essential because it allows the requirements to be refined and rewritten, and it is exactly where the cycle closes: dissemination produces feedback that flows back to planning and direction, starting the cycle again.



The essential point is that the cycle is not merely a sequence of actions performed on data. It starts from requirements and runs through collection, processing, and analysis to produce an output, but that output is both *used* by the consumers for their purpose *and* turned into feedback that improves the CTI process so it can start again. Often the results of analysis open up a new interpretation of what the adversary is doing, which lets you pose new questions — and any new question requires a fresh phase of planning and direction.

Priority Intelligence Requirements

The intelligence cycle is built on top of questions: someone asks a question, and the CTI process is constructed to answer it. These questions are expressed in a formal shape called a **Priority Intelligence Requirement (PIR)**.

> A PIR is a question whose answer should **change a decision** that a **named person** has to make, **by a date**.

Each PIR directs a decision. It is owned by its consumer: the **owner defines** the PIR, the **CTI team serves** it, and the owner then **consumes** the outputs. The relationship is a one-directional flow — a client poses a question, the question is written as a PIR, the CTI process is fuelled by that PIR, and the resulting intelligence is returned to that client.

The Five Properties of a Good PIR

A well-formed PIR has five properties.

Tied to a decision. You can name the decision it informs and the person who makes it. A PIR with no bounded decision produces non-actionable intelligence. For example, "should we upgrade our EDR to the next version, or stick with the current one for another year?" is a concrete, typically budget-related decision to which a question can be attached.

Specific. It must be a clear question with a well-defined direction, bounded by sector, geography, technology, or time — not a general topic. "Ransomware" is not a PIR because it does not clarify the intent of the requirement; "which of the top five ransomware groups would our EDR fail to detect at initial access?" is a question with a clear intent.

Answerable. It must be answerable with the data you have or can realistically obtain, because the answer will come from analysing collected data. If a question requires acquiring data you cannot access, you cannot answer it and the PIR is useless.

Time-bound. It has a deadline, because the decision it serves has one. This stems directly from being tied to a decision: a decision you could take whenever you like, with no deadline, is in practice not a decision, because it has no practical impact on the organisation. All decisions are time-bounded; since decisions are directed by intelligence and intelligence is produced by PIRs, PIRs inherit the same time frame. The deadline must also be *realistic*: if you ask the CTI team to answer within two weeks and they cannot gather the needed information that fast, the bound is unrealistic and the process cannot produce something that directs the decision in time. Equally, a question with no clear time frame at all is useless, because there is no point in time at which the intelligence will drive a decision.

Falsifiable. It must be possible to state in advance what evidence would settle the question — including evidence that would *contradict* the expected answer. The stakeholder will take a decision on the output but does not yet know what that output will be, so the PIR must clearly define the kind of evidence the process is expected to produce for that evidence to be useful in taking the decision.

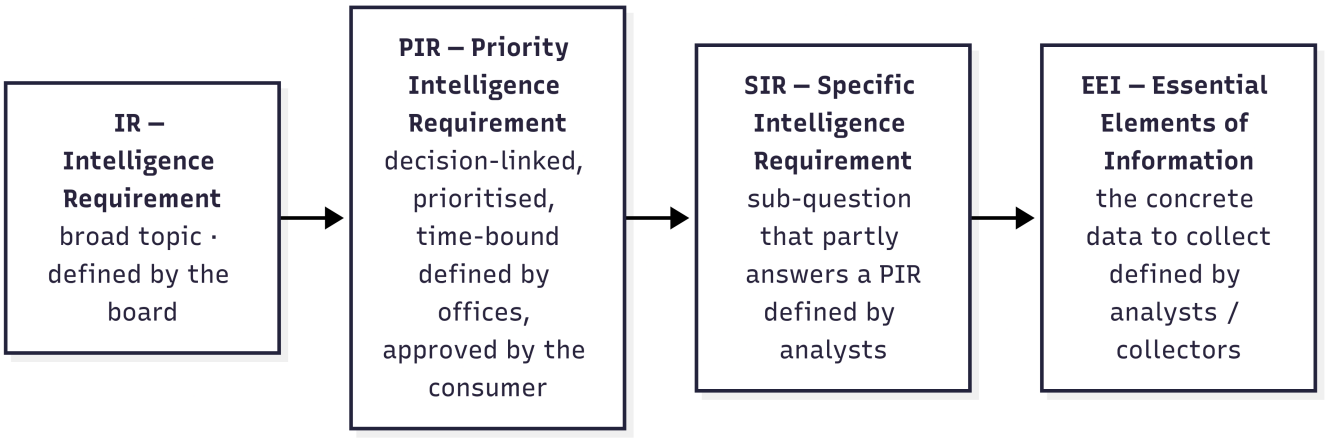
The Requirements Hierarchy: IR, PIR, SIR, EEI

PIRs sit inside a hierarchy of requirements that descends from broad organisational interest to the concrete data to be collected.

Level	What it is	Example	Written by
IR — Intelligence Requirement	A broad topic the organisation needs to understand	Understand threats to our European payment operations	CTI lead with stakeholders
PIR — Priority Intelligence Req.	A prioritised, decision-linked question with a deadline	Which ransomware groups have targeted EU payment processors in the last 12 months, and how do they gain initial access?	CTI lead, approved by consumer
SIR — Specific Intelligence Req.	A sub-question that partly answers a PIR	Which of those groups use valid accounts for initial access?	Analyst
EEI — Essential Elements of Info.	The concrete observable to be collected	Leak-site victim lists, IR reports, IAB listings for our sector	Analyst / collector

The flow of this hierarchy, and the reason priority sits where it does, is as follows. The **board** defines broad **Intelligence Requirements** — topics of strategic importance, such as understanding a phenomenon on a larger scale. These IRs percolate down to the various offices. Inside each office, people take the generic requirements, analyse them, and decide which aspects matter most for their office and which matter less — in doing so they assign **priorities** and express **Priority Intelligence Requirements** derived from the general ones. Starting from a PIR, the CTI team may further subdivide it into more specific requirements (**SIRs**) when a question cannot be answered by a single instance of the process and must be broken into sub-questions whose answers, taken together, answer the original.

Finally, from the specific requirements the team derives the list of **Essential Elements of Information** — the data that must actually be collected.



Why speak of *priority* intelligence requirements rather than plain intelligence requirements? Because priority is not based on the *importance* of a requirement but on your **ability to take a useful decision by its deadline**. Everyone works with bounded resources and cannot do everything. If a question needs an answer within a two-month window because the decision it drives has that deadline, the intelligence needed to drive that decision has a higher priority than intelligence feeding a decision you will take in a year. At the IR level the requirements are general and not necessarily tied to a precise deadline; priorities are assigned lower down, by the offices, on the basis of deadlines. If you are unable to define priorities — to turn general requirements into prioritised PIRs fed to the CTI team — running the process effectively becomes very difficult.

Well-Formed vs Poorly-Formed PIRs

The difference between a question and a usable PIR is best seen by contrast.

Poorly written	Well formed
What are the current cyber threats?	Which threat actors have targeted EU retail payment processors since January 2026, and by what initial-access vector?
Is APT29 a threat to us?	Has any actor using the TTPs attributed to APT29 been observed against organisations running our identity stack, in the last 18 months?
Monitor the dark web.	Are credentials belonging to our domains being offered on infostealer log markets, and at what volume, month by month?
Are we secure against ransomware?	Which of the top five ransomware groups by victim count would our current EDR coverage fail to detect at the initial-access stage?
Give us intelligence on our supply chain.	Which of our 20 most critical suppliers have disclosed a breach or appeared on a leak site in the past 24 months?

"What are the current cyber threats?" is a question but not a good PIR: there is no clear decision attached, no clear time bound on the answer. Its improved form — "which actors have targeted EU retail payment processors since January 2026, and by what initial-access vector?" — carries an implicit decision (whether any actor uses specific access vectors to target us as a payment processor) and defines its time-boundness directly in the wording. Likewise, "give us intelligence on our supply chain" lacks falsifiability — it does not define what kind of evidence is expected — whereas "which of our 20 most critical suppliers have disclosed a breach or appeared on a leak site in the past 24 months?" clearly states the evidence and could drive a decision such as tightening security clauses in supplier contracts.

Where PIRs Come From

PIRs derive from the more generic intelligence requirements, but those in turn should express questions about something of genuine interest to the organisation. Such questions arise from a few recurring sources.

The first is **crown jewel analysis**. You ask which assets the organisation cannot afford to lose — the assets that, if lost, would cause it to fail its core mission. For a university, a first answer is the data on students' academic careers: a university's central mission is to teach and then to certify, with a degree and a mark, that a person studied; losing that information means failing the mission, so it is a crown jewel that must be protected at almost any cost. Research data is another asset that cannot be lost without undermining the institution's credibility as a research body. Crucially, this is not a black-and-white classification — there are degrees of impact from losing different assets — and assets need not be data: for a factory producing goods, the crown jewel may be the ability to keep the production line running twenty-four hours a day; for another organisation it may be strategic market information. Having listed the crown jewels and asked who might target them, you place questions and define PIRs on that basis.

The second is **threat modelling of the organisation**. Here you look at your organisation from an abstract standpoint and list all the threats that are *credible* against it, given its sector, geography, size, technology stack, public profile, and partners — the factors that determine who is plausibly interested in you. A small truffle producer in the Lazio region, for instance, might face the standard ransomware actor like anyone else, but also a larger competitor in the same sector trying to steal information about the good spots in the countryside where the truffles are found. Threat modelling is widely used for organisations in the IT sector and less commonly outside it, but the approach is general enough to apply to any organisation; the resulting model helps define PIRs, since each credible threat actor becomes something to check for in the outside world.

The third is **lessons from past incidents** — typically the most under-used source of requirements, and a fundamental one. Whoever attacked you before, if only partly successful, will try again. This is especially true for victims of ransomware who chose to pay: paying signals to the attacker a readiness to be victimised again, and studies have shown that organisations which paid a ransom are frequently victims again within a short time. Understanding what went wrong before, and who attacked you, is essential preparation for the future.

The fourth is **stakeholders**, internal and external. If they struggle with decisions that touch external cyber threats, interviewing them to understand their needs is a good way to collect meaningful PIRs — most will not know what to ask for until you ask them.

A Common Error

This last point carries a warning. Interviewing broadly can easily leave you with pages and pages of questions, but these are *priority* intelligence requirements, not generic ones: an effective CTI process answers a handful of prioritised questions in a given time frame, not an endless list. Each requirement spawns its own instance of the CTI process — its own data to collect, technical assets to set up, processing, and analysis — so every single PIR is expensive, and prioritising is what keeps the process sustainable.

The most common error in practice runs in exactly the wrong direction. Rather than starting from requirements, many implementations start broadly **from data**. Someone is hired, told to build a CTI process, and — because collecting PIRs is difficult and slow, and they want to look effective from day one

— does the only immediately practical thing: they ask for the list of available data, collect it, and start analysing without any specific question in mind. The output is a report measured by its weight — "I've already produced a 360-page report in six months" — full of data linked to no context and no question, and therefore essentially useless. Producing intelligence first and looking for a use for it afterwards rarely yields anything valuable; even when some use can be found, the process is flawed from the start, because more important questions that should have been answered first are still sitting unaddressed.

The discipline is the same as in any engineering practice: a technical team would not write an application without first collecting requirements, and a good application is the one built by correctly identifying and analysing requirements. Useful cyber threat intelligence is, in the same way, the intelligence obtained by first defining requirements and only then collecting and analysing the data that serves them.

Data Collection

Once the requirements define what must be known, they also determine which data must be collected. That data comes from a wide range of sources, each with its own strengths and weaknesses.

Source	What it gives you	Strength	Weakness
Internal telemetry	What actually happened to you	Ground truth, high relevance	Limited to your own visibility
Own incident history	Who has already targeted you	Most predictive source you have	Small sample, often undocumented
Sharing communities	Peer observations, early warning	Sector relevance, trust	Requires reciprocity and effort
Vendor reporting	Deep analysis of actors and campaigns	Expertise, scale of telemetry	Marketing incentive, naming chaos
Open feeds	Indicators at volume	Free, immediate	Noisy, stale, rarely contextualised
Commercial feeds	Curated indicators, finished reporting	Coverage, SLA	Cost; overlap; opaque methodology
Underground / dark web	Access brokers, leaks, chatter	Early signal on intent	Legal and ethical exposure
Honeypots, sinkholes, scanning	Adversary infrastructure at scale	Original observation	Operational cost

The richest and most relevant sources begin inside the organisation. A reasonably well-organised, medium-sized enterprise has **internal IT assets** that produce large quantities of data: network switches generate logs, logs are collected by an intrusion detection system that feeds a SIEM used for alerting and monitoring, EDR and antivirus agents on the hosts produce events, and firewalls raise alarms. This internal data is immediately available to feed the process for those PIRs that can make use of it. **Own incident history** — all the lessons learned and technical details of previous incidents — is part of this internal picture and is among the most predictive data an organisation possesses.

From outside the organisation, **sharing communities** (such as ISACs, the subject of their own dedicated treatment) provide data that is fundamental for understanding what is happening across peers. **Vendor reporting** lets you buy or obtain deep analyses of actors and campaigns. Data can also be acquired from less official sources such as the **underground or dark web**, where some malicious actors share part of their information as a function of their business. The clearest example is the **initial access**

broker — an actor whose sole purpose is to compromise systems to obtain access, do nothing else with it, and then sell that access, with a given level of privilege, to someone else. Initial access brokers advertise the assets they want to sell, more or less openly, on underground forums. This division of labour can explain otherwise puzzling incident timelines: an intrusion may show an initial compromise in one month and a full compromise only weeks later, with nothing in between — a silence consistent with one actor compromising a machine, gaining administrator privileges, and then stopping to resell that access on the black market to the actor who ultimately carried out the attack.

A final source is rarer and more expensive: the deliberate collection of attacker behaviour through **honeypots**. A honeypot is a fake network set up specifically to lure an attacker into attacking it instead of the real network. Rather than stopping the attack, you *divert* it into an emulated environment that resembles the real network closely enough to be credible, and you let the attacker continue to act against the false target, for the sole purpose of studying how they would have acted against your real systems. This is difficult and costly and is rarely feasible for an ordinary organisation. Because the only purpose of a honeypot is to collect information that feeds CTI, honeypots are often run by the very companies whose main mission is producing CTI feeds: they collect data from their clients where permitted, and they also run a few honeypots — rather like fishing, hoping an attacker will be distracted enough to fall into the net and reveal what they are capable of.

Processing

Collected data is not yet usable. Before analysis it must be preprocessed through a sequence of fairly standard data-management steps.

Normalisation. The same data may arrive from different sources using different standards and formats — this is especially true of raw data from internal assets, where different vendors use different log formats. One representation per object type is imposed, with consistent timestamps in UTC and consistent actor naming.

De-duplication. The same indicator may arrive from several feeds; removing duplicates keeps metrics clean and avoids double-counting.

Defanging of dangerous data. Sometimes data must be *modified* because it is itself dangerous. Consider a phishing email containing a link ("click here to change your password before it is too late") that points to an asset controlled by the attacker. Before inserting it into the CTI process, the link is altered — for example by inserting brackets or asterisks that break it — so that it cannot be clicked accidentally by anyone handling the data.

Enrichment. A great deal of raw data can be augmented with further information, and enrichment is largely what turns data into information. Given an IP address, for example, you can query the web to obtain the autonomous system it maps to, the owner of the containing IP block, and the service provider actually controlling it; similar enrichment (geolocation, WHOIS, passive DNS, reputation, first-and-last-seen) applies to domain names and other observables.

Aging and decay. A piece of data does not last forever. This matters most for low-level observables such as IP addresses and fully qualified domain names, which are useful only for a limited time because attackers rotate them quickly. If today the attacker controls a compromised machine in the UK, that information may be good only for the next 48 hours or a week: once the address starts appearing in public databases as linked to malicious activity, the attacker simply moves to a different one. Keeping such an indicator in the knowledge base forever would, taken to its logical end, mean eventually flagging

the entire IP address space. A decay model therefore reduces confidence over time and eventually retires the indicator automatically.

Quality control. Feeds routinely contain the defender's own sinkholes, public DNS resolvers, and CDN addresses — data that is irrelevant or actively harmful if acted upon — and this must be filtered out.

Making Intelligence Actionable

With the data collected and processed, the final step is to transform it into **actionable intelligence** — to decide which defensive countermeasures are worth putting in place. Understanding *why* a particular countermeasure is useful requires returning to the behaviour of the threat actor, and specifically to the assumption that the actor is **rational**.

A rational attacker will use the **cheapest technique** in their possession that achieves the intended goal, and will try to **reuse** that technique as much as possible. This reuse has two dimensions, because for most actors you are not the only target. An attacker typically has several targets, so each technique in their arsenal must be **scalable across targets** (horizontal, in space): a virus used against you should be usable, with minimal changes, against another company in your sector. A technique must also be **scalable over time**: if the attacker uses a technique today and you manage to block it today, they want to reuse that same technique tomorrow against someone else, because reusing it lowers their own cost. The attacker therefore builds a **toolbox** in which each tool is made as reusable as possible — exactly as in a well-run software-engineering process, where reuse of what you already own and control is maximised. This assumption is not incidental; it is the foundation of the methodologies used to turn raw data into intelligence that drives a decision.

The Economics of Attack

Techniques differ sharply in how cheap and reusable they are for the attacker, and this is what makes some attacks far more common than others.

Inexpensive, valuable, scalable, repeatable	**Costly, valueless, unscalable, unrepeatable**
Phishing	Web vulnerabilities
Credential reuse	0-day exploits
Known vulnerabilities **with** public exploits	Known vulnerabilities **without** public exploits
Office macros	Embedded devices
Spyware	Crypto weaknesses
Vendor compromise	Insider threat

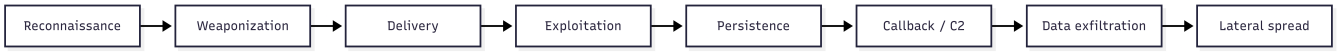
On the cheap side, **phishing** is extremely scalable: sending phishing emails to a large list of targets costs essentially nothing, and even slightly tailoring them to be more credible is very cheap. **Credential reuse** is similarly cheap — the black market is full of databases containing tens of thousands of compromised credentials sold for very little; most are useless because passwords have changed, but they are so cheap that finding even a couple that still work is worthwhile.

On the expensive side, consider an attacker who finds that a target runs a web application with a SQL-injection vulnerability on a specific page. Finding that vulnerability and writing the exploit code that turns it into the advantage the attacker wants is expensive — and it is **not repeatable**, because moving to a different target means a different website where that particular vulnerability is absent. **Zero-day exploitation code** is repeatable in principle but *extinct* in practice: using it risks burning it. Exploiting

a vulnerability that is publicly listed but for which **no public exploit code exists** is also costly, because the code is not simple to produce — although the price of this category is dropping quickly, since large language models and similar tools now help produce such code much faster, shifting the boundary between "cheap" and "expensive." **Crypto weaknesses** sit firmly on the expensive side; the attacker must invest heavily to gain any advantage. The attacker knows exactly what is in their box, the value of each tool, and the skill each requires, and will do their best to reuse the scalable, repeatable tools as much as possible.

Where to Impose Cost: the Kill Chain as a Frame

This economic picture tells the defender *where* to act. The attack unfolds as a sequence of phases, and at each phase the attacker employs one — sometimes more than one — technique against you. This sequence is the **Intrusion Kill Chain**, which serves as the reference model for the rest of the analysis: it is not merely a way to depict an attack but a methodology for analysing intrusions in order to extract actionable intelligence. Its phases run from reconnaissance through to the attacker's final objective.



At each step you may decide how to act against the attacker — but **not all steps are equal**. For some steps the attacker uses a highly scalable technique; for others, an expensive, non-repeatable one. Suppose the attacker uses **phishing** in the delivery phase. You must defend against phishing, but defending against it is *expensive for you precisely because it is cheap for the attacker*: they change the email at each new attempt and grow ever more proficient at writing credible messages, so even when you block the next phishing email at your mail server, the price they pay to modify it and try again is tiny — your heavy investment buys a defence that is quickly circumvented.

Now suppose instead that the attacker has designed a **specific protocol to exfiltrate data** — dividing the data into chunks, encrypting it, and sending it to a bespoke botnet. That capability is effective, but it also lets you define very precise monitoring conditions and rules to identify the exfiltration, block it, and ensure no data leaves your compromised assets. The crucial point is the asymmetry: for the attacker, overcoming this defence is **extremely expensive**, because to defeat your monitoring rules they must rewrite a large portion of their code, stand up a new botnet, change the encryption mechanism, and re-engineer all the data-preparation logic — then test and redeploy it from scratch.

Different countermeasures therefore impose very different costs on the adversary, and you want to be most effective **exactly where the attacker relies on techniques that are non-scalable, non-repeatable, or expensive** — because that is where you cause the most damage. For the defender, "causing damage" means imposing a larger cost on the attacker if they want to try again. The ultimate goal of the whole game is to **alter its economics**: to make the price the attacker must pay next time so large that they no longer see any advantage in succeeding against you.

This is also why the CTI process has a clearly bounded scope. It is **effective only against adversaries who are cost-bound**. If an attacker has access to an unlimited supply of resources, this approach will not work, because they can always substitute whatever you block with the next top-level technique. The kind of adversary the CTI process is genuinely effective against is the **rational attacker who reuses techniques as much as possible** — because that reuse is what gives you a clear target: by understanding those techniques and blocking exactly the ones that are expensive for the attacker to replace, you impose the greatest cost and gain the most durable advantage.

Indicators

The fundamental element of intelligence in this analysis is the **indicator**: any piece of information that objectively describes an intrusion. Indicators come in three kinds, distinguished by how they are formed.

An **atomic** indicator cannot be broken down into smaller parts and still retain its meaning in the context of an intrusion. An IP address is the classic example: split it and the pieces describe nothing. Email addresses and vulnerability identifiers are likewise atomic.

A **computed** indicator is derived from data involved in the incident through some processing step. The archetype is a **hash value** (fingerprint) of a malware sample, produced by applying a hash function to the file.

A **behavioural** indicator is a collection of atomic and computed indicators assembled together, qualified by quantity and often by combinatorial logic. For example: *"the intruder used a backdoor (identified by a given hash) that generated network traffic matching a particular regular expression, at a certain frequency, toward a particular IP address."* Such an indicator binds several lower-level indicators — some atomic, some computed — into a description of behaviour.

Atomic and computed indicators are **low-level**. The hash of a malware sample is different in character from a domain name, because a domain name already brings together derived information — for instance, the set of IP addresses it resolves to. Within the intelligence lifecycle, indicators serve two purposes. Some drive **immediate action**: the fingerprint of a malware sample can be pushed straight into a company's EDR configuration so the malware is detected and blocked at once. Others support **long-term decisions** about how to protect the organisation.

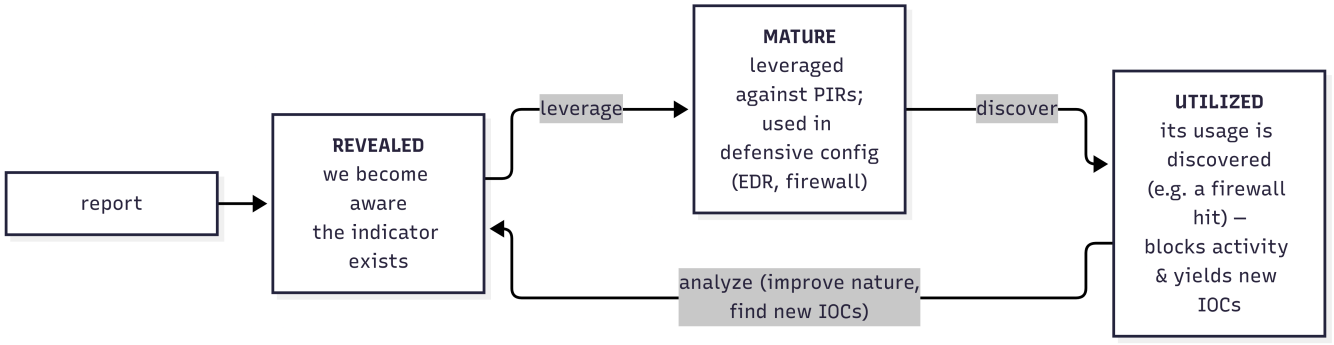
The Lifecycle of an Indicator

Beyond their nature, indicators have a lifecycle, and the states of that lifecycle may arrive from different sources.

As soon as the organisation becomes aware that an indicator exists, it is **revealed** — reported into the knowledge base. The analyst may then choose to **leverage** it, matching it against the defined PIRs; at that point the indicator becomes **mature** — it is taken into account by the defensive strategy and used, for example, to alter the configuration of an EDR or a firewall. When an indicator is mature, the organisation is actively empowering its systems with the knowledge the indicator carries.

Once mature, two things can happen. The indicator may **never be used** — it stays in the knowledge base but nothing further is ever seen that relates to it. For a hash fingerprint, this is good news: that sample was seen only once. Alternatively, the indicator becomes **utilized**, once its usage is actually discovered. Suppose a mature indicator is an IP address mapped to command-and-control activity for a given adversary, and a firewall log line later shows an attempted connection from one of our PCs to that address: the indicator has been utilized, and it has proven genuinely useful.

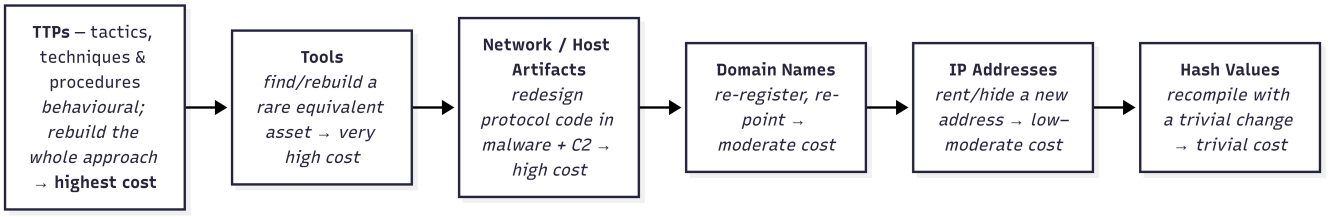
That single utilisation yields two distinct advantages. First, it **blocks** malicious activity — a compromised machine we did not know about was trying to exfiltrate to an adversary. Second, it reveals a **new source of further indicators**: seeing that log line, the analyst locates the machine, detaches it from the network, and analyses its internals to see which process attempted the connection, deriving still more indicators of compromise. In this way, the transition from mature to utilized often lets analysis **improve the nature** of an indicator or **identify new** indicators of compromise. The *size* of an organisation's knowledge — not its quality, but its breadth — is measured by the number of significant indicators it holds about a specific incident or threat actor.



The Pyramid of Pain

Indicators do not all have the same value, so the value of an indicator must somehow be measured. The most correct way would be to look at the results of *utilized* indicators — to check which ones actually protected us from incidents — but this has a fatal drawback: it tells us an indicator's worth only *after* it has been used, whereas we need to know the worth *before* utilisation, so that we can set priorities and take better-informed decisions about which actions to perform. We therefore need a way to estimate an indicator's value in advance, without knowing whether it will ever be used. The model that does this is the **Pyramid of Pain**.

The idea is intuitive: indicators of different natures are mapped to different layers, and an indicator's **value is proxied by how much pain — how much cost — it imposes on the attacker** when we act on it. The higher the layer, the greater the pain.



At the **bottom** sit **hash values** (fingerprints), and their position has two meanings. First, it is easy to collect enormous numbers of them: vast collections of hash indicators exist online, because countless malware samples are produced daily and each, once detected, has its fingerprint loaded into a database — a continuously growing collection. Second, their defensive value is low. Suppose we fingerprint a sample found on a compromised machine and load that hash into the EDR so any program with the same fingerprint is blocked. After a few days of silence the attacker, hearing no beacon, concludes something was blocked; one hypothesis they will try is that the sample was discovered and its hash loaded into an EDR. They may even *confirm* this if an analyst made the mistake of uploading the sample to VirusTotal, which fingerprints it into its own database and signals to the attacker that the sample is now identified as malware. The attacker then takes the source code, changes a couple of lines, recompiles, and obtains an identical malware with a *different* fingerprint — rendering our hash-based rule useless. The cost of defeating a hash indicator is therefore minimal. Hashes have **low value but exist in huge quantities**, and the defensive activity around them is almost entirely automated: EDRs continuously fetch new hash collections from open databases to stay current.

Moving **up** the pyramid, value increases. Changing the **IP address** of a command-and-control centre is not very expensive today, but it costs more than editing code: the attacker must obtain an address from a different provider, conceal their presence there, or buy a compromised machine on the dark web. So an IP indicator protects us for longer and imposes a larger cost on the attacker to regain effectiveness. **Domain names** sit higher still. **Network and host artifacts** — for example, a malware that beacons over DNS always embedding the same fixed value in its requests — are more painful again: to defeat such

an indicator the attacker must redesign part of the communication protocol, changing code in *both* the malware and the C2. **Tools** are near the top: if the attacker relies on an exploitation framework built on the dark web and we block its use, they must source a similar, relatively rare asset and re-tool, which is very hard. At the **top** are **tactics, techniques and procedures (TTPs)** — high-level behavioural indicators that describe the attacker's *approach* (for instance, always infecting targets via phishing emails from a specific domain or with specific body content) and that bind together several lower-level indicators. Hampering a tactic forces the attacker to revise their whole approach from scratch, imposing the largest cost of all.

The reason cost is the unifying measure is that, against a rational, cost-effective attacker, the only way to stop them is to make the cost of success larger than the reward — at which point they stop. The aim, uncomfortable as it sounds, is to **discourage** rather than absolutely block. It is the logic of protecting an apartment in a big city: you improve your lock not to make burglary impossible, but so that your lock is better than the neighbour's, pushing the attacker toward an easier target. It is not glamorous, but it is what works.

Indicators of Compromise vs Indicators of Attack

There is a second way to read this pyramid, by dividing indicators into two families — families that are not sharply defined, so the boundary remains open to discussion.

An **Indicator of Compromise (IOC)** is any information that accurately describes a *specific* intrusion: the sample found on the compromised PC, the domain it used to beacon over DNS. IOCs are used **reactively**, as a countermeasure to attacks. Knowing a domain is used to signal a compromised machine, for example, you set up a DNS proxy in front of your resolver to intercept such requests and either block them or **sinkhole** them — answering with a local IP address so the malware beacons to a machine you control, buying time to observe the compromised host's behaviour without the attacker realising it has been detected.

An **Indicator of Attack (IOA)** lives at the top of the pyramid and includes TTPs. It does not necessarily describe one specific incident; it describes *how an attacker tends to behave* against a target. IOAs are used **proactively**, in threat hunting — to check for the presence of an attacker inside an environment that currently shows no signs of a known attack. If a particular actor is known to compromise a machine and then scan the local network for the domain controller, you set up rules and activities to actively search your network for signs of that behaviour; finding none suggests you were probably not a victim of that attack.

Tools and network/host artifacts straddle the border between the two families. The use of a scanner such as nmap after a foothold can be read as an IOC (a pure trace of compromise) or as an IOA (a behavioural step the attacker performs) — the same thing seen from two perspectives: that of the SOC team configuring countermeasures against ongoing attacks, and that of the threat-hunting team looking for undetected intrusions.

The Pyramid of Pain, then, attaches only a **proxy value** to an indicator, not its real value — which can be known only once the indicator has been utilised, something we would rather avoid, since utilisation means the attacker nearly succeeded. All indicators are useful, at different levels of abstraction: those at the bottom impose low cost and have low effect on the attacker but are cheap to obtain and easy to automate, while those at the top are far more effective against the attacker but far more complex for us to obtain. The analysis techniques considered next exist precisely to **distil these high-level indicators starting from low-level ones**.

Mapping Indicators to the Kill Chain

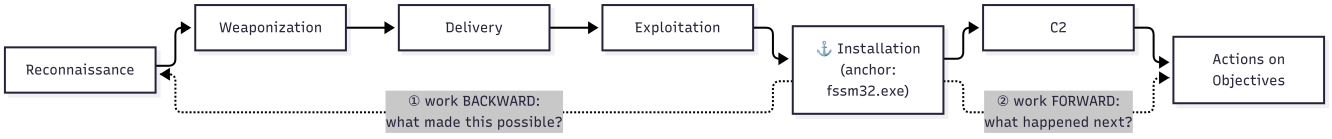
The way to distil high-level knowledge from low-level indicators is to organise them on the **Intrusion Kill Chain**. Any indicator can be mapped to the specific phase at which it can be observed. The fingerprint of a malware sample, for instance, maps naturally to the **Installation** phase, because that is the stage at which the sample can actually be seen on the host (reported, for example, as a file name at a path). Mapped across all phases, the indicators of a single intrusion form a picture like this:

Phase	Indicators (worked example)
Reconnaissance	Recipient list; benign file `tcnom.pdf`
Weaponization	Trivial encryption algorithm: Key 1
Delivery	`dn...etto@yahoo.com`; downstream IP `60.abc.xyz.215`; subject "AIAA Technical Committees"; email body
Exploitation	`CVE-2009-0658`; shellcode
Installation	`C:\...\fssm32.exe`; `C:\...\IEUpd.exe`; `C:\...\IEXPLORE.hlp`
C2	`202.abc.xyz.7`; HTTP request
Actions on Objectives	N/A

The difficulty is that when we first obtain an indicator for an ongoing incident, we hold only a **single point** in the sequence of events that make up the intrusion. Finding malware on a compromised PC tells us the sample is there, but we know the attacker took several steps *before* to land it there, and that the malware may have taken steps *after* — communicating with the C2, acting on objectives. At this stage we know only that one point. The task is to improve our knowledge around it.

Reconstructing the Attack

Kill-chain-based analysis starts from an **anchor**: the indicator we obtained somehow — found by chance, delivered by a monitoring device, or passed on by a colleague at another company. We place the anchor on the chain and then work in **two directions**, backward and forward.



Working backward proceeds by asking what the attacker must have done to achieve the result we see. For the malware to be installed on that machine, it was probably installed through some weakness. The first hypothesis — the operator installed something — can be checked by asking them; the answer ("I installed nothing, there were no updates") does not close the investigation but *rules out* that branch. The next hypothesis is that the attacker exploited a vulnerability in authorised software already on the machine; since we manage the machine we have the list of installed software and patch levels, so we check the web for known vulnerabilities and may arrive at two candidate vulnerabilities consistent with the unauthorised installation. We then ask how such a vulnerability was exploited — some software was run — and how *that* arrived: via a malicious link or a malicious email attachment. Both can be checked, because a managed machine gives us the full browser history (scan the last few days for suspicious activity) and access to the user's inbox (check recent mail for malicious items). Step by step, this traces the intrusion back to the Reconnaissance phase.

The engine of this process is the **falsifiable hypothesis**. Every hypothesis must come with a clear way to be negated: a fact which, if found true, would make the hypothesis definitely false. This is what lets us periodically rule out hypotheses that are not credible, until only credible ones remain. For each credible hypothesis we reason, "if this is exactly what happened, this kind of attacker activity should have left *this* kind of trace in our systems" — and that prompts us to **enlarge the inspection** to other assets that might hold further evidence. Discovering a suspicious link was clicked leads us to analyse email attachments; checking the inbox, we find a suspicious spreadsheet loaded with suspicious macros — a clear sign of malicious activity and a **new indicator** to place on the chain.

Working forward applies the same logic after the anchor: did the malware send messages to a C2? Did it perform other actions on the local machine or on other machines on the network? Each question prompts us to enlarge the inspection to other logs, reports, and machines that may contain further evidence.

At each phase we identify the **gaps** in our analysis. At the beginning everything is a gap; for every hypothesis that has not been falsified, we look for evidence, and when we find it the gap closes and we gain a new indicator. Some gaps remain open, because the attacker works to conceal their activity. Crucially, the *absence* of a log line about a malicious network action does not prove the action did not happen — it simply means the gap remains a gap: unproven, but still possible.

By the end of this repeatable reconstruction we may have collected a great deal: that besides `fssm32.exe` the malware installed a couple more files; that it contacted a given IP address over HTTP; a gap at Actions on Objectives; that the exploited vulnerability was an old 2009 one left unpatched; that delivery came by email with a given subject, from a given address, with a recoverable body; that the attachment exploiting the DLL was encrypted with a free algorithm whose key we recovered; and that the list of recipients came from a PDF left available on our own website. **Every one of these is an indicator of compromise.** Even at this low level of structure they are already **actionable**: we can check how many other PCs run the same vulnerable software version and launch a patching procedure; block any email from that sender; load the EDR with the fingerprints of those files; and configure the firewall to report or block requests to that site. From a single piece of information, a repeatable process has produced actionable intelligence that supports decisions.

Course of Action

The actions available to protect against attacks presenting the reconstructed characteristics are collectively called the **course of action**, and for each phase of the chain there are six possible strategies, conventionally the **six D's**. They form a spectrum from the merely observational to the actively offensive.

Detect — become aware that someone is again using a known indicator against us. This is the most basic action and can start right at the Delivery phase: configure the mail server so that any email matching the characteristics rings a bell and alarms someone.

Deny — prevent the event from happening. Knowing is important, but blocking is more important. Here the matching attachment is stripped and the mail delivered without it, carrying a red warning that it contained something malicious; or, for an exploitation, the vulnerable software is simply patched so the vulnerability is gone and the attack is wholly blocked.

Disrupt — make the event fail while it is executing, when you could not prevent it. The user opens the attachment, but the EDR sees a strange process trying to overwrite another process's memory, flags it as highly suspicious, and terminates it. The attack was working and was interrupted mid-course.

Degrade — reduce the effectiveness of an attack you can neither deny nor disrupt. This is not always achievable, but it is common against denial-of-service: you cannot block the flood, but you can reconfigure the load balancers so that requests from the offending autonomous system are directed to a single machine, leaving the rest of the cluster free for legitimate users. The attacker is not stopped, but their effectiveness is degraded — and regaining it costs them more.

Deceive — let the attacker continue and believe they are succeeding, while rerouting their effort onto a controlled infrastructure where the impact is nullified. The malicious email is intercepted and moved into a simulated environment where an automated process opens it and detonates the attachment inside a virtual, false system seeded with fake information, so that an attacker aiming to steal data steals worthless data. This is typically implemented with honeypots.

Destroy — remove the adversary's capability by acting against them: launching a denial-of-service against the C2, or finding a vulnerability in the C2 and destroying it. These offensive actions are, at least in Italy and across Europe, **completely unlawful**, so this strategy is generally unavailable to ordinary organisations or individuals. The picture differs elsewhere: in the United States recent legislation lets private companies, under specific authorisation, take adversarial action against recognised attackers, and a few firms already do so; a European proposal to change the rules exists but remains distant. A legal grey area arises from hiring a provider based in a country without such restrictions — whether this makes the commissioning Italian organisation liable is unclear and would be decided at trial. In practice, defensive work stops at the first five strategies; destruction is left to law-enforcement agencies, which have the proper legal basis.

These strategies combine with the kill-chain phases into the **Course of Action matrix**, where each cell names a concrete technique.

Phase	Detect	Deny	Disrupt	Degrade	Deceive	Destroy
Reconnaissance	Web analytics	Firewall ACL	—	—	—	—
Weaponization	NIDS	NIPS	—	—	—	—
Delivery	Vigilant user	Proxy filter	In-line AV	Queuing	—	—
Exploitation	HIDS	Patch	DEP	—	—	—
Installation	HIDS	"chroot" jail	AV	—	—	—
C2	NIDS	Firewall ACL	NIPS	Tarpit	DNS redirect	—
Actions on Objectives	Audit log	—	—	Quality of Service	Honeypot	—

Reading the matrix means, for each phase and indicator, finding a technique that supplies one of the strategies. To *detect* exploitation of a vulnerability on a host, install a host-based intrusion detection system or an EDR that rings an alarm when exploitation occurs; to *deny* it, patch the software so the vulnerability disappears. (**DEP**, Data Execution Prevention, is the disrupt-stage mechanism that blocks execution when the CPU instruction pointer lands in a memory region marked as data — the same principle by which code regions are executable but not writable.) For the Reconnaissance phase, where the attacker found a forgotten PDF on our website, web analytics can *detect* suspicious access to seldom-used published files.

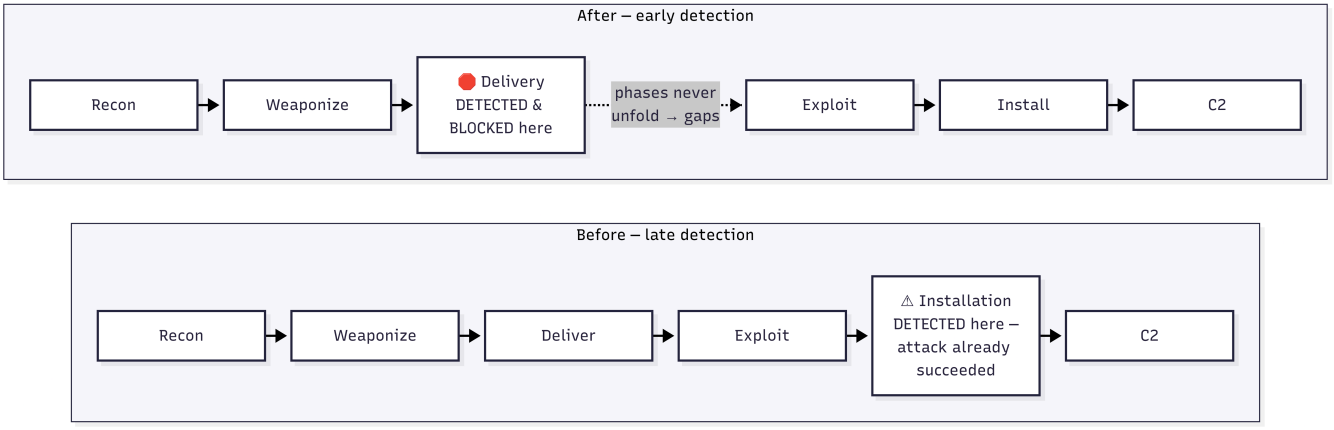
The important point is what this achieves. We are taking the knowledge acquired as the victim of an incident — data collected against PIRs such as "how can I detect an incoming attack with these characteristics?" and "how can I block it?" — mapping it to the kill chain to see which phases each indicator represents, filling the gaps, and then using the indicators to identify effective countermeasures that prevent a similar attack from succeeding again. The matrix then becomes a **prioritisation tool**: going down the list, some countermeasures are mere configuration changes to apply now, while others (a feature our IDS lacks and we cannot afford) will not be implemented — and that is how we decide where to invest money, time, and experience to improve our ability to block that kind of attack.

Measuring Effectiveness Over Time

Once a subset of solutions is implemented, their effectiveness must be checked and monitored continuously. Over time, some countermeasures prove ineffective and must be improved or replaced; others are present but never triggered. Each still has meaning, because it was chosen on an objective basis — the data collected — and whether it ends up useful depends on what the attacker actually does. Measuring effectiveness per phase, period after period, is what lets defensive strategies evolve: tracking, across successive reviews, which phases are covered by detection, which by mitigation, and where newly leveraged indicators extend coverage.

Reconstructing the Kill Chain: Moving Detection Earlier

The countermeasures derived above do not merely repel the next attack; they **move the point of detection earlier** along the chain. In the first case we detected the intrusion only at Installation, and the attack was in fact successful — we even found activity toward the command-and-control. After applying what we learned, a very similar phishing email arriving next time is detected and blocked at the **Delivery** phase: the mail server's rule quarantines it, and there is no impact. The SOC team is delighted.



The Synthesis Problem

Blocking early is a defensive success but an **intelligence problem**. From the threat analyst's point of view this is a *new* incident, and we should extract as much from it as possible — yet the only indicator we have is the mail server's alert that a rule-triggering email was quarantined. We can extract information about the email's content, and hypothesise about Weaponization and Reconnaissance, but every phase *after* detection is an empty gap. This time the gaps exist not because evidence is hidden, but because the attack **never unfolded**: no evidence was created, since we blocked it in advance and therefore could not observe how it would have behaved.

The analyst must somehow **synthesise** the missing information. One option is **deception**: using a detonation/sandbox environment to let the malware execute beyond the detection point and observe

what it does. Two caveats limit this. First, a simulated environment always differs from the real one — a large organisation's hundreds or thousands of hosts can only be approximated — so the malware may take a different execution route than it would have taken for real. Second, and more seriously, the attacker may **detect the deception**; an attacker who realises they have been diverted into a sandbox may simply delete everything, or, worse, deliberately exhibit misleading behaviour, so that we believe we are observing the real attack while in fact the attacker is playing with us.

The other option is **static synthesis**: take the malicious attachment and analyse it to understand how it was built, how it works, and what it would have done — recovering, where possible, which vulnerability it targeted and extracting the payload (possibly obfuscated). This is limited when the malware is highly modular and was delivered only with the exploitation code and a minimal installation stub, because most of the payload would have been downloaded during Installation from an external address that we cannot easily obtain. Synthesis therefore yields **less reliable** information about the attacker's techniques than observing a fully unfolded attack.

From Incident to Campaign

So far this concerns a single attack, but the adversaries worth studying are **persistent** — motivated to attack us again and again. When we face multiple attacks, a question arises that we set aside for now: given two attacks, how do we know whether they come from the *same* actor or from two different ones? Assuming for the moment that they share a source, correlating information across a persistent adversary's attacks is clearly valuable: such an attacker is highly motivated against us, so to impose cost intelligently we must learn as much as possible about them and build an **adversary profile**, exploring every way they might act against us. What we study is then no longer a single incident but a **campaign**.

We continue to use the kill chain, correlating indicators observed from attack to attack and looking for **repeated patterns** of the same IOCs. Consider three successive attempts. The first succeeds and is fully reconstructed. The **second** is blocked at Delivery and differs only slightly — the sender address and the encryption algorithm (Key 1) are unchanged, while the recipient set, the subject, and the downstream IP change, and the target list this time comes from a different document we had published on the web. The **third** keeps the sender and downstream but makes a *big* change at Exploitation: a recent zero-day PowerPoint exploit, only three days old, with the exploitation code entirely replaced. Comparing these intrusions side by side across the chain reveals the attacker's **motivation level**: between the first and second attempts they changed only minimal details, showing limited willingness to invest; in the third they adopted completely different techniques — likely sourcing the zero-day exploit code on the black market — showing they are highly motivated to succeed against us. The general pattern is that the attacker first changes cheap, low-value details and, finding them ineffective, moves up to higher-value (more expensive) changes.

Commonalities and Attribution

Continuing this game of collecting and correlating indicators across intrusions, some indicators **remain stable** from one attempt to the next. These are the **commonalities** (inflection points), and they matter enormously: if the attacker keeps trying, changing techniques, yet leaves certain indicators unchanged, those indicators tell us the attacker is **not willing to invest in modifying them** — so they become a kind of **profile**, a signature of the attacker. Every new intrusion presenting that same limited set of indicator values raises the probability that it comes from the same attacker, which is exactly how we answer the attribution question posed earlier.

In practice there are pitfalls. Two intrusions sharing the **same C2 IP address** do *not* necessarily share an attacker: that address may belong to a compromised machine whose access was sold on the black

market, so multiple actors independently use it to hide their infrastructure. Likewise, two incidents exploiting the **same vulnerability** are not thereby the same attacker: when a fresh CVE is published with its patch, there is a rush of *different* actors racing to weaponise it — today one can even ask a local AI model to build the exploit code and then scan the whole IPv4 space for still-vulnerable servers — so a shared vulnerability indicates only that everyone knew about it. What genuinely characterises a specific actor is their **behaviour**. This is precisely why behavioural indicators sit at the top of the Pyramid of Pain: behaviours express the *nature* of an adversary, changing one's nature is extremely difficult, and so behaviour serves as a common fingerprint.

Uses of the Kill Chain

Beyond planning operational countermeasures, kill-chain analysis supports several further uses. **Coverage assessment**: do our defensive processes map across the whole chain and against the activities the attacker is known to perform — are all indicators covered, are there gaps, are the solutions strong enough? **Effectiveness measurement**: for each countermeasure, exactly how effective was it against the attempts we received? **Post-mortem analysis**: after a successful breach, use the chain to understand precisely how the attack unfolded and learn, phase by phase, what could have been done better. **Investment planning** follows from all three.

The post-mortem of the 2013 Target breach is the canonical illustration. In **Reconnaissance**, supplier-portal and facilities-management information was publicly available; in **Weaponization/Delivery**, a classic phishing campaign carried password-logging malware (in a PDF or Office attachment) to Fazio, an HVAC vendor; in **Exploitation/Installation**, the attackers used a compromised default password to move internally from an external billing system to the point-of-sale devices, where RAM-scraping and exfiltration malware was installed, retaining access for over a month; in **Actions on Objectives**, data was exfiltrated in plaintext to a server in Russia — as many as 110 million customer records. The lessons map cleanly to the chain: a weak vendor gave the foothold, missed anti-intrusion warnings allowed installation and lateral movement, and weak internal controls allowed the final theft.

Limits of the Kill Chain

The kill chain is widely used but remains a limited methodology. Its central limitation is that it offers only **one kind of view** — the *temporal* evolution of an attack, phase by phase — and it assumes the attacker passes through all phases from Reconnaissance to the final goal. It does **not correlate** the nature of the attacker, the nature of the victim, and the assets the attacker uses. It is built to organise **indicators of compromise**, but an IOC is merely evidence stemming from the *consequences* of a technique, and such evidence is often incomplete or not detailed enough to reveal the underlying tactics — which is exactly why climbing the stages of the Pyramid of Pain is so hard. The model is also clearly **focused on malware-based attacks**; mapping a denial-of-service attack onto it is possible but forces the instrument to do something it was not designed for. Finally, the chain is used to organise intelligence analysis to rebuild the attacker's playbook, yet attackers do **not always follow a precise playbook** — they sometimes improvise with whatever opportunity arises, without weighing cost-effectiveness. The playbook assumption gives us a well-defined setting in which the analysis techniques are effective, but they are only as effective as that precondition holds, and it does not always hold. For these reasons CTI analysts complement the kill chain with other models that look at incidents from different angles — chief among them the Diamond Model.

The Diamond Model

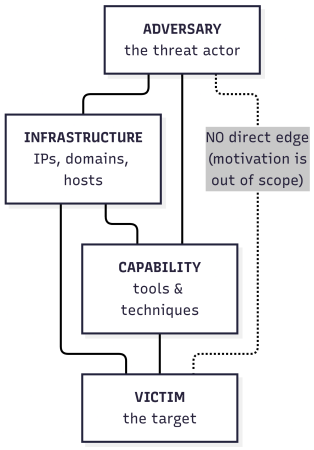
The Diamond Model is the model most commonly used alongside the kill chain, and from one angle it is even simpler. Its defining move is to **drop the temporal link** between the phases of an attack entirely and instead focus on the **relationships** between the attacker, the victim, and the capabilities and infrastructure the attacker uses against that victim. These relationships are described through **security events**. Events are then connected within a single intrusion through **activity threads**, and grouped into **activity groups** — the latter being, today, the main approach to the *attribution* of attacks, an area where much work remains but which is already standard practice.

Events and Core Features

Each intrusion **event** is represented by **four core features**, captured by a single foundational sentence:

> For every intrusion event there exists an **adversary** taking a step toward an intended goal by using a **capability** over **infrastructure** against a **victim**.

The sentence binds the four features together: the adversary acts against the victim, but the action is *mediated* — exercised through capabilities that operate over infrastructure, both controlled by the adversary. An event, moreover, defines a discrete, **time-bound** activity restricted to a specific phase, and because the four core features rarely describe an event completely, a diamond can carry additional **meta features** attached as text, key–value pairs, or whatever is needed.



The core features are joined by **edges**, but — crucially — the edges do **not** connect every vertex to every other. In particular, there is a **missing link between adversary and victim**. Since the model lets you analyse only relationships backed by an edge, it deliberately does **not** help you establish the direct adversary–victim relationship, and therefore does not help you study the attacker's **motivation**: that is outside its scope. This is because the Diamond Model operates at the *operational* level, where the attacker's motivation has no clear, direct consequence for how you improve your defences.

Each core feature has internal structure. The **adversary** is the threat actor — a person, organisation, or country — that has the motivation to act against the victim; it is further split into the **adversary operator** (the actual hacker conducting the intrusion) and the **adversary customer** (the entity that benefits, which may or may not be the same party). The **victim** is the target where the attack inflicts its direct impact — an organisation, an individual, a PC, an IP address, even a process — and divides into the **persona** (the individual or organisation targeted) and the **asset** (the attack surface: networks, systems, hosts, email and IP addresses, social-media accounts). The set of a victim's vulnerabilities and exposures susceptible to exploitation are its **victim susceptibilities**, a sub-feature of the victim. The **capability** is everything in the attacker's toolchain — every technique, tactic, or tool — and comprises the **capability capacity** (all the vulnerabilities and exposures a given capability can exploit, regardless of victim) and the **adversary arsenal** (the attacker's complete set of capabilities); a command-and-control facility is typically a specific kind of capability. Some capabilities impact the victim directly;

others need support from **infrastructure** — the physical and logical communication structures used to deliver a capability, maintain control of it, and effect results from the victim (IP addresses, hostnames, compromised assets, domain names). Infrastructure is **directly controlled**, **controlled by an intermediary**, or provided by **public infrastructure** providers.

The Event Tuple and Meta Features

Formally, an event is a tuple pairing each feature with a **confidence** value, reflecting that intelligence is always a judgement under uncertainty:

```
E = ⟨ ⟨Adversary, Conf⟩, ⟨Capability, Conf⟩, ⟨Infrastructure, Conf⟩, ⟨Victim, Conf⟩,
      ⟨Timestamp_start, Conf⟩, ⟨Timestamp_end, Conf⟩, ⟨Phase, Conf⟩, ⟨Result, Conf⟩,
      ⟨Direction, Conf⟩, ⟨Methodology, Conf⟩, ⟨Resources, Conf⟩ ⟩
```

Any element can be broken down further; the victim, for example, expands into ⟨Organization⟩, ⟨Host IP Address⟩, ⟨Hostname⟩, ⟨Application⟩, ⟨TCP Port⟩, each with its own confidence. Beyond the four core features, the **meta features** complete the description of an event:

Meta feature	Meaning
Timestamp	When the event occurred; may include start/end timestamps
Phase	The logical role the event plays in an activity thread (you may reuse kill-chain jargon)
Result	Post-conditions: qualitative `⟨Success, Failure, Unknown⟩`; informative `Confidentiality | Integrity | Availability-compromised`; plus details of the specific compromised assets
Direction	Victim-to-Infrastructure, Infrastructure-to-Victim, Infrastructure-to-Infrastructure, Adversary-to-Infrastructure, Infrastructure-to-Adversary, Bidirectional, or Unknown
Methodology	The general class of activity (e.g. spear-phish email, content-delivery attack, SYN flood) — preferably from an existing taxonomy
Resources	All supporting elements the event depends on: software, knowledge, information, hardware, funds, facilities, access

These meta features can themselves be expanded as needed. As an illustration of mapping, a diamond modelling the **Exploitation** phase of an intrusion would place *someone* as the adversary, a specific PC as the victim, and knowledge of the exploitation code for a vulnerability as the capability — with the kill-chain phase recorded as a meta feature, since the two models are used together rather than in isolation.

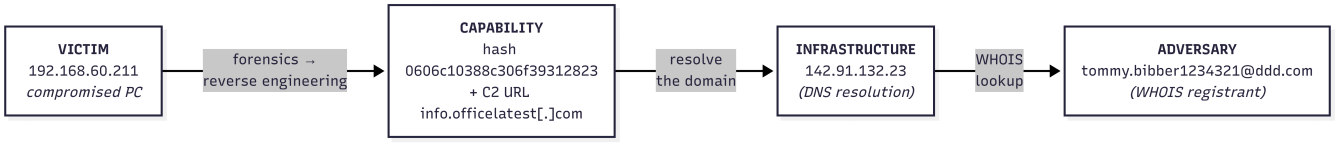
Two Useful Extensions

Two feature extensions enrich the basic diamond. The **social-political** extension describes the relationship between the adversary and its victims — their intent and persistence — and is useful for defining a *shared threat-space* with other victims who face the same adversary. The **technology** extension describes the technology that connects and enables the infrastructure and the capability to operate and communicate. Geometrically, these two extensions correspond to the diamond's two axes: the social-political axis runs between adversary and victim, and the technology axis runs between infrastructure and capability.

Analytical Pivoting

Analysis with the Diamond Model proceeds through **analytical pivoting**. You start from an initial indicator, map it to the vertex it belongs to, and then **use the edges** to guide your search for new evidence relating the connected vertices; each new piece of evidence is mapped onto its vertex, and you pivot again. You stop only when there is no way to pivot to another vertex and collect further information — at which point the diamond is as complete as the available evidence allows.

A worked example makes the mechanism concrete. The initial indicator is the **IP address of a compromised local machine**, `192.168.60.211`, mapped to the **Victim** (the asset directly receiving the impact). Performing a forensic analysis of that machine, we find an unauthorised running process; we compute its hash, `0606c10388c306f39312823`, and map it to **Capability**, because that hash represents a piece of software the attacker can use against us. Looking *inside* the malware, we find a URL, `info.officelatest[.]com`. Where does it map? Not to Infrastructure, as one might expect of a domain name, but to **Capability** — because this domain is the malware's command-and-control, and a C2 is a capability the attacker uses to control the implant. From the domain we pivot to **Infrastructure** by resolving it through DNS to the IP address `142.91.132.23`, a machine the attacker controls. Finally we pivot toward the **Adversary**: querying the WHOIS database for the domain's registration, we find it was registered one week ago to the address `tommy.bibber1234321@ddd.com` — which, even if it belongs to a compromised third party, represents our adversary at this stage.



Pivoting can begin from **any** of the four features, depending on the indicators available. Starting from the Adversary is the hardest, because it is difficult to gather information from there; the usual practice is to start from the Victim and move around through Capability and Infrastructure, collecting as much as possible.

Activity Threads

Looking at a whole incident, you typically find **several events**, and for each you build a diamond that collects only the indicators related to that event. You then take this collection of diamonds and **organise them temporally using the kill chain** — because the diamonds represent the preconditions the attacker needed to perform subsequent activities (exploitation, for instance, is the precondition for beaconing and exfiltration toward the C2). A set of events organised this way, each as its own diamond threaded along the kill chain, is an **activity thread**, and this representation lets you see graphically which preconditions link one activity to another, including across different threads.

Correlation along and across threads works in two directions. **Vertical correlation**, within a single thread, means identifying knowledge gaps, filling them with new knowledge, and establishing causal relationships among events. **Horizontal correlation**, across threads, is the analytic process of causally linking events between vertical threads across adversary–victim pairs: identifying common gaps between threads, using knowledge from one thread to fill a gap in another, and identifying common features across victims — which can lead to the creation of an activity group. As before, where the analysis reveals gaps — say, no evidence for the Weaponization or Installation phases — those gaps become the basis for **new Priority Intelligence Requirements**, aimed at collecting the information needed to place events in the missing steps.

Attack Graphs

Activity threads support a powerful form of preliminary assessment. You first combine information from the system and network administrators — a map of the network, the hosts and their characteristics, the software installed, how systems interact, and the business dependencies among them — into a model of your system. You then overlay the **crown jewels**: the assets (data, services) you must protect and where they sit on the network. From this you build an **attack graph** representing all the potential paths an attacker could take from the initial attack surface to the crown jewels.

These are *potential* paths, but when you later observe a real attack and study it through the Diamond Model and kill chain, you can **map the observed activity onto the attack graph**, confirming that the attacker is following some of the paths you studied in advance. This is useful in two ways. **A priori**, it helps you decide how best to protect a crown jewel: if every path to a crown jewel must pass through a single common point — a **choke point** — then protecting that one point best prevents the attacker from ever reaching the objective, and that is where you invest. **During incident response**, with the attacker already inside and moving toward a crown jewel, you map the compromised hosts onto the attack graph and read off, from the last point reached, all the possible next steps; you then take the right measures — in the worst case, shutting down exactly those systems — to block the spread toward the objective. In both cases the intelligence, extracted from data acquired through appropriate requirements, drives a decision that is the **most effective at the smallest cost**, rather than panicking and shutting everything down for lack of information.

Activity Groups

The final step groups events and threads into **activity groups**, the mechanism used for attribution — identifying actors and dividing the events of different incidents among different actors. Conceptually it is exactly what the kill-chain campaign analysis did: look at commonalities and differences and use them to organise events, now within the richer Diamond representation. Knowledge about activity groups is extracted through a defined process: **(1) Analytic Problem** — state the problem to be solved by grouping; **(2) Feature Selection** — choose the event features and adversary processes that will form the basis of classification and clustering; **(3) Creation** — form activity groups from the set of events and threads; **(4) Growth** — classify new events into the groups as they flow in; **(5) Analysis** — analyse the groups to address the analytic problem; **(6) Redefinition** — periodically redefine the groups to keep them accurate.

Operationally, an **Activity Group Creation (AGC)** function takes the whole set of events and threads and groups those that share defining characteristics — a common process ordering (for example A → B → C), shared IP infrastructure, shared capabilities — into the **activity groups (AGS)**, while events and threads that match no group are set aside as **outliers**. The result is a partition of observed activity into candidate actors, each characterised by a recurring process and reused infrastructure or capabilities — the raw material for attribution and for anticipating how a given actor will act next.

CHAPTER IF_3: Yara Rules

Introduction: What YARA Is and Why It Matters

YARA is a mature, well-established technology for pattern matching over arbitrary data. It is not a new methodology nor an experimental research prototype; it is a battle-tested instrument that has been a staple of malware analysis and incident response workflows for well over a decade. Understanding YARA is less about learning a novel concept and more about mastering a precise, expressive language for describing *what suspicious or interesting data looks like*, and then letting a fast engine find that data wherever it hides.

The most useful mental model is to think of YARA as **"grep on steroids."** The classic Unix `grep` utility searches for textual patterns inside streams of data. YARA generalizes this idea dramatically: instead of being limited to simple text searches inside a single stream, it lets an analyst *program* the search. Patterns can be combined, nested, and constrained with logical conditions, so the matching logic can be as simple as "does this byte sequence appear anywhere?" or as sophisticated as "does this file begin with a specific magic number **and** contain at least three of these ten indicators **and** import these particular library functions?" This programmability is precisely what elevates YARA above an ordinary search tool.

What YARA Is Used For

YARA addresses a recurring need in security operations: you are confronted with some unknown artifact — a suspicious file, a memory dump, a block of bytes extracted from a compromised system — and you want to determine quickly whether it matches something you already understand. The tool serves several distinct purposes that are worth separating clearly.

The first purpose is **triage against known threats**. Over time, an analyst or an organization accumulates substantial experience with many malware families. Some of these families are closely related to one another; adversaries frequently re-release essentially the same malicious code under new names, with only cosmetic changes. The question then becomes: *whenever I encounter a new observable, can I immediately tell whether it is something I already know, or something closely related to it?* YARA lets you encode your prior knowledge into reusable matching rules so that recognizing a returning threat becomes automatic rather than a fresh investigation each time. This reuse of hard-won prior knowledge is one of the central operational benefits of the tool.

The second purpose is **exploratory searching over large datasets**. Because YARA is fast, it can be turned loose on very large volumes of data — the entirety of a memory dump, or a substantial quantity of bytes read directly from a storage device on a seized system — to search for a new piece of information whose presence you suspect but have not yet located. Here YARA is not confirming a known threat so much as helping you discover where something of interest resides inside an overwhelming amount of raw data.

Beyond these two uses, YARA offers something more organizational and strategic: it helps you **maintain a structured library of matching rules** for different threats, each rule grounded in observed Indicators of Compromise (IoCs). Rather than keeping scattered notes and ad-hoc signatures, an analyst can curate a coherent collection of rules — effectively a living catalog of file signatures and behavioral fingerprints — that encodes institutional knowledge about the threat landscape and can be shared, versioned, and extended.

It is equally important to understand **what YARA is not**. YARA is *not* an antivirus product, and it is *not* a host-based intrusion detection system. It does not run continuously in the background monitoring a machine, it does not quarantine files, and it does not make autonomous blocking decisions. It is a classification and search engine that you point at data when you want to ask a specific question. Its defining virtues are that it is simultaneously **easy to use, flexible, and very powerful** — a combination that is rare and that explains its longevity. It is also open source, which has contributed both to its wide adoption and to the existence of various extensions; the discussion here adheres to the official, standard syntax.

How YARA Is Deployed and Run

YARA is distributed as a tool that you download and compile for your own operating system. In practice you interact with it through a command-line interface. The essential invocation takes one or more rule files and applies them to a target, where the target may be a single file or an entire directory tree.

The conceptual usage is as follows:

```
yara [OPTION]... [RULEFILE]... FILE
```

The `RULEFILE` arguments are the rules you have written — the descriptions of what you are looking for. The final `FILE` argument is the thing you want to check against those rules. The engine's job is to report **which of the provided rules match the given target.** When the target is a directory and the search should descend into nested folders, recursive searching is enabled explicitly; this is the scenario in which you point YARA at an entire file system subtree and let it look for matches across everything inside it.

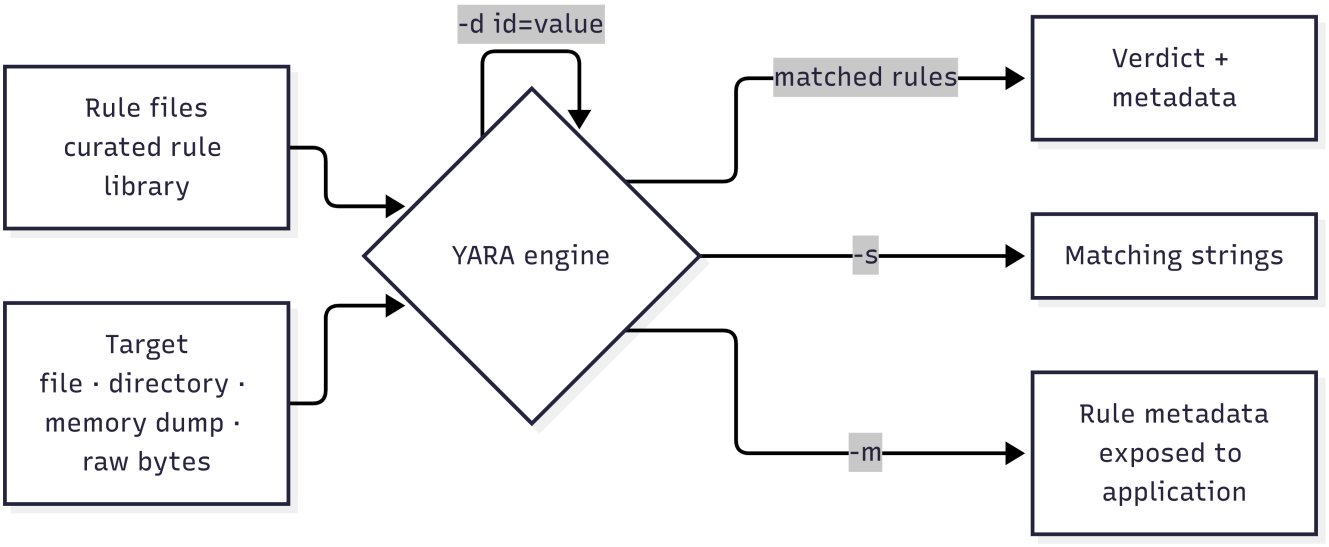
A practical operational note follows from the fact that rules have a cost. In real deployments you normally accumulate a large collection of rules, and you do not necessarily run all of them against all of your data every time. Running the full collection can be slow, so you often select a relevant subset — by rule name or by tag — to keep performance acceptable. The decision of which rules to run against which data is part of the analyst's judgment, informed by context and by the performance budget available.

The command-line options that shape YARA's behavior are summarized below. These options let you filter which rules are considered, control what information is printed on a match, define values that the rules can reference from outside, and tune how the search is performed.

Option	Meaning
`-t <tag>`	Print rules tagged as `<tag>` and ignore the rest. Can be used more than once.
`-i <identifier>`	Print rules named `<identifier>` and ignore the rest. Can be used more than once.
`-n`	Print only *not satisfied* rules (negate the result).
`-g`	Print tags.
`-m`	Print metadata.
`-s`	Print matching strings.
`-d <identifier>=<value>`	Define an external variable.
`-r`	Recursively search directories.
`-f`	Fast matching mode.
`-v`	Show version information.

Two of these deserve emphasis because they reappear conceptually throughout the rest of this chapter. The `-s` option prints the actual strings that caused a match, which is invaluable when you need to understand *why* a rule fired rather than merely *that* it fired. The `-d` option defines an **external variable** — a value supplied from outside the rule at invocation time. This mechanism allows a rule to reason about information that is not contained in the file itself but is instead part of the execution context, a capability that becomes important when writing rules that depend on run-time facts such as a file's path on disk.

The overall workflow can be visualized as a simple pipeline, in which a curated collection of rules is applied by the engine to one or more targets, producing a verdict about which rules matched and what metadata those matches carry.



This framing — a programmable, fast engine that applies a reusable library of expressive rules to arbitrary data and reports structured results — is the foundation on which every subsequent concept builds. The remainder of this chapter develops the YARA rule language itself, starting from the minimal rule and progressing to the sophisticated real-world fingerprints used to detect specific malware families, obfuscation schemes, cryptographic routines, and packing techniques.

The Anatomy of a YARA Rule

Every YARA rule is a self-contained description of something you want to recognize. The language is deliberately small, which is a large part of why it is approachable, yet the few constructs it provides compose into very expressive fingerprints. A rule is organized into named sections, and understanding what each section is for — and, just as importantly, what each section does *not* do — is the key to reading and writing rules fluently.

The Minimal Rule: A Name and a Condition

At an absolute minimum, a rule must have two things: a **name** and a **condition**. Nothing else is strictly required. The name identifies the rule; the condition is the Boolean expression that decides whether the rule matches a given target.

The simplest possible rule is one whose condition is a constant:

```
rule dummy { condition: false }
```

This rule is syntactically valid and will compile and run, but it does nothing useful: because its condition is permanently `false`, it never matches anything. Its logical opposite is equally trivial:

```
rule dummy { condition: true }
```

This rule matches on *anything*, since its condition is always satisfied. Although both of these are degenerate, it would be a mistake to dismiss the "always true" form as entirely useless. A rule that is guaranteed to evaluate can still serve a real purpose: it can be used simply to confirm that YARA was

able to read and parse the provided file at all — in effect, "did the engine successfully receive a valid file to process?" So even the trivial condition has a modest diagnostic role.

`filesize` and the Notion of Built-in Keywords

Rules become useful the moment their conditions reference facts about the target. YARA provides a vocabulary of **built-in keywords** — special reserved words that the engine recognizes and resolves automatically. The canonical introductory example is `filesize`, which evaluates to the size of the target being scanned. You should think of `filesize` as a predefined built-in variable that the engine exposes to you for free.

Suppose you want a rule that fires on any file larger than 500 kilobytes. You simply write a condition that compares the built-in `filesize` against the desired threshold:

```
rule over_500kb { condition: filesize > 500KB }
```

Here `filesize` is not something you declared; it is one of YARA's special words, and the engine substitutes the actual size of the file when it evaluates the rule. This same pattern — a rule whose name is descriptive and whose condition is a single comparison against a built-in value — is the smallest step beyond the trivial rules, and it already demonstrates the shape of every rule you will ever write: *a named predicate over properties of the data.*

YARA ships with a collection of such predefined built-in facilities, and crucially it also lets you bring in much more capability by **importing libraries** (modules). These imported modules provide, for example, the ability to parse structured file formats or to compute over data with specific algorithms. The built-in vocabulary is therefore only a starting point; the language is designed to be extended with richer, domain-specific knowledge through imports, a mechanism explored in detail later in this chapter.

The Standard Structure of a Rule

Real rules are organized into three sections beyond the name: **meta**, **strings**, and **condition**. The following rule is the canonical example used to illustrate the full structure. It is somewhat artificial, chosen for clarity rather than realism, but every element in it corresponds to a construct you will use constantly.

```
rule silent_banker : banker {

    meta:
        description = "This is just an example"
        threat_level = 3
        in_the_wild = true

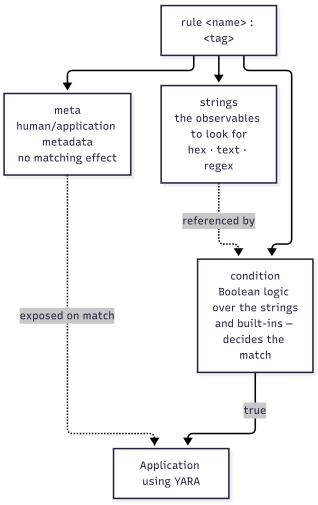
    strings:
        $a = {6A 40 68 00 30 00 00 6A 14 8D 91}
        $b = {8D 4D B0 2B C1 83 C0 27 99 6A 4E 59 F7 F9}
        $c = "UVODFRYSIHLNWPEJXQZAKCBGMT"

    condition:
```

```
        all of them
}
```

Before dissecting the sections, note the rule header itself: `rule silent_banker : banker`. The token after the colon, `banker`, is a **tag**. A tag is a label attached to the rule, and you can think of it as a way of categorizing a rule as belonging to a broader set — here, the family of banking-related rules. Tags are how you later ask YARA to run only a particular category of rules (recall the `-t <tag>` command-line option). Conceptually this gives a rule the flavor of an *extension* of, or a *member of*, a larger prior collection or specification, letting you organize many rules under shared classifications.

The overall layout of a rule can be pictured as follows, with the header on top and the three bodies beneath it:



The `meta` Section

The `meta` section is a set of arbitrary **key–value pairs**. You may name the keys pretty much however you like, subject only to a small set of reserved names you must avoid. The values may be **strings, integers, decimals, or booleans**.

The defining characteristic of the `meta` section is that it has **no effect on matching whatsoever**. The engine does not evaluate meta values when deciding whether a rule fires; they carry no actionable logical meaning for the matching process. Their entire purpose is to convey information — historically, information for the humans who read and maintain the rule, and increasingly information that automated tooling (including AI-assisted analysis pipelines) can consume. In earlier eras this content would simply have been a comment for the analyst; the `meta` section formalizes it into structured, machine-readable annotations.

In the example, each key illustrates a typical kind of annotation. The `description` tells you, in plain language, what the rule is doing. The `threat_level` encodes a severity grading; the specific scale depends entirely on the threat-intelligence pipeline you operate within, and organizations assign such levels according to their own conventions. Related metadata often captured in this section includes how widely the finding may be shared — a confidentiality or distribution marking indicating which entities are permitted to receive the information. The `in_the_wild = true` annotation is a meaningful hint about the rule's provenance: it suggests that the indicators encoded here were drawn from a real malware sample actually observed in the wild, as opposed to a synthetic signature an analyst merely constructed from first principles. In other words, it tells a downstream reader that this fingerprint reflects something genuinely seen in operation.

The practical payoff comes at match time. When a rule matches a target, its `meta` information is **exposed to the application that is using YARA.** The engine effectively says to the calling application: "this rule matched, and here is the high-level, human-meaningful context about what that match signifies." A tool built on top of YARA can therefore react intelligently to a hit — escalating based on `threat_level`, respecting a sharing marking, or displaying the `description` to an analyst — all without hard-coding that knowledge, because the rule carries its own self-description.

The `strings` Section

The `strings` section is where you define the **observables** — the concrete patterns the engine will look for inside the target. Each observable is bound to a variable name beginning with a dollar sign, such as `$a`, `$b`, or `$c`. These names are how the condition later refers to the individual patterns.

YARA supports three fundamentally different ways of specifying an observable, and the example deliberately shows two of them.

The first is the **hexadecimal byte pattern**, written between curly braces. Both `$a = {6A 40 68 00 30 00 00 6A 14 8D 91}` and `$b = {8D 4D B0 2B C1 83 C0 27 99 6A 4E 59 F7 F9}` are of this kind. These are *raw sequences of bytes* — not text. There is no notion of a string terminator here; each pair of hexadecimal digits is simply one byte that must appear, in order, in the data. This representation is the natural choice when the thing you are matching is binary in nature, such as a specific run of machine-code bytes or a fixed binary structure, where interpreting the bytes as printable characters would be meaningless.

The second is the **text string**, written between double quotes. The observable `$c = "UVODFRYSIHLNWPEJXQZAKCBGMT"` is of this kind. It is expressed as readable text purely as a convenience, but it is important to understand that at the engine level this is *still a search for bytes* — specifically the byte values that encode each of those characters. A text string and the equivalent hex byte pattern are simply two ways of expressing the same underlying thing. Which you choose is a matter of what is convenient for the task at hand: when the content is genuinely textual and printable, the quoted form is far easier to read and write; when the content is raw, binary, or taken from a disassembly, the hexadecimal form is clearer and more honest about what is really there.

The third form, which the example does not use but which the language fully supports, is the **regular expression**, written between forward slashes (`/.../`). Regular expressions give you enormous flexibility to describe families of patterns rather than a single fixed sequence. They are powerful but notoriously tricky: even experienced analysts often find them easier to read than to author correctly, and a poorly written expression can match far more or far less than intended. Used judiciously, however, regex observables let you capture variable content elegantly.

A critical refinement of the `strings` section is the ability to describe patterns that contain **holes** — regions where you know that *some* bytes appear but you do not care about their exact values, or you do not even know how many bytes there are. This matters enormously when matching code. The same logical instruction compiled into two different binaries may share an identical opcode while differing in its operand, because the operand encodes an address, and an address that is valid in one program is meaningless in another. If you insisted on matching the operand bytes exactly, your rule would be brittle and would fail across samples. The solution is to **mask** the variable portion: you fix the stable opcode bytes and leave the operand bytes as "don't care," so that the invariant part of the instruction is what drives the match while the address bytes are ignored.

YARA provides two mechanisms for this. The first is the single-byte wildcard, written as `?`, which stands for any value in that position — conceptually "there is a byte here, but I do not know or care what it is." The second, more powerful, is the **jump**, written with square brackets, which specifies a variable-length gap. A jump lets you say "between these two fixed patterns there are at least x and at most y arbitrary bytes," expressing a range of lengths for the unknown region. This is exactly what you need when you know two landmarks in the data are separated by an uncertain amount of filler. Rather than enumerating every possible combination of intervening bytes by hand, you declare the minimum and maximum size of the gap and let the engine handle the rest. These masking and jump facilities are what make hex patterns practical for real-world code matching, and they recur prominently in the advanced examples later in this chapter.

The `condition` Section

The `condition` section is the logical heart of the rule. It is a Boolean expression that references the observables defined in `strings` (and any built-ins or imported facts) and evaluates to true or false, determining whether the rule matches. In the `silent_banker` example the condition is `all of them`, which requires that *all* of the defined strings — `$a`, `$b`, and `$c` — be present in the target before the rule will fire.

The expressive power of conditions comes from a small set of quantifiers and the ability to combine observables with ordinary Boolean operators. The most common conditional forms are summarized below.

Condition	Meaning
`any of them`	The rule matches anything containing *any* of the strings defined in the rule (at least one is enough).
`all of them`	The rule matches only if *all* of the defined strings are present in the content.
`3 of them`	The rule matches anything containing *at least three* of the defined strings.
`$a and 3 of ($s*)`	Match content that contains string `$a` **and** at least three strings whose variable name begins with `$s`.

Each of these captures a distinct intent. `any of them` is the most permissive: a single hit suffices, which is appropriate when any one of several alternative indicators is individually conclusive. `all of them` is the most demanding, used when the simultaneous presence of every indicator is what gives you confidence. The counting form, `N of them`, occupies the valuable middle ground. Its purpose is to express *sufficient confidence without demanding completeness.* Imagine you have encoded ten indicators for a threat; requiring all ten might be too strict, because real samples vary and may not exhibit every indicator, while requiring just one might produce spurious matches. By writing, say, `3 of them`, you declare that finding any three of the ten is enough evidence to conclude a genuine match and not a coincidental one. This directly tunes the balance between catching real threats and avoiding false positives.

The final form illustrates two additional capabilities working together. First, you can quantify over a *subset* of strings selected by a name prefix: `($s*)` denotes the set of all observables whose names start with `$s`, so `3 of ($s*)` means "at least three of the strings in that named family." Second, you can freely combine such quantified expressions with Boolean operators — here `$a and 3 of ($s*)` requires both that the specific string `$a` be present and that at least three of the `$s`-prefixed strings be present. More generally, you can wire observables together with logical connectives (`and`, `or`, and so on) to build

conditions of arbitrary sophistication, mixing individual string references, counting quantifiers, built-in facts like `filesize`, and grouping as needed. It is this compositional freedom in the condition — far more than any single observable — that gives YARA its characteristic blend of simplicity and power.

With the anatomy of a rule fully established — a descriptive name and optional tag, non-functional but richly informative metadata, a set of hex, text, or regex observables with masking and jumps, and a Boolean condition that quantifies and combines them — we are equipped to read real rules that solve concrete detection problems.

A First Realistic Example: Detecting Phishing PDFs

The rules seen so far were either trivial or deliberately artificial. The following rule is the first that solves a genuine, practical detection problem, and studying it introduces several constructs that are indispensable in real work: **magic numbers**, **positional anchoring with the `at` operator**, **string modifiers** (`ascii`, `wide`, `nocase`), and the **occurrence-counting operator `#`**. The rule detects a PDF document that contains only a small number of links — a configuration that is a common hallmark of phishing.

```
rule pdf_1.7_contains_few_links {

    meta:
        author = "Sean Whalen"
        last_updated = "2017-06-08"
        tlp = "white"
        category = "malicious"
        confidence = "medium"
        killchain_phase = "exploit"
        description = "A PDFv1.7 that contains one or two links - a common phishing
tactic"

    strings:
        $pdf_magic = {25 50 44 46}
        $s_anchor_tag = "<a " ascii wide nocase
        $s_uri = /\(http.+\)/ ascii wide nocase

    condition:
        $pdf_magic at 0 and (#s_anchor_tag == 1 or (#s_uri > 0 and #s_uri < 3))
}
```

The Detection Rationale

The intuition behind the rule is worth stating plainly, because it shapes every technical choice that follows. A PDF that contains just one or two links is, counterintuitively, *more* suspicious than one containing many. A document packed with links is very often entirely legitimate — consider a scientific article whose bibliography lists dozens of references, or any richly cross-linked document. Such a document is unlikely to be a targeted phishing lure. By contrast, a PDF whose entire payload is a single clickable link, or perhaps two, is precisely the shape of a phishing attempt: the attacker's goal is to get the victim to click one specific malicious destination, so the document carries little else. A file matching

this profile is therefore not necessarily malicious on its own, but it is exactly the kind of artifact an analyst would want to flag and hand off to additional tooling for deeper scanning.

A subtlety reinforces why the link must be *clickable* rather than merely present as text. If an `http` string appeared only as plain, non-clickable text — again, think of a reference list — its mere presence would not be strong evidence of phishing. The danger arises when the link is live and the user can activate it with a click. This also connects to why phishing links are so effective: attackers frequently register look-alike destinations that differ from a legitimate address by a single, visually ambiguous character — a lowercase "l," a capital "I," and the digit "1" are easily confused, and a victim glancing at a crafted domain may believe it legitimate and act on it, sometimes with very costly consequences. The rule's job is to surface documents whose link content fits the sparse, action-oriented pattern of such lures.

The Metadata, Read as Threat Intelligence

The `meta` section here is far richer than in the earlier example and reads like a proper threat-intelligence record. The `author` and `last_updated` fields establish provenance and currency. The `tlp = "white"` field uses the Traffic Light Protocol to mark the rule's sharing restrictions; a "white" marking indicates the information may be freely shared. The `category = "malicious"` and `confidence = "medium"` fields classify the finding and temper it with an honest statement of certainty. The `killchain_phase = "exploit"` field situates the detection within the exploitation stage of an intrusion lifecycle. None of these values influence matching — the `meta` section never does — but together they give any downstream system or analyst the full context needed to act on a hit appropriately.

Magic Numbers and the `$pdf_magic` Observable

The first observable, `$pdf_magic = {25 50 44 46}`, is a **magic number**: a short, fixed sequence of bytes that characterizes a particular file format. File formats conventionally begin with such a signature so that tools can identify the type of a file by inspecting its opening bytes rather than trusting its extension. The byte sequence `25 50 44 46` is the standard signature that marks a file as a PDF. In practice, an analyst who needs a magic number either knows it from experience or looks it up, since these signatures are well documented. Encoding the magic number as the first indicator ensures the rule only considers files that genuinely are PDFs.

Positional Anchoring with `at 0`

A magic number is only meaningful when it appears in the correct place — at the very start of the file. It would be a mistake to accept a file merely because the bytes `25 50 44 46` appear *somewhere* inside it, since such a coincidence carries no significance. YARA provides the **`at` operator** to anchor an observable to a specific offset. The expression `$pdf_magic at 0` requires that the magic bytes occur **at offset 0** — the current file position zero, the file's absolute beginning. This is the analyst's way of insisting "these bytes must be the first bytes of the file, not merely present somewhere within it."

This notion of anchoring to a precise location is a recurring theme. When you care about *where* an indicator appears and not just *whether* it appears, you combine the string reference with a positional constraint. The design philosophy is to treat observables like bricks: you specify which bytes you want and, where it matters, in which places, and you assemble these positioned requirements into the overall condition.

String Modifiers: `ascii`, `wide`, and `nocase`

Both link-related observables carry the modifiers `ascii wide nocase`, and these three modifiers are essential tools for robust text matching. They solve the problem that the "same" text can be represented in a file in more than one way.

The `ascii` modifier instructs YARA to look for the string encoded as ordinary single-byte ASCII characters — the representation most people assume by default. The `wide` modifier instructs YARA to also look for the string in a two-byte-per-character encoding, in which each character is followed by a zero byte; this is the form in which text is commonly stored in Windows and in many document formats, and a rule that checked only for the ASCII form would silently miss text stored this way. Specifying both `ascii` and `wide` tells the engine to match the pattern in either representation. The `nocase` modifier makes the match case-insensitive, so that variations in letter case — `<a` , `<A` , and so on — are all treated as matches. Together these modifiers make the text observables resilient to encoding and casing differences that an attacker (or merely an ordinary document generator) might introduce.

The Link Observables

With the modifiers understood, the two link observables are straightforward. The observable `$s_anchor_tag = "<a "` looks for the opening of an HTML-style anchor tag — the `<a` construct that normally precedes a hyperlink's attributes and destination. The observable `$s_uri = /\(http.+\)/` is a regular expression that looks for an `http`-style URI enclosed in parentheses: the pattern reads, loosely, as "an opening parenthesis, the text `http`, one or more further characters, and a closing parenthesis." This regular expression is intentionally simple and will match any parenthesized string that begins with `http`, which is sufficient to recognize the presence of a link reference in the document's structure.

It is worth acknowledging that this slide version of the rule is simplified relative to what a production rule might do. A more rigorous rule could verify that an anchor tag actually *references* a given URI, tying the two observables together structurally. The version shown deliberately avoids that complexity: it treats the presence of an anchor tag and the presence of a small number of URIs as independent indicators, which keeps the logic clear at the cost of some precision.

Counting Occurrences with `#` and the Final Condition

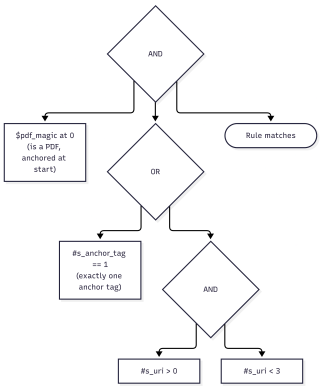
The condition introduces the final new construct: the **occurrence-counting operator**, written as `#`. Prefixing a string's name with `#` yields the *number of times* that string occurs in the target, rather than a mere true/false for its presence. Thus `#s_anchor_tag` is the count of anchor-tag matches and `#s_uri` is the count of URI matches. This counting capability is what lets the rule express "few links" precisely.

The complete condition is:

```
$pdf_magic at 0 and (#s_anchor_tag == 1 or (#s_uri > 0 and #s_uri < 3))
```

It decomposes into two conjoined requirements. The first, `$pdf_magic at 0`, confirms the file really is a PDF by requiring the magic number at the very start. The second, the parenthesized expression, encodes the "few links" criterion in two alternative ways joined by `or`. The first alternative, `#s_anchor_tag == 1`, is satisfied when there is **exactly one** anchor tag. The second alternative, `#s_uri > 0 and #s_uri < 3`, is satisfied when the number of URI matches is **greater than zero and fewer than three** — that is, either one or two URIs. The rule therefore fires when the file is a genuine PDF **and** it contains either exactly one anchor tag or between one and two parenthesized `http` links. A file with zero links fails both alternatives, and a file with three or more URIs also fails the second alternative, so the rule precisely isolates the sparse-link profile characteristic of phishing.

The logical structure of the condition is shown below.



This example consolidates the core toolkit of practical YARA authoring. A rule confirms a file's type with an anchored magic number, describes the textual indicators of interest with encoding- and case-resilient modifiers, counts how many times those indicators occur, and combines positional, counting, and Boolean logic into a single condition that captures a nuanced real-world intuition — here, that *sparsity* of links, not their abundance, is the signal of malice.

Imports and Modules

The rules examined so far reasoned only about the raw byte content of a target. But many of the questions an analyst most wants to ask are not about raw bytes at all — they are about *structured, semantic* properties of a file: which library functions does this executable import? How large is each of its sections? What is the cryptographic hash of its contents? Answering such questions by hand with byte patterns and offset arithmetic would be extraordinarily cumbersome. Consider trying to locate an executable's import table purely through byte offsets: you would be writing conditions like "at some offset, plus some other offset, plus yet another number," and the expression would grow long, fragile, and nearly unreadable.

YARA solves this with **modules**, brought into a rule file with the `import` statement. A module is a library that teaches the engine how to parse and reason about a particular kind of structure, exposing convenient functions and fields that you can reference directly in conditions. Several modules can be imported, and each grants the rule a vocabulary of semantic queries appropriate to its domain.

The `pe` Module

The most important module for Windows malware analysis is `pe`, which understands the Portable Executable file format — the format of Windows programs. Once imported, it lets a rule interrogate the executable's structure directly, without the analyst having to compute offsets into the file. The following rule illustrates this by inferring suspicious *behavior* from an executable's imports and a telltale string.

```
import "pe"

rule suspicious_pe {

    strings:
        $str = "procmon"

    condition:
        pe.imports( "kernel32.dll", "CreateProcessA" ) and
        pe.imports( "wininet.dll", "HttpSendRequestA" ) and
```

```
        $str
}
```

The central new construct is `pe.imports(dll, function)`, a function provided by the `pe` module that evaluates to true when the executable imports the named `function` from the named dynamic-link library. The rule uses it twice. The first check, `pe.imports("kernel32.dll", "CreateProcessA")`, is true when the program imports `CreateProcessA` — the Windows API for launching a new process — from the core system library `kernel32.dll`. The second, `pe.imports("wininet.dll", "HttpSendRequestA")`, is true when the program imports `HttpSendRequestA` from `wininet.dll`; this API performs network communication, allowing a program to send HTTP requests without launching a visible browser.

Notice an important stylistic point: there is **no byte observable for the imports.** The import checks are expressed entirely within the condition through the module's semantic query. The only entry in the `strings` section is `$str = "procmon"`, a plain text observable. The condition requires all three facts simultaneously — the ability to create processes, the ability to send data over the network, and the presence of the string `procmon`.

The reasoning that makes this combination suspicious is behavioral. "Procmon" refers to Process Monitor, a legitimate utility whose job is to monitor the activity of processes on a system in real time. A genuine process-monitoring tool has no business sending data out to the web. An artifact that presents itself as procmon yet combines process-creation and network-send capabilities is anomalous: it may well be a trojanized or infected version of the tool, masquerading as a benign utility while exfiltrating data or contacting a remote host. The rule does not detect a known byte signature at all; instead it **infers malicious semantics** by combining structural facts (the imports) with an identity hint (the string). This ability to reason about what a program is *capable of doing*, rather than merely what bytes it contains, is the defining value of modules.

References: Rules That Build on Other Rules

As a rule collection grows, a new problem emerges: duplication. Many higher-level detections are really combinations of smaller, independently meaningful observations, and you do not want to rewrite the same low-level logic inside every rule that needs it. Common behaviors, and combinations of small individual patterns, should be expressed once and reused. YARA supports this directly by letting **rules reference other rules**: a rule's condition can name another rule, and that name evaluates to true exactly when the referenced rule matches.

The following set of rules demonstrates both a second module, `hash`, and the composition of rules by reference.

```
import "hash"

rule RULE_EVILMD5 {
    meta:
        description = "Suspicious MD5 Hash"
    condition:
        hash.md5(0, filesize) == "3bb34a700e8d21acfdfe0f09208a7c01"
}

rule RULE_EVILPATH {
```

```
    meta:
        description = "Suspicious File Path"
    condition:
        path icontains "/appdata/roaming/"
}

rule RULE_SUSP_BHV {
    meta:
        description = "Suspicious Behavior"
    condition:
        RULE_EVILMD5 and RULE_EVILPATH
}
```

`RULE_EVILMD5` and the `hash` Module

The `hash` module works analogously to the `pe` module, but it computes cryptographic digests over regions of the target. The condition `hash.md5(0, filesize) == "3bb34a700e8d21acfdfe0f09208a7c01"` instructs the engine to compute the MD5 digest of the file's contents starting at offset `0` and spanning `filesize` bytes — that is, the MD5 of the entire file — and to compare it against a specific known-bad hash value. That reference hash would typically come from a prior investigation in which the exact malicious file was identified.

This is an *exact-match* signature at its purest: it fires if and only if the scanned file is byte-for-byte identical to the one whose hash was recorded. Such a rule is maximally precise and produces essentially no false positives, but it is also maximally brittle — changing a single byte of the file yields a completely different hash and defeats the rule entirely. Hash-based rules are therefore excellent for pinning down a specific, known artifact, but they do not generalize across variants.

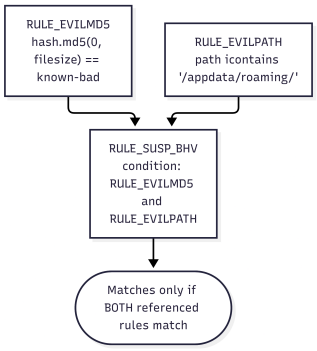
`RULE_EVILPATH`, `icontains`, and External Variables

`RULE_EVILPATH` checks a property of the file's *location* rather than its contents: `path icontains "/appdata/roaming/"`. The `icontains` operator is a case-insensitive substring test, true when the left operand contains the right operand regardless of letter case. The motivation is that on Windows, the `AppData\Roaming` area is a frequent home for persistence mechanisms — autostart entries, user-specific settings, and files dropped into the Start Menu's startup locations — so a file residing there is worth flagging.

There is, however, an important correctness subtlety in this rule. The identifier `path` is **not** a standard YARA built-in keyword in the way `filesize` is. A file's path is not part of the file's byte content; it is a fact about the *execution context* in which the scan runs. To reason about such context-dependent information, YARA uses **external variables**. You declare that the rule expects a variable named `path`, and you supply its value from outside the engine at invocation time — concretely, by passing it on the command line with the `-d` option, for example `-d path=<current path>`. The engine then substitutes the provided value wherever `path` appears. This is the practical purpose of external variables introduced earlier: they let a rule incorporate run-time facts that cannot be read from the file itself, and the file path is the archetypal example.

`RULE_SUSP_BHV`: Composition by Reference

The third rule, `RULE_SUSP_BHV`, contains no strings and no module calls of its own. Its condition is simply `RULE_EVILMD5 and RULE_EVILPATH` — the names of the two preceding rules. It matches precisely when **both** referenced rules match: when the file has the suspicious hash *and* it resides in a suspicious path. This is rule composition in action. Two independent, individually meaningful observations are combined into a single higher-level rule expressing a more confident conclusion — "suspicious behavior" — without re-implementing either underlying check.



The architectural lesson generalizes well beyond this toy example. By factoring detections into small, reusable rules — each capturing one coherent indicator — and then referencing them from composite rules that express behaviors or threat profiles, an analyst builds a maintainable, layered rule library. Low-level rules capture atomic facts (a hash, a path, an import, a byte pattern); higher-level rules combine those facts into meaningful verdicts. This modularity mirrors good software design and is what keeps a large rule collection comprehensible and extensible as it grows.

Example #1: Detecting Zeus by Its HTTP Request Pattern

We now turn to a rule drawn from the real world rather than a textbook. It targets **Zeus**, one of the most consequential banking trojans in the history of malware, and it detects the family through distinctive elements of the HTTP request pattern Zeus uses to communicate. Studying it ties together nearly everything established so far — a PE check, a string family with `all of`, alternative indicators with `or` — and it offers an instructive window into both why network and string indicators are so valuable and why Zeus in particular was so difficult to analyze at the code level.

```
import "pe"

rule Windows_Malware_Zeus : Zeus_1134 {

    meta:
        author = "Xylitol xylitol@malwareint.com"
        date = "2014-03-03"
        description = "Match first two bytes, protocol and string present in Zeus
1.1.3.4"
        reference = "http://www.xylibox.com/2014/03/zeus-1134.html"

    strings:
        $mz = {4D 5A}
        $protocol1 = "X_ID: "
        $protocol2 = "X_OS: "
        $protocol3 = "X_BV: "
        $stringR1 = "InitializeSecurityDescriptor"
```

```
        $stringR2 = "Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; SV1)"

    condition:
        ($mz at 0 and all of ($protocol*) and ($stringR1 or $stringR2))
}
```

The rule's tag, `Zeus_1134`, records that this fingerprint targets Zeus version 1.1.3.4 specifically, and the `meta` section provides authorship, a date, a human-readable description of the matching strategy, and a reference URL documenting the sample. As always, the metadata steers no logic; it is context for whoever acts on a hit.

What This Rule Is Really Looking At

It is important to frame what the rule is examining. It is not merely scanning an inert data file in the abstract; it is searching the malware's own body for the artifacts of its behavior — in particular, the character strings the malware embeds and uses when it communicates with its operators. Malware frequently carries, in plain form within its binary, the fixed strings it will later send over the network or the names of the routines it relies upon. These embedded strings are a goldmine for detection precisely because they are stable identifiers of the family's machinery.

The PE Check: `$mz at 0`

The observable `$mz = {4D 5A}` is the two-byte sequence `4D 5A`, which are the ASCII letters **"MZ."** Every Windows executable begins with these two bytes; they are the opening of the DOS header that prefixes a Portable Executable, and they serve as the magic marker identifying the file as a Windows program. Requiring `$mz at 0` — the bytes "MZ" at offset zero — confirms that the artifact under examination really is a Windows executable rather than, say, an ELF binary, a smaller script, or some other file type. (In a complete PE, these two bytes are followed by further header fields and, deeper in the file, the actual "PE" signature that marks the start of the PE header proper; for the purpose of this rule, "MZ" at the very start is sufficient to establish that we are dealing with a Windows executable.) This is the same anchored-magic-number technique seen in the PDF rule, now applied to confirm an executable's identity.

The Protocol Strings: `all of ($protocol*)`

The three protocol observables — `$protocol1 = "X_ID: "`, `$protocol2 = "X_OS: "`, and `$protocol3 = "X_BV: "` — are the distinctive header fields of Zeus's custom communication protocol. When Zeus phones home, its HTTP requests carry these characteristic fields, and their simultaneous presence is a strong indicator of the family. The condition requires `all of ($protocol*)`, which, using the named-prefix set selection introduced earlier, means every observable whose name begins with `$protocol` must be present. All three protocol headers must therefore appear for this part of the condition to hold, reflecting the fact that they travel together as a set in genuine Zeus traffic.

The Alternative Indicators: `$stringR1 or $stringR2`

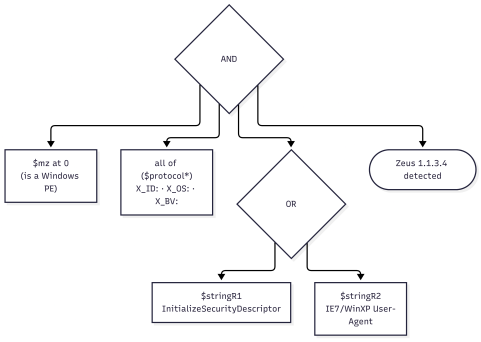
The remaining two observables are alternative corroborating indicators, and the condition requires that **at least one** of them be present via `($stringR1 or $stringR2)`. The first, `$stringR1 = "InitializeSecurityDescriptor"`, is the name of a Windows API routine that the malware references. The second, `$stringR2 = "Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; SV1)"`, is a complete HTTP **User-Agent** string.

That User-Agent deserves commentary, because it reveals a deliberate evasion tactic. By embedding a legitimate-looking browser User-Agent, Zeus disguises its network traffic as ordinary web browsing. The string identifies the client as Internet Explorer 7 running on Windows NT 5.1 — the internal version number corresponding to **Windows XP** — with the `SV1` token indicating the hardened "Security Version 1" browser configuration of that era. In a network-traffic detector, a request bearing a standard, familiar User-Agent is likely to blend in and pass unnoticed, which is exactly the point.

It is worth noting, though, that hard-coding a single fixed User-Agent is not an especially sophisticated form of disguise. Precisely because the value is constant, it becomes a reliable signature — the very property this rule exploits. A more careful adversary would vary the User-Agent and keep it closely aligned with whatever legitimate software is prevalent on the victim population, so that no single value stands out. The reason the rule can afford to match on *either* `$stringR1` *or* `$stringR2` rather than requiring both is that different Zeus variants exhibited one indicator or the other; the author judged that the presence of either, in combination with the protocol headers and the PE check, was sufficient evidence of this family.

The Complete Condition

Putting the pieces together, the rule fires when the artifact is a Windows executable, carries all three of Zeus's protocol header strings, and contains at least one of the two corroborating strings:



Why String-Based Detection Matters: Zeus's Virtual-Machine Obfuscation

A deeper lesson lurks behind this rule. Zeus was not only historically important for its impact; it was technically notable as one of the earliest samples to employ an advanced anti-analysis technique known as **virtual-machine (VM) based obfuscation**, sometimes called virtualization obfuscation. Understanding it explains *why* analysts lean so heavily on string and network indicators for such families.

Under normal circumstances, a program is a sequence of native machine instructions that a disassembler can translate back into readable assembly, giving the analyst a clear view of the code's logic. VM-based obfuscation breaks this entirely. The malware author designs a *custom assembly language* — an instruction set that exists nowhere but inside this one piece of malware — and translates the original code into bytecode for that invented language. The shipped malware then consists of two parts: an **interpreter** (a small virtual machine) that knows how to execute the custom bytecode, and the **original logic re-expressed as a program in that custom bytecode.**

The consequence for an analyst is severe. Disassembling the binary no longer yields the program's real logic; it yields only the interpreter's dispatch loop and an opaque blob of custom bytecode. Every function of the real malware has been converted into a sequence of operations in a language the analyst must first reverse-engineer before any of the actual behavior becomes legible. Defeating this kind of

obfuscation — recovering the original semantics from a custom virtual machine — is extremely hard and is, in the general case, an open problem.

Zeus's implementation was sophisticated but not insurmountable; with sufficient patience an analyst could reconstruct the obscured semantics, and a common fallback is to ignore the tangled code and instead observe the program's **API calls**, inferring what the malware is trying to accomplish from the system services it invokes. But the broader takeaway is this: when a family's *code* is deliberately made unreadable, its *strings and network behavior* often remain exposed and stable, because the malware must still speak a recognizable protocol and still reference real system routines to function. For a large, well-studied family like Zeus, recognizing it through these surface indicators is therefore far simpler and more reliable than fighting through its code-level obfuscation — which is exactly the pragmatic strategy this YARA rule embodies.

Example #2: Identifying PE Files Correctly

Earlier rules confirmed that a file was an executable by checking for the two bytes "MZ" at offset 0. That check is a useful first approximation, but it is weak: the bytes "MZ" at the start merely indicate the old DOS-style header and do not, by themselves, prove the file is a genuine Portable Executable. A far more reliable identification verifies *both* the DOS signature at the start *and* the actual "PE" signature at the location the file itself says the PE header lives. Building that rule requires understanding the layout of the PE format, why it carries a DOS header at all, and a powerful new YARA capability: **indirection**, the ability to follow a pointer stored inside the file.

The DOS Heritage of the PE Format

To understand why a Windows executable begins with "MZ" and a block of DOS-era code, one must recall that Windows did not originate as a standalone operating system. In its early incarnations — through Windows 3.1 and Windows for Workgroups 3.11 — Windows was itself a *program* that you launched from within MS-DOS. For compatibility, Windows executables therefore had to coexist with the DOS world, and this legacy is baked permanently into the PE file format.

The result is that every PE file begins with a **DOS header** followed by a small **DOS stub** program. The header starts with the magic bytes `4D 5A` ("MZ"). The stub is a tiny real-mode program whose only job is to run when someone mistakenly executes the file under plain DOS; it prints the familiar message **"This program cannot be run in DOS mode."** and exits. This is the compatibility mechanism in action: a modern Windows program run under DOS will not crash incomprehensibly — DOS reads the stub, prints the message, and declines to run the real program, which it cannot understand. The genuine Windows code — the actual PE header and everything after it — lives further into the file, and the DOS header contains a field that tells the loader exactly where to find it.

The Header Layout and the Pointer at Offset 0x3C

Consider the opening bytes of a real PE file, laid out sixteen bytes per row (each row's offset is shown on the left, and the printable interpretation on the right):

```
00000000  4d 5a 90 00 03 00 00 00 04 00 00 00 ff ff 00 00  |MZ..............|
00000010  b8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00  |........@.......|
00000020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
00000030  00 00 00 00 00 00 00 00 00 00 00 00 c8 00 00 00  |................|
00000040  0e 1f ba 0e 00 b4 09 cd 21 b8 01 4c cd 21 54 68  |........!..L.!Th|
00000050  69 73 20 70 72 6f 67 72 61 6d 20 63 61 6e 6e 6f  |is program canno|
```

```
00000060  74 20 62 65 20 72 75 6e 20 69 6e 20 44 4f 53 20  |t be run in DOS |
00000070  6d 6f 64 65 2e 0d 0d 0a 24 00 00 00 00 00 00 00  |mode....$.......|
                            ...
000000c0  00 00 00 00 00 00 00 00 50 45 00 00 4c 01 03 00  |........PE..L...|
```

Two landmarks matter. At the very top, offset 0 holds `4D 5A` — "MZ." Reading across the dump, rows `0x40` through `0x70` contain the DOS-stub code (`0e 1f ba 0e ...`) and the "This program cannot be run in DOS mode." message. Then, down at offset `0xC8` , the bytes `50 45 00 00` appear — that is **"PE"** followed by two zero bytes, the real signature of the PE header. Immediately after it, `4C 01` encodes the target machine (`0x014C` , meaning Intel i386).

The crucial question is: how does the loader know the PE header is at `0xC8` and not somewhere else? The answer lies in a specific field of the DOS header located at **offset `0x3C`** . This four-byte field — known as `e_lfanew` — holds the offset at which the PE header begins. To find it in the dump, count sixteen bytes per row: offset `0x30` begins the last row shown before the stub, and twelve bytes into that row (positions `0x3C` through `0x3F`) we find `c8 00 00 00` . That value is the pointer to the PE header.

Little-Endian: Reading the Pointer Value

The four bytes `c8 00 00 00` must be interpreted as a single 32-bit integer, and here the **endianness** matters. x86 systems store multi-byte integers in **little-endian** order, meaning the least significant byte comes first. The byte sequence `c8 00 00 00` therefore represents the integer `0x000000C8` — simply `0xC8` , or 200 in decimal. And indeed, jumping to offset `0xC8` in the file lands exactly on the `50 45 00 00` ("PE") signature. The DOS header's field at `0x3C` thus acts as a pointer: it is a value stored in the file that tells you where to look next.

Indirection in YARA: `uint32()` and `at`

This is where YARA's **indirection** capability becomes essential. A naive rule would hard-code the PE header's location, but the PE header does not live at a fixed offset — its position varies from file to file, which is precisely why the format stores a pointer to it at `0x3C` . To write a general rule, you must *read* that pointer and then *follow* it. YARA lets you do exactly this.

The function `uint32(offset)` reads a 32-bit, little-endian unsigned integer from the given offset in the file. Thus `uint32(0x3c)` reads the four bytes at offset `0x3C` and yields the integer they encode — in our example, `0xC8` . This computed value can then itself be used as an offset. The expression `$pe at (uint32(0x3c))` therefore means: "the string `$pe` must appear at the offset obtained by reading the 32-bit pointer stored at `0x3C` ." You are no longer matching at a fixed location; you are matching at a location the file itself computes for you. This is indirection — following an offset stored in the data — and it mirrors how the operating system's own loader navigates the file.

The `is_PE_file` Rule

With indirection understood, the robust PE-identification rule is compact and elegant:

```
rule is_PE_file {

    strings:
        $mz = "MZ"
        $pe = "PE"
```

```
    condition:
        ($mz at 0) and
        ($pe at (uint32(0x3c)))
}
```

It makes two demands. First, `$mz at 0` requires the DOS signature "MZ" at the file's very beginning. Second, and far more discriminating, `$pe at (uint32(0x3c))` requires the "PE" signature to appear at exactly the offset that the file's own `e_lfanew` field points to. A file can only satisfy the second condition if it has a well-formed DOS header whose pointer genuinely leads to a valid PE signature — a property that a mere "MZ" at the start does not guarantee. This rule therefore identifies true Portable Executables with confidence, and it demonstrates indirection as a general technique: whenever a format stores the location of one structure inside another, YARA can read the pointer and follow it.



File Offsets Versus Virtual Addresses: RVA and the Base Address

One final concept generalizes well beyond this rule and is foundational for later work on imports, shared objects, and unpacking. The pointer value `0xC8` we just followed is a **file offset** — a position measured from the start of the file on disk. But Windows internals make extensive use of a *different* kind of offset once a program is loaded into memory, and conflating the two is a common source of error.

When the operating system loads an executable into memory, it places it at some **base address**. A program is never loaded at address 0, because the lowest region of the address space (the "zero page") is reserved; a typical base address for a classic executable is `0x400000`. Many structures inside a PE — among them the import tables analyzed during reverse engineering and unpacking — are referenced not by file offsets but by offsets *relative to this base address*. Such an offset is called a **Relative Virtual Address (RVA)**. To find where a structure actually resides in memory, the loader computes:

$$\text{virtual address} = \text{base address} + \text{RVA}$$

So a stored value like `0xC8`, considered as an RVA rather than a file offset, would correspond to the in-memory address `0x400000 + 0xC8` when loaded at base `0x400000`. The same numeric offset therefore has two possible meanings depending on context: as a file offset it locates bytes on disk, and as an RVA it locates bytes in the loaded image relative to wherever the loader placed it. Keeping this distinction clear is essential when a rule or an analysis must follow Windows' pervasive web of relative offsets, and it is why understanding base addresses and RVAs accompanies any serious work with the PE format.

More Advanced Cases: Recognizing Code Patterns

Everything so far has matched either textual indicators or well-defined structural facts. The most demanding — and most powerful — use of YARA is matching **patterns in machine code**, where the goal is to recognize a characteristic sequence of instructions despite the variability that real code introduces. This section works through a complete, realistic case: an obfuscated string hidden inside code, recovered by reading the bytes as x86 instructions and then captured with a flexible YARA pattern. It is the single best illustration of why wildcards, jumps, and regular expressions exist.

An Obfuscated String Hidden in Code

Consider the following region of a binary, shown as a hex dump with the ASCII interpretation on the right:

```
00415393  C6 45 CC 53  C6 45 CD 6F  C6 45 CE 66  C6 45 CF 74   .E.S.E.o.E.f.E.t
004153A3  C6 45 D0 77  C6 45 D1 61  C6 45 D2 72  C6 45 D3 65   .E.w.E.a.E.r.E.e
004153B3  C6 45 D4 5C  C6 45 D5 4D  C6 45 D6 69  C6 45 D7 63   .E.\.E.M.E.i.E.c
004153C3  C6 45 D8 72  C6 45 D9 6F  C6 45 DA 73  C6 45 DB 6F   .E.r.E.o.E.s.E.o
004153D3  C6 45 DC 66  C6 45 DD 74  C6 45 DE 5C  C6 45 DF 57   .E.f.E.t.E.\.E.W
004153E3  C6 45 E0 69  C6 45 E1 6E  C6 45 E2 64  C6 45 E3 6F   .E.i.E.n.E.d.E.o
004153F3  C6 45 E4 77  C6 45 E5 73  C6 45 E6 5C  C6 45 E7 43   .E.w.E.s.E.\.E.C
00415403  C6 45 E8 75  C6 45 E9 72  C6 45 EA 72  C6 45 EB 65   .E.u.E.r.E.r.E.e
00415413  C6 45 EC 6E  C6 45 ED 74  C6 45 EE 56  C6 45 EF 65   .E.n.E.t.E.V.E.e
00415423  C6 45 F0 72  C6 45 F1 73  C6 45 F2 69  C6 45 F3 6F   .E.r.E.s.E.i.E.o
00415433  C6 45 F4 6E  C6 45 F5 5C  C6 45 F6 52  C6 45 F7 75   .E.n.E.\.E.R.E.u
00415443  C6 45 F8 6E                                         .E.n
```

At first glance the ASCII column is unhelpful — it shows a monotonous `.E.S.E.o.E.f...` in which a recurring "E" and scattered dots obscure anything meaningful. A naive strings-extraction tool would find nothing recognizable here. Yet a recognizable string *is* present; it is simply concealed by the structure of the code. The task is to find the regularity in the bytes and understand what the code is doing.

Finding the Regularity

Scanning the dump reveals strong patterns. Reading the bytes four at a time, the **first two bytes of every group are identical: `C6 45`**. The **third byte increases steadily** — `CC, CD, CE, CF, D0, D1, D2, ...` — behaving exactly like a counter incrementing by one at each step. The **fourth byte is irregular**, varying from group to group with no obvious arithmetic pattern.

The misleading "E" in the ASCII column now has an explanation: the second byte of every group is `45`, and `0x45` is the ASCII code for the letter "E." The printable rendering therefore stamps an "E" into every group, and the surrounding non-printable bytes (`C6`, and the counter bytes) render as dots. The real content of interest is the *fourth* byte of each group — the irregular one — but to be sure, we must stop guessing from the ASCII view and instead interpret the bytes as what they actually are: **x86 machine instructions.**

Decoding the Opcode: `C6 45` Is a Single-Byte `MOV`

The repeating `C6 45` prefix is the key. Using an x86 opcode map — a chart that organizes the one-byte opcodes by their high and low nibbles — the byte `C6` falls in the region of the `MOV` family of instructions. To pin down precisely which `MOV`, one consults the processor's instruction reference. The relevant entry is:

> **`C6 /0` — `MOV r/m8, imm8`** — *Move imm8 to r/m8.*

So `C6` is the opcode for a `MOV` that moves an **8-bit immediate value (a single byte) into an 8-bit destination** (`r/m8`). The `/0` notation indicates that this opcode uses the `reg` field of the following ModR/M byte as an opcode extension with value 0, rather than to select a second register.

The second byte, `45`, is that **ModR/M byte**, and it encodes the destination. Decoding `0x45` as bits gives `01 000 101`, which splits into three fields: `mod = 01`, `reg = 000`, and `r/m = 101`. The `reg = 000` matches the required `/0` extension. The combination `mod = 01` with `r/m = 101` denotes, per the

32-bit addressing-forms table, the effective address **`[EBP] + disp8`** — the `EBP` register plus a single signed displacement byte. The `mod = 01` specifically signals that exactly one displacement byte (`disp8`) follows the ModR/M byte.

Putting this together, each four-byte group is a complete instruction of the form:

```
mov byte ptr [ebp + disp8], imm8
```

The four bytes decompose as follows:

Byte	Field	Example	Meaning
`C6`	opcode	`C6`	`MOV r/m8, imm8` — move a 1-byte immediate into an 8-bit destination
`45`	ModR/M	`mod=01, reg=/0, r/m=101`	destination is `[EBP] + disp8`
3rd byte	`disp8`	`CC`	signed displacement from `EBP` — the index into the buffer
4th byte	`imm8`	`53`	the immediate byte being stored

The mystery now resolves completely. The third byte is the **`disp8` displacement** — the index of where in a local buffer the byte is written. Because it is a signed 8-bit value, `0xCC` is negative (−52), placing the buffer in the function's local stack frame just below `EBP` ; a disassembler such as IDA would name this local variable and render the instructions as `mov [ebp+SubKey], ...` , `mov [ebp+SubKey+1], ...` , and so on, with `SubKey` equal to the first displacement `0xCC` . The displacement increments by one each instruction because the code is filling consecutive positions of the buffer. The fourth byte is the **`imm8` immediate** — the actual character being stored. The code is copying a string into a stack buffer **one byte at a time.**

Reconstructing the Hidden String

Collecting the fourth byte (`imm8`) of every group and interpreting each as an ASCII character reconstructs the concealed string:

```
53 6F 66 74 77 61 72 65 5C 4D 69 63 72 6F 73 6F 66 74 5C 57 69 6E 64 6F 77 73 5C
 S  o  f  t  w  a  r  e  \  M  i  c  r  o  s  o  f  t  \  W  i  n  d  o  w  s  \
43 75 72 72 65 6E 74 56 65 72 73 69 6F 6E 5C 52 75 6E
 C  u  r  r  e  n  t  V  e  r  s  i  o  n  \  R  u  n
```

The hidden string is:

```
Software\Microsoft\Windows\CurrentVersion\Run
```

This is the well-known Windows registry key used to achieve **persistence**: programs listed under `...\CurrentVersion\Run` are launched automatically every time the user logs in. Its presence is a strong indicator that the code is installing itself to survive reboots.

Why This Technique Defeats String Scanning

The reason the malware builds the string with a long series of single-byte `MOV` instructions, rather than storing it as an ordinary string literal, is precisely to **conceal it from static analysis**. If the string `Software\Microsoft\Windows\CurrentVersion\Run` were stored contiguously in the binary, any simple strings-extraction tool would find it instantly and flag the persistence behavior. By scattering each character into a separate instruction's immediate operand, the contiguous string never appears anywhere in the file — the letters are interleaved with opcode and displacement bytes — and naive string scanning comes up empty. This "stack string" construction was historically common, especially in the era before friendly compilers and disassemblers, when authors would assemble strings character by character on the stack specifically because the tooling of the day would not display them. Recovering such a string requires understanding the code, exactly as we just did.

Writing the Signature

The payoff of all this analysis is that the construction leaves a highly characteristic *code* pattern that YARA can match, even though the string itself is invisible. The signature reasoning is:

- **`0xC6 0x45` is constant** — it is the fixed opcode and ModR/M byte, present in every instruction of the sequence.
- **`disp8` (the index) is variable**, but restricted to a single byte.
- **The character (`imm8`) is variable**, but also restricted to a single byte.

The sequence therefore looks like `C6 45 ?? ??` repeated, where each `??` is one arbitrary byte:

```
C6 45 ?? ??  C6 45 ?? ??  C6 45 ...
```

One could write the two variable bytes with single-byte wildcards (`??`), but a cleaner approach uses **jumps**. A jump, written in square brackets, specifies a run of variable-length content; `[2]` means "exactly two arbitrary bytes here," and the general form `[x-y]` means "between x and y arbitrary bytes." Using jumps, the rule becomes:

```
rule single_byte_mov {

    strings:
        $a = { C6 45 [2] C6 45 [2] C6 45 }

    condition:
        $a
}
```

Here `$a` matches the fixed `C6 45`, then skips exactly two variable bytes (the `disp8` and `imm8`), then `C6 45` again, another two-byte skip, and a final `C6 45`. This confirms the presence of three consecutive single-byte `MOV` instructions of the right form — enough to recognize the pattern, though it only spans a few characters of the hidden string.

To match the **full** repeated sequence of arbitrary length, a **regular expression** is more suitable and more flexible:

```
$a = /(\xc6\x45..){3,}/
```

This regex reads as: the two literal bytes `\xc6\x45`, followed by `..` — two arbitrary bytes in regex notation — all grouped together and repeated `{3,}`, meaning **three or more times**. It therefore captures a run of at least three (and as many as exist) consecutive `C6 45 ?? ??` instructions, matching the entire stack-string construction however long it is. This regex form is the better choice when you want to recognize the whole pattern rather than just a fixed-size fragment, and it exemplifies why regular-expression observables are indispensable for flexible code matching.

The broader methodology demonstrated here is the heart of advanced YARA authoring: identify the *invariant* part of a code construct (the opcode and addressing bytes that never change), treat the *variable* part (operands, displacements, immediates) as wildcards or jumps, and express the repetition with a jump count or a regex quantifier. The result is a rule that recognizes a behavior — here, building a persistence string on the stack — robustly across samples, regardless of the specific characters involved.

More Advanced Cases: Matching Algorithmic Constants

A second powerful strategy for code recognition exploits the **constants** embedded in algorithms. Many well-known algorithms are defined by specific, fixed numeric values, and when a program implements such an algorithm it must carry those values literally in its code or data. Those literals become excellent fingerprints, because they are both distinctive and semantically meaningful — finding them tells you not merely that *some* code is present, but that code implementing a *particular* algorithm is present.

Why Constants Make Good Signatures

The guiding principle is to **look for constants that are important to an algorithm**, and to prefer the longest distinctive sequences available, because the longer the sequence, the lower the probability of a coincidental match and therefore the fewer false positives. Several categories of algorithmic constant are especially valuable:

The **static substitution box (S-box) of DES** is a large, fixed table of values that any DES implementation must contain. Its sheer size and specificity make it almost impossible to encounter by chance, so matching it is strong evidence of DES. Similarly, the **initialization and transform constants of MD5** are fixed values baked into the MD5 algorithm, and the **polynomial used in a Cyclic Redundancy Check (CRC)** is a defining constant of a particular CRC variant. Recognizing any of these lets an analyst conclude that code is performing a well-known cryptographic or integrity operation — unless, of course, the relevant data has itself been obscured, which remains a limitation of the approach.

Pitfalls When Matching Constants

Three practical hazards complicate the matching of numeric constants, and each calls for a more conservative, flexible rule.

The first is **endianness**. x86 systems store multi-byte values in little-endian order, so the value `0x1234` is stored in memory as the bytes `0x34 0x12`, not `0x12 0x34`. But data does not always stay in native order: values sent over a network are conventionally transmitted in big-endian ("network byte order"), so a constant captured from network traffic — or from a memory dump of data destined for the network — may appear with its bytes reversed relative to how it sits in CPU registers. A rule author must therefore be mindful of which representation applies. A conservative tactic is to **break long numbers into smaller pieces and account for both the little-endian and big-endian variants**, so the rule matches regardless of byte order.

The second pitfall is **register and operand variability**. The same algorithm compiled in two contexts may not use the same registers; one build might operate on `EAX`, another on a different register, so the exact instruction bytes differ even though the logic is identical. As with operand addresses seen earlier, the remedy is to mask the bytes that encode the register selection and match only the invariant parts.

The third pitfall is **compiler optimization**. A compiler may transform the straightforward code an author wrote into a different but equivalent instruction sequence, and heavy optimization can reshape patterns substantially. A rule written against the unoptimized form may silently fail on an optimized build. The practical advice is to be flexible — overly narrow rules break over time and force costly rewrites as malware and toolchains evolve — and, where it matters, to maintain both an optimized and an unoptimized variant of the pattern. Recognizing these transformations requires experience and familiarity with how compilers alter code.

Case Study: A Linear Congruential Generator in PoisonIvy

A concrete and instructive example is a pseudo-random number generator found in the **PoisonIvy** malware. It implements a **Linear Congruential Generator (LCG)**, one of the classic recurrences for producing a sequence of pseudo-random numbers. An LCG is defined by:

$$x_{n+1} = (a \cdot x_n + c) \bmod m$$

Each new value is obtained from the previous one by multiplying by a constant multiplier a, adding a constant increment c, and reducing modulo m. The specific constants a, c, and m characterize a particular generator, and it is exactly these constants that give us something to match.

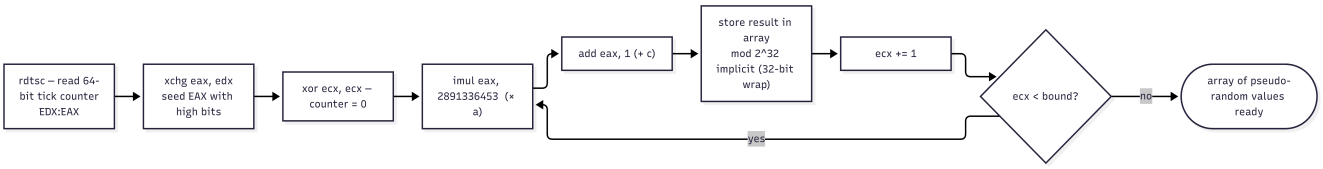
The relevant disassembly is:

```
00000DA5 rand_init:
00000DA5        lea    esi, [ebp+base]
00000DAB        rdtsc                       ; seed with CPU tick counter
00000DAD        xchg   eax, edx
00000DAE        xor    ecx, ecx
00000DB0 rand_loop:                         ; LCG  x := (x * 2891336453 + 1) mod 2^32
00000DB0        imul   eax, 2891336453      ; { 05 4B 56 AC } little-endian
00000DB6        add    eax, 1
00000DB9        mov    [esi+ecx*4+8D9h], eax
00000DC0        add    ecx, 1
00000DC3        cmp    ecx, 34
00000DC6        jb     short rand_loop
```

Seeding the generator. The routine first prepares a destination pointer with `lea esi, [ebp+base]`, then executes `rdtsc`. The `RDTSC` instruction reads the processor's **timestamp counter** — a continuously incrementing count of CPU clock ticks — and returns its 64-bit value split across two registers: the low 32 bits in `EAX` and the high 32 bits in `EDX`. Because the timestamp is essentially unpredictable at fine granularity, it makes a convenient seed. The subsequent `xchg eax, edx` swaps the two halves so that the **high-order bits** (originally in `EDX`) end up in `EAX`, which is the register the generator will use. Conceptually, if one imagines the 64-bit counter like a long clock reading, the high half holds the slowly-changing portion (the "hours and minutes") while the low half holds the rapidly-changing fractional part; the code deliberately seeds from the upper half. The entropy of such a seed is not

enormous, but high entropy is not the goal — the aim is merely a non-constant starting point. Finally `xor ecx, ecx` zeroes `ECX`, which will serve as the loop counter.

The generation loop. `rand_loop` implements the LCG recurrence directly. `imul eax, 2891336453` multiplies the current state by the multiplier $a = 2891336453$. `add eax, 1` adds the increment $c = 1$. The modulus $m = 2^{32}$ is implicit: because the arithmetic is performed in 32-bit registers, results automatically wrap around modulo 2^{32}, so no explicit modulo operation is needed. The resulting pseudo-random value is stored with `mov [esi+ecx*4+8D9h], eax` into an array (indexed by `ECX`, scaled by 4 for 32-bit elements), after which `add ecx, 1` advances the counter, `cmp ecx, 34` tests it against the fixed bound, and `jb short rand_loop` repeats until the array is filled.



The constant and its little-endian encoding. The multiplier $a = 2891336453$ in hexadecimal is `0xAC564B05`. Because x86 stores immediates in little-endian order, this value appears in the instruction's bytes as `05 4B 56 AC` — precisely the byte sequence annotated in the disassembly. This large, specific constant is the strongest available fingerprint, since there is no ordinary reason for an unrelated program to contain exactly this four-byte value.

The `has_rng` Rule

Two features of this routine are distinctive enough to build a rule around: the use of `RDTSC` and the presence of the LCG multiplier constant. `RDTSC` encodes to the two opcode bytes `0F 31`, and the constant is `05 4B 56 AC`. The corresponding rule requires both:

```
rule has_rng {

    strings:
        $a = { 0F 31 }        // rdtsc
        $b = { 05 4B 56 AC }  // static value of parameter

    condition:
        all of them
}
```

The observable `$a` matches the `RDTSC` instruction and `$b` matches the little-endian constant; `all of them` requires both to be present. Together they target the two aspects of the routine that are not easy to vary — a hardware instruction used for seeding and a specific large algorithmic constant — which is exactly the "look for distinctive constants" strategy in practice.

False Positives and the Precision–Recall Balance

The slide's own warning — *watch out for false positives!* — is important, and it illustrates a general truth about constant-based rules. Neither indicator is individually conclusive. `RDTSC` (`0F 31`) is a very commonly used instruction, appearing in all sorts of legitimate and malicious programs, so matching it alone would be nearly meaningless. The four-byte constant `05 4B 56 AC` is more distinctive, but even it could appear by coincidence somewhere in a large binary, or arise from two unrelated adjacent instructions whose bytes happen to align. Requiring *both* with `all of them` sharply reduces the false-

positive rate compared to either alone, but it does not eliminate the possibility, since the two observables could still co-occur in a file for unrelated reasons without being part of this generator.

The rule can be tightened further by constraining the **relative position** of the two indicators — for example, requiring that `$a` and `$b` appear within a limited distance of one another (say, within twenty or thirty bytes), which is reasonable when the code is laid out consecutively as here. Doing so raises precision by demanding that the two findings be genuinely part of the same code region rather than incidental neighbors across the file. But there is a limit: making a rule too precise risks missing legitimate variants, so every tightening trades **recall** (catching all real instances) against **precision** (avoiding false alarms). This precision–recall balance is a recurring theme, and the disciplined way to manage it is empirical: write the rule, run it against your curated collection of analyzed samples, and observe whether it is too tight (missing known matches) or too loose (flagging benign files), adjusting accordingly. Experience with this feedback loop is what ultimately produces well-calibrated rules.

Inferring Semantics from Imports: Keylogger Detection

A theme that recurs throughout advanced YARA authoring is combining structural facts — especially imports — to **infer what a program is capable of doing**, even when no single indicator is damning. The following rule applies this idea to detect a probable keylogger by combining a persistence indicator with two revealing API imports.

```
import "pe"

rule potential_keylogger {

    strings:
        $autorun = "Software\Microsoft\Windows\CurrentVersion\Run" wide ascii

    condition:
        pe.imports("User32.dll", "SetWindowsHookExA")
        and
        pe.imports("User32.dll", "RegisterHotKey")
        and
        $autorun
}
```

The `$autorun` observable is the same `...\CurrentVersion\Run` registry key encountered earlier — the classic persistence location — here given the `wide ascii` modifiers so it is matched in both single-byte and two-byte-per-character encodings. Its presence suggests the program intends to install itself to run automatically at login.

The two import checks reveal the program's behavioral capabilities. `SetWindowsHookExA` is a `User32.dll` function that lets a program **register callbacks that fire when specific events occur**, either for a single application or for the entire system. This is the fundamental mechanism used to implement keyloggers: by installing the appropriate hook, a program can intercept and record keyboard input, mouse movement, and other system-wide events. `RegisterHotKey`, also from `User32.dll`, registers global hotkeys. Neither function is malicious in isolation — legitimate software uses both — but the *combination* of event-hooking capability, hotkey registration, and a persistence mechanism paints a coherent and suspicious picture.

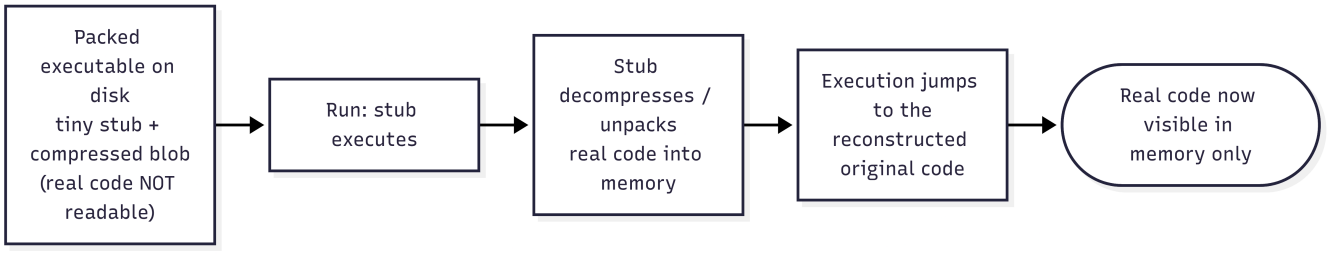
The condition requires all three together: the program imports `SetWindowsHookExA`, imports `RegisterHotKey`, and contains the autorun registry path. Any one of these alone would be weak evidence, but their conjunction strongly suggests a program that installs itself persistently and hooks system events to capture user activity — the behavioral profile of a keylogger. This is the same inferential strategy seen in the `suspicious_pe` and Zeus rules, applied to a new behavior: YARA is used not to match a known signature but to **reason about intent from capability.**

Detecting Packed Executables

The final and most sophisticated detection problem concerns **packed executables**. Packing is among the most common techniques malware uses to frustrate analysis, and recognizing it is a routine part of triage.

What Packing Is and Why It Is Used

Packing is a technique that hides a program's real code so that it is not available to static analysis. When a packed executable is opened and inspected on disk, its genuine instructions are not present in readable form; instead, the real code is **extracted at run time**, during execution, and only then becomes available in memory. A useful analogy is a self-extracting archive — like a zip file that unpacks itself — except that the packed artifact is an executable rather than an archive, and the extraction happens entirely **inside memory within a single program**. A packed executable begins with a small stub of code whose job is to decompress (or otherwise reconstruct) the real code and expand it into memory, after which execution transfers to the now-revealed original program.



Historically, packing served several purposes. In an era of slow network connections, compressing an executable reduced download size and time, while still delivering a single clickable program rather than an archive the user would have to extract manually. Packing was also used to **protect intellectual property**: by hiding the code and sometimes embedding license or environment checks that must pass before the real code is revealed, developers shielded their programs from reverse engineering. Today a whole industry of commercial packers exists precisely to armor code against analysis, keeping the program functional while making static inspection fruitless, because the code that actually runs is only materialized at run time in the addresses where it expects to reside.

The analytic response to packing is **unpacking**: a process that drives the program's execution to the point where all of its code has been reconstructed in memory, then captures a copy of that revealed code so it can be analyzed normally. (The full unpacking methodology is a substantial topic of its own.) Before unpacking, though, it is valuable simply to *detect* that a file is packed, and YARA supports both heuristic and signature-based detection.

Heuristic Detection: `possibly_packed`

The heuristic approach looks for the structural side effects that packing tends to leave behind, without relying on any specific packer's identity. It uses both the `pe` module and the `math` module.

```
import "pe"
import "math"

rule possibly_packed {

    meta:
        description = "Rule to detect binaries with a big difference between
                       section size and section vsize (after unpack). Also, it
                       includes a big entropy and executable flags"

    condition:
        for any i in ( 0..pe.number_of_sections - 1):(
            (pe.sections[i].virtual_size > pe.sections[i].raw_data_size*2)
            and
            pe.sections[i].characteristics & pe.SECTION_MEM_EXECUTE
            and
            math.entropy(pe.sections[i].raw_data_offset,
                         pe.sections[i].raw_data_size) >= 7
        )
}
```

The `for any ... in` quantifier. The condition introduces an iteration construct: `for any i in (0..pe.number_of_sections - 1):( ... )`. This iterates the index `i` over every section of the executable — from `0` to one less than the total number of sections — and the whole expression is true if the parenthesized predicate holds for **at least one** section. In other words, the rule fires if *any* section exhibits all three packing indicators simultaneously. (A `for all` form exists analogously for requiring every section to satisfy a predicate.)

Indicator 1 — a mismatch between virtual size and raw size. `pe.sections[i].virtual_size > pe.sections[i].raw_data_size*2` compares two quantities a PE records for each section: the **virtual size** (`virtual_size`), which is how much memory the section requests when loaded, and the **raw data size** (`raw_data_size`), which is how many bytes the section actually occupies on disk. In a normal program these are roughly comparable. In a packed program they diverge sharply: a section may contain only a small amount of data on disk — perhaps a few hundred bytes of stub — yet declare that it needs a much larger region in memory, because the real instructions are stored elsewhere (compressed) and will be expanded into that space at run time. Requiring the virtual size to exceed *twice* the raw size captures this telltale inflation.

Indicator 2 — executable permissions. `pe.sections[i].characteristics & pe.SECTION_MEM_EXECUTE` tests, via a bitwise AND, whether the section's characteristics flags include the "memory executable" bit. The point is to focus on sections that will hold *code*. If the size mismatch occurred in a non-executable data section — such as a BSS (uninitialized data) section, which legitimately requests memory it does not occupy on disk — that would be perfectly normal and would produce a false positive. By insisting the suspicious section be executable, the rule aims at the scenario that actually matters: a code section that will be filled in at run time. (This indicator is not foolproof, since execute permissions can be set in various ways, but it meaningfully narrows the focus.)

Indicator 3 — high entropy. `math.entropy(pe.sections[i].raw_data_offset, pe.sections[i].raw_data_size) >= 7` computes, using the `math` module, the **Shannon entropy** of

the section's on-disk bytes. Entropy measures randomness, ranging from 0 to 8 for byte data; ordinary code and text have moderate entropy, while **compressed or encrypted data has very high entropy**, approaching the maximum of 8. Packed sections contain compressed (and often encrypted) payloads, so they exhibit high entropy. A threshold of `>= 7` flags sections whose byte distribution is close to random — a strong signal of compression or encryption consistent with packing.

Taken together, the rule says: *if any executable section both requests far more memory than it occupies on disk and contains high-entropy (compressed-looking) bytes, the binary is probably packed.* This is a robust, packer-agnostic heuristic that recognizes the fingerprints of packing regardless of which tool produced it.

Signature Detection: The `UPX` Rule

The heuristic above detects packing in general; a complementary approach detects a *specific* packer by its distinctive marks. The simplest and earliest widely used packer is **UPX**, and it leaves obvious traces. UPX conventionally renames the executable's sections to `UPX0`, `UPX1`, and so on, and the string `UPX` typically appears in the file.

```
import "pe"

rule UPX {

    strings:
        $upx = "UPX"

    condition:
        pe.sections[0].name contains "UPX0"
        and
        pe.sections[1].name contains "UPX1"
        and
        $upx
}
```

The rule checks three marks: the first section's name contains `UPX0`, the second section's name contains `UPX1`, and the literal string `UPX` is present somewhere in the file. The `pe.sections[i].name` field exposes each section's name, and `contains` performs a substring test. When all three hold, the file bears the classic signature of a UPX-packed executable.

Caveats and the Role of Analyst Judgment

Signature-based packer detection comes with important caveats that reinforce earlier lessons about false positives and analytic skepticism. Because UPX is open source, malware authors frequently **customize** it — altering its stub, renaming its sections, or otherwise modifying it — so a file that looks like UPX may not behave like standard UPX. A clean, standard UPX file can be decompressed trivially with a single command (`upx -d`), but a customized or deliberately disguised variant will cause that command to fail, revealing that it is not genuine UPX. Conversely, a malware author might *mimic* UPX's section naming as a false flag, hoping an analyst will waste effort attempting a routine UPX unpack. Such mimicry — section names like `UPX0` / `UPX1` on something that is not really UPX — is a known tactic, including among sophisticated threat actors who repurpose familiar packers.

The lesson is that a match from a rule like this is not a conclusion but a lead. The rule will faithfully report "this looks like UPX," and it is then the analyst's responsibility — exercising human judgment — to confirm whether it truly is UPX, is a customized variant, or is an impostor. A detection that "smells like UPX" must still be validated by actually attempting to unpack it and observing the result. This interplay between automated pattern matching and human verification is the proper working model for all of YARA: the tool rapidly surfaces candidates and encodes accumulated knowledge, while the analyst supplies the contextual reasoning needed to turn a match into a confident finding.

This completes the arc of YARA's capabilities, from the trivial rule with a constant condition to sophisticated fingerprints that confirm file types through indirection, recognize obfuscated stack-strings by decoding machine code, identify algorithms by their constants, infer behavior from imports, and detect packing through both entropy-based heuristics and packer-specific signatures. In every case the same philosophy applies: describe the invariant essence of what you are looking for, express it flexibly enough to survive real-world variation, combine indicators to balance precision against recall, and validate the results against ground truth.